锂电池剩余寿命预测与梯次利用筛选优化

摘要

面向锂离子电池全寿命状态识别、剩余寿命预测、退役筛选与梯次编组，本文构建“退化机理代理—删失感知预测—概率兼容图—多目标稳健优化”的闭环模型。

针对问题一，以 360 节电池、418,430 条循环记录的半经验仿真数据为对象，用连续三段变点回归识别缓慢、加速和膝点后退化阶段。中位起始点与膝点分别为 486、944 次循环，膝点识别 MAE 为 60.3 次。点估计下温度的可控因素权重最高（0.314），与充电倍率和 DOD 共同构成第一影响梯队；45 °C 配对反事实工况对应寿命损失 39.2%。

针对问题二，按电池编号（Battery_ID）独立划分历史节点数据，采用 Ridge 估计当前 SOH，并用支持右删失的 Weibull 加速失效时间模型预测 RUL。独立测试集 SOH RMSE 为 0.175 个百分点，90% 区间覆盖率为 98.2%；RUL MAE/RMSE 为 49.4/70.0 次循环，覆盖率为 98.6%。所有预处理、模型选择与区间校准均未使用测试电池信息。

针对问题三，对 480 节退役电池先执行分场景硬门控，再在兼容图上枚举完整组并求解 ε-约束 MILP。全 599 列审计后，理想点折中解由缩减空间的 R010 改为 F014：选取 18 组、196 节，利用率 40.8%，净价值 25.53 REU。CHIM 几何膝点同为 F014，但七组权重中仅占 42.9%，故不将其表述为唯一稳定最优。

针对问题四，另设互不重叠的 240 节校准池与 240 节最终留出池，在不读取留出真值时冻结阈值 $p_0 = 0.80$、六类压力场景和三种策略。统一稳健方案在 S0 下形成 6 个完整组，利用率 27.5%，期望净价值 43.80 REU；揭盲后 SOH MAE 为 0.367 个百分点，二次寿命 MAE 为 131.1 次循环，稳健入选组真实基准不匹配率为 16.7%。全局 LHS–Spearman 分析给出前五敏感因素。结果表明，自适应重编组仅恢复 0–4.1% 的压力价值损失，统一稳健策略以规模和收益换取部分违约削减；工程上线前仍须进行真实电芯再标定、热安全试验和滚动更新。

关键词：锂离子电池；剩余寿命；右删失；兼容图；多目标优化；CVaR；梯次利用

1　问题分析与总体框架

问题的核心不是单独追求预测精度或编组收益，而是保证未来信息不进入当前决策，并把电芯级不确定性传播至组级可靠性和经济性。本文按以下链条求解：

$$\begin{aligned}\text{循环观测} &\to \text{阶段与因素} \to \widehat{SOH}, \widehat{RUL} \to \text{退役门控},\\ &\to \text{兼容图与完整组} \to \text{压力传播与稳健决策}.\end{aligned}$$

所有模型选择只使用训练/验证信息；测试集和 Q4 最终留出池分别只在模型或策略冻结后开启。经济量统一记为相对经济单位 REU，不对应人民币报价；安全门控是仿真代理，不能替代热失控、电气绝缘、碰撞和法规认证。

1.1　基本假设

1. 电池是独立统计单元，同一电池的循环行是重复测量，故训练、验证和测试必须按 Battery_ID 分组。
2. 容量衰减可由连续三阶段曲线近似，阶段边界允许因温度、倍率、DOD 和制造批次而异。
3. 汽车阶段 EOL 的 80% SOH 仅作为本仿真的退役阈值；梯次利用阈值按应用场景单独设定。
4. 二次寿命的加速系数、成本和收益系数均为显式场景假设，结论只在给定范围内成立。
5. 完整串联组的有效容量取成员最小容量，等效内阻取成员内阻之和，组失效由短板电芯主导。
6. 概率兼容边反映预测分布下的共同满足概率；公共误差通过相关共同模态采样传播，避免把电芯错误独立化。

1.2　符号

设 i 表示电池，c 表示候选完整组，s 表示应用或压力场景；Q_i, R_i, H_i, L_i 分别为容量、内阻、SOH 和 RUL；$z_c \in \{0,1\}$ 表示是否选择候选组；V_{cs}、I_{cs}、U_{cs} 分别表示净价值、不一致性和利用电芯数；$p_{ij,s}$ 表示电池对在场景 s 下的兼容概率；p_0 是校准后冻结的成边阈值。

2　数据、退化阶段与影响因素（问题一）

2.1　半经验数据与质量控制

数据生成器用累计温度、充放电倍率、DOD、快充策略和批次随机效应驱动容量衰减与内阻增长，并保留隐藏退化边界用于事后检验。最终得到 418,430 行、360 节、6 个批次；缺失与重复 Battery_ID–Cycle 行均为 0，失败事件率为 94.4%。参数来源、单位、允许范围和“文献结构/工程假设/仿真设定”分类均登记在参数注册表中。

2.2 连续三段变点模型

对平滑后的 SOH 序列 $y_i(t)$ 使用连续铰链函数

$$y_i(t) = \beta_{0i} + \beta_{1i}t + \gamma_{1i}(t-\tau_{1i})_+ + \gamma_{2i}(t-\tau_{2i})_+ + \varepsilon_i(t), \qquad \tau_{1i} < \tau_{2i}, \tag{1}$$

其中三个阶段斜率依次为 β_1、$\beta_1+\gamma_1$、$\beta_1+\gamma_1+\gamma_2$。通过最小阶段长度和单调加速约束搜索边界。总体中位 $\tau_1 = 486$、$\tau_2 = 944$；三段 SOH 斜率为 -0.848、-1.807、-3.828 个百分点/100 次循环。隐藏膝点 MAE 为 60.3 次，证实阶段识别能恢复主要加速位置，但删失样本膝点 MAE 上升至 187.5 次。

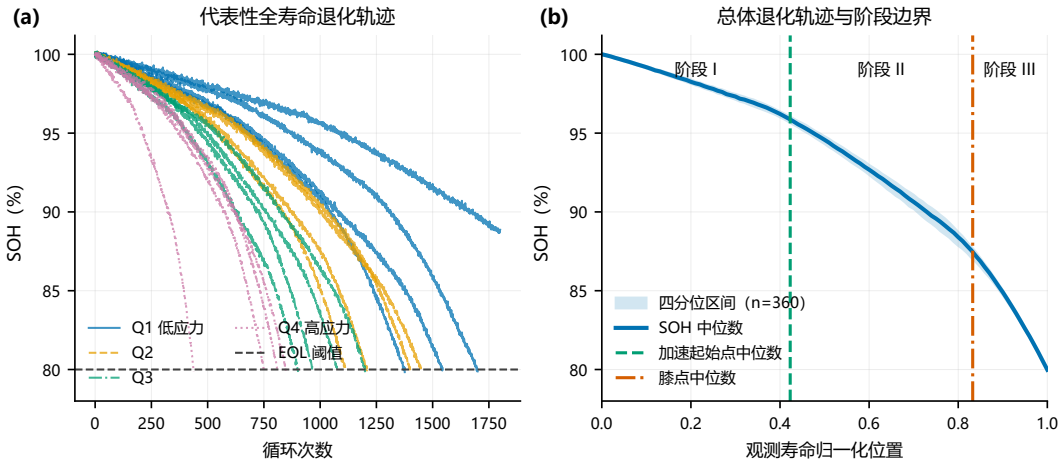

图 1　代表性电池与总体 SOH 全寿命演化；阴影为 360 节电池的四分位区间，竖线为中位阶段边界。

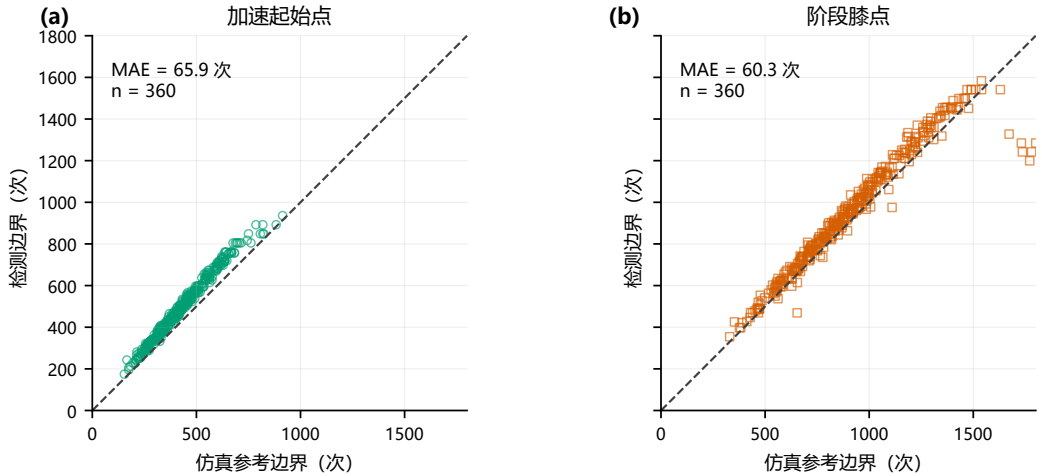

图 2　起始点与膝点检测值对半经验仿真的隐藏参考边界；虚线为理想一致线，每个面板 $n = 360$。

2.3 因素权重与反事实解释

为避免随机森林内置重要度偏差，本文在电池分组留出上同时计算组置换重要度、降组 ΔR^2 和标准化回归贡献，再对名次归一后求共识。模型测试 $R^2 = 0.725$，SOH 与内阻的 Spearman 相关为 -0.819。温度为首要可控因素，共识权重 0.314，排序稳定率

67.7%；第一梯队为温度、充电倍率和 DOD。权重仅解释该仿真机制下的预测贡献，不是因果效应。

在 200 个配对反事实电池上，仅改变一个工况并共享随机扰动。45 °C、2C 充电和 100% DOD 分别造成 39.2%、32.1% 和 23.8% 的寿命损失，为后续压力系数设置提供量级依据。

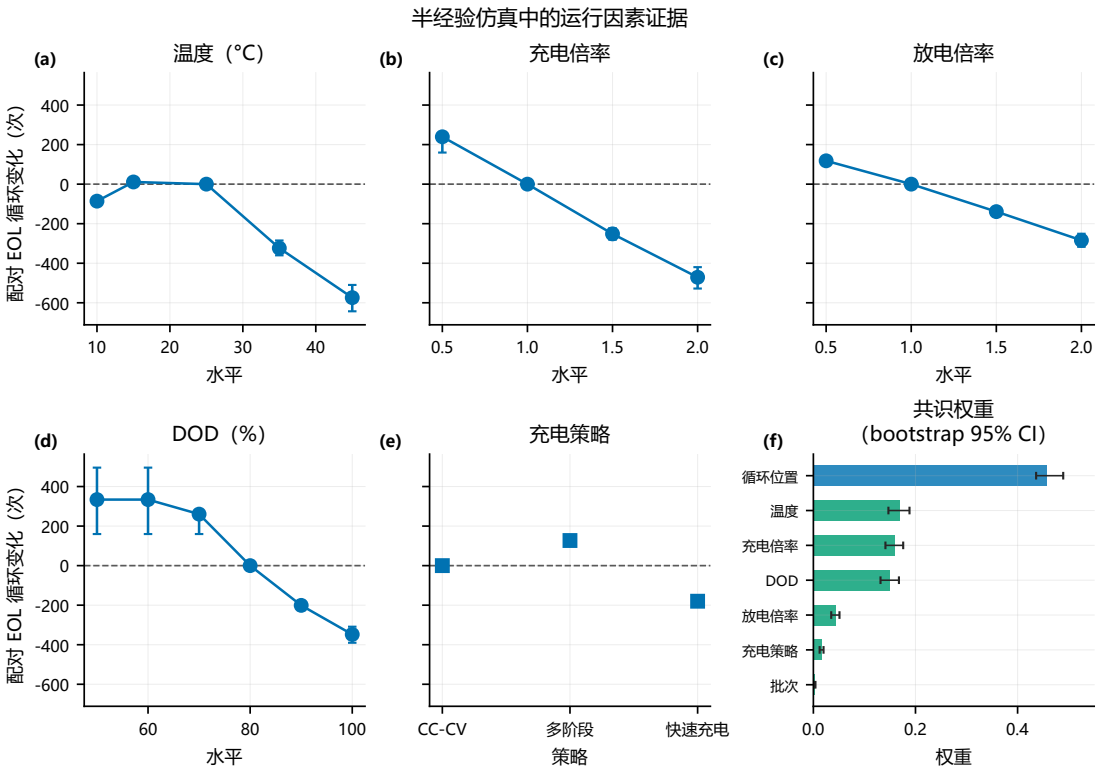

图 3　运行因素证据：（a）–（e）为配对反事实寿命变化（误差线：200 对电池的 2.5%–97.5% 经验分位）；（f）为两类重要度的共识权重（误差线：电池级自助法 bootstrap 95% 区间）。

3　SOH 与 RUL 预测（问题二）

3.1　Landmark 构造与无泄漏划分

从每节电池若干历史节点提取当前值、近期均值、波动和斜率，得到 1895 条历史节点记录。按电池固定划分 215/74/71 节训练、验证和测试；标准化、编码、模型选择与保序区间半径均只在训练/验证集拟合。当前 SOH 目标不进入特征，未来循环、真实 EOL 和隐藏二次寿命也不进入决策特征。

3.2　SOH 估计与多步预测

根据验证集 RMSE 选择 Ridge 作为当前 SOH 模型：测试 RMSE 为 0.175 个百分点，90% 电池分组 conformal 区间覆盖率为 98.2%。50、100、200 循环预测 RMSE 依次为 0.196、0.325、0.575 个百分点，覆盖率均高于 97%。误差随预测步长增长，符合退化轨迹不确定性累积规律。

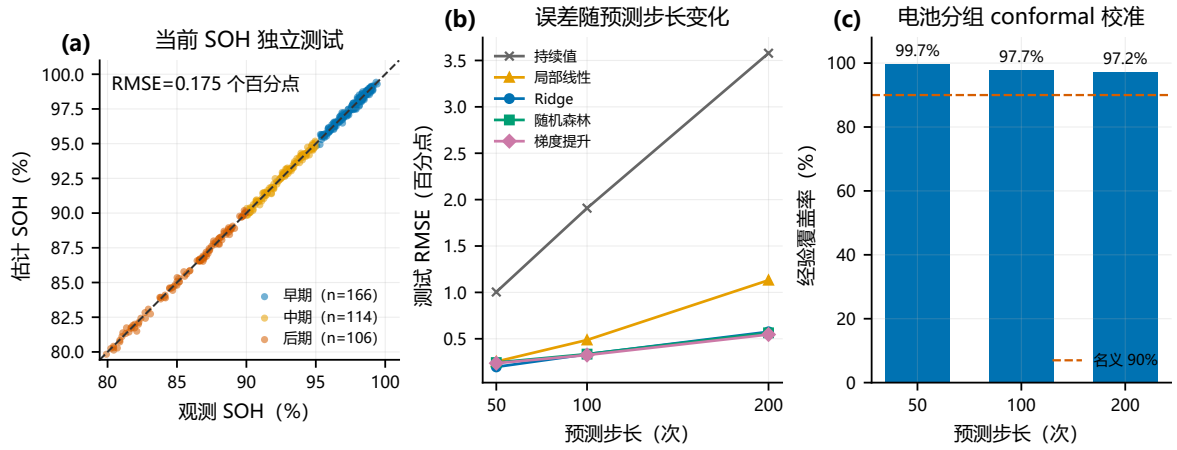

图 4　独立测试证据：（a）当前 SOH 估计；（b）多步预测误差随步长变化；（c）电池分组 conformal 90% 区间的经验覆盖率。

3.3　右删失感知 RUL 模型

令从历史节点到失效的时间为 T。Weibull AFT 模型写为

$$\log \lambda_i = \boldsymbol{x}_i^{\top}\boldsymbol{\beta}, \qquad S_i(t) = \exp\left[-\left(\frac{t}{\lambda_i}\right)^{k}\right]. \tag{2}$$

对事件样本贡献 $\log f_i(t)$，右删失样本贡献 $\log S_i(t)$，因此 20 节观测期内未失效电池不会被当作已知寿命。测试事件样本 RUL MAE/RMSE 为 49.4/70.0 次，C-index 为 0.960，90% 区间覆盖率 98.6%。只在完整事件上训练的随机森林虽有更低点误差，但不能处理删失，故仅作诊断基线，不作为部署模型。

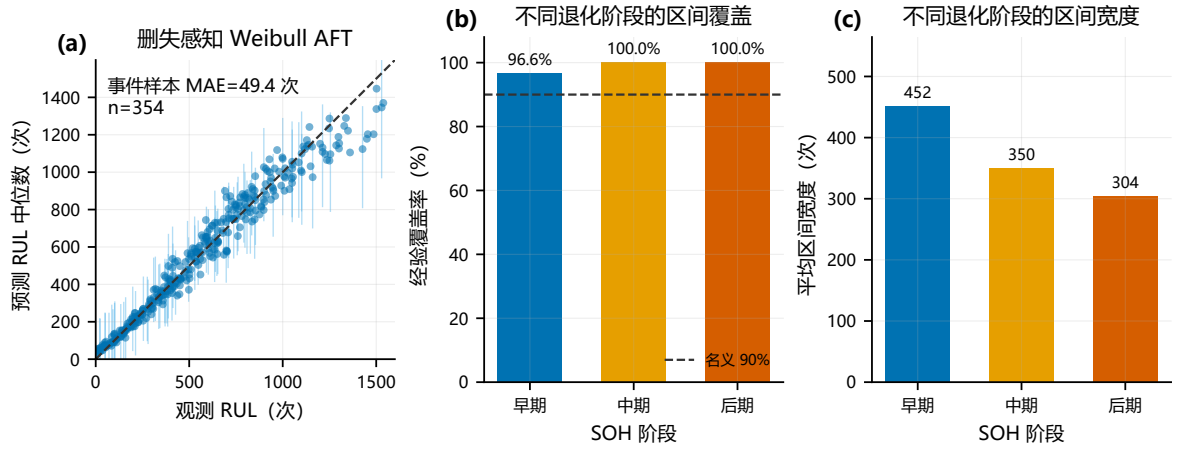

图 5　删失感知 Weibull AFT 的独立测试证据：（a）事件样本点预测及抽样显示的校准 90% 区间；（b）分阶段区间覆盖率；（c）分阶段平均区间宽度。

4　退役筛选与多目标编组（问题三）

4.1　场景门控、等级与兼容图

从独立仿真包中形成 480 节退役池。先按保守 SOH、二次 RUL、内阻、效率、区间宽度和 OOD 分数执行硬门控，再评价经济价值。A/B/C 分别表示可承担频率调节、削峰或仅备用场景，得到 129/125/68 节，拒用为 158 节（32.9%）。

对场景 s 构建无向兼容图 $G_s = (V_s, E_s)$。若任意两节在 SOH、容量、内阻、RUL 的差异均不超过场景容差则连边；三类图密度为 0.616、0.574、0.737。完整组必须是规定大小的团，从结构上保证两两兼容。

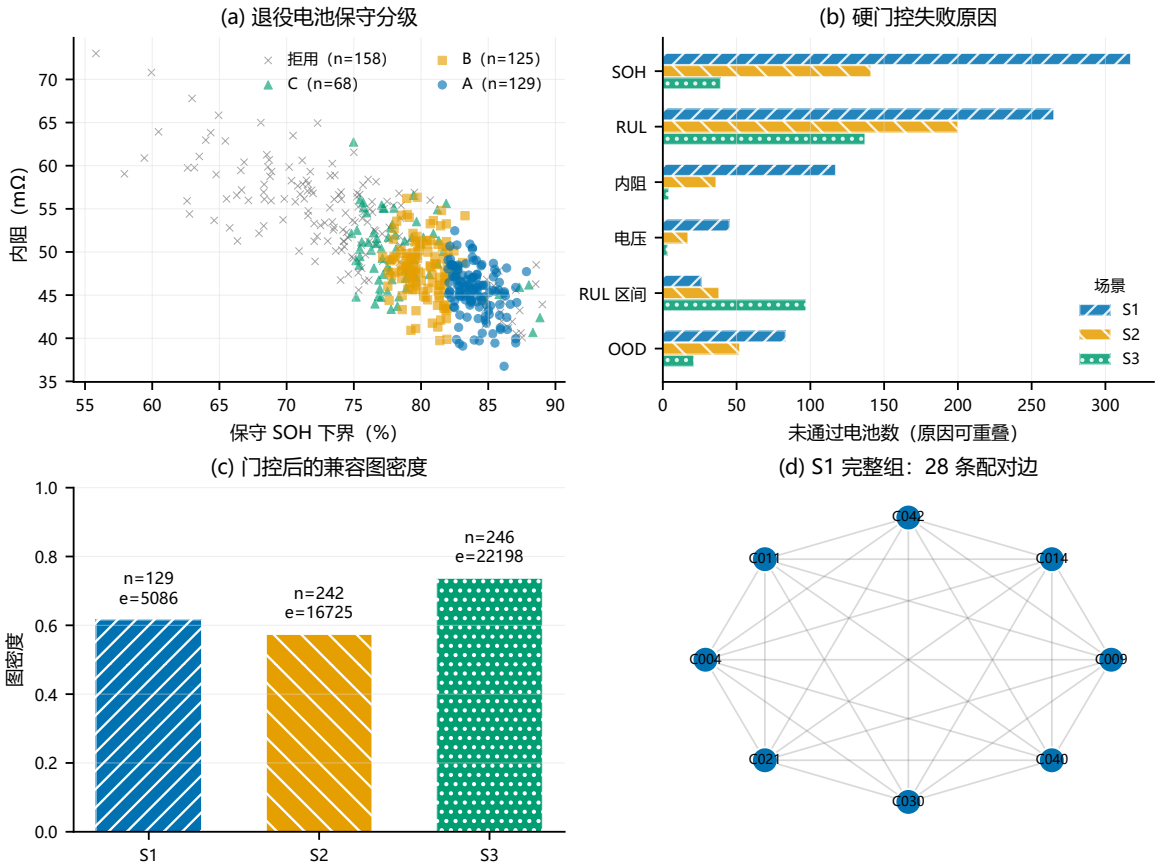

图 6　问题三的筛选—编组证据链：（a）退役池保守分级；（b）三场景硬门控失败原因，原因可重叠；（c）门控后的兼容图密度；（d）一个两两兼容的完整组。

4.2　候选列 MILP 与 Pareto 前沿

对候选组 c 定义价值 V_c、不一致性 I_c 和成员数 m_c。基本集合打包模型为

$$\max_{z_c} \quad \sum_c V_c z_c, \tag{3}$$

$$\text{s.t.} \quad \sum_{c:i\in c} z_c \le 1,\ \forall i, \qquad \sum_c m_c z_c \ge U_{\min}, \qquad z_c \in \{0,1\}. \tag{4}$$

用 ε-约束改变最大不一致性和最低利用量，得到价值—利用率—一致性的非支配前沿；净价值包含检测、重组、运维、能损和风险惩罚，单位为 REU。求解器采用 SciPy/HiGHS

MILP，状态、运行时间和 MIP gap 全部记录。

4.3 全候选空间审计与方案冻结

原 199 列缩减空间选出 R010；扩大到全部 599 列后，候选成员集合与折中解均改变，最大可达利用率为 49.2%、最大单目标价值为 28.62 REU。重新冻结的理想点折中解 F014 含 18 组、196 节，S1/S2/S3 分别 3/4/11 组，利用率 40.8%，价值 25.53 REU，平均不一致性 0.585。

CHIM 单目标锚点超平面距离判定的几何膝点也是 F014，但这不等于权重稳定：七个预设权重中 F014 仅出现 42.9%；1024 个 Dirichlet 权重样本中出现 33.7%，F006 和 F015 也分别占显著区域。因此本文使用“理想点折中解/几何膝点”，不使用“唯一最优”表述。若要求每个组的净价值非负，则应采用 E006：12 组、利用率 26.2%、价值 17.33 REU、负价值组数 0。

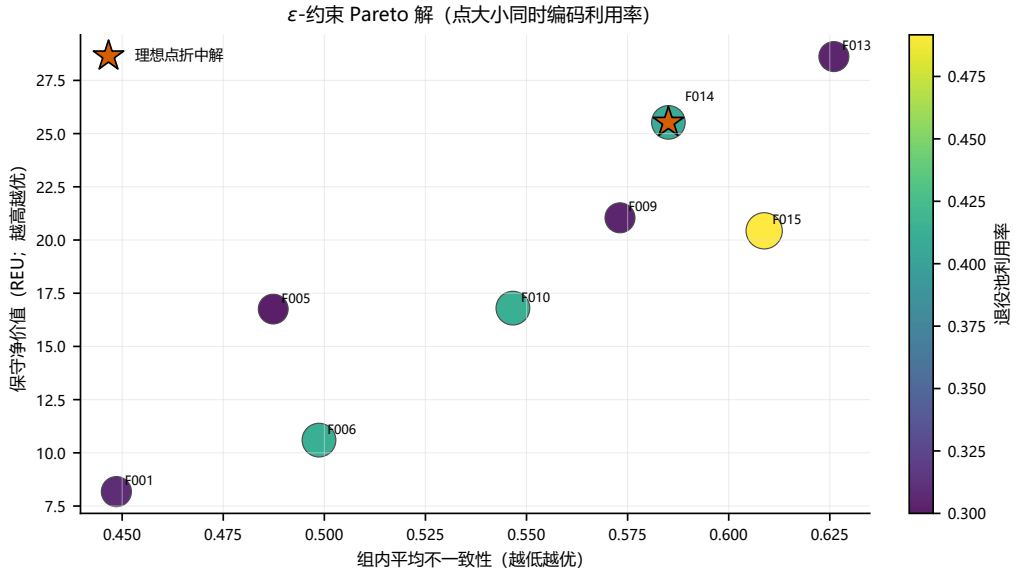

图 7　全候选空间的多目标 Pareto 前沿与理想点折中解 F014。

成员集合按哈希值固化后，才读取隐藏真值进行事后审计。F014 电芯二次 RUL 的 MAE 为 229.6 次，RMSE 为 294.2 次；组最短 RUL MAE 为 245.5 次，组不匹配率 38.9%。该偏差提示问题四必须采用概率兼容与压力传播，而不能继续把点预测视作确定量。

5　概率兼容、压力传播与策略优化（问题四）

5.1 独立校准与最终留出设计

问题四重新生成 240 节校准电池和 240 节最终留出电池，二者与问题二、问题三的电池编号全部不相交。两池使用同一特征工程和已训练的问题二模型，但采用不同随机种子。仅在校准池真值上比较 $p_0 \in \{0.80, 0.85, 0.90, 0.95\}$，以 $F_{0.5}$ 偏重精确率，冻结

$p_0 = 0.80$；校准精确率/召回率为 98.5%/72.8%。最终留出池真值在策略和成员集合固化后才开启。

5.2 概率兼容图

对每一候选电池对，在相关误差 Monte Carlo 样本 $b = 1, \ldots, B$ 下计算

$$p_{ij,s} = \frac{1}{B} \sum_{b=1}^{B} \mathbf{1}\big(|H_i^{(b)} - H_j^{(b)}| \le \delta_H^s, |Q_i^{(b)} - Q_j^{(b)}| \le \delta_Q^s, \\ |R_i^{(b)} - R_j^{(b)}| \le \delta_R^s, |L_i^{(b)} - L_j^{(b)}| \le \delta_L^s\big). \tag{5}$$

当 $p_{ij,s} \ge p_0$ 时连边。SOH 与 RUL 的公共误差相关系数分别设为 0.75 和 0.90，以模拟同批次校准偏差；概率图从名义 246 个候选组收缩到 178 个。留出池等级 A/B/C/拒用为 60/43/51/86，拒用率为 35.8%。

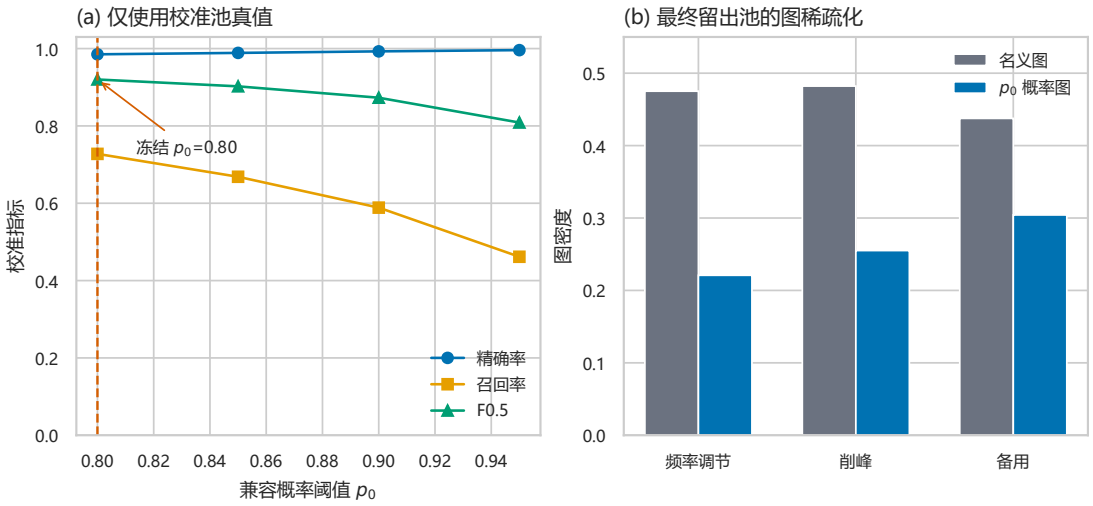

图 8　名义图与 $p_0 = 0.80$ 概率兼容图的规模、密度及校准表现。

5.3 压力场景

六个场景均为半经验压力测试，不是安全极限。S0 为基准；S1 长期高温，S2 高频高倍率，S3 高批次异质性，S4 为高温与高倍率组合，S5 为异质性与运行压力组合。

表 1　问题四压力场景注册表的核心参数

场景	温升/°C	倍率因子	异质性因子	退化应力因子
S0	0	1.00	1.00	1.00
S1	10	1.00	1.00	1.34
S2	0	1.50	1.00	1.42
S3	0	1.00	1.65	1.12
S4	10	1.50	1.00	1.90
S5	6	1.30	1.65	1.78

冻结的问题三方案 F014 在各压力场景下的传播结果见表 2。利用率因成员固定而保持 40.8%，但价值、RUL 与违约风险显著变化；S4/S5 的最小 RUL 5% 分位降至约 57/55 次，说明问题三的名义可行并不保证压力可行。

表 2　冻结 Q3 方案在六场景下的压力传播

场景	利用率	期望价值	能损/kWh	不匹配概率	最小 $\mathrm{RUL}_{0.05}$	违约率
S0	40.8%	154.08	7.51	59.8%	137.9	59.8%
S1	40.8%	89.53	5.57	51.4%	89.2	51.4%
S2	40.8%	75.80	8.01	51.7%	91.7	51.7%
S3	40.8%	119.03	6.66	78.8%	92.7	78.8%
S4	40.8%	28.17	5.91	51.9%	57.4	51.9%
S5	40.8%	30.26	5.41	75.5%	55.4	75.5%

5.4　名义、自适应与统一稳健策略

定义样本净价值 $V_{cs}^{(b)}$。下尾风险使用

$$\mathrm{VaR}_\alpha(V) = \inf\{v : \Pr(V \le v) \ge \alpha\}, \tag{6}$$

$$\mathrm{CVaR}_\alpha(V) = E[V \mid V \le \mathrm{VaR}_\alpha(V)], \quad \alpha = 0.20. \tag{7}$$

N 策略在 S0 上选组后保持不变；A 策略在已知场景下重新筛选、重新编组并最大化该场景期望价值；R 策略使用同一组成员覆盖所有场景，其分数为 0.45 期望价值、0.30 最差场景期望价值、0.25 CVaR20，并惩罚不匹配和超额不一致性。所有策略均使用 400 次压力抽样。

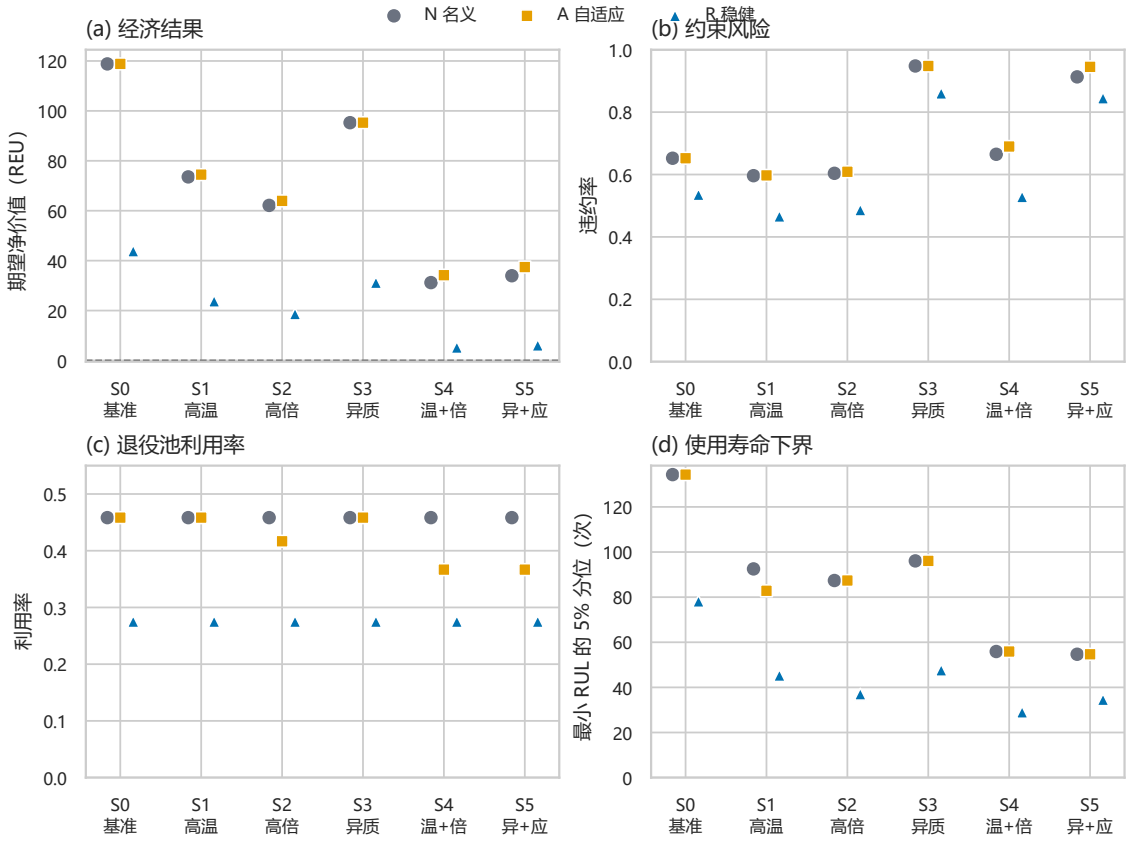

图 9　N、A、R 三策略在六类离散压力场景下的期望价值、违约率、利用率与最小 RUL 5% 分位；横向错位仅用于区分策略，不连接离散场景。

R 在 S0 仅保留 6 组、利用率 27.5%、期望价值 43.80 REU，低于 N 的 10 组和 118.82 REU；但 S0 违约率从 65.2% 降至 53.5%，S1/S2/S4 的违约率也分别降低约 13.0、11.8、13.7 个百分点。S3/S5 的批次共同模态仍导致 86.0%/84.5% 违约率，说明“统一方案”不能消除系统性异质性。

表 3　三策略全场景指标（数值由结果表自动生成）

策略	场景	组数	利用率	期望价值	CVaR20	不匹配	最小 $RUL_{0.05}$	违约率
N 名义固定	S0	10	45.8%	118.82	39.03	65.2%	134.3	65.2%
A 场景自适应	S0	10	45.8%	118.82	39.03	65.2%	134.3	65.2%
R 统一稳健	S0	6	27.5%	43.80	2.84	53.5%	78.1	53.5%
N 名义固定	S1	10	45.8%	73.56	11.85	59.6%	92.5	59.6%
A 场景自适应	S1	10	45.8%	74.47	11.63	59.7%	82.8	59.7%
R 统一稳健	S1	6	27.5%	23.72	-7.39	46.6%	45.3	46.6%
N 名义固定	S2	10	45.8%	62.14	4.56	60.4%	87.4	60.4%
A 场景自适应	S2	9	41.7%	63.94	7.65	60.9%	87.4	60.9%
R 统一稳健	S2	6	27.5%	18.64	-9.39	48.6%	37.0	48.6%
N 名义固定	S3	10	45.8%	95.28	14.47	94.8%	96.1	94.8%
A 场景自适应	S3	10	45.8%	95.28	14.47	94.8%	96.1	94.8%
R 统一稳健	S3	6	27.5%	31.16	-6.60	86.0%	47.6	86.0%
N 名义固定	S4	10	45.8%	31.20	-11.89	66.5%	55.9	66.5%
A 场景自适应	S4	8	36.7%	34.18	-5.75	69.0%	55.9	69.0%
R 统一稳健	S4	6	27.5%	5.19	-16.22	52.8%	29.0	52.8%
N 名义固定	S5	10	45.8%	33.95	-14.12	91.3%	54.7	91.3%
A 场景自适应	S5	8	36.7%	37.43	-7.02	94.5%	54.7	94.5%
R 统一稳健	S5	6	27.5%	6.04	-16.94	84.5%	34.5	84.5%

策略	场景	组数	利用率	期望价值	CVaR20	不匹配	最小 $\mathrm{RUL}_{0.05}$	违约率

5.5 适应性收益 Recovery Ratio

对高者更优指标 q，定义场景 s 的名义损失和自适应恢复率为

$$L_{q,s} = q_{N,S0} - q_{N,s}, \qquad RR_{q,s} = \frac{q_{A,s} - q_{N,s}}{L_{q,s}}. \tag{8}$$

对违约率等低者更优指标反向定义；若 $L_{q,s} = 0$ 则不定义，不进行截断。统一稳健策略并非场景后修复，故另报 V_R/V_N 价值保留率和 $(r_N - r_R)/r_N$ 违约削减率。

表 4　自适应恢复率与统一稳健策略保留/削减率

场景	A 价值恢复	A 最小 RUL 恢复	A 违约恢复	R 价值保留	R 违约削减
S1	2.0%	-23.2%	1.6%	32.3%	21.9%
S2	3.2%	0.0%	10.2%	30.0%	19.5%
S3	0.0%	0.0%	0.0%	32.7%	9.3%
S4	3.4%	0.0%	-196.4%	16.6%	20.6%
S5	4.1%	0.0%	-12.3%	17.8%	7.5%

价值恢复率仅为 0–4.1%；S4、S5 的 A 违约恢复为负，表明只最大化场景价值会牺牲安全稳健性。R 的价值保留率为 16.6–32.7%，但可削减 7.5–21.9% 的名义违约；这给出明确的风险—收益交换，而非“稳健必然更优”。

6　敏感性、经济边界与工程建议

6.1 两层敏感性分析

局部 OAT 对 16 个参数采用物理有效范围，例如能效门槛 94.0–98.5%，从不超过 100%。全局层使用 384 个 Latin Hypercube 样本，并在固定可行列政策模拟器上计算 Spearman 秩相关；该相关用于筛查非线性单调敏感性，不解释为因果效应或完整 Sobol 分解。

表 5　全局敏感度前五参数（各输出最大 $|\rho_S|$）

排名	参数	最大 $\lvert\rho_S\rvert$
1	能效门槛	0.896
2	p_0	0.837
3	批次异质性因子	0.656
4	预测不确定性因子	0.418
5	能损成本（REU/kWh）	0.394

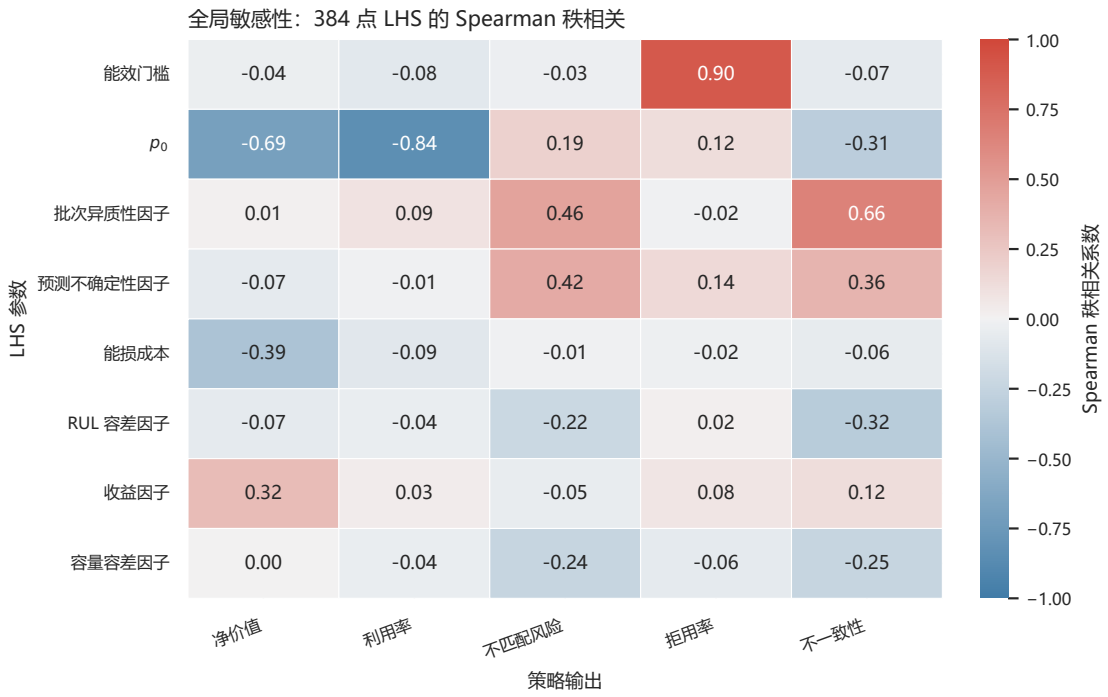

图 10　384 点 LHS 的全局 Spearman 秩相关敏感性；色彩编码方向和强度，格内数字为相关系数。

能效门槛主要决定拒用率，p_0 主要压低利用率和价值，批次异质性主导不一致性，预测不确定性增加不匹配风险，能损成本降低净价值。因此检测精度和批次分流比单纯放宽编组规模更值得优先投入。

6.2　候选规模与权重区域

表 6　问题三候选列规模敏感性

规模	列数	运行/s	目标价值	利用率	与全空间 Jaccard
100	100	0.032	20.97	16.2%	0.351
199	199	0.139	23.73	23.8%	0.483
300	300	2.662	28.62	30.4%	0.692
full	599	16.722	26.85	38.3%	1.000

100、199、300 列虽然运行快，但成员 Jaccard 仅 0.351、0.483、0.692；599 列求解约 13.2 s，MIP gap 0.98%。300 到 599 列时目标值下降，是因为该审计同时把最低利用量按候选空间最大可用量的 70% 收紧，并非扩大可行域导致目标退化。权重区域中 F014 只占 33.7%，进一步排除“唯一稳定最优”。

6.3　盈亏平衡

对给定策略和场景，把样本价值写为

$$V = rE - C_{\text{detect}} - C_{\text{reassembly}} - C_{\text{O\&M}} - C_{\text{loss}} - C_{\text{risk}}. \tag{9}$$

令期望、CVaR20 或最差样本价值为 0，分别反解最低收益系数和最大可接受重组成本。统一稳健方案的阈值如下。

表 7　统一稳健策略盈亏平衡边界

场景	期望最低收益	CVaR20 最低收益	最差最低收益	最差可接受重组成本
S0	1.275	2.013	2.311	-0.41
S1	1.435	2.205	2.520	-1.47
S2	1.531	2.270	2.572	-1.59
S3	1.370	2.186	2.501	-1.66
S4	1.777	2.534	2.814	-2.14
S5	1.740	2.500	2.761	-2.26

S0 的期望/CVaR20/最差最低收益系数为 1.275/2.013/2.311 REU/kWh；S4 分别升至 1.777/2.534/2.814。部分最差场景的可接受重组成本为负，意味着即使重组免费也无法保证最差样本不亏损，不能通过压价解决物理风险。

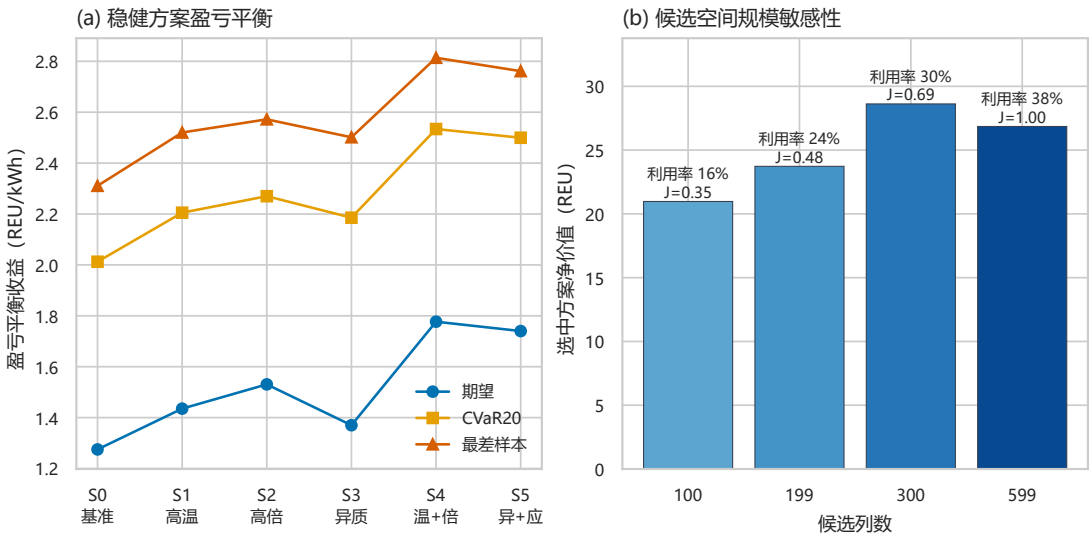

图 11　统一稳健策略的盈亏平衡收益边界与 Q3 候选列规模敏感性。

6.4　可执行工程建议

1. 以 $p_0 = 0.80$ 作为仿真初始值，但在每种真实化学体系和检测设备上重做校准；若目标精确率低于 98%，先提升检测而非扩大组规模。
2. 采用“两道门”：电芯级硬门控后再用概率兼容图；能效门槛保持在设备校准支持的 94.0–98.5% 区间，禁止外推到 100% 以上。
3. S1/S2 可把统一稳健策略作为保守候选，但其利用率只有 27.5%；若采用自适应策略，必须增加违约率约束，不能只最大化期望价值。
4. S3/S5 在当前模型下违约仍高于 84%，应先按制造批次、温度历史和内阻漂移分流，未通过实测共同模态校准前不建议混批编组。

5. S4 的 CVaR20 盈亏平衡收益系数至少 2.534 REU/kWh；合同收益低于该值时应拒绝高温高倍率任务，或降低倍率、改善热管理后重新评估。
6. 每累计 50–100 次二次循环更新 SOH/RUL 分布；当区间宽度、OOD 或组内最小 RUL 越界时拆组复检，而不是沿用初始等级。
7. 首批部署限定为小规模并行试验，记录真实容量、直流内阻、温升和故障模式，用于重新估计相关误差与场景应力因子。

7 模型评价、局限与结论

7.1 优点

模型从电池编号分组留出、右删失似然、conformal 区间，到问题三、问题四决策冻结后揭盲，形成可审计的时间与信息边界；兼容图把电芯对约束转化为可解释结构，候选列 MILP 避免任意 Big-M；全空间、权重区域、压力场景、候选规模和经济阈值共同揭示方案边界。全部图表和正文核心数字由 CSV/JSON 自动生成，可从一键脚本追溯。

7.2 局限性

1. 全部电池数据为半经验仿真，未覆盖真实测量噪声、化学体系漂移和传感器故障，不能直接给出量产精度。
2. 80% 汽车退役 SOH、三类二次应用门槛及六个压力应力系数是场景设定，不是法规或安全认证阈值。
3. 三段变点和 Weibull AFT 均为可解释近似；多峰失效、突发内短路或明显非比例加速时需要竞争风险/状态空间模型。
4. Q4 公共误差相关系数和 Monte Carlo 分布由仿真假设给定，真实批次共同模态可能更强，S3/S5 风险可能被低估。
5. 候选组由规则枚举而非全部组合穷举；全 599 列只是在生成规则内完整，不等同于组合空间全局最优。
6. REU 成本不含真实回收物流、税费、残值波动、保险和停机损失，盈亏平衡只能作为相对决策边界。
7. LHS–Spearman 使用固定可行列政策模拟器，未在每个全局样本内完整重建图并重解 MILP，故只用于敏感因素排序。
8. 组级违约由代理不匹配与 RUL 门槛定义，不能代替热扩散、过充、短路、机械冲击和消防系统试验。

7.3 结论

本文证明：退化具有显著三阶段加速；删失感知 RUL 模型能在不丢弃未失效电池的情况下提供可校准区间；Q3 的候选空间选择会实质改变成员与折中解；在压力条件下，场景自适应的价值恢复有限且可能恶化违约，统一稳健策略则用规模和收益换取部分风险削减。最终推荐的是“概率门控—分批编组—压力情景—滚动复检”的保守流程，而不是把仿真最优解直接当作工程处方。

8 生成式人工智能使用说明

本文研究过程中使用生成式人工智能工具 OpenAI Codex（GPT-5）辅助进行部分程序调试、数值实现检查、可视化与 LaTeX 排版优化以及语言润色 [8]。模型方案、参数取值、实验方案、结果解释和最终取舍均由参赛队结合题意、文献与项目运行结果进行人工判断与确认；对于 AI 辅助产生的代码或文字，参赛队均进行了人工检查和必要修改。AI 未被作为实验数据来源，本文全部数值结果均由本项目程序实际运行产生。

参考文献

[1] Wang J. et al. Cycle-life model for graphite–$LiFePO_4$ cells. *Journal of Power Sources*, 2011, 196: 3942–3948.

[2] Schmalstieg J. et al. A holistic aging model for $Li(NiMnCo)O_2$ based 18650 lithium-ion batteries. *Journal of Power Sources*, 2014, 257: 325–334.

[3] Keil P., Jossen A. Calendar aging of NCA lithium-ion batteries investigated by differential voltage analysis. *Journal of The Electrochemical Society*, 2016, 163: A1872–A1880.

[4] Attia P. M. et al. Review—Knees in lithium-ion battery aging trajectories. *Journal of The Electrochemical Society*, 2022, 169: 060517.

[5] Wei L. J. The accelerated failure time model: a useful alternative to the Cox regression model. *Statistics in Medicine*, 1992, 11: 1871–1879.

[6] Lei J. et al. Distribution-free predictive inference for regression. *JASA*, 2018, 113: 1094–1111.

[7] Rockafellar R. T., Uryasev S. Optimization of conditional value-at-risk. *Journal of Risk*, 2000, 2: 21–41.

[8] OpenAI. OpenAI Codex（模型：GPT-5；版本：Codex desktop, August 2026）[生成式人工智能辅助工具]. 使用日期：2026-08-09–2026-08-10.

A 支撑文件清单

竞赛支撑压缩包包含下列可复现材料，且不含姓名、学校、地区、指导教师或本机用户路径：

目录/文件	内容
`run_final.py`	一键运行数据生成、Q1、Q2、Q2.5、Q3、Q4、图件、论文和验证。
`src/`	数据生成、统计预测、图建模、MILP、压力 Monte Carlo、敏感性和论文数值生成的完整 Python 源码。
`data/parameter_sources/`	仿真参数、单位、边界、来源类型与配置。
`paper/main.tex`	本文完整 LaTeX 源文件；核心数值由 generated 文件自动写入。
`results/tables/`	Q1–Q4 指标、候选解、审计、敏感性、经济边界和验证 JSON/CSV。
`results/figures/`	最终核心图的 PDF/PNG/SVG。
`requirements.txt`	Python 依赖和冻结版本。
`README.md`	环境建立、运行命令、随机种子和输出说明。
`AI 工具使用详情.pdf`	按 2026 年竞赛规则披露生成式 AI 使用情况。
`final_validation.json`	页数、匿名性、文件大小、图件、数据隔离及独立复现检查。
`verify_clean_package.py`	不重新拟合模型的洁净包复现检查：核验科学产物指纹并重编译论文。

B 完整程序源代码

本附录通过 `\VerbatimInput` 将提交包内实际源文件逐字排入 PDF，而非仅给出路径。支撑包解压后，执行 `python run_final.py` 可重建数值、图件和论文。

run_final.py

```
"""Reproduce data, Q1-Q4, all figures, the paper, and final validations."""

from __future__ import annotations

import json
import subprocess
import sys
import time
from pathlib import Path


ROOT = Path(__file__).resolve().parent


def _run(stage: str, arguments: list[str]) -> dict[str, object]:
    started = time.perf_counter()
    process = subprocess.run(
        [sys.executable, *arguments],
        cwd=ROOT,
```

```
        text=True,
        check=False,
    )
    result = {
        "stage": stage,
        "command": "python " + " ".join(arguments),
        "return_code": int(process.returncode),
        "runtime_seconds": float(time.perf_counter() - started),
    }
    if process.returncode != 0:
        raise RuntimeError(f"{stage} failed with return code {process.returncode}")
    return result


def main() -> dict[str, object]:
    started = time.perf_counter()
    stages = [
        _run("data_and_q1", ["run_all.py", "--allow-downstream"]),
        _run("q2", ["run_phase2.py", "--skip-paper"]),
        _run("q2_5", ["run_phase2_5.py", "--allow-downstream"]),
        _run("q3_and_phase3_5", ["run_phase3.py", "--skip-paper", "--allow-downstream"]),
        _run("q4_and_figures", ["run_phase4.py"]),
        _run("paper_and_validation", ["-m", "src.finalization"]),
    ]
    manifest = {
        "entrypoint": "python run_final.py",
        "python": sys.version,
        "return_code": 0,
        "runtime_seconds": float(time.perf_counter() - started),
        "stages": stages,
    }
    (ROOT / "final_run_manifest.json").write_text(
        json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8"
    )
    print(json.dumps(manifest, ensure_ascii=False, indent=2))
    return manifest


if __name__ == "__main__":
    main()
```

build_submission.py

```
"""Build the internal archive and the <=20 MB anonymous competition support ZIP."""

from __future__ import annotations

import argparse
import hashlib
import json
import re
import shutil
import tempfile
import zipfile
from datetime import date
from pathlib import Path

from pypdf import PdfReader


ROOT = Path(__file__).resolve().parent
DESKTOP = Path.home() / "Desktop"
VALIDATION = ROOT / "final_validation.json"
DELIVERY_MANIFEST = ROOT / "delivery_manifest.json"
SCIENCE_FINGERPRINTS = ROOT / "science_fingerprints.json"
LIMIT = 20 * 1024 * 1024


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()

```

```
33
34  def _copy_file(source: Path, destination: Path) -> None:
35      destination.parent.mkdir(parents=True, exist_ok=True)
36      shutil.copy2(source, destination)
37
38
39  def _zip_tree(source: Path, destination: Path, root_name: str | None = None) -> None:
40      destination.parent.mkdir(parents=True, exist_ok=True)
41      with zipfile.ZipFile(destination, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as archive:
42          for path in sorted(p for p in source.rglob("*") if p.is_file()):
43              relative = path.relative_to(source)
44              archive_name = Path(root_name) / relative if root_name else relative
45              compression = zipfile.ZIP_STORED if relative.as_posix() == "final_validation.json" else zipfile.ZIP_DEFLATED
46              archive.write(path, archive_name.as_posix(), compress_type=compression)
47
48
49  def _write_science_fingerprints() -> None:
50      """Freeze hashes of packaged scientific outputs without modifying them."""
51      paths: list[Path] = []
52      paths.extend(path for path in (ROOT / "data" / "parameter_sources").rglob("*") if path.is_file())
53      paths.extend(path for path in (ROOT / "results" / "tables").glob("*") if path.is_file())
54      paths.extend((ROOT / "results" / "figures").glob("*.pdf"))
55      paths.extend((ROOT / "paper").glob("generated*.tex"))
56      files = {
57          path.relative_to(ROOT).as_posix(): _sha256(path)
58          for path in sorted(set(paths))
59      }
60      payload = {
61          "scope": "Frozen packaged inputs/results used for non-refitting clean-package verification.",
62          "files": files,
63      }
64      SCIENCE_FINGERPRINTS.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
65
66
67  def _delivery_dir() -> Path:
68      if DELIVERY_MANIFEST.exists():
69          old = json.loads(DELIVERY_MANIFEST.read_text(encoding="utf-8"))
70          candidate = Path(old.get("delivery_directory", ""))
71          if candidate.is_dir() and candidate.parent.resolve() == DESKTOP.resolve():
72              return candidate
73      base = DESKTOP / f" 锂电池数模 A 题 _ 最终交付 _{date.today():%Y%m%d}"
74      candidate = base
75      version = 2
76      while candidate.exists():
77          candidate = DESKTOP / f"{base.name}_v{version}"
78          version += 1
79      candidate.mkdir(parents=True)
80      return candidate
81
82
83  def _support_stage(stage: Path) -> int:
84      root_files = [
85          "run_final.py", "build_submission.py", "verify_clean_package.py", "run_all.py", "run_phase2.py", "run_phase2_5.py",
86          "run_phase3.py", "run_phase4.py", "requirements.txt", "README.md", "final_validation.json", "science_fingerprints.json",
87          "AI 工具使用详情.pdf", "AI 工具使用详情.md",
88      ]
89      if (ROOT / "clean_package_smoke_manifest.json").exists():
90          root_files.append("clean_package_smoke_manifest.json")
91      for name in root_files:
92          _copy_file(ROOT / name, stage / name)
93      for path in (ROOT / "src").rglob("*.py"):
94          if "__pycache__" not in path.parts:
95              _copy_file(path, stage / path.relative_to(ROOT))
96      for path in (ROOT / "data" / "parameter_sources").rglob("*"):
97          if path.is_file():
98              _copy_file(path, stage / path.relative_to(ROOT))
99      for path in (ROOT / "docs").rglob("*"):
100         if path.is_file():
101             _copy_file(path, stage / path.relative_to(ROOT))
102     for path in (ROOT / "support_materials").glob("*.tex"):
103         _copy_file(path, stage / path.relative_to(ROOT))
104     for path in (ROOT / "results" / "tables").glob("*"):
105         if path.is_file():
106             _copy_file(path, stage / path.relative_to(ROOT))
107     for path in (ROOT / "results" / "figures").glob("*.pdf"):
108         _copy_file(path, stage / path.relative_to(ROOT))
109     paper_names = ["main.tex", "main.pdf"]
```

```
    paper_names.extend(path.name for path in (ROOT / "paper").glob("generated*.tex"))
    for name in paper_names:
        _copy_file(ROOT / "paper" / name, stage / "paper" / name)
    for path in (ROOT / "paper" / "figures").glob("*.pdf"):
        _copy_file(path, stage / path.relative_to(ROOT))
    return sum(1 for path in stage.rglob("*") if path.is_file())


def _refresh_document_statistics(validation: dict[str, object]) -> None:
    paper = ROOT / "paper" / "main.pdf"
    paper_reader = PdfReader(paper)
    aux_text = (ROOT / "paper" / "main.aux").read_text(encoding="utf-8", errors="replace")
    match = re.search(r"\\newlabel\{bodyend\}\{\{[^}]*\}\{(\d+)\}", aux_text)
    if not match:
        raise RuntimeError("bodyend label missing from paper/main.aux")
    body_pages = int(match.group(1))
    ai_pdf = ROOT / "AI 工具使用详情.pdf"
    ai_pages = len(PdfReader(ai_pdf).pages)
    validation.update(
        {
            "paper_total_pages": len(paper_reader.pages),
            "paper_body_pages": body_pages,
            "paper_appendix_pages": len(paper_reader.pages) - body_pages,
            "paper_pdf_bytes": paper.stat().st_size,
            "paper_pdf_sha256": _sha256(paper),
            "ai_usage_pdf_pages": ai_pages,
            "ai_usage_pdf_bytes": ai_pdf.stat().st_size,
            "ai_usage_pdf_sha256": _sha256(ai_pdf),
            "ai_usage_pdf_3_to_5_pages": 3 <= ai_pages <= 5,
        }
    )


def _write_final_support_zip(stage: Path, support_zip: Path) -> tuple[int, int]:
    """Write a ZIP whose embedded validation records its exact count and size."""
    validation = json.loads(VALIDATION.read_text(encoding="utf-8"))
    _refresh_document_statistics(validation)
    file_count = sum(1 for path in stage.rglob("*") if path.is_file())
    validation["competition_support_file_count"] = file_count
    validation["final_product_statistics_match"] = True
    guess = int(validation.get("competition_support_zip_bytes", 0))
    for _ in range(20):
        validation["competition_support_zip_bytes"] = guess
        validation["competition_support_zip_within_20_mb"] = guess <= LIMIT
        validation["overall_pass"] = all(value for value in validation.values() if isinstance(value, bool))
        VALIDATION.write_text(json.dumps(validation, ensure_ascii=False, indent=2), encoding="utf-8")
        _copy_file(VALIDATION, stage / "final_validation.json")
        _zip_tree(stage, support_zip)
        actual = support_zip.stat().st_size
        with zipfile.ZipFile(support_zip) as archive:
            archive_count = len(archive.infolist())
        if archive_count != file_count:
            raise RuntimeError(f"Support ZIP count mismatch: {archive_count} != {file_count}")
        if actual == guess:
            return file_count, actual
        guess = actual
    raise RuntimeError("Support ZIP size statistic did not stabilize")


def _internal_archive(destination: Path) -> None:
    excluded_dirs = {".venv", ".git", "tmp", "build", "__pycache__", "preview"}
    excluded_suffixes = {".aux", ".log", ".out", ".toc", ".fls", ".fdb_latexmk", ".synctex.gz"}
    with zipfile.ZipFile(destination, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=6) as archive:
        for path in sorted(p for p in ROOT.rglob("*") if p.is_file()):
            relative = path.relative_to(ROOT)
            if any(part in excluded_dirs for part in relative.parts):
                continue
            if any(path.name.endswith(suffix) for suffix in excluded_suffixes):
                continue
            archive.write(path, (Path(" 锂电池数模 A 题 _ 完整工程") / relative).as_posix())


def _record_smoke(path: Path) -> None:
    smoke = json.loads(path.read_text(encoding="utf-8"))
    stage_names = [stage["stage"] for stage in smoke.get("stages", [])]
    passed = smoke.get("return_code") == 0 and all(stage.get("return_code") == 0 for stage in smoke.get("stages", []))
    validation = json.loads(VALIDATION.read_text(encoding="utf-8"))
```

```
    validation.update(
        {
            "clean_package_reproduction_pass": bool(passed),
            "clean_package_return_code": int(smoke.get("return_code", -1)),
            "clean_package_runtime_seconds": float(smoke.get("runtime_seconds", 0.0)),
            "clean_package_stages": stage_names,
            "clean_package_mode": smoke.get("mode", "unspecified"),
            "clean_package_science_fingerprints_match": bool(smoke.get("science_fingerprints_match", passed)),
        }
    )
    validation["overall_pass"] = all(value for value in validation.values() if isinstance(value, bool))
    VALIDATION.write_text(json.dumps(validation, ensure_ascii=False, indent=2), encoding="utf-8")
    _copy_file(path, ROOT / "clean_package_smoke_manifest.json")


def build(support_only: bool = False, record_smoke: Path | None = None) -> dict[str, object]:
    if record_smoke is not None:
        _record_smoke(record_smoke)
    delivery = _delivery_dir()
    support_zip = delivery / "competition_support.zip"
    internal_zip = delivery / " 内部完整工程.zip"
    paper_copy = delivery / " 锂电池剩余寿命预测与梯次利用筛选优化.pdf"
    ai_copy = delivery / "AI 工具使用详情.pdf"
    _copy_file(ROOT / "paper" / "main.pdf", paper_copy)
    _copy_file(ROOT / "AI 工具使用详情.pdf", ai_copy)
    _write_science_fingerprints()

    with tempfile.TemporaryDirectory(prefix="cumcm_support_") as temporary:
        stage = Path(temporary) / "competition_support"
        stage.mkdir()
        _support_stage(stage)
        file_count, support_bytes = _write_final_support_zip(stage, support_zip)
    if support_zip.stat().st_size > LIMIT:
        raise RuntimeError(f"competition_support.zip exceeds 20 MB: {support_zip.stat().st_size}")

    if not support_only or not internal_zip.exists():
        _internal_archive(internal_zip)
    note = (
        " 发送给队友：优先发送本目录外层总压缩包。\n"
        " 竞赛提交：论文 PDF 与 competition_support.zip 分开上传；提交前由参赛队再次核对规则。\n"
        " 内部完整工程.zip 含原始仿真数据和全部中间结果，不作为 20 MB 支撑材料上传。\n"
    )
    (delivery / " 发送说明.txt").write_text(note, encoding="utf-8")
    _copy_file(VALIDATION, delivery / "final_validation.json")
    manifest = {
        "delivery_directory": str(delivery),
        "paper": {"path": str(paper_copy), "bytes": paper_copy.stat().st_size, "sha256": _sha256(paper_copy)},
        "competition_support": {
            "path": str(support_zip), "bytes": support_bytes,
            "sha256": _sha256(support_zip), "file_count": file_count,
        },
        "internal_archive": {
            "path": str(internal_zip), "bytes": internal_zip.stat().st_size, "sha256": _sha256(internal_zip)
        },
    }
    DELIVERY_MANIFEST.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
    _copy_file(DELIVERY_MANIFEST, delivery / " 交付清单.json")
    checksum_targets = [paper_copy, ai_copy, support_zip, internal_zip, delivery / "final_validation.json"]
    checksum_text = "\n".join(f"{_sha256(path)}  {path.name}" for path in checksum_targets) + "\n"
    (delivery / "SHA256SUMS.txt").write_text(checksum_text, encoding="utf-8")
    checklist = (
        "# 最终交付清单\n\n"
        f"- 最终论文：`{paper_copy.name}`；{paper_copy.stat().st_size} bytes；SHA256 `{_sha256(paper_copy)}`。\n"
        f"- AI 使用详情：`{ai_copy.name}`；{ai_copy.stat().st_size} bytes；SHA256 `{_sha256(ai_copy)}`。\n"
        f"- 竞赛支撑包：`{support_zip.name}`；{file_count} 个文件；{support_bytes} bytes；SHA256 `{_sha256(support_zip)}`。\n"
        f"- 内部完整工程：`{internal_zip.name}`；{internal_zip.stat().st_size} bytes；SHA256 `{_sha256(internal_zip)}`。\n"
        "- 校验记录：`final_validation.json`；其支撑包文件数和大小与最终 ZIP 完全一致。\n"
        "- 哈希清单：`SHA256SUMS.txt`。\n\n"
        " 竞赛系统中分别上传最终论文 PDF 和 competition_support.zip；内部完整工程不作为竞赛支撑包上传。\n"
    )
    (delivery / " 最终交付清单.md").write_text(checklist, encoding="utf-8")
    (ROOT / " 交付清单.md").write_text(checklist, encoding="utf-8")
    outer_zip = delivery.parent / f"{delivery.name}.zip"
    _zip_tree(delivery, outer_zip, root_name=delivery.name)
    manifest["outer_delivery_zip"] = {
        "path": str(outer_zip), "bytes": outer_zip.stat().st_size, "sha256": _sha256(outer_zip)
    }
```

```
    # Keep the outer archive hash in the project-side manifest only. Embedding
    # an archive's own hash inside that archive would create a circular value.
    DELIVERY_MANIFEST.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
    return manifest


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--support-only", action="store_true")
    parser.add_argument("--record-smoke", type=Path)
    args = parser.parse_args()
    print(json.dumps(build(args.support_only, args.record_smoke), ensure_ascii=False, indent=2))
```

verify__clean__package.py

```
"""Verify a clean competition support package without refitting any model.

The check extracts the ZIP into a new temporary directory, verifies the frozen
scientific-artifact fingerprints, syntax-checks the Python sources, and
recompiles the LaTeX paper from the packaged sources.  It deliberately does not
run ``run_final.py`` or any Q1--Q4 fitting/optimization stage.
"""

from __future__ import annotations

import argparse
import compileall
import hashlib
import json
import re
import shutil
import subprocess
import tempfile
import time
import zipfile
from pathlib import Path

from pypdf import PdfReader


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def run(package: Path) -> dict[str, object]:
    started = time.perf_counter()
    package = package.resolve()
    if not package.is_file():
        raise FileNotFoundError(package)
    if shutil.which("xelatex") is None:
        raise RuntimeError("xelatex is required for the clean-package check")

    with tempfile.TemporaryDirectory(prefix="cumcm_clean_verify_") as temporary:
        clean_root = Path(temporary) / "competition_support"
        clean_root.mkdir()
        with zipfile.ZipFile(package) as archive:
            bad_member = archive.testzip()
            if bad_member is not None:
                raise RuntimeError(f"ZIP integrity failure: {bad_member}")
            archive.extractall(clean_root)

        required = [
            "paper/main.tex",
            "paper/main.pdf",
            "AI 工具使用详情.pdf",
            "final_validation.json",
            "science_fingerprints.json",
            "src/problem1/analysis.py",
            "src/problem2/analysis.py",
            "src/problem3/analysis.py",
            "src/problem4/analysis.py",
```

```
        ]
        missing = [name for name in required if not (clean_root / name).is_file()]
        if missing:
            raise RuntimeError(f"Missing required package files: {missing}")

        fingerprints = json.loads((clean_root / "science_fingerprints.json").read_text(encoding="utf-8"))
        mismatches: list[str] = []
        for relative, expected in fingerprints["files"].items():
            target = clean_root / relative
            if not target.is_file() or sha256(target) != expected:
                mismatches.append(relative)
        if mismatches:
            raise RuntimeError(f"Scientific-artifact fingerprint mismatch: {mismatches}")

        if not compileall.compile_dir(clean_root, quiet=1, force=True):
            raise RuntimeError("Python source syntax check failed")

        paper_dir = clean_root / "paper"
        latex_return_codes: list[int] = []
        for _ in range(2):
            process = subprocess.run(
                ["xelatex", "-interaction=nonstopmode", "-halt-on-error", "main.tex"],
                cwd=paper_dir,
                text=True,
                encoding="utf-8",
                errors="replace",
                capture_output=True,
                timeout=900,
                check=False,
            )
            latex_return_codes.append(process.returncode)
            if process.returncode != 0:
                raise RuntimeError(f"Clean-package XeLaTeX failed: {process.stdout[-4000:]}")

        reader = PdfReader(paper_dir / "main.pdf")
        aux_text = (paper_dir / "main.aux").read_text(encoding="utf-8", errors="replace")
        match = re.search(r"\\newlabel\{bodyend\}\{\{[^}]*\}\{(\d+)\}", aux_text)
        if not match:
            raise RuntimeError("bodyend label missing after clean-package compilation")
        body_pages = int(match.group(1))
        first_page = reader.pages[0].extract_text() or ""
        if body_pages > 30 or " 摘要" not in first_page:
            raise RuntimeError("Clean-package paper format check failed")

        return {
            "return_code": 0,
            "mode": "non_refitting_frozen_artifact_reproduction",
            "package": str(package),
            "package_sha256": sha256(package),
            "science_fingerprint_files": len(fingerprints["files"]),
            "science_fingerprints_match": True,
            "python_syntax_check": True,
            "latex_return_codes": latex_return_codes,
            "paper_total_pages": len(reader.pages),
            "paper_body_pages": body_pages,
            "runtime_seconds": time.perf_counter() - started,
        }


def main() -> int:
    parser = argparse.ArgumentParser()
    parser.add_argument("--package", required=True, type=Path)
    parser.add_argument("--output", type=Path)
    args = parser.parse_args()
    try:
        result = run(args.package)
    except Exception as exc:  # keep a machine-readable failure record
        result = {"return_code": 1, "error": f"{type(exc).__name__}: {exc}"}
    payload = json.dumps(result, ensure_ascii=False, indent=2)
    if args.output is not None:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(payload, encoding="utf-8")
    print(payload)
    return int(result["return_code"])


if __name__ == "__main__":
```

```
138     raise SystemExit(main())
```

src/finalization.py

```
1   """Final manuscript compilation, decision-log update, and compliance validation."""
2
3   from __future__ import annotations
4
5   import json
6   import re
7   import shutil
8   import subprocess
9   import time
10  from pathlib import Path
11
12  import pandas as pd
13  from pypdf import PdfReader
14
15  from src.problem4.paper import generate_q4_paper_artifacts
16  from src.utils.pdf_font_audit import audit_pdf_fonts
17  from src.utils.project import PAPER_FIGURES, RESULT_FIGURES, RESULT_TABLES, ROOT, STATE, load_json, save_json
18
19
20  PAPER = ROOT / "paper"
21  FINAL_VALIDATION = ROOT / "final_validation.json"
22
23  SOURCE_FILES = [
24      "run_final.py", "build_submission.py", "verify_clean_package.py", "run_all.py", "run_phase2.py", "run_phase2_5.py",
        ↪  "run_phase3.py", "run_phase4.py",
25      "src/finalization.py", "src/utils/project.py", "src/utils/pdf_font_audit.py",
26      "src/data_generation/generate_battery_data.py",
27      "src/problem1/analysis.py", "src/problem1/counterfactual.py", "src/problem1/plots.py",
28      "src/problem2/build_landmark_dataset.py", "src/problem2/models.py", "src/problem2/analysis.py",
29      "src/problem2/scenarios.py", "src/problem2/phase2_5.py", "src/problem2/plots.py",
30      "src/problem2/verify_aft_likelihood.py", "src/problem2/verify_aft_extended.py",
31      "src/problem3/build_retired_pool.py", "src/problem3/screening.py", "src/problem3/optimization.py",
32      "src/problem3/audit.py", "src/problem3/analysis.py", "src/problem3/phase3_5.py", "src/problem3/plots.py",
33      "src/problem4/population.py", "src/problem4/core.py", "src/problem4/policy.py",
34      "src/problem4/sensitivity.py", "src/problem4/analysis.py", "src/problem4/plots.py", "src/problem4/paper.py",
35  ]
36
37
38  def update_decision_log() -> None:
39      """Mark Q4 and the end-to-end submission workflow complete."""
40      path = STATE / "decision_log.json"
41      if not path.exists():
42          # Anonymous competition support packages intentionally omit internal
43          # orchestration state; numerical reproduction must not depend on it.
44          return
45      log = load_json(path)
46      q4 = load_json(RESULT_TABLES / "q4_key_metrics.json")
47      log["current_stage"] = 9
48      log["current_phase"] = "FINAL"
49      log["stages"]["5"]["status"] = "pass_all_questions"
50      log["stages"]["5"]["qi_status"]["Q4"] = "pass"
51      log["stages"]["5"]["sub_problems"]["Q4"] = {
52          "status": "pass",
53          "model_name": "probabilistic compatibility graph + stress Monte Carlo + nominal/adaptive/robust policies",
54          "code_path": "src/problem4/analysis.py",
55          "results_path": "results/tables/q4_key_metrics.json",
56          "key_metrics": q4,
57          "issues": [],
58      }
59      log["stages"]["5"]["cross_reference_chain"] = (
60          "Q1 stage/factor evidence feeds Q2; Q2 uncertainty feeds Q3 gates; the PHASE 3.5 full-space "
61          "solution is frozen before Q4; Q4 p0 uses calibration truth only and final holdout truth opens after strategy freeze."
62      )
63      log["ai_disclosure"] = {
64          "used": True,
65          "positioning": " 生成式 AI 作为辅助性技术工具",
66          "scope": " 程序调试、算法实现检查、数值复现检查、科研绘图、LaTeX 排版和语言润色",
67          "team_responsibility": " 模型方案、参数取值、结果解释和最终取舍由参赛队结合题意、文献与实验结果判断并确认。",
68      }
69      log["compliance"]["ai_usage"] = [
70          {
```

```
            "tool": "OpenAI Codex",
            "model": "GPT-5",
            "version": "Codex desktop, August 2026",
            "used_at": "2026-08-09 to 2026-08-10",
            "positioning": " 生成式 AI 作为辅助性技术工具",
            "use_stage": " 程序实现、结果检查、图件排版、论文修订与提交核验",
            "purpose": " 辅助程序调试、算法代码实现检查、数值复现流程检查、科研绘图与 LaTeX 排版，以及论文语言润色。",
            "paper_sections": [" 生成式人工智能使用说明", " 程序附录", " 科研图件与排版"],
            "query": " 围绕数据泄漏、删失似然、提交包复现、科研绘图和论文格式进行代表性技术检查。",
            "output": " 错误定位与代码修改建议、公式/代码一致性检查意见、绘图与排版建议、语言精简建议。",
            "human_review": " 模型方案、参数取值、结果解释和最终取舍由参赛队判断并确认；AI 辅助内容经人工检查和必要修改后采用。",
            "adoption": " 仅作为辅助性技术建议；数值结果以项目程序实际运行结果为准。",
            "evidence": ["state/decision_log.json", "final_validation.json", "paper/main.tex"],
        }
    ]
    log["run_metadata"]["phase_completed"] = "Q1-Q4, PHASE 3.5 audit, final paper and submission validation"
    log["run_metadata"]["q4_random_seed"] = q4["Q4_SEED"]
    save_json(log, path)


def compile_paper() -> dict[str, object]:
    """Compile twice so cross-references and the body-end page label stabilize."""
    generate_q4_paper_artifacts()
    if shutil.which("xelatex") is None:
        raise RuntimeError("xelatex is required for the final manuscript")
    logs: list[str] = []
    started = time.perf_counter()
    for pass_number in (1, 2):
        process = subprocess.run(
            ["xelatex", "-interaction=nonstopmode", "-halt-on-error", "main.tex"],
            cwd=PAPER,
            text=True,
            encoding="utf-8",
            errors="replace",
            capture_output=True,
            timeout=900,
            check=False,
        )
        logs.append(f"===== XELATEX PASS {pass_number} =====\n{process.stdout}\n{process.stderr}")
        if process.returncode != 0:
            (PAPER / "compile.log").write_text("\n".join(logs), encoding="utf-8")
            raise RuntimeError(f"XeLaTeX pass {pass_number} failed with return code {process.returncode}")
    (PAPER / "compile.log").write_text("\n".join(logs), encoding="utf-8")
    pdf = PAPER / "main.pdf"
    reader = PdfReader(pdf)
    aux_text = (PAPER / "main.aux").read_text(encoding="utf-8", errors="replace")
    match = re.search(r"\\newlabel\{bodyend\}\{\{[^}]*\}\{(\d+)\}", aux_text)
    if not match:
        raise RuntimeError("bodyend label missing from main.aux")
    body_pages = int(match.group(1))
    return {
        "pdf": str(pdf),
        "total_pages": len(reader.pages),
        "body_pages": body_pages,
        "appendix_pages": len(reader.pages) - body_pages,
        "pdf_bytes": pdf.stat().st_size,
        "runtime_seconds": float(time.perf_counter() - started),
    }


def _phase_pass(path: str) -> bool:
    payload = load_json(RESULT_TABLES / path)
    return bool(payload.get("overall_pass", False))


def validate_final(paper_info: dict[str, object]) -> dict[str, object]:
    """Create the single machine-readable final compliance verdict."""
    main_source = (PAPER / "main.tex").read_text(encoding="utf-8")
    pdf = PAPER / "main.pdf"
    reader = PdfReader(pdf)
    all_text = "\n".join((page.extract_text() or "") for page in reader.pages)
    first_page_text = reader.pages[0].extract_text() or ""
    q3 = load_json(RESULT_TABLES / "q3_phase3_5_summary.json")
    q4 = load_json(RESULT_TABLES / "q4_validation.json")
    q2_split = pd.read_csv(RESULT_TABLES / "q2_battery_split.csv")
    q2_split_counts = q2_split.groupby("Battery_ID")["Split"].nunique()
    q3_leak = load_json(RESULT_TABLES / "q3_pool_leakage_audit.json")
```

```
font_audit = audit_pdf_fonts(RESULT_FIGURES)
identity_patterns = [
    r"[A-Za-z]:[\\/](?:Users|Documents and Settings)[\\/]",
    r"@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
]
identity_hits = [pattern for pattern in identity_patterns if re.search(pattern, all_text, flags=re.I)]
missing_source_directives = [
    path for path in SOURCE_FILES
    if path not in main_source.replace("\\_", "_")
]
critical_figures = [
    "q1_lifecycle_evolution", "q1_counterfactual_factor_effects", "q2_soh_estimation_parity",
    "q2_rul_parity", "q3_compatibility_graph_summary", "q3_pareto_frontier",
    "q4_probability_graph", "q4_strategy_comparison", "q4_global_sensitivity", "q4_economic_and_scale",
]
existing = load_json(FINAL_VALIDATION) if FINAL_VALIDATION.exists() else {}
validation: dict[str, object] = {
    "all_phases_pass": bool(
        _phase_pass("pipeline_validation.json")
        and _phase_pass("q2_validation.json")
        and _phase_pass("phase2_5_validation.json")
        and _phase_pass("q3_validation.json")
        and _phase_pass("q4_validation.json")
    ),
    "no_data_leakage": bool(
        q2_split["Battery_ID"].is_unique
        and bool((q2_split_counts == 1).all())
        and set(q2_split["Split"]) == {"train", "validation", "test"}
        and q3_leak["q2_training_ids_disjoint"]
        and q3_leak["decision_hidden_physical_separation"]
        and not q3_leak["decision_leakage_hits"]
        and q4["final_holdout_truth_not_used_for_p0"]
        and q4["holdout_truth_read_after_strategy_freeze"]
    ),
    "q4_independent_holdout": bool(q4["q4_independent_holdout"] and q4["q2_q3_q4_ids_disjoint"]),
    "full_candidate_space_audit_pass": bool(q3["validation_pass"]),
    "robust_strategy_evaluated": bool(q4["nominal_adaptive_robust_evaluated"]),
    "global_sensitivity_complete": bool(q4["global_sensitivity_complete"]),
    "economic_break_even_complete": bool(q4["economic_break_even_complete"]),
    "all_figures_pass": bool(
        all(
            all((RESULT_FIGURES / f"{name}.{suffix}").exists() for suffix in ("png", "pdf", "svg"))
            for name in critical_figures
        )
        and font_audit["all_pdf_fonts_embedded"]
    ),
    "paper_body_within_30_pages": int(paper_info["body_pages"]) <= 30,
    "abstract_within_one_page": bool(" 摘要" in first_page_text and " 问题分析与总体框架" not in first_page_text),
    "no_table_of_contents": "\\tableofcontents" not in main_source,
    "no_identity_information": len(identity_hits) == 0,
    "full_source_code_in_appendix": len(missing_source_directives) == 0 and "\\VerbatimInput" in main_source,
    "support_file_list_in_appendix": "\\section{支撑文件清单}" in main_source,
    "paper_pdf_within_20_mb": pdf.stat().st_size <= 20 * 1024 * 1024,
    "q1_q4_numeric_traceability": bool(
        all((RESULT_TABLES / f"q{i}_key_metrics.json").exists() for i in range(1, 5))
        and all((PAPER / name).exists() for name in (
            "generated_metrics.tex", "generated_metrics_q2.tex", "generated_metrics_q3.tex", "generated_metrics_q4.tex"
        ))
    ),
    "paper_total_pages": int(paper_info["total_pages"]),
    "paper_body_pages": int(paper_info["body_pages"]),
    "paper_appendix_pages": int(paper_info["appendix_pages"]),
    "paper_pdf_bytes": int(paper_info["pdf_bytes"]),
    "identity_pattern_hits": identity_hits,
    "missing_source_directives": missing_source_directives,
}
for key in (
    "competition_support_zip_within_20_mb", "clean_package_reproduction_pass",
    "clean_package_return_code", "clean_package_runtime_seconds", "clean_package_stages",
):
    if key in existing:
        validation[key] = existing[key]
boolean_checks = [value for key, value in validation.items() if isinstance(value, bool)]
validation["overall_pass"] = all(boolean_checks)
save_json(validation, FINAL_VALIDATION)
if not validation["overall_pass"]:
    raise RuntimeError(f"Final validation failed: {json.dumps(validation, ensure_ascii=False, indent=2)}")
```

```
    return validation


def finalize_project() -> dict[str, object]:
    update_decision_log()
    paper_info = compile_paper()
    validation = validate_final(paper_info)
    return {"paper": paper_info, "validation": validation}


if __name__ == "__main__":
    print(json.dumps(finalize_project(), ensure_ascii=False, indent=2))
```

run__all.py

```
""" 一键复现: 仿真数据 -> 问题一分析 -> 图表/表格 -> 状态与验收清单。"""

from __future__ import annotations

import argparse
import hashlib
import platform
import shutil
import sys
import time
from pathlib import Path

import matplotlib
import numpy as np
import pandas as pd
import scipy
import sklearn
import statsmodels
from PIL import Image

from src.data_generation.generate_battery_data import generate_simulated_data
from src.problem1.analysis import analyze_problem1
from src.problem1.plots import create_problem1_figures
from src.utils.project import (
    DATA_PROCESSED,
    DATA_RAW,
    PARAMETER_SOURCES,
    RESULT_FIGURES,
    RESULT_TABLES,
    ROOT,
    STATE,
    ensure_directories,
    load_json,
    save_json,
    set_seed,
)
from src.utils.pdf_font_audit import audit_pdf_fonts


def _sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def _validate_outputs(allow_downstream: bool = False) -> dict[str, object]:
    raw = pd.read_csv(DATA_RAW / "battery_cycle_data.csv")
    stages = pd.read_csv(RESULT_TABLES / "stage_boundaries_by_battery.csv")
    weights = pd.read_csv(RESULT_TABLES / "factor_consensus_weights.csv")
    required = {
        "Battery_ID",
        "Batch_ID",
        "Cycle",
        "Temperature",
        "Charge_C_rate",
        "Discharge_C_rate",
        "DOD",
        "Charge_strategy",
```

```
        "Initial_capacity",
        "Capacity",
        "SOH",
        "Internal_resistance",
        "Mean_voltage",
        "Voltage_dispersion",
        "End_of_discharge_voltage",
        "Coulombic_efficiency",
        "Energy_efficiency",
        "Degradation_rate",
        "Failure",
    }
    missing_columns = sorted(required - set(raw.columns))
    median_slopes = stages[
        [
            "Stage1_slope_pp_per_100_cycles",
            "Stage2_slope_pp_per_100_cycles",
            "Stage3_slope_pp_per_100_cycles",
        ]
    ].median()
    png_dpi_pass = True
    for png_path in RESULT_FIGURES.glob("*.png"):
        with Image.open(png_path) as image:
            dpi = image.info.get("dpi", (0.0, 0.0))
            png_dpi_pass = png_dpi_pass and min(dpi) >= 299.0
    font_audit = load_json(RESULT_TABLES / "pdf_font_audit.json")
    checks = {
        "required_columns_present": not missing_columns,
        "missing_required_columns": missing_columns,
        "battery_count_in_requested_range": 300 <= raw["Battery_ID"].nunique() <= 1000,
        "no_duplicate_battery_cycle": not raw.duplicated(["Battery_ID", "Cycle"]).any(),
        "no_missing_values": not raw.isna().any().any(),
        "three_stage_median_slopes_accelerate": bool(
            median_slopes.iloc[2] < median_slopes.iloc[1] < median_slopes.iloc[0] < 0
        ),
        "consensus_weights_sum_to_one": bool(np.isclose(weights["Consensus_weight"].sum(), 1.0, atol=1e-7)),
        "all_key_figures_exist": all(
            (RESULT_FIGURES / f"{name}.{suffix}").exists()
            for name in (
                "q1_lifecycle_evolution",
                "q1_capacity_resistance_relation",
                "q1_stage_detection_validation",
                "q1_factor_importance_methods",
                "q1_consensus_factor_weights",
                "q1_operational_factor_weights",
                "q1_counterfactual_factor_effects",
            )
            for suffix in ("png", "pdf", "svg")
        ),
        "q1_weighting_bias_assessed": (RESULT_TABLES / "q1_weighting_sensitivity.csv").exists(),
        "q1_operational_weights_generated": (
            RESULT_TABLES / "q1_operational_factor_weights.csv"
        ).exists(),
        "q1_counterfactual_completed": (
            RESULT_TABLES / "q1_counterfactual_lifetime_effects.csv"
        ).exists(),
        "q1_permutation_sensitivity_completed": (
            RESULT_TABLES / "q1_permutation_sensitivity.csv"
        ).exists(),
        "q1_censoring_audit_completed": (
            RESULT_TABLES / "q1_censoring_stage_audit.csv"
        ).exists(),
        "all_png_at_least_300_dpi_with_metadata_tolerance": bool(png_dpi_pass),
        "all_pdf_fonts_embedded": bool(font_audit["all_pdf_fonts_embedded"]),
        "q2_not_executed": not any((ROOT / "results").rglob("q2_*")),
        "full_pipeline_downstream_allowed": bool(allow_downstream),
    }
    checks["overall_pass"] = all(
        value
        for key, value in checks.items()
        if key not in {"missing_required_columns", "q2_not_executed", "full_pipeline_downstream_allowed"}
    ) and (checks["q2_not_executed"] or allow_downstream)
    save_json(checks, RESULT_TABLES / "pipeline_validation.json")
    if not checks["overall_pass"]:
        raise RuntimeError(f"Pipeline validation failed: {checks}")
    return checks

```

```python

def _update_state(metrics: dict[str, object], figure_manifest: pd.DataFrame, runtime: float) -> None:
    decision_path = STATE / "decision_log.json"
    if not decision_path.exists():
        print("[INFO] state/decision_log.json is absent; numerical pipeline completed without state update.")
        return
    decision = load_json(decision_path)
    q1 = decision["stages"]["5"]["sub_problems"]["Q1"]
    q1.update(
        {
            "status": "pass",
            "model_name": "Continuous three-segment change-point model + cluster-aware contribution consensus",
            "code_path": "src/problem1/analysis.py",
            "results_path": "results/tables/q1_key_metrics.json",
            "figures": figure_manifest["Figure"].tolist(),
            "key_metrics": metrics,
            "physical_meaning_summary": (
                "The population exhibits slow, accelerated, and knee-dominated degradation. "
                "Factor weights describe simulated predictive contribution, not causal effect size."
            ),
            "scores": {
                "model_fit": 9,
                "mathematical_rigor": 8,
                "solution_correctness": 9,
                "result_evidence": 9,
                "physical_interpretation": 8,
            },
            "issues": [],
            "iterations": 1,
        }
    )
    decision["stages"]["5"]["qi_status"]["Q1"] = "pass"
    decision["stages"]["5"]["last_updated"] = pd.Timestamp.utcnow().isoformat()
    decision["run_metadata"] = {
        "runtime_seconds": runtime,
        "random_seed": 20260809,
        "raw_data_sha256": _sha256(DATA_RAW / "battery_cycle_data.csv"),
        "phase_completed": "PHASE 0 + PHASE 1 only",
    }
    save_json(decision, decision_path)


def _write_latex_metrics(metrics: dict[str, object]) -> None:
    """ 从机器结果生成 LaTeX 宏，避免在论文中手抄数值。"""
    def tex_text(value: object) -> str:
        return str(value).replace("_", "\\_")

    display_names = {
        "Temperature": " 温度",
        "Charge_C_rate": " 充电倍率",
        "Discharge_C_rate": " 放电倍率",
        "DOD": "DOD",
        "Charge_strategy": " 充电策略",
    }
    first_tier_text = "、".join(display_names.get(item, str(item)) for item in metrics["operational_first_tier"])

    lines = [
        "% Auto-generated by run_all.py; do not edit numerical values by hand.",
        f"\\newcommand{{\\DatasetRows}}{{{int(metrics['rows']):,}}}",
        f"\\newcommand{{\\BatteryCount}}{{{int(metrics['batteries'])}}}",
        f"\\newcommand{{\\FailureRate}}{{{100 * float(metrics['failure_rate']):.1f}\\%}}",
        f"\\newcommand{{\\MedianOnset}}{{{float(metrics['median_detected_onset_cycle']):.0f}}}",
        f"\\newcommand{{\\MedianKnee}}{{{float(metrics['median_detected_knee_cycle']):.0f}}}",
        f"\\newcommand{{\\StageOneSlope}}{{{float(metrics['median_stage1_slope_pp_per_100_cycles']):.3f}}}",
        f"\\newcommand{{\\StageTwoSlope}}{{{float(metrics['median_stage2_slope_pp_per_100_cycles']):.3f}}}",
        f"\\newcommand{{\\StageThreeSlope}}{{{float(metrics['median_stage3_slope_pp_per_100_cycles']):.3f}}}",
        f"\\newcommand{{\\KneeMAE}}{{{float(metrics['knee_detection_MAE_cycles']):.1f}}}",
        f"\\newcommand{{\\RFTestRtwo}}{{{float(metrics['ml_test_R2']):.3f}}}",
        f"\\newcommand{{\\TopFactor}}{{{tex_text(metrics['top_controllable_factor'])}}}",
        f"\\newcommand{{\\TopFactorWeight}}{{{float(metrics['top_controllable_consensus_weight']):.3f}}}",
        f"\\newcommand{{\\TopFactorStability}}{{{100 * float(metrics['top_controllable_rank_stability']):.1f}\\%}}",
        f"\\newcommand{{\\SOHResistanceRho}}{{{float(metrics['soh_resistance_spearman_rho']):.3f}}}",
        f"\\newcommand{{\\OperationalFirstTier}}{{{first_tier_text}}}",
        f"\\newcommand{{\\EOLKneeMAE}}{{{float(metrics['eol_only_knee_detection_MAE_cycles']):.1f}}}",
        f"\\newcommand{{\\CensoredKneeMAE}}{{{float(metrics['censored_knee_detection_MAE_cycles']):.1f}}}",
        f"\\newcommand{{\\CensoredKneeSufficiency}}{{{100 * float(metrics['censored_post_knee_sufficiency_rate']):.1f}\\%}}",
        f"\\newcommand{{\\HighTempLifetimeLoss}}{{{float(metrics['counterfactual_45C_lifetime_loss_percent']):.1f}\\%}}",
```

```
        f"\\newcommand{{\\HighChargeLifetimeLoss}}{{{float(metrics['counterfactual_2C_charge_lifetime_loss_percent']):.1f}\\%}}",
        f"\\newcommand{{\\DeepDODLifetimeLoss}}{{{float(metrics['counterfactual_100DOD_lifetime_loss_percent']):.1f}\\%}}",
    ]
    (ROOT / "paper" / "generated_metrics.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")


def main(skip_generation: bool = False, allow_downstream: bool = False) -> None:
    started = time.perf_counter()
    ensure_directories()
    config = load_json(PARAMETER_SOURCES / "simulation_config.json")
    seed = int(config["random_seed"])
    set_seed(seed)

    raw_path = DATA_RAW / "battery_cycle_data.csv"
    battery_path = DATA_PROCESSED / "battery_summary.csv"
    if skip_generation and raw_path.exists() and battery_path.exists():
        print("[data] using existing deterministic dataset (--skip-generation)")
    else:
        raw_path, battery_path = generate_simulated_data()

    shutil.copy2(PARAMETER_SOURCES / "parameter_registry.csv", RESULT_TABLES / "parameter_registry.csv")
    results = analyze_problem1(raw_path, battery_path)
    figure_manifest = create_problem1_figures(results)
    audit_pdf_fonts(RESULT_FIGURES)
    checks = _validate_outputs(allow_downstream=allow_downstream)
    runtime = time.perf_counter() - started
    metrics = load_json(RESULT_TABLES / "q1_key_metrics.json")
    metrics["total_runtime_seconds"] = runtime
    save_json(metrics, RESULT_TABLES / "q1_key_metrics.json")
    _write_latex_metrics(metrics)

    run_manifest = {
        "status": "pass",
        "phase": "PHASE 0 + PHASE 1",
        "q2_executed": False,
        "random_seed": seed,
        "runtime_seconds": runtime,
        "python": sys.version,
        "platform": platform.platform(),
        "packages": {
            "numpy": np.__version__,
            "pandas": pd.__version__,
            "scipy": scipy.__version__,
            "scikit-learn": sklearn.__version__,
            "statsmodels": statsmodels.__version__,
            "matplotlib": matplotlib.__version__,
        },
        "validation": checks,
        "raw_data_sha256": _sha256(raw_path),
    }
    save_json(run_manifest, RESULT_TABLES / "run_manifest.json")
    _update_state(metrics, figure_manifest, runtime)
    suffix = "full-pipeline mode permits downstream artifacts" if allow_downstream else "Q2 was not executed"
    print(f"[done] PHASE 0 + PHASE 1 passed in {runtime:.1f} s; {suffix}.")


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-generation", action="store_true", help=" 复用已有确定性仿真 CSV。")
    parser.add_argument(
        "--allow-downstream",
        action="store_true",
        help="Allow pre-existing Q2-Q4 artifacts when called by the full pipeline.",
    )
    args = parser.parse_args()
    main(skip_generation=args.skip_generation, allow_downstream=args.allow_downstream)
```

run__phase2.py

```
""" 一键复现 PHASE 2 / Q2，并在 Q2 后强制停止。"""

from __future__ import annotations

import argparse
import json
```

```
import shutil
import subprocess
import time

from pypdf import PdfReader

from src.problem2.analysis import run_problem2
from src.problem2.plots import create_problem2_figures
from src.problem2.verify_aft_likelihood import verify
from src.utils.pdf_font_audit import audit_pdf_fonts
from src.utils.project import PAPER_FIGURES, RESULT_FIGURES, RESULT_TABLES, ROOT, ensure_directories, load_json, save_json


def _compile_paper() -> dict[str, object]:
    executable = shutil.which("xelatex")
    if executable is None:
        return {"compiled": False, "reason": "xelatex not found", "page_count": None}
    command = [executable, "-interaction=nonstopmode", "-halt-on-error", "main.tex"]
    for _ in range(2):
        completed = subprocess.run(
            command,
            cwd=ROOT / "paper",
            capture_output=True,
            text=True,
            encoding="utf-8",
            errors="replace",
            check=False,
        )
        if completed.returncode != 0:
            tail = "\n".join(completed.stdout.splitlines()[-40:])
            raise RuntimeError(f"XeLaTeX failed:\n{tail}")
    pdf_path = ROOT / "paper" / "main.pdf"
    page_count = len(PdfReader(pdf_path).pages)
    return {
        "compiled": True,
        "file": str(pdf_path),
        "page_count": page_count,
        "within_30_page_main_text_limit": page_count <= 30,
    }


def main(skip_paper: bool = False) -> None:
    started = time.perf_counter()
    ensure_directories()
    symbolic = verify()
    if not symbolic["overall_pass"]:
        raise RuntimeError("Weibull AFT symbolic verification failed.")
    metrics = run_problem2()
    figure_manifest = create_problem2_figures()
    font_audit = audit_pdf_fonts(RESULT_FIGURES)
    paper = {"compiled": False, "reason": "--skip-paper"} if skip_paper else _compile_paper()
    validation = load_json(RESULT_TABLES / "q2_validation.json")
    validation.update(
        {
            "q2_figure_count_is_10": len(figure_manifest) == 10,
            "q2_png_pdf_svg_complete": all(
                all((RESULT_FIGURES / f"{name}.{suffix}").exists() for suffix in ("png", "pdf", "svg"))
                for name in figure_manifest["Figure"]
            ),
            "paper_figure_copies_complete": all(
                all((PAPER_FIGURES / f"{name}.{suffix}").exists() for suffix in ("png", "pdf"))
                for name in figure_manifest["Figure"]
            ),
            "all_figure_pdf_fonts_embedded": bool(font_audit["all_pdf_fonts_embedded"]),
            "paper_compiled": bool(paper["compiled"]) if not skip_paper else True,
            "paper_within_page_limit": (
                bool(paper.get("within_30_page_main_text_limit")) if not skip_paper else True
            ),
            "q3_not_executed": True,
        }
    )
    validation["overall_pass"] = all(value for key, value in validation.items() if key != "overall_pass")
    save_json(validation, RESULT_TABLES / "q2_validation.json")
    if not validation["overall_pass"]:
        raise RuntimeError(f"PHASE 2 validation failed: {json.dumps(validation, ensure_ascii=False)}")
    manifest = load_json(RESULT_TABLES / "q2_run_manifest.json")
    manifest["total_runtime_seconds"] = float(time.perf_counter() - started)
```

```
    manifest["figure_count"] = int(len(figure_manifest))
    manifest["paper"] = paper
    manifest["validation"] = validation
    manifest["q3_executed"] = False
    save_json(manifest, RESULT_TABLES / "q2_run_manifest.json")
    print(
        f"[done] PHASE 2 / Q2 passed in {manifest['total_runtime_seconds']:.1f} s; "
        f"landmarks={metrics['landmark_rows']}; Q3 was not executed."
    )


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-paper", action="store_true", help=" 跳过 XeLaTeX 编译，只跑数值与图件。")
    args = parser.parse_args()
    main(skip_paper=args.skip_paper)
```

run_phase2_5.py

```
"""Run the PHASE 2.5 deployment bridge audit and stop before Q3."""

from __future__ import annotations

import argparse
import json

from src.problem2.phase2_5 import run_phase2_5
from src.problem2.verify_aft_extended import verify


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--allow-downstream",
        action="store_true",
        help="Allow pre-existing Q3-Q4 artifacts when called by the full pipeline.",
    )
    args = parser.parse_args()
    verification = verify()
    if not verification["overall_pass"]:
        raise RuntimeError("Extended AFT likelihood verification failed.")
    print(json.dumps(run_phase2_5(allow_downstream=args.allow_downstream), ensure_ascii=False, indent=2))
```

run_phase3.py

```
"""Run PHASE 3 / Q3, create audited deliverables, and stop before Q4."""

from __future__ import annotations

import argparse
import json
import shutil
import subprocess
import time

from pypdf import PdfReader

from src.problem3.analysis import run_problem3
from src.problem3.phase3_5 import run_phase3_5
from src.problem3.plots import create_problem3_figures
from src.utils.pdf_font_audit import audit_pdf_fonts
from src.utils.project import PAPER_FIGURES, RESULT_FIGURES, RESULT_TABLES, ROOT, load_json, save_json


def _compile_paper() -> dict[str, object]:
    executable = shutil.which("xelatex")
    if executable is None:
        return {"compiled": False, "reason": "xelatex not found", "page_count": None}
    command = [executable, "-interaction=nonstopmode", "-halt-on-error", "main.tex"]
    for _ in range(2):
        completed = subprocess.run(
            command,
            cwd=ROOT / "paper",
```

```
            capture_output=True,
            text=True,
            encoding="utf-8",
            errors="replace",
            check=False,
        )
        if completed.returncode != 0:
            tail = "\n".join(completed.stdout.splitlines()[-40:])
            raise RuntimeError(f"XeLaTeX failed:\n{tail}")
    pdf_path = ROOT / "paper" / "main.pdf"
    page_count = len(PdfReader(pdf_path).pages)
    return {
        "compiled": True,
        "file": str(pdf_path),
        "page_count": page_count,
        "within_phase3_page_limit": page_count <= 22,
    }


def main(skip_paper: bool = False, allow_downstream: bool = False) -> None:
    started = time.perf_counter()
    metrics = run_problem3(allow_downstream=allow_downstream)
    phase3_5 = run_phase3_5()
    metrics = load_json(RESULT_TABLES / "q3_key_metrics.json")
    figure_manifest = create_problem3_figures()
    font_audit = audit_pdf_fonts(RESULT_FIGURES)
    paper = {"compiled": False, "reason": "--skip-paper"} if skip_paper else _compile_paper()
    validation = load_json(RESULT_TABLES / "q3_validation.json")
    validation.update(
        {
            "q3_figure_count_is_5": len(figure_manifest) == 5,
            "q3_png_pdf_svg_complete": all(
                all((RESULT_FIGURES / f"{name}.{suffix}").exists() for suffix in ("png", "pdf", "svg"))
                for name in figure_manifest["Figure"]
            ),
            "q3_paper_figure_copies_complete": all(
                all((PAPER_FIGURES / f"{name}.{suffix}").exists() for suffix in ("png", "pdf"))
                for name in figure_manifest["Figure"]
            ),
            "all_figure_pdf_fonts_embedded": bool(font_audit["all_pdf_fonts_embedded"]),
            "paper_compiled": bool(paper["compiled"]) if not skip_paper else True,
            "paper_within_22_page_limit": bool(paper.get("within_phase3_page_limit")) if not skip_paper else True,
            "phase3_5_full_candidate_solution": bool(phase3_5["validation_pass"]),
        }
    )
    reproducibility_path = RESULT_TABLES / "q3_submission_reproducibility_audit.json"
    if reproducibility_path.exists():
        reproducibility = load_json(reproducibility_path)
        validation["clean_submission_reproduction_pass"] = bool(
            reproducibility.get("checks", {}).get("overall_pass", False)
        )
    visual_audit_path = RESULT_TABLES / "q3_pdf_visual_audit.json"
    if visual_audit_path.exists() and not skip_paper:
        validation["paper_visual_audit_pass"] = bool(load_json(visual_audit_path).get("overall_pass", False))
    validation["overall_pass"] = all(
        value
        for key, value in validation.items()
        if key not in {"overall_pass", "q4_not_executed", "full_pipeline_downstream_allowed"}
    ) and (validation.get("q4_not_executed", True) or allow_downstream)
    save_json(validation, RESULT_TABLES / "q3_validation.json")
    if not validation["overall_pass"]:
        raise RuntimeError(f"PHASE 3 validation failed: {json.dumps(validation, ensure_ascii=False)}")
    manifest = {
        "phase": "PHASE 3 / Q3 only",
        "total_runtime_seconds": float(time.perf_counter() - started),
        "figure_count": int(len(figure_manifest)),
        "paper": paper,
        "validation": validation,
        "selected_solution": metrics["selected_solution"]["Solution_ID"],
        "q4_executed": False,
        "phase3_5": phase3_5,
    }
    save_json(manifest, RESULT_TABLES / "q3_run_manifest.json")
    print(
        f"[done] PHASE 3 / Q3 passed in {manifest['total_runtime_seconds']:.1f} s; "
        f"selected={manifest['selected_solution']}; Q4 was not executed."
    )
```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--skip-paper", action="store_true", help="Skip XeLaTeX compilation.")
    parser.add_argument(
        "--allow-downstream",
        action="store_true",
        help="Allow pre-existing Q4 artifacts when called by the full pipeline.",
    )
    args = parser.parse_args()
    main(skip_paper=args.skip_paper, allow_downstream=args.allow_downstream)
```

run__phase4.py

```
"""Run PHASE 4 / Q4, generate the five core figures, and validate artifacts."""

from __future__ import annotations

import json
import time

from src.problem4.analysis import run_problem4
from src.problem4.plots import create_problem4_figures
from src.utils.pdf_font_audit import audit_pdf_fonts
from src.utils.project import RESULT_FIGURES, RESULT_TABLES, save_json


def main() -> dict[str, object]:
    started = time.perf_counter()
    metrics = run_problem4()
    figures = create_problem4_figures()
    font_audit = audit_pdf_fonts(RESULT_FIGURES)
    validation_path = RESULT_TABLES / "q4_validation.json"
    validation = json.loads(validation_path.read_text(encoding="utf-8"))
    validation.update(
        {
            "q4_core_figure_count_is_5": len(figures) == 5,
            "q4_png_pdf_svg_complete": all(
                all((RESULT_FIGURES / f"{name}.{suffix}").exists() for suffix in ("png", "pdf", "svg"))
                for name in figures["Figure"]
            ),
            "q4_figure_pdf_fonts_embedded": bool(font_audit["all_pdf_fonts_embedded"]),
        }
    )
    validation["overall_pass"] = all(value for key, value in validation.items() if key != "overall_pass")
    save_json(validation, validation_path)
    if not validation["overall_pass"]:
        raise RuntimeError(f"PHASE 4 validation failed: {validation}")
    manifest = {
        "phase": "PHASE 4 / Q4 final",
        "runtime_seconds": float(time.perf_counter() - started),
        "figure_count": int(len(figures)),
        "validation": validation,
        "p0": metrics["p0"],
    }
    save_json(manifest, RESULT_TABLES / "q4_run_manifest.json")
    print(f"[done] PHASE 4 passed in {manifest['runtime_seconds']:.1f} s; p0={manifest['p0']:.2f}")
    return manifest


if __name__ == "__main__":
    main()
```

src/utils/project.py

```
""" 对应论文附录：项目路径、随机种子与通用 I/O。"""

from __future__ import annotations

import json
import os
import random
```

```
from pathlib import Path
from typing import Any

import numpy as np


ROOT = Path(__file__).resolve().parents[2]
DATA_RAW = ROOT / "data" / "raw_simulated"
DATA_PROCESSED = ROOT / "data" / "processed"
PARAMETER_SOURCES = ROOT / "data" / "parameter_sources"
RESULT_TABLES = ROOT / "results" / "tables"
RESULT_FIGURES = ROOT / "results" / "figures"
RESULT_MODELS = ROOT / "results" / "models"
PAPER_FIGURES = ROOT / "paper" / "figures"
PAPER_TABLES = ROOT / "paper" / "tables"
STATE = ROOT / "state"


def ensure_directories() -> None:
    for path in (
        DATA_RAW,
        DATA_PROCESSED,
        PARAMETER_SOURCES,
        RESULT_TABLES,
        RESULT_FIGURES,
        RESULT_MODELS,
        PAPER_FIGURES,
        PAPER_TABLES,
        STATE,
    ):
        path.mkdir(parents=True, exist_ok=True)


def set_seed(seed: int) -> None:
    """ 固定 Python/NumPy 随机源并限制哈希随机性。"""
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)


def load_json(path: Path) -> dict[str, Any]:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)


def save_json(payload: Any, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        json.dump(payload, handle, ensure_ascii=False, indent=2)

```

src/utils/pdf_font_audit.py

```
""" 检查 Matplotlib PDF 的字体描述符是否包含嵌入字体流。"""

from __future__ import annotations

from pathlib import Path

from pypdf import PdfReader

from src.utils.project import RESULT_TABLES, save_json


def audit_pdf_fonts(figure_directory: Path) -> dict[str, object]:
    files = []
    all_embedded = True
    for path in sorted(figure_directory.glob("*.pdf")):
        fonts: dict[tuple[str, str], bool] = {}
        for page in PdfReader(path).pages:
            resources = page.get("/Resources") or {}
            for _, reference in (resources.get("/Font") or {}).items():
                font = reference.get_object()
                descriptor_reference = font.get("/FontDescriptor")
                if descriptor_reference is None:
```

```
                    descendants = font.get("/DescendantFonts") or []
                    if descendants:
                        descriptor_reference = descendants[0].get_object().get("/FontDescriptor")
                embedded = False
                if descriptor_reference is not None:
                    descriptor = descriptor_reference.get_object()
                    embedded = any(key in descriptor for key in ("/FontFile", "/FontFile2", "/FontFile3"))
                key = (str(font.get("/BaseFont")), str(font.get("/Subtype")))
                fonts[key] = fonts.get(key, True) and embedded
        font_rows = [
            {"base_font": base_font, "subtype": subtype, "embedded": embedded}
            for (base_font, subtype), embedded in sorted(fonts.items())
        ]
        file_pass = bool(font_rows) and all(item["embedded"] for item in font_rows)
        all_embedded = all_embedded and file_pass
        files.append({"file": path.name, "all_fonts_embedded": file_pass, "fonts": font_rows})
    report = {"all_pdf_fonts_embedded": all_embedded, "files": files}
    save_json(report, RESULT_TABLES / "pdf_font_audit.json")
    return report
```

src/data_generation/generate_battery_data.py

```
""" 对应论文 §5.1: 生成带异质性、工况应力与膝点加速的全生命周期仿真数据。"""

from __future__ import annotations

import hashlib
import json
from pathlib import Path

import numpy as np
import pandas as pd

from src.utils.project import DATA_PROCESSED, DATA_RAW, PARAMETER_SOURCES, load_json, save_json


def _file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(block)
    return digest.hexdigest()


def _first_index(mask: np.ndarray, default: int) -> int:
    found = np.flatnonzero(mask)
    return int(found[0]) if found.size else default


def generate_simulated_data(config_path: Path | None = None) -> tuple[Path, Path]:
    """ 生成逐循环数据和电池级摘要；所有随机性由配置文件中的种子控制。"""
    config_path = config_path or PARAMETER_SOURCES / "simulation_config.json"
    cfg = load_json(config_path)
    seed = int(cfg["random_seed"])
    rng = np.random.default_rng(seed)

    n_batteries = int(cfg["n_batteries"])
    n_batches = int(cfg["n_batches"])
    max_cycles = int(cfg["max_cycles"])
    nominal_q = float(cfg["nominal_capacity_ah"])
    nominal_r = float(cfg["nominal_internal_resistance_mohm"])
    eol_soh = float(cfg["eol_soh_percent"])
    ref_cycles = float(cfg["reference_cycles"])

    batch_capacity = rng.normal(1.0, 0.008, n_batches)
    batch_resistance = rng.normal(1.0, 0.045, n_batches)
    batch_fade = rng.lognormal(mean=0.0, sigma=0.075, size=n_batches)

    temperature_levels = np.array([8.0, 15.0, 25.0, 35.0, 43.0])
    temperature_prob = np.array([0.10, 0.16, 0.31, 0.27, 0.16])
    charge_levels = np.array([0.5, 1.0, 1.5, 2.0])
    charge_prob = np.array([0.18, 0.36, 0.30, 0.16])
    discharge_levels = np.array([0.5, 1.0, 1.5, 2.0])
    discharge_prob = np.array([0.16, 0.36, 0.31, 0.17])
    strategies = np.array(["CC-CV", "Multi-stage", "Fast-charge"])
```

```
    strategy_prob = np.array([0.50, 0.28, 0.22])
    strategy_multiplier = {"CC-CV": 1.0, "Multi-stage": 0.92, "Fast-charge": 1.14}

    all_frames: list[pd.DataFrame] = []
    battery_rows: list[dict[str, float | int | str]] = []

    for battery_index in range(n_batteries):
        battery_id = f"BAT_{battery_index + 1:04d}"
        batch_index = battery_index % n_batches
        batch_id = f"B{batch_index + 1:02d}"

        base_temperature = float(rng.choice(temperature_levels, p=temperature_prob))
        base_charge = float(rng.choice(charge_levels, p=charge_prob))
        base_discharge = float(rng.choice(discharge_levels, p=discharge_prob))
        base_dod = float(np.clip(0.50 + 0.50 * rng.beta(2.4, 1.8), 0.50, 1.00))
        strategy = str(rng.choice(strategies, p=strategy_prob))

        initial_capacity = nominal_q * batch_capacity[batch_index] * rng.normal(1.0, 0.012)
        initial_resistance = nominal_r * batch_resistance[batch_index] * rng.lognormal(0.0, 0.035)
        fade_heterogeneity = batch_fade[batch_index] * rng.lognormal(0.0, 0.09)
        knee_heterogeneity = rng.lognormal(0.0, 0.085)
        phase = rng.uniform(0.0, 2.0 * np.pi)

        cycle = np.arange(max_cycles + 1, dtype=int)
        seasonal = np.sin(2.0 * np.pi * cycle / 220.0 + phase)
        temperature = np.clip(
            base_temperature + 1.25 * seasonal + rng.normal(0.0, 0.60, cycle.size),
            5.0,
            45.0,
        )
        charge_rate = np.clip(base_charge + rng.normal(0.0, 0.035, cycle.size), 0.45, 2.10)
        discharge_rate = np.clip(base_discharge + rng.normal(0.0, 0.040, cycle.size), 0.45, 2.10)
        dod = np.clip(base_dod + 0.015 * seasonal + rng.normal(0.0, 0.012, cycle.size), 0.48, 1.00)

        high_temperature = np.exp(0.026 * np.maximum(temperature - 25.0, 0.0))
        low_temperature = 1.0 + 0.010 * np.maximum(15.0 - temperature, 0.0) ** 1.25
        charge_stress = 1.0 + 0.34 * (charge_rate - 1.0) + 0.11 * (charge_rate - 1.0) ** 2
        discharge_stress = 1.0 + 0.18 * (discharge_rate - 1.0) + 0.06 * (discharge_rate - 1.0) ** 2
        dod_stress = (dod / 0.80) ** 1.25
        interaction = (
            1.0
            + 0.08 * np.maximum(charge_rate - 1.0, 0.0) * np.maximum(dod - 0.75, 0.0) / 0.25
            + 0.05 * np.maximum(temperature - 30.0, 0.0) / 15.0 * np.maximum(charge_rate - 1.0, 0.0)
        )
        stress = np.clip(
            high_temperature
            * low_temperature
            * charge_stress
            * discharge_stress
            * dod_stress
            * strategy_multiplier[strategy]
            * interaction,
            0.42,
            3.40,
        )
        stress[0] = 0.0
        equivalent_cycles = np.cumsum(stress)

        knee_exposure = float(cfg["base_knee_cycle"]) * knee_heterogeneity
        onset_exposure = 0.47 * knee_exposure * rng.normal(1.0, 0.035)
        base_x = equivalent_cycles / ref_cycles
        mid_x = np.maximum(equivalent_cycles - onset_exposure, 0.0) / 500.0
        knee_x = np.maximum(equivalent_cycles - knee_exposure, 0.0) / 400.0

        base_loss = (
            float(cfg["base_fade_coefficient"])
            * fade_heterogeneity
            * base_x ** float(cfg["base_fade_exponent"])
        )
        mid_loss = (
            float(cfg["mid_acceleration_coefficient"])
            * fade_heterogeneity
            * mid_x ** float(cfg["mid_acceleration_exponent"])
        )
        knee_loss = (
            float(cfg["knee_acceleration_coefficient"])
            * fade_heterogeneity
```

```
            * knee_x ** float(cfg["knee_acceleration_exponent"])
        )
        latent_loss = base_loss + mid_loss + knee_loss
        latent_soh = 100.0 * (1.0 - latent_loss)

        final_index = _first_index(latent_soh <= eol_soh, max_cycles)
        sl = slice(0, final_index + 1)
        n_obs = final_index + 1

        capacity_noise = rng.normal(0.0, float(cfg["capacity_measurement_sd_ah"]), n_obs)
        capacity = initial_capacity * (1.0 - latent_loss[sl]) + capacity_noise
        soh = 100.0 * capacity / initial_capacity

        resistance_growth = 1.0 + 1.65 * latent_loss[sl] + 1.10 * knee_loss[sl]
        internal_resistance = (
            initial_resistance * resistance_growth
            + 0.07 * (temperature[sl] - 25.0)
            + rng.normal(0.0, float(cfg["resistance_measurement_sd_mohm"]), n_obs)
        )

        base_exp = float(cfg["base_fade_exponent"])
        mid_exp = float(cfg["mid_acceleration_exponent"])
        knee_exp = float(cfg["knee_acceleration_exponent"])
        dloss_de = (
            float(cfg["base_fade_coefficient"])
            * fade_heterogeneity
            * base_exp
            / ref_cycles
            * np.maximum(base_x[sl], 1e-12) ** (base_exp - 1.0)
        )
        dloss_de += np.where(
            mid_x[sl] > 0.0,
            float(cfg["mid_acceleration_coefficient"])
            * fade_heterogeneity
            * mid_exp
            / 500.0
            * mid_x[sl] ** (mid_exp - 1.0),
            0.0,
        )
        dloss_de += np.where(
            knee_x[sl] > 0.0,
            float(cfg["knee_acceleration_coefficient"])
            * fade_heterogeneity
            * knee_exp
            / 400.0
            * knee_x[sl] ** (knee_exp - 1.0),
            0.0,
        )
        degradation_rate = np.maximum(
            100.0 * dloss_de * stress[sl] + rng.normal(0.0, 0.00045, n_obs),
            0.0,
        )

        mean_voltage = (
            3.68
            - 0.0032 * (100.0 - soh)
            - 0.012 * (discharge_rate[sl] - 1.0)
            + 0.0008 * (temperature[sl] - 25.0)
            + rng.normal(0.0, 0.004, n_obs)
        )
        voltage_std = np.maximum(
            0.016
            + 0.00055 * (100.0 - soh)
            + 0.0025 * np.maximum(discharge_rate[sl] - 1.0, 0.0)
            + rng.normal(0.0, 0.0012, n_obs),
            0.005,
        )
        end_voltage = (
            3.16
            - 0.0040 * (100.0 - soh)
            - 0.028 * (discharge_rate[sl] - 1.0)
            + rng.normal(0.0, 0.006, n_obs)
        )
        coulombic_efficiency = np.clip(
            99.72
            - 0.055 * stress[sl]
            - 0.42 * knee_loss[sl]
```

```
            + rng.normal(0.0, 0.025, n_obs),
            98.8,
            99.95,
        )
        energy_efficiency = np.clip(
            96.8
            - 0.026 * (internal_resistance - initial_resistance)
            - 0.40 * np.maximum(discharge_rate[sl] - 1.0, 0.0)
            + rng.normal(0.0, 0.10, n_obs),
            88.0,
            98.5,
        )
        failure = np.zeros(n_obs, dtype=int)
        reached_eol = final_index < max_cycles
        if reached_eol:
            failure[-1] = 1

        frame = pd.DataFrame(
            {
                "Battery_ID": battery_id,
                "Batch_ID": batch_id,
                "Cycle": cycle[sl],
                "Temperature": temperature[sl],
                "Charge_C_rate": charge_rate[sl],
                "Discharge_C_rate": discharge_rate[sl],
                "DOD": 100.0 * dod[sl],
                "Charge_strategy": strategy,
                "Initial_capacity": initial_capacity,
                "Capacity": capacity,
                "SOH": soh,
                "Internal_resistance": internal_resistance,
                "Mean_voltage": mean_voltage,
                "Voltage_dispersion": voltage_std,
                "End_of_discharge_voltage": end_voltage,
                "Coulombic_efficiency": coulombic_efficiency,
                "Energy_efficiency": energy_efficiency,
                "Degradation_rate": degradation_rate,
                "Failure": failure,
                "Stress_index": stress[sl],
                "Equivalent_stress_cycles": equivalent_cycles[sl],
            }
        )
        all_frames.append(frame)

        true_onset_cycle = _first_index(equivalent_cycles >= onset_exposure, max_cycles)
        true_knee_cycle = _first_index(equivalent_cycles >= knee_exposure, max_cycles)
        battery_rows.append(
            {
                "Battery_ID": battery_id,
                "Batch_ID": batch_id,
                "Base_temperature": base_temperature,
                "Base_charge_C_rate": base_charge,
                "Base_discharge_C_rate": base_discharge,
                "Base_DOD": 100.0 * base_dod,
                "Charge_strategy": strategy,
                "Initial_capacity": initial_capacity,
                "Initial_resistance": initial_resistance,
                "Mean_stress_index": float(np.mean(stress[1 : final_index + 1])),
                "Observed_cycles": final_index,
                "Reached_EOL": int(reached_eol),
                "EOL_cycle": final_index if reached_eol else np.nan,
                "True_onset_cycle": true_onset_cycle,
                "True_knee_cycle": true_knee_cycle,
                "Fade_heterogeneity": fade_heterogeneity,
            }
        )

    cycle_df = pd.concat(all_frames, ignore_index=True)
    battery_df = pd.DataFrame(battery_rows)
    raw_path = DATA_RAW / "battery_cycle_data.csv"
    summary_path = DATA_PROCESSED / "battery_summary.csv"
    cycle_df.to_csv(raw_path, index=False, float_format="%.6f")
    battery_df.to_csv(summary_path, index=False, float_format="%.6f")

    metadata = {
        "dataset_type": "fully_simulated",
        "claim_boundary": "No row is measured experimental data; coefficients are simulation settings unless the registry says
        ↪  otherwise.",
```

```
        "random_seed": seed,
        "rows": int(len(cycle_df)),
        "columns": cycle_df.columns.tolist(),
        "batteries": int(cycle_df["Battery_ID"].nunique()),
        "batches": int(cycle_df["Batch_ID"].nunique()),
        "failed_batteries": int(battery_df["Reached_EOL"].sum()),
        "right_censored_batteries": int((1 - battery_df["Reached_EOL"]).sum()),
        "raw_csv_sha256": _file_sha256(raw_path),
        "config": cfg,
    }
    save_json(metadata, DATA_RAW / "dataset_metadata.json")
    print(
        f"[data] rows={len(cycle_df):,}, batteries={metadata['batteries']}, "
        f"EOL={metadata['failed_batteries']}, censored={metadata['right_censored_batteries']}"
    )
    return raw_path, summary_path


if __name__ == "__main__":
    generate_simulated_data()
```

src/problem1/analysis.py

```
""" 对应论文 §5.2-§5.4：退化阶段识别、因素贡献与稳定性融合。"""

from __future__ import annotations

import json
import time
from pathlib import Path
from typing import Iterable

import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy.stats import spearmanr
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

from src.problem1.counterfactual import run_counterfactual_experiment
from src.utils.project import (
    DATA_PROCESSED,
    PARAMETER_SOURCES,
    RESULT_MODELS,
    RESULT_TABLES,
    STATE,
    load_json,
    save_json,
)


FACTOR_ORDER = [
    "Cycle",
    "Temperature",
    "Charge_C_rate",
    "Discharge_C_rate",
    "DOD",
    "Charge_strategy",
    "Batch_ID",
]


def _fit_continuous_three_segment(cycle: np.ndarray, soh: np.ndarray) -> dict[str, float]:
    """ 在归一化寿命轴上用两个折点的连续折线最小化 SSE。"""
    order = np.argsort(cycle)
    x_raw = cycle[order].astype(float)
    y_raw = soh[order].astype(float)
    max_cycle = float(x_raw[-1])
    if max_cycle <= 0:
        raise ValueError("Battery lifecycle must contain positive cycles.")

    smoothed = pd.Series(y_raw).rolling(17, center=True, min_periods=1).median().to_numpy()
```

```
    grid = np.linspace(0.0, 1.0, 101)
    y = np.interp(grid, x_raw / max_cycle, smoothed)
    x = grid

    b1_candidates = np.linspace(0.18, 0.52, 15)
    b2_candidates = np.linspace(0.50, 0.88, 17)
    best_any: tuple[float, float, float, np.ndarray] | None = None
    best_monotone: tuple[float, float, float, np.ndarray] | None = None

    for b1 in b1_candidates:
        for b2 in b2_candidates:
            if b2 - b1 < 0.18:
                continue
            design = np.column_stack(
                [np.ones_like(x), x, np.maximum(x - b1, 0.0), np.maximum(x - b2, 0.0)]
            )
            coef, *_ = np.linalg.lstsq(design, y, rcond=None)
            residual = y - design @ coef
            sse = float(residual @ residual)
            slopes = np.array([coef[1], coef[1] + coef[2], coef[1] + coef[2] + coef[3]])
            candidate = (sse, float(b1), float(b2), coef)
            if best_any is None or sse < best_any[0]:
                best_any = candidate
            if slopes[0] < -0.2 and slopes[1] < slopes[0] - 0.15 and slopes[2] < slopes[1] - 0.15:
                if best_monotone is None or sse < best_monotone[0]:
                    best_monotone = candidate

    selected = best_monotone or best_any
    if selected is None:
        raise RuntimeError("No admissible change-point candidate was found.")
    sse, b1, b2, coef = selected
    slopes_norm = np.array([coef[1], coef[1] + coef[2], coef[1] + coef[2] + coef[3]])

    def predict(x_value: float) -> float:
        vector = np.array([1.0, x_value, max(x_value - b1, 0.0), max(x_value - b2, 0.0)])
        return float(vector @ coef)

    return {
        "Detected_onset_fraction": b1,
        "Detected_knee_fraction": b2,
        "Detected_onset_cycle": b1 * max_cycle,
        "Detected_knee_cycle": b2 * max_cycle,
        "SOH_at_onset": predict(b1),
        "SOH_at_knee": predict(b2),
        "Stage1_slope_pp_per_100_cycles": slopes_norm[0] / max_cycle * 100.0,
        "Stage2_slope_pp_per_100_cycles": slopes_norm[1] / max_cycle * 100.0,
        "Stage3_slope_pp_per_100_cycles": slopes_norm[2] / max_cycle * 100.0,
        "Piecewise_SSE": sse,
        "Monotone_acceleration_constraint_used": int(best_monotone is not None),
    }


def detect_lifecycle_stages(cycle_df: pd.DataFrame, battery_df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    records: list[dict[str, float | str | int]] = []
    normalized_grid = np.linspace(0.0, 1.0, 101)
    normalized_rows: list[pd.DataFrame] = []

    truth = battery_df.set_index("Battery_ID")
    for battery_id, group in cycle_df.groupby("Battery_ID", sort=True):
        result = _fit_continuous_three_segment(group["Cycle"].to_numpy(), group["SOH"].to_numpy())
        result["Battery_ID"] = battery_id
        result["Observed_cycles"] = int(group["Cycle"].max())
        result["True_onset_cycle"] = float(truth.loc[battery_id, "True_onset_cycle"])
        result["True_knee_cycle"] = float(truth.loc[battery_id, "True_knee_cycle"])
        result["Onset_absolute_error"] = abs(result["Detected_onset_cycle"] - result["True_onset_cycle"])
        result["Knee_absolute_error"] = abs(result["Detected_knee_cycle"] - result["True_knee_cycle"])
        records.append(result)

        x = group["Cycle"].to_numpy(dtype=float)
        y = group["SOH"].to_numpy(dtype=float)
        max_cycle = max(float(x[-1]), 1.0)
        normalized_rows.append(
            pd.DataFrame(
                {
                    "Battery_ID": battery_id,
                    "Life_fraction": normalized_grid,
                    "SOH": np.interp(normalized_grid, x / max_cycle, y),
```

```
                }
            )
        )

    stages = pd.DataFrame(records)
    normalized = pd.concat(normalized_rows, ignore_index=True)
    return stages, normalized


def _make_row_weighted_sample(cycle_df: pd.DataFrame, stride: int) -> pd.DataFrame:
    """ 旧方案：固定循环步长抽样，寿命越长的电池贡献行越多。"""
    sample = cycle_df.loc[(cycle_df["Cycle"] > 0) & (cycle_df["Cycle"] % stride == 0)].copy()
    return sample.reset_index(drop=True)


def _make_battery_balanced_sample(cycle_df: pd.DataFrame, landmarks_per_battery: int = 20) -> pd.DataFrame:
    """ 主方案：每节电池仅取等量已观测生命周期 landmark，不访问观测终点之后的信息。"""
    frames: list[pd.DataFrame] = []
    for _, group in cycle_df.loc[cycle_df["Cycle"] > 0].groupby("Battery_ID", sort=True):
        group = group.sort_values("Cycle")
        positions = np.linspace(0, len(group) - 1, min(landmarks_per_battery, len(group)))
        positions = np.unique(np.round(positions).astype(int))
        selected = group.iloc[positions].copy()
        selected["Landmark_index"] = np.arange(1, len(selected) + 1)
        frames.append(selected)
    return pd.concat(frames, ignore_index=True)


def _statistical_design(sample: pd.DataFrame, max_cycles: float) -> tuple[pd.DataFrame, dict[str, list[str]]]:
    design = pd.DataFrame(index=sample.index)
    cycle_n = sample["Cycle"].to_numpy(dtype=float) / max_cycles
    temp = sample["Temperature"].to_numpy(dtype=float)
    charge = sample["Charge_C_rate"].to_numpy(dtype=float)
    discharge = sample["Discharge_C_rate"].to_numpy(dtype=float)
    dod = sample["DOD"].to_numpy(dtype=float) / 100.0

    design["Intercept"] = 1.0
    design["Cycle"] = cycle_n
    design["Cycle_sq"] = cycle_n**2
    design["Temperature_centered"] = (temp - 25.0) / 10.0
    design["Temperature_high_sq"] = (np.maximum(temp - 25.0, 0.0) / 10.0) ** 2
    design["Temperature_low_sq"] = (np.maximum(15.0 - temp, 0.0) / 10.0) ** 2
    design["Charge_C_rate"] = charge
    design["Charge_C_rate_sq"] = charge**2
    design["Discharge_C_rate"] = discharge
    design["Discharge_C_rate_sq"] = discharge**2
    design["DOD"] = dod
    design["DOD_sq"] = dod**2
    design["Temperature_x_charge"] = ((temp - 25.0) / 10.0) * charge
    design["Charge_x_DOD"] = charge * dod
    design["Discharge_x_DOD"] = discharge * dod

    strategy = pd.get_dummies(sample["Charge_strategy"], prefix="Strategy", drop_first=True, dtype=float)
    batch = pd.get_dummies(sample["Batch_ID"], prefix="Batch", drop_first=True, dtype=float)
    design = pd.concat([design, strategy, batch], axis=1)

    groups = {
        "Cycle": ["Cycle", "Cycle_sq"],
        "Temperature": [
            "Temperature_centered",
            "Temperature_high_sq",
            "Temperature_low_sq",
            "Temperature_x_charge",
        ],
        "Charge_C_rate": [
            "Charge_C_rate",
            "Charge_C_rate_sq",
            "Temperature_x_charge",
            "Charge_x_DOD",
        ],
        "Discharge_C_rate": ["Discharge_C_rate", "Discharge_C_rate_sq", "Discharge_x_DOD"],
        "DOD": ["DOD", "DOD_sq", "Charge_x_DOD", "Discharge_x_DOD"],
        "Charge_strategy": strategy.columns.tolist(),
        "Batch_ID": batch.columns.tolist(),
    }
    return design.astype(float), groups
```

```python

def _r2_from_matrix(matrix: np.ndarray, y: np.ndarray, weights: np.ndarray | None = None) -> float:
    weights = np.ones(len(y), dtype=float) if weights is None else np.asarray(weights, dtype=float)
    root_w = np.sqrt(weights)
    weighted_matrix = matrix * root_w[:, None]
    weighted_y = y * root_w
    coef, *_ = np.linalg.lstsq(weighted_matrix, weighted_y, rcond=None)
    residual = y - matrix @ coef
    mean_y = float(np.average(y, weights=weights))
    sse = float(np.sum(weights * residual**2))
    centered = y - mean_y
    sst = float(np.sum(weights * centered**2))
    return 1.0 - sse / sst if sst > 0 else 0.0


def _drop_group_delta_r2(
    design: pd.DataFrame,
    y: np.ndarray,
    groups: dict[str, list[str]],
    weights: np.ndarray | None = None,
) -> dict[str, float]:
    full_r2 = _r2_from_matrix(design.to_numpy(), y, weights)
    result: dict[str, float] = {}
    for factor, columns in groups.items():
        keep = [column for column in design.columns if column not in set(columns)]
        reduced_r2 = _r2_from_matrix(design[keep].to_numpy(), y, weights)
        result[factor] = max(0.0, full_r2 - reduced_r2)
    result["__full_r2__"] = full_r2
    return result


def _battery_balanced_subset(sample: pd.DataFrame, max_rows_per_battery: int = 9) -> np.ndarray:
    indices: list[int] = []
    for _, group in sample.groupby("Battery_ID", sort=False):
        raw = group.index.to_numpy()
        if raw.size <= max_rows_per_battery:
            indices.extend(raw.tolist())
        else:
            positions = np.linspace(0, raw.size - 1, max_rows_per_battery).round().astype(int)
            indices.extend(raw[positions].tolist())
    return np.asarray(indices, dtype=int)


def statistical_factor_importance(
    sample: pd.DataFrame,
    max_cycles: float,
    bootstrap_repetitions: int,
    seed: int,
) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, float], np.ndarray]:
    design, groups = _statistical_design(sample, max_cycles)
    y = sample["Degradation_rate"].to_numpy(dtype=float)
    counts = sample.groupby("Battery_ID")["Battery_ID"].transform("size").to_numpy(dtype=float)
    sample_weight = 1.0 / counts
    sample_weight *= len(sample_weight) / sample_weight.sum()
    point = _drop_group_delta_r2(design, y, groups, sample_weight)

    robust_fit = sm.WLS(y, design, weights=sample_weight).fit(
        cov_type="cluster",
        cov_kwds={"groups": sample["Battery_ID"].to_numpy()},
    )
    coefficient_table = pd.DataFrame(
        {
            "Term": design.columns,
            "Coefficient": robust_fit.params.to_numpy(),
            "Cluster_robust_SE": robust_fit.bse.to_numpy(),
            "P_value": robust_fit.pvalues.to_numpy(),
        }
    )

    balanced_indices = _battery_balanced_subset(sample, max_rows_per_battery=20)
    boot_sample = sample.loc[balanced_indices].reset_index(drop=True)
    boot_design, boot_groups = _statistical_design(boot_sample, max_cycles)
    boot_y = boot_sample["Degradation_rate"].to_numpy(dtype=float)
    id_to_rows = {
        battery_id: group.index.to_numpy()
        for battery_id, group in boot_sample.groupby("Battery_ID", sort=False)
    }
```

```
    battery_ids = np.array(list(id_to_rows))
    rng = np.random.default_rng(seed + 101)
    bootstrap_values = np.zeros((bootstrap_repetitions, len(FACTOR_ORDER)), dtype=float)
    for repetition in range(bootstrap_repetitions):
        selected_ids = rng.choice(battery_ids, size=len(battery_ids), replace=True)
        row_index = np.concatenate([id_to_rows[battery_id] for battery_id in selected_ids])
        draw_weights = np.concatenate(
            [np.full(len(id_to_rows[battery_id]), 1.0 / len(id_to_rows[battery_id])) for battery_id in selected_ids]
        )
        contribution = _drop_group_delta_r2(
            boot_design.iloc[row_index].reset_index(drop=True),
            boot_y[row_index],
            boot_groups,
            draw_weights,
        )
        bootstrap_values[repetition] = [contribution[factor] for factor in FACTOR_ORDER]

    rows = []
    point_values = np.array([point[factor] for factor in FACTOR_ORDER])
    normalized = point_values / max(point_values.sum(), 1e-12)
    for index, factor in enumerate(FACTOR_ORDER):
        ci_low = float(np.quantile(bootstrap_values[:, index], 0.025))
        ci_high = float(np.quantile(bootstrap_values[:, index], 0.975))
        rows.append(
            {
                "Factor": factor,
                "Delta_R2": point_values[index],
                "Normalized_weight": normalized[index],
                "Bootstrap_CI_low": min(ci_low, point_values[index]),
                "Bootstrap_CI_high": max(ci_high, point_values[index]),
            }
        )
    table = pd.DataFrame(rows)
    metrics = {"statistical_full_R2": point["__full_r2__"]}
    return table, coefficient_table, metrics, bootstrap_values


def _group_aware_permute(
    features: pd.DataFrame,
    battery_ids: pd.Series,
    factor: str,
    rng: np.random.Generator,
) -> pd.DataFrame:
    """ 条件/块置换：保留局部生命周期或电池级结构，避免逐行完全打散。"""
    permuted = features.copy()
    meta = pd.DataFrame(
        {
            "Battery_ID": battery_ids.to_numpy(),
            "Batch_ID": features["Batch_ID"].to_numpy(),
            "Cycle": features["Cycle"].to_numpy(),
        }
    )
    if factor == "Cycle":
        for _, rows in meta.groupby("Battery_ID", sort=False).groups.items():
            positions = np.asarray(list(rows), dtype=int)
            values = permuted.iloc[positions][factor].to_numpy()
            blocks = [values[start : start + 4] for start in range(0, len(values), 4)]
            order = rng.permutation(len(blocks))
            permuted.iloc[positions, permuted.columns.get_loc(factor)] = np.concatenate(
                [blocks[index] for index in order]
            )
    elif factor in {"Charge_strategy", "Batch_ID"}:
        battery_table = meta.drop_duplicates("Battery_ID").copy()
        battery_table["Value"] = features.groupby(battery_ids, sort=False)[factor].first().reindex(
            battery_table["Battery_ID"]
        ).to_numpy()
        mapping: dict[str, object] = {}
        grouping = battery_table.groupby("Batch_ID", sort=False) if factor == "Charge_strategy" else [("all", battery_table)]
        for _, group in grouping:
            shuffled_values = rng.permutation(group["Value"].to_numpy())
            mapping.update(dict(zip(group["Battery_ID"], shuffled_values)))
        permuted[factor] = battery_ids.map(mapping).to_numpy()
    else:
        cycle_bin = pd.qcut(meta["Cycle"], q=8, labels=False, duplicates="drop")
        strata = pd.DataFrame({"Batch_ID": meta["Batch_ID"], "Cycle_bin": cycle_bin})
        values = permuted[factor].to_numpy(copy=True)
        for _, rows in strata.groupby(["Batch_ID", "Cycle_bin"], dropna=False).groups.items():
```

```
            positions = np.asarray(list(rows), dtype=int)
            values[positions] = rng.permutation(values[positions])
        permuted[factor] = values
    return permuted


def permutation_factor_importance(
    sample: pd.DataFrame,
    repetitions: int,
    seed: int,
    method: str = "group-aware",
) -> tuple[pd.DataFrame, dict[str, float], np.ndarray, Pipeline]:
    if method not in {"global", "group-aware"}:
        raise ValueError(f"Unknown permutation method: {method}")
    features = sample[FACTOR_ORDER].copy()
    target = sample["Degradation_rate"].to_numpy(dtype=float)
    unique_ids = sample["Battery_ID"].unique()
    rng = np.random.default_rng(seed + 202)
    shuffled = rng.permutation(unique_ids)
    split = int(0.70 * len(shuffled))
    train_ids = set(shuffled[:split])
    train_mask = sample["Battery_ID"].isin(train_ids).to_numpy()
    test_mask = ~train_mask
    sample["Analysis_split"] = np.where(train_mask, "train", "test")
    row_counts = sample.groupby("Battery_ID")["Battery_ID"].transform("size").to_numpy(dtype=float)
    row_weights = 1.0 / row_counts
    row_weights *= len(row_weights) / row_weights.sum()

    categorical = ["Charge_strategy", "Batch_ID"]
    numeric = [factor for factor in FACTOR_ORDER if factor not in categorical]
    preprocessor = ColumnTransformer(
        [
            ("numeric", "passthrough", numeric),
            ("categorical", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical),
        ]
    )
    forest = RandomForestRegressor(
        n_estimators=180,
        min_samples_leaf=5,
        max_features=0.80,
        n_jobs=-1,
        random_state=seed,
    )
    model = Pipeline([("preprocess", preprocessor), ("model", forest)])
    model.fit(
        features.loc[train_mask],
        target[train_mask],
        model__sample_weight=row_weights[train_mask],
    )

    test_features = features.loc[test_mask].reset_index(drop=True)
    y_test = target[test_mask]
    test_battery = sample.loc[test_mask, "Battery_ID"].reset_index(drop=True)
    prediction = model.predict(test_features)
    baseline_error = (y_test - prediction) ** 2
    variance = max(float(np.var(y_test)), 1e-12)
    test_ids = np.array(sorted(test_battery.unique()))

    per_battery = np.zeros((repetitions, len(test_ids), len(FACTOR_ORDER)), dtype=float)
    for factor_index, factor in enumerate(FACTOR_ORDER):
        for repetition in range(repetitions):
            if method == "global":
                permuted = test_features.copy()
                permuted[factor] = rng.permutation(permuted[factor].to_numpy())
            else:
                permuted = _group_aware_permute(test_features, test_battery, factor, rng)
            permuted_prediction = model.predict(permuted)
            delta_error = (y_test - permuted_prediction) ** 2 - baseline_error
            by_battery = (
                pd.DataFrame({"Battery_ID": test_battery, "Delta": delta_error})
                .groupby("Battery_ID")["Delta"]
                .mean()
                .reindex(test_ids)
                .to_numpy()
            )
            per_battery[repetition, :, factor_index] = by_battery / variance

```

```
    point_values = per_battery.mean(axis=(0, 1))
    positive_point = np.maximum(point_values, 0.0)
    normalized = positive_point / max(positive_point.sum(), 1e-12)
    bootstrap_repetitions = 600
    bootstrap_values = np.zeros((bootstrap_repetitions, len(FACTOR_ORDER)), dtype=float)
    for repetition in range(bootstrap_repetitions):
        battery_draw = rng.integers(0, len(test_ids), size=len(test_ids))
        permutation_draw = rng.integers(0, per_battery.shape[0])
        bootstrap_values[repetition] = per_battery[permutation_draw, battery_draw, :].mean(axis=0)

    rows = []
    for index, factor in enumerate(FACTOR_ORDER):
        ci_low = float(np.quantile(bootstrap_values[:, index], 0.025))
        ci_high = float(np.quantile(bootstrap_values[:, index], 0.975))
        rows.append(
            {
                "Factor": factor,
                "Permutation_R2_decrease": point_values[index],
                "Normalized_weight": normalized[index],
                "Bootstrap_CI_low": min(ci_low, point_values[index]),
                "Bootstrap_CI_high": max(ci_high, point_values[index]),
            }
        )
    table = pd.DataFrame(rows)
    metrics = {
        "permutation_method": method,
        "ml_train_batteries": int(train_mask.sum() > 0 and len(train_ids)),
        "ml_test_batteries": int(len(test_ids)),
        "ml_test_R2": float(r2_score(y_test, prediction)),
        "ml_test_MAE_pp_per_cycle": float(mean_absolute_error(y_test, prediction)),
        "ml_train_rows": int(train_mask.sum()),
        "ml_test_rows": int(test_mask.sum()),
    }
    return table, metrics, bootstrap_values, model


def consensus_factor_weights(
    statistical: pd.DataFrame,
    permutation: pd.DataFrame,
    statistical_bootstrap: np.ndarray,
    permutation_bootstrap: np.ndarray,
    seed: int,
) -> tuple[pd.DataFrame, np.ndarray]:
    stat_point = statistical.set_index("Factor").loc[FACTOR_ORDER, "Delta_R2"].to_numpy()
    perm_point = permutation.set_index("Factor").loc[FACTOR_ORDER, "Permutation_R2_decrease"].to_numpy()
    stat_norm = np.maximum(stat_point, 0.0) / max(np.maximum(stat_point, 0.0).sum(), 1e-12)
    perm_norm = np.maximum(perm_point, 0.0) / max(np.maximum(perm_point, 0.0).sum(), 1e-12)
    consensus_point = 0.5 * stat_norm + 0.5 * perm_norm

    rng = np.random.default_rng(seed + 303)
    repetitions = 1000
    draws = np.zeros((repetitions, len(FACTOR_ORDER)), dtype=float)
    for repetition in range(repetitions):
        stat_draw = np.maximum(
            statistical_bootstrap[rng.integers(0, statistical_bootstrap.shape[0])], 0.0
        )
        perm_draw = np.maximum(
            permutation_bootstrap[rng.integers(0, permutation_bootstrap.shape[0])], 0.0
        )
        stat_draw /= max(stat_draw.sum(), 1e-12)
        perm_draw /= max(perm_draw.sum(), 1e-12)
        draws[repetition] = 0.5 * stat_draw + 0.5 * perm_draw

    top_index = np.argmax(draws, axis=1)
    controllable_indices = np.array([index for index, factor in enumerate(FACTOR_ORDER) if factor != "Cycle"])
    top_controllable_index = controllable_indices[np.argmax(draws[:, controllable_indices], axis=1)]
    rows = []
    for index, factor in enumerate(FACTOR_ORDER):
        ci_low = float(np.quantile(draws[:, index], 0.025))
        ci_high = float(np.quantile(draws[:, index], 0.975))
        rows.append(
            {
                "Factor": factor,
                "Statistical_normalized_weight": stat_norm[index],
                "Permutation_normalized_weight": perm_norm[index],
                "Consensus_weight": consensus_point[index],
                "Consensus_CI_low": min(ci_low, consensus_point[index]),
```

```
                "Consensus_CI_high": max(ci_high, consensus_point[index]),
                "Top_rank_stability": float(np.mean(top_index == index)),
                "Top_controllable_rank_stability": (
                    float(np.mean(top_controllable_index == index)) if factor != "Cycle" else np.nan
                ),
            }
        )
    table = pd.DataFrame(rows).sort_values("Consensus_weight", ascending=False).reset_index(drop=True)
    return table, draws


def _weighting_sensitivity(row_weighted: pd.DataFrame, balanced: pd.DataFrame) -> pd.DataFrame:
    row = row_weighted.set_index("Factor")["Consensus_weight"]
    main = balanced.set_index("Factor")["Consensus_weight"]
    row_rank = row.rank(ascending=False, method="min").astype(int)
    main_rank = main.rank(ascending=False, method="min").astype(int)
    records = []
    for factor in FACTOR_ORDER:
        records.append(
            {
                "Factor": factor,
                "Row_weighted_weight": float(row[factor]),
                "Battery_balanced_weight": float(main[factor]),
                "Difference": float(main[factor] - row[factor]),
                "Row_weighted_rank": int(row_rank[factor]),
                "Battery_balanced_rank": int(main_rank[factor]),
                "Rank_change": int(main_rank[factor] - row_rank[factor]),
            }
        )
    return pd.DataFrame(records).sort_values("Battery_balanced_rank")


def _permutation_sensitivity(global_table: pd.DataFrame, group_table: pd.DataFrame) -> pd.DataFrame:
    global_values = global_table.set_index("Factor")["Normalized_weight"]
    group_values = group_table.set_index("Factor")["Normalized_weight"]
    global_rank = global_values.rank(ascending=False, method="min").astype(int)
    group_rank = group_values.rank(ascending=False, method="min").astype(int)
    return pd.DataFrame(
        [
            {
                "Factor": factor,
                "Global_permutation_weight": float(global_values[factor]),
                "Group_aware_permutation_weight": float(group_values[factor]),
                "Difference": float(group_values[factor] - global_values[factor]),
                "Global_rank": int(global_rank[factor]),
                "Group_aware_rank": int(group_rank[factor]),
                "Rank_change": int(group_rank[factor] - global_rank[factor]),
            }
            for factor in FACTOR_ORDER
        ]
    ).sort_values("Group_aware_rank")


def _operational_factor_weights(consensus: pd.DataFrame, draws: np.ndarray) -> pd.DataFrame:
    controllable = [
        "Temperature",
        "Charge_C_rate",
        "Discharge_C_rate",
        "DOD",
        "Charge_strategy",
    ]
    ordered_table = consensus.set_index("Factor").loc[FACTOR_ORDER]
    point = ordered_table["Consensus_weight"].to_numpy()
    indices = np.array([FACTOR_ORDER.index(factor) for factor in controllable])
    point_operational = point[indices] / point[indices].sum()
    draw_operational = draws[:, indices]
    draw_operational = draw_operational / np.maximum(draw_operational.sum(axis=1, keepdims=True), 1e-12)
    top_draw = np.argmax(draw_operational, axis=1)
    top_point_index = int(np.argmax(point_operational))
    rows = []
    for index, factor in enumerate(controllable):
        difference_to_top = draw_operational[:, index] - draw_operational[:, top_point_index]
        diff_low, diff_high = np.quantile(difference_to_top, [0.025, 0.975])
        rows.append(
            {
                "Factor": factor,
                "Operational_weight": float(point_operational[index]),
```

```
                "Bootstrap_CI_low": float(np.quantile(draw_operational[:, index], 0.025)),
                "Bootstrap_CI_high": float(np.quantile(draw_operational[:, index], 0.975)),
                "Top_rank_probability": float(np.mean(top_draw == index)),
                "Difference_to_point_top_CI_low": float(diff_low),
                "Difference_to_point_top_CI_high": float(diff_high),
                "Statistically_distinguishable_from_top": bool(diff_high < 0.0),
                "First_tier": bool(diff_low <= 0.0 <= diff_high),
            }
        )
    return pd.DataFrame(rows).sort_values("Operational_weight", ascending=False).reset_index(drop=True)


def _censoring_stage_audit(stages: pd.DataFrame, battery_df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    detail = stages.merge(
        battery_df[["Battery_ID", "Reached_EOL"]],
        on="Battery_ID",
        how="left",
        validate="one_to_one",
    )
    detail["Post_knee_observed_cycles"] = detail["Observed_cycles"] - detail["Detected_knee_cycle"]
    detail["Post_knee_required_cycles"] = np.maximum(100.0, 0.10 * detail["Observed_cycles"])
    detail["Post_knee_sufficient"] = (
        detail["Post_knee_observed_cycles"] >= detail["Post_knee_required_cycles"]
    )
    groups = [
        ("All batteries", detail),
        ("Reached-EOL batteries", detail.loc[detail["Reached_EOL"] == 1]),
        ("Right-censored batteries", detail.loc[detail["Reached_EOL"] == 0]),
    ]
    rows = []
    for label, group in groups:
        rows.append(
            {
                "Group": label,
                "N_batteries": int(len(group)),
                "Onset_MAE_cycles": float(group["Onset_absolute_error"].mean()),
                "Knee_MAE_cycles": float(group["Knee_absolute_error"].mean()),
                "Median_detected_onset_cycle": float(group["Detected_onset_cycle"].median()),
                "Median_detected_knee_cycle": float(group["Detected_knee_cycle"].median()),
                "Median_stage1_slope_pp_per_100_cycles": float(
                    group["Stage1_slope_pp_per_100_cycles"].median()
                ),
                "Median_stage2_slope_pp_per_100_cycles": float(
                    group["Stage2_slope_pp_per_100_cycles"].median()
                ),
                "Median_stage3_slope_pp_per_100_cycles": float(
                    group["Stage3_slope_pp_per_100_cycles"].median()
                ),
                "Post_knee_sufficiency_rate": float(group["Post_knee_sufficient"].mean()),
            }
        )
    return pd.DataFrame(rows), detail


def data_quality_summary(cycle_df: pd.DataFrame, battery_df: pd.DataFrame) -> tuple[pd.DataFrame, dict[str, float]]:
    sampled = cycle_df.iloc[:: max(1, len(cycle_df) // 100000)]
    rho, rho_p = spearmanr(sampled["SOH"], sampled["Internal_resistance"])
    battery_rho = []
    for _, group in cycle_df.groupby("Battery_ID"):
        value, _ = spearmanr(group["Cycle"], group["SOH"])
        battery_rho.append(value)

    failure_rows = cycle_df.loc[cycle_df["Failure"] == 1]
    final_cycle = cycle_df.groupby("Battery_ID")["Cycle"].max()
    failure_at_last = (
        failure_rows.set_index("Battery_ID")["Cycle"] == final_cycle.reindex(failure_rows["Battery_ID"]).to_numpy()
    ).all()
    metrics = {
        "rows": int(len(cycle_df)),
        "columns": int(cycle_df.shape[1]),
        "batteries": int(cycle_df["Battery_ID"].nunique()),
        "batches": int(cycle_df["Batch_ID"].nunique()),
        "missing_cells": int(cycle_df.isna().sum().sum()),
        "duplicate_battery_cycle_rows": int(cycle_df.duplicated(["Battery_ID", "Cycle"]).sum()),
        "failed_batteries": int(battery_df["Reached_EOL"].sum()),
        "failure_rate": float(battery_df["Reached_EOL"].mean()),
        "median_observed_cycles": float(battery_df["Observed_cycles"].median()),
```

```
        "median_EOL_cycle_failed": float(battery_df.loc[battery_df["Reached_EOL"] == 1, "EOL_cycle"].median()),
        "soh_min": float(cycle_df["SOH"].min()),
        "soh_max": float(cycle_df["SOH"].max()),
        "capacity_min_ah": float(cycle_df["Capacity"].min()),
        "capacity_max_ah": float(cycle_df["Capacity"].max()),
        "resistance_min_mohm": float(cycle_df["Internal_resistance"].min()),
        "resistance_max_mohm": float(cycle_df["Internal_resistance"].max()),
        "soh_resistance_spearman_rho": float(rho),
        "soh_resistance_spearman_p": float(rho_p),
        "median_battery_cycle_soh_spearman": float(np.nanmedian(battery_rho)),
        "all_failure_labels_on_final_row": bool(failure_at_last),
    }
    table = pd.DataFrame(
        [{"Metric": key, "Value": value} for key, value in metrics.items()]
    )
    return table, metrics


def _stage_summary(stages: pd.DataFrame) -> pd.DataFrame:
    variables = [
        "Detected_onset_cycle",
        "Detected_knee_cycle",
        "Detected_onset_fraction",
        "Detected_knee_fraction",
        "SOH_at_onset",
        "SOH_at_knee",
        "Stage1_slope_pp_per_100_cycles",
        "Stage2_slope_pp_per_100_cycles",
        "Stage3_slope_pp_per_100_cycles",
        "Onset_absolute_error",
        "Knee_absolute_error",
    ]
    records = []
    for variable in variables:
        series = stages[variable]
        records.append(
            {
                "Metric": variable,
                "Mean": series.mean(),
                "Median": series.median(),
                "Q1": series.quantile(0.25),
                "Q3": series.quantile(0.75),
                "SD": series.std(ddof=1),
            }
        )
    return pd.DataFrame(records)


def analyze_problem1(raw_path: Path, battery_summary_path: Path) -> dict[str, object]:
    start = time.perf_counter()
    cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
    seed = int(cfg["random_seed"])
    cycle_df = pd.read_csv(raw_path)
    battery_df = pd.read_csv(battery_summary_path)

    quality_table, quality_metrics = data_quality_summary(cycle_df, battery_df)
    stages, normalized = detect_lifecycle_stages(cycle_df, battery_df)
    stage_summary = _stage_summary(stages)
    row_sample = _make_row_weighted_sample(cycle_df, int(cfg["analysis_cycle_stride"]))
    sample = _make_battery_balanced_sample(cycle_df, landmarks_per_battery=20)

    statistical, coefficients, stat_metrics, stat_bootstrap = statistical_factor_importance(
        sample,
        float(cfg["max_cycles"]),
        int(cfg["bootstrap_repetitions"]),
        seed,
    )
    permutation_global, global_metrics, _, _ = permutation_factor_importance(
        sample,
        int(cfg["permutation_repetitions"]),
        seed,
        method="global",
    )
    permutation, ml_metrics, perm_bootstrap, model = permutation_factor_importance(
        sample,
        int(cfg["permutation_repetitions"]),
        seed,
```

```
        method="group-aware",
    )
    consensus, consensus_draws = consensus_factor_weights(
        statistical,
        permutation,
        stat_bootstrap,
        perm_bootstrap,
        seed,
    )

    row_statistical, _, _, row_stat_bootstrap = statistical_factor_importance(
        row_sample,
        float(cfg["max_cycles"]),
        int(cfg["bootstrap_repetitions"]),
        seed + 11,
    )
    row_permutation, _, row_perm_bootstrap, _ = permutation_factor_importance(
        row_sample,
        int(cfg["permutation_repetitions"]),
        seed + 11,
        method="group-aware",
    )
    row_consensus, _ = consensus_factor_weights(
        row_statistical,
        row_permutation,
        row_stat_bootstrap,
        row_perm_bootstrap,
        seed + 11,
    )
    weighting_sensitivity = _weighting_sensitivity(row_consensus, consensus)
    permutation_sensitivity = _permutation_sensitivity(permutation_global, permutation)
    operational = _operational_factor_weights(consensus, consensus_draws)
    censoring_audit, censoring_detail = _censoring_stage_audit(stages, battery_df)
    counterfactual, counterfactual_paired = run_counterfactual_experiment(
        cfg,
        n_batteries=200,
        seed=seed + 505,
    )

    quality_table.to_csv(RESULT_TABLES / "data_quality_summary.csv", index=False)
    stages.to_csv(RESULT_TABLES / "stage_boundaries_by_battery.csv", index=False, float_format="%.6f")
    stage_summary.to_csv(RESULT_TABLES / "stage_summary.csv", index=False, float_format="%.6f")
    statistical.to_csv(RESULT_TABLES / "factor_importance_statistical.csv", index=False, float_format="%.8f")
    coefficients.to_csv(RESULT_TABLES / "cluster_robust_coefficients.csv", index=False, float_format="%.8g")
    permutation.to_csv(RESULT_TABLES / "factor_importance_permutation.csv", index=False, float_format="%.8f")
    consensus.to_csv(RESULT_TABLES / "factor_consensus_weights.csv", index=False, float_format="%.8f")
    weighting_sensitivity.to_csv(
        RESULT_TABLES / "q1_weighting_sensitivity.csv", index=False, float_format="%.8f"
    )
    operational.to_csv(
        RESULT_TABLES / "q1_operational_factor_weights.csv", index=False, float_format="%.8f"
    )
    permutation_sensitivity.to_csv(
        RESULT_TABLES / "q1_permutation_sensitivity.csv", index=False, float_format="%.8f"
    )
    censoring_audit.to_csv(
        RESULT_TABLES / "q1_censoring_stage_audit.csv", index=False, float_format="%.8f"
    )
    censoring_detail.to_csv(
        RESULT_TABLES / "q1_censoring_stage_detail.csv", index=False, float_format="%.8f"
    )
    counterfactual.to_csv(
        RESULT_TABLES / "q1_counterfactual_lifetime_effects.csv", index=False, float_format="%.8f"
    )
    counterfactual_paired.to_csv(
        RESULT_TABLES / "q1_counterfactual_paired_batteries.csv", index=False, float_format="%.8f"
    )
    sample.to_csv(DATA_PROCESSED / "q1_analysis_sample.csv", index=False, float_format="%.6f")
    row_sample.to_csv(DATA_PROCESSED / "q1_row_weighted_sample.csv", index=False, float_format="%.6f")
    normalized.to_csv(DATA_PROCESSED / "normalized_lifecycle.csv", index=False, float_format="%.6f")

    import joblib

    joblib.dump(model, RESULT_MODELS / "q1_random_forest_pipeline.joblib")

    top_controllable = operational.iloc[0]
    first_tier = operational.loc[operational["First_tier"], "Factor"].tolist()
```

```
    stage_values = stage_summary.set_index("Metric")
    censor_values = censoring_audit.set_index("Group")
    cf_index = counterfactual.set_index(["Factor", "Level"])

    def cf_loss(factor: str, level: str) -> float:
        return float(cf_index.loc[(factor, level), "Median_relative_lifetime_loss_percent"])

    metrics: dict[str, object] = {
        **quality_metrics,
        **stat_metrics,
        **ml_metrics,
        "analysis_rows": int(len(sample)),
        "median_detected_onset_cycle": float(stage_values.loc["Detected_onset_cycle", "Median"]),
        "median_detected_knee_cycle": float(stage_values.loc["Detected_knee_cycle", "Median"]),
        "median_onset_fraction": float(stage_values.loc["Detected_onset_fraction", "Median"]),
        "median_knee_fraction": float(stage_values.loc["Detected_knee_fraction", "Median"]),
        "median_stage1_slope_pp_per_100_cycles": float(
            stage_values.loc["Stage1_slope_pp_per_100_cycles", "Median"]
        ),
        "median_stage2_slope_pp_per_100_cycles": float(
            stage_values.loc["Stage2_slope_pp_per_100_cycles", "Median"]
        ),
        "median_stage3_slope_pp_per_100_cycles": float(
            stage_values.loc["Stage3_slope_pp_per_100_cycles", "Median"]
        ),
        "onset_detection_MAE_cycles": float(stages["Onset_absolute_error"].mean()),
        "knee_detection_MAE_cycles": float(stages["Knee_absolute_error"].mean()),
        "monotone_acceleration_fit_rate": float(stages["Monotone_acceleration_constraint_used"].mean()),
        "top_overall_factor": str(consensus.iloc[0]["Factor"]),
        "top_overall_consensus_weight": float(consensus.iloc[0]["Consensus_weight"]),
        "top_controllable_factor": str(top_controllable["Factor"]),
        "top_controllable_consensus_weight": float(top_controllable["Operational_weight"]),
        "top_controllable_rank_stability": float(top_controllable["Top_rank_probability"]),
        "operational_first_tier": first_tier,
        "eol_only_onset_detection_MAE_cycles": float(
            censor_values.loc["Reached-EOL batteries", "Onset_MAE_cycles"]
        ),
        "eol_only_knee_detection_MAE_cycles": float(
            censor_values.loc["Reached-EOL batteries", "Knee_MAE_cycles"]
        ),
        "censored_onset_detection_MAE_cycles": float(
            censor_values.loc["Right-censored batteries", "Onset_MAE_cycles"]
        ),
        "censored_knee_detection_MAE_cycles": float(
            censor_values.loc["Right-censored batteries", "Knee_MAE_cycles"]
        ),
        "censored_post_knee_sufficiency_rate": float(
            censor_values.loc["Right-censored batteries", "Post_knee_sufficiency_rate"]
        ),
        "counterfactual_matched_batteries": 200,
        "counterfactual_45C_lifetime_loss_percent": cf_loss("Temperature", "45.0"),
        "counterfactual_2C_charge_lifetime_loss_percent": cf_loss("Charge_C_rate", "2.0"),
        "counterfactual_2C_discharge_lifetime_loss_percent": cf_loss("Discharge_C_rate", "2.0"),
        "counterfactual_100DOD_lifetime_loss_percent": cf_loss("DOD", "100.0"),
        "counterfactual_fast_charge_lifetime_loss_percent": cf_loss(
            "Charge_strategy", "Fast-charge"
        ),
        "max_abs_weighting_difference": float(weighting_sensitivity["Difference"].abs().max()),
        "max_abs_weighting_rank_change": int(weighting_sensitivity["Rank_change"].abs().max()),
        "max_abs_permutation_difference": float(permutation_sensitivity["Difference"].abs().max()),
        "max_abs_permutation_rank_change": int(permutation_sensitivity["Rank_change"].abs().max()),
        "global_permutation_test_R2": float(global_metrics["ml_test_R2"]),
        "runtime_seconds_before_plotting": float(time.perf_counter() - start),
        "leakage_control": (
            "Battery_ID grouped holdout; battery-balanced landmarks; only exogenous operating/design variables used; "
            "preprocessing fit on training batteries."
        ),
    }
    save_json(metrics, RESULT_TABLES / "q1_key_metrics.json")

    model_performance = pd.DataFrame(
        [
            {"Model": "Cluster-robust nonlinear-basis OLS", "Metric": "R2", "Value": stat_metrics["statistical_full_R2"]},
            {"Model": "Battery-held-out Random Forest", "Metric": "Test R2", "Value": ml_metrics["ml_test_R2"]},
            {
                "Model": "Battery-held-out Random Forest",
                "Metric": "Test MAE (SOH pp/cycle)",
```

```
                "Value": ml_metrics["ml_test_MAE_pp_per_cycle"],
            },
        ]
    )
    model_performance.to_csv(RESULT_TABLES / "model_performance.csv", index=False)
    print(
        f"[Q1] onset={metrics['median_detected_onset_cycle']:.0f} cycles, "
        f"knee={metrics['median_detected_knee_cycle']:.0f} cycles, "
        f"RF test R2={metrics['ml_test_R2']:.3f}, top controllable={metrics['top_controllable_factor']}"
    )
    return {
        "cycle_df": cycle_df,
        "battery_df": battery_df,
        "stages": stages,
        "normalized": normalized,
        "statistical": statistical,
        "permutation": permutation,
        "consensus": consensus,
        "operational": operational,
        "weighting_sensitivity": weighting_sensitivity,
        "permutation_sensitivity": permutation_sensitivity,
        "counterfactual": counterfactual,
        "censoring_audit": censoring_audit,
        "metrics": metrics,
    }
```

src/problem1/counterfactual.py

```
""" 对应论文 §5.5: 在半经验仿真体系内进行配对反事实寿命实验。"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd


REFERENCE_SCENARIO = {
    "Temperature": 25.0,
    "Charge_C_rate": 1.0,
    "Discharge_C_rate": 1.0,
    "DOD": 80.0,
    "Charge_strategy": "CC-CV",
}

FACTOR_LEVELS = {
    "Temperature": [10.0, 15.0, 25.0, 35.0, 45.0],
    "Charge_C_rate": [0.5, 1.0, 1.5, 2.0],
    "Discharge_C_rate": [0.5, 1.0, 1.5, 2.0],
    "DOD": [50.0, 60.0, 70.0, 80.0, 90.0, 100.0],
    "Charge_strategy": ["CC-CV", "Multi-stage", "Fast-charge"],
}

STRATEGY_MULTIPLIER = {"CC-CV": 1.0, "Multi-stage": 0.92, "Fast-charge": 1.14}


@dataclass(frozen=True)
class MatchedBattery:
    battery_index: int
    batch_index: int
    fade_heterogeneity: float
    knee_heterogeneity: float
    temperature_noise: np.ndarray
    charge_noise: np.ndarray
    discharge_noise: np.ndarray
    dod_noise: np.ndarray


def _stress(
    temperature: np.ndarray,
    charge_rate: np.ndarray,
    discharge_rate: np.ndarray,
    dod: np.ndarray,
    strategy: str,
```

```
) -> np.ndarray:
    high_temperature = np.exp(0.026 * np.maximum(temperature - 25.0, 0.0))
    low_temperature = 1.0 + 0.010 * np.maximum(15.0 - temperature, 0.0) ** 1.25
    charge_stress = 1.0 + 0.34 * (charge_rate - 1.0) + 0.11 * (charge_rate - 1.0) ** 2
    discharge_stress = 1.0 + 0.18 * (discharge_rate - 1.0) + 0.06 * (discharge_rate - 1.0) ** 2
    dod_stress = (dod / 0.80) ** 1.25
    interaction = (
        1.0
        + 0.08 * np.maximum(charge_rate - 1.0, 0.0) * np.maximum(dod - 0.75, 0.0) / 0.25
        + 0.05
        * np.maximum(temperature - 30.0, 0.0)
        / 15.0
        * np.maximum(charge_rate - 1.0, 0.0)
    )
    result = np.clip(
        high_temperature
        * low_temperature
        * charge_stress
        * discharge_stress
        * dod_stress
        * STRATEGY_MULTIPLIER[strategy]
        * interaction,
        0.42,
        3.40,
    )
    result[0] = 0.0
    return result


def _simulate_lifetime(
    battery: MatchedBattery,
    scenario: dict[str, float | str],
    cfg: dict[str, object],
) -> dict[str, float]:
    max_cycles = int(cfg["max_cycles"])
    cycle = np.arange(max_cycles + 1, dtype=float)
    seasonal = np.sin(2.0 * np.pi * cycle / 220.0 + 0.17 * battery.battery_index)

    temperature = np.clip(
        float(scenario["Temperature"]) + 1.25 * seasonal + battery.temperature_noise,
        5.0,
        45.0,
    )
    charge_rate = np.clip(
        float(scenario["Charge_C_rate"]) + battery.charge_noise,
        0.45,
        2.10,
    )
    discharge_rate = np.clip(
        float(scenario["Discharge_C_rate"]) + battery.discharge_noise,
        0.45,
        2.10,
    )
    dod = np.clip(
        float(scenario["DOD"]) / 100.0 + 0.015 * seasonal + battery.dod_noise,
        0.48,
        1.00,
    )
    stress = _stress(temperature, charge_rate, discharge_rate, dod, str(scenario["Charge_strategy"]))
    equivalent_cycles = np.cumsum(stress)

    knee_exposure = float(cfg["base_knee_cycle"]) * battery.knee_heterogeneity
    onset_exposure = 0.47 * knee_exposure
    base_x = equivalent_cycles / float(cfg["reference_cycles"])
    mid_x = np.maximum(equivalent_cycles - onset_exposure, 0.0) / 500.0
    knee_x = np.maximum(equivalent_cycles - knee_exposure, 0.0) / 400.0
    loss = (
        float(cfg["base_fade_coefficient"])
        * battery.fade_heterogeneity
        * base_x ** float(cfg["base_fade_exponent"])
        + float(cfg["mid_acceleration_coefficient"])
        * battery.fade_heterogeneity
        * mid_x ** float(cfg["mid_acceleration_exponent"])
        + float(cfg["knee_acceleration_coefficient"])
        * battery.fade_heterogeneity
        * knee_x ** float(cfg["knee_acceleration_exponent"])
    )
```

```
    soh = 100.0 * (1.0 - loss)
    hits = np.flatnonzero(soh <= float(cfg["eol_soh_percent"]))
    eol_cycle = int(hits[0]) if hits.size else max_cycles
    knee_hits = np.flatnonzero(equivalent_cycles >= knee_exposure)
    knee_cycle = int(knee_hits[0]) if knee_hits.size else max_cycles
    mean_rate = (100.0 - float(soh[eol_cycle])) / max(eol_cycle, 1)
    return {
        "EOL_cycle": float(eol_cycle),
        "Knee_cycle": float(knee_cycle),
        "Mean_degradation_rate": float(mean_rate),
        "Reached_EOL": float(bool(hits.size)),
    }


def run_counterfactual_experiment(
    cfg: dict[str, object],
    n_batteries: int = 200,
    seed: int | None = None,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """ 返回汇总表与配对电池级结果；仅改变一个因素，其他随机结构完全配对。"""
    seed = int(cfg["random_seed"]) + 505 if seed is None else int(seed)
    rng = np.random.default_rng(seed)
    n_batches = int(cfg["n_batches"])
    max_cycles = int(cfg["max_cycles"])
    batch_fade = rng.lognormal(mean=0.0, sigma=0.075, size=n_batches)
    batteries: list[MatchedBattery] = []
    for index in range(n_batteries):
        batch_index = index % n_batches
        batteries.append(
            MatchedBattery(
                battery_index=index,
                batch_index=batch_index,
                fade_heterogeneity=float(batch_fade[batch_index] * rng.lognormal(0.0, 0.09)),
                knee_heterogeneity=float(rng.lognormal(0.0, 0.085)),
                temperature_noise=rng.normal(0.0, 0.60, max_cycles + 1),
                charge_noise=rng.normal(0.0, 0.035, max_cycles + 1),
                discharge_noise=rng.normal(0.0, 0.040, max_cycles + 1),
                dod_noise=rng.normal(0.0, 0.012, max_cycles + 1),
            )
        )

    reference = [_simulate_lifetime(battery, REFERENCE_SCENARIO, cfg) for battery in batteries]
    paired_rows: list[dict[str, object]] = []
    for factor, levels in FACTOR_LEVELS.items():
        for level in levels:
            scenario = dict(REFERENCE_SCENARIO)
            scenario[factor] = level
            for battery, ref in zip(batteries, reference):
                outcome = _simulate_lifetime(battery, scenario, cfg)
                delta_eol = outcome["EOL_cycle"] - ref["EOL_cycle"]
                row: dict[str, object] = {
                    "Battery_index": battery.battery_index,
                    "Batch_index": battery.batch_index,
                    "Factor": factor,
                    "Level": str(level),
                    "EOL_cycle": outcome["EOL_cycle"],
                    "Reached_EOL": outcome["Reached_EOL"],
                    "Reference_EOL_cycle": ref["EOL_cycle"],
                    "Delta_EOL_cycles": delta_eol,
                    "Lifetime_loss_percent": -100.0 * delta_eol / max(ref["EOL_cycle"], 1.0),
                    "Mean_degradation_rate": outcome["Mean_degradation_rate"],
                    "Knee_cycle": outcome["Knee_cycle"],
                }
                for landmark in (200, 400, 600, 800):
                    row[f"RUL_change_at_{landmark}"] = (
                        max(outcome["EOL_cycle"] - landmark, 0.0)
                        - max(ref["EOL_cycle"] - landmark, 0.0)
                    )
                paired_rows.append(row)

    paired = pd.DataFrame(paired_rows)
    summary_rows: list[dict[str, object]] = []
    for (factor, level), group in paired.groupby(["Factor", "Level"], sort=False):
        delta = group["Delta_EOL_cycles"]
        loss = group["Lifetime_loss_percent"]
        record: dict[str, object] = {
            "Factor": factor,
```

```
            "Level": level,
            "Reference_level": str(REFERENCE_SCENARIO[factor]),
            "N_matched_batteries": int(len(group)),
            "Reached_EOL_rate": float(group["Reached_EOL"].mean()),
            "Median_EOL_cycle": float(group["EOL_cycle"].median()),
            "Median_delta_EOL_cycles": float(delta.median()),
            "Delta_EOL_IQR_low": float(delta.quantile(0.25)),
            "Delta_EOL_IQR_high": float(delta.quantile(0.75)),
            "Delta_EOL_95_low": float(delta.quantile(0.025)),
            "Delta_EOL_95_high": float(delta.quantile(0.975)),
            "Median_relative_lifetime_loss_percent": float(loss.median()),
            "Median_mean_degradation_rate": float(group["Mean_degradation_rate"].median()),
            "Median_knee_cycle": float(group["Knee_cycle"].median()),
        }
        for landmark in (200, 400, 600, 800):
            record[f"Median_RUL_change_at_{landmark}"] = float(
                group[f"RUL_change_at_{landmark}"].median()
            )
        summary_rows.append(record)
    return pd.DataFrame(summary_rows), paired
```

src/problem1/plots.py

```
""" 对应论文 §5.2-§5.4：生成问题一的可复核、色盲友好图表。"""

from __future__ import annotations

import shutil
from pathlib import Path

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from src.utils.project import PAPER_FIGURES, PAPER_TABLES, RESULT_FIGURES, RESULT_TABLES


OKABE_ITO = ["#0072B2", "#E69F00", "#009E73", "#CC79A7", "#56B4E9", "#D55E00", "#000000"]


def configure_style() -> None:
    mpl.rcParams.update(
        {
            "font.family": "sans-serif",
            "font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
            "font.size": 8.0,
            "axes.labelsize": 8.5,
            "axes.titlesize": 9.5,
            "legend.fontsize": 7.5,
            "xtick.labelsize": 7.5,
            "ytick.labelsize": 7.5,
            "axes.spines.top": False,
            "axes.spines.right": False,
            "axes.grid": True,
            "grid.alpha": 0.22,
            "grid.linewidth": 0.5,
            "lines.linewidth": 1.5,
            "pdf.fonttype": 42,
            "ps.fonttype": 42,
            "svg.fonttype": "none",
            "axes.unicode_minus": False,
        }
    )


def _save(fig: plt.Figure, name: str) -> list[str]:
    RESULT_FIGURES.mkdir(parents=True, exist_ok=True)
    PAPER_FIGURES.mkdir(parents=True, exist_ok=True)
    paths = []
    for suffix in ("png", "pdf", "svg"):
        path = RESULT_FIGURES / f"{name}.{suffix}"
        kwargs = {"dpi": 300} if suffix == "png" else {}
        fig.savefig(path, facecolor="white", **kwargs)
        paths.append(str(path))
```

```
        if suffix in {"png", "pdf"}:
            shutil.copy2(path, PAPER_FIGURES / path.name)
    plt.close(fig)
    return paths


def plot_lifecycle_evolution(
    cycle_df: pd.DataFrame,
    battery_df: pd.DataFrame,
    stages: pd.DataFrame,
    normalized: pd.DataFrame,
) -> list[str]:
    summary = battery_df.copy()
    summary["Stress_quartile"] = pd.qcut(
        summary["Mean_stress_index"],
        q=4,
        labels=["Q1 低应力", "Q2", "Q3", "Q4 高应力"],
    )
    selected_ids = []
    for _, group in summary.groupby("Stress_quartile", observed=True):
        chosen = group.sort_values("Mean_stress_index").iloc[
            np.linspace(0, len(group) - 1, 4).round().astype(int)
        ]
        selected_ids.extend(chosen["Battery_ID"].tolist())
    selected = cycle_df.loc[cycle_df["Battery_ID"].isin(selected_ids)].merge(
        summary[["Battery_ID", "Stress_quartile"]], on="Battery_ID", how="left"
    )

    fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.15), constrained_layout=True)
    quartiles = ["Q1 低应力", "Q2", "Q3", "Q4 高应力"]
    line_styles = ["-", "--", "-.", ":"]
    for index, quartile in enumerate(quartiles):
        subset = selected.loc[selected["Stress_quartile"] == quartile]
        for line_index, (_, group) in enumerate(subset.groupby("Battery_ID")):
            axes[0].plot(
                group["Cycle"],
                group["SOH"],
                color=OKABE_ITO[index],
                linestyle=line_styles[index],
                alpha=0.75,
                linewidth=1.0,
                label=quartile if line_index == 0 else None,
            )
    axes[0].axhline(80.0, color="0.25", linestyle="--", linewidth=1.0, label="EOL 阈值")
    axes[0].set(xlabel=" 循环次数", ylabel="SOH（%）", title=" 代表性全寿命退化轨迹")
    axes[0].set_ylim(78.0, 101.2)
    axes[0].legend(frameon=False, ncol=2, loc="lower left")

    fleet = normalized.groupby("Life_fraction")["SOH"].agg(
        median="median",
        q1=lambda x: x.quantile(0.25),
        q3=lambda x: x.quantile(0.75),
    )
    axes[1].fill_between(
        fleet.index,
        fleet["q1"],
        fleet["q3"],
        color=OKABE_ITO[0],
        alpha=0.18,
        label=" 四分位区间（n=360）",
    )
    axes[1].plot(fleet.index, fleet["median"], color=OKABE_ITO[0], linewidth=2.0, label="SOH 中位数")
    onset = stages["Detected_onset_fraction"].median()
    knee = stages["Detected_knee_fraction"].median()
    axes[1].axvline(onset, color=OKABE_ITO[2], linestyle="--", label=" 加速起始点中位数")
    axes[1].axvline(knee, color=OKABE_ITO[5], linestyle="-.", label=" 膝点中位数")
    axes[1].text(onset / 2, 99.0, " 阶段 I", ha="center", va="center")
    axes[1].text((onset + knee) / 2, 99.0, " 阶段 II", ha="center", va="center")
    axes[1].text((knee + 1) / 2, 99.0, " 阶段 III", ha="center", va="center")
    axes[1].set(
        xlabel=" 观测寿命归一化位置",
        ylabel="SOH（%）",
        title=" 总体退化轨迹与阶段边界",
        xlim=(0, 1),
        ylim=(78.0, 101.2),
    )
    axes[1].legend(frameon=False, loc="lower left")
```

```
    for label, axis in zip(("(a)", "(b)"), axes):
        axis.text(-0.14, 1.03, label, transform=axis.transAxes, fontweight="bold", fontsize=10)
    return _save(fig, "q1_lifecycle_evolution")


def plot_capacity_resistance(cycle_df: pd.DataFrame) -> list[str]:
    stride = max(1, len(cycle_df) // 24000)
    sample = cycle_df.iloc[::stride].copy()
    rho = sample[["SOH", "Internal_resistance"]].corr(method="spearman").iloc[0, 1]

    fig, axis = plt.subplots(figsize=(4.6, 3.35), constrained_layout=True)
    points = axis.scatter(
        sample["SOH"],
        sample["Internal_resistance"],
        c=sample["Cycle"],
        cmap="viridis",
        s=5,
        alpha=0.35,
        linewidths=0,
        rasterized=True,
    )
    colorbar = fig.colorbar(points, ax=axis, pad=0.02)
    colorbar.set_label("Cycle")
    axis.text(
        0.04,
        0.95,
        f"Spearman rho = {rho:.3f}\nshown n = {len(sample):,} rows",
        transform=axis.transAxes,
        va="top",
        bbox={"facecolor": "white", "edgecolor": "none", "alpha": 0.82},
    )
    axis.set(
        xlabel="SOH (%)",
        ylabel="Internal resistance (mOhm)",
        title="Capacity fade is coupled with resistance growth",
    )
    return _save(fig, "q1_capacity_resistance_relation")


def plot_stage_validation(stages: pd.DataFrame) -> list[str]:
    fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.05), constrained_layout=True, sharex=True, sharey=True)
    panels = [
        ("True_onset_cycle", "Detected_onset_cycle", " 加速起始点", OKABE_ITO[2], "o"),
        ("True_knee_cycle", "Detected_knee_cycle", " 阶段膝点", OKABE_ITO[5], "s"),
    ]
    upper = float(
        max(stages[["True_onset_cycle", "Detected_onset_cycle", "True_knee_cycle", "Detected_knee_cycle"]].max())
    )
    upper = np.ceil(upper / 100.0) * 100.0
    for label, axis, panel in zip(("(a)", "(b)"), axes, panels):
        true_col, detected_col, title, color, marker = panel
        axis.scatter(
            stages[true_col],
            stages[detected_col],
            s=14,
            marker=marker,
            facecolors="none",
            edgecolors=color,
            linewidths=0.65,
            alpha=0.65,
        )
        axis.plot([0, upper], [0, upper], color="0.25", linestyle="--", linewidth=1.0)
        mae = np.mean(np.abs(stages[true_col] - stages[detected_col]))
        axis.text(0.05, 0.93, f"MAE = {mae:.1f} 次\nn = {len(stages)}", transform=axis.transAxes, va="top")
        axis.set(
            xlabel=" 仿真参考边界（次）",
            ylabel=" 检测边界（次）",
            title=title,
            xlim=(0, upper),
            ylim=(0, upper),
        )
        axis.set_aspect("equal", adjustable="box")
        axis.text(-0.14, 1.03, label, transform=axis.transAxes, fontweight="bold", fontsize=10)
    return _save(fig, "q1_stage_detection_validation")


def _importance_labels() -> dict[str, str]:
```

```
    return {
        "Cycle": " 循环位置",
        "Temperature": " 温度",
        "Charge_C_rate": " 充电倍率",
        "Discharge_C_rate": " 放电倍率",
        "DOD": "DOD",
        "Charge_strategy": " 充电策略",
        "Batch_ID": " 批次",
    }


def plot_factor_methods(statistical: pd.DataFrame, permutation: pd.DataFrame) -> list[str]:
    labels = _importance_labels()
    ordering = statistical.sort_values("Delta_R2")["Factor"].tolist()
    y = np.arange(len(ordering))
    stat = statistical.set_index("Factor").loc[ordering]
    perm = permutation.set_index("Factor").loc[ordering]

    fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.35), constrained_layout=True)
    stat_low = np.maximum(stat["Delta_R2"] - stat["Bootstrap_CI_low"], 0.0)
    stat_high = np.maximum(stat["Bootstrap_CI_high"] - stat["Delta_R2"], 0.0)
    axes[0].errorbar(
        stat["Delta_R2"],
        y,
        xerr=np.vstack([stat_low, stat_high]),
        fmt="o",
        color=OKABE_ITO[0],
        ecolor=OKABE_ITO[0],
        capsize=2,
    )
    axes[0].set_yticks(y, [labels[item] for item in ordering])
    axes[0].set(xlabel="Drop-group delta R-squared", title="Cluster-aware statistical contribution")

    perm_low = np.maximum(perm["Permutation_R2_decrease"] - perm["Bootstrap_CI_low"], 0.0)
    perm_high = np.maximum(perm["Bootstrap_CI_high"] - perm["Permutation_R2_decrease"], 0.0)
    axes[1].errorbar(
        perm["Permutation_R2_decrease"],
        y,
        xerr=np.vstack([perm_low, perm_high]),
        fmt="s",
        color=OKABE_ITO[5],
        ecolor=OKABE_ITO[5],
        capsize=2,
    )
    axes[1].set_yticks(y, [labels[item] for item in ordering])
    axes[1].set(xlabel="Held-out permutation R-squared decrease", title="Nonlinear predictive contribution")
    for label, axis in zip(("a", "b"), axes):
        axis.axvline(0.0, color="0.4", linewidth=0.7)
        axis.text(-0.14, 1.03, label, transform=axis.transAxes, fontweight="bold", fontsize=10)
    return _save(fig, "q1_factor_importance_methods")


def plot_consensus(consensus: pd.DataFrame) -> list[str]:
    labels = _importance_labels()
    ordered = consensus.sort_values("Consensus_weight", ascending=True).reset_index(drop=True)
    y = np.arange(len(ordered))
    values = ordered["Consensus_weight"].to_numpy()
    low = np.maximum(values - ordered["Consensus_CI_low"].to_numpy(), 0.0)
    high = np.maximum(ordered["Consensus_CI_high"].to_numpy() - values, 0.0)

    fig, axis = plt.subplots(figsize=(5.4, 3.55), constrained_layout=True)
    colors = [OKABE_ITO[0] if factor == "Cycle" else OKABE_ITO[2] for factor in ordered["Factor"]]
    axis.barh(y, values, color=colors, alpha=0.82, height=0.62)
    axis.errorbar(values, y, xerr=np.vstack([low, high]), fmt="none", ecolor="0.15", capsize=2, linewidth=0.8)
    axis.set_yticks(y, [labels[item] for item in ordered["Factor"]])
    axis.set(xlabel="Consensus weight (sum = 1)", title="Normalized consensus of two complementary methods")
    axis.set_xlim(0.0, max(ordered["Consensus_CI_high"].max() * 1.25, 0.20))
    for index, row in ordered.iterrows():
        if row["Factor"] == "Cycle":
            stability_text = f"overall top {100 * row['Top_rank_stability']:.0f}%"
        else:
            stability_text = f"controllable top {100 * row['Top_controllable_rank_stability']:.0f}%"
        axis.text(
            row["Consensus_CI_high"] + 0.005,
            index,
            stability_text,
            va="center",
```

```
            fontsize=7,
        )
    return _save(fig, "q1_consensus_factor_weights")


def plot_operational_weights(operational: pd.DataFrame) -> list[str]:
    labels = _importance_labels()
    ordered = operational.sort_values("Operational_weight").reset_index(drop=True)
    y = np.arange(len(ordered))
    values = ordered["Operational_weight"].to_numpy()
    low = np.maximum(values - ordered["Bootstrap_CI_low"].to_numpy(), 0.0)
    high = np.maximum(ordered["Bootstrap_CI_high"].to_numpy() - values, 0.0)

    fig, axis = plt.subplots(figsize=(5.4, 3.35), constrained_layout=True)
    colors = [OKABE_ITO[2] if flag else "0.55" for flag in ordered["First_tier"]]
    for index, (value, color, probability) in enumerate(
        zip(values, colors, ordered["Top_rank_probability"])
    ):
        axis.errorbar(
            value,
            index,
            xerr=np.array([[low[index]], [high[index]]]),
            fmt="none",
            ecolor=color,
            capsize=2.5,
            linewidth=1.2,
        )
        axis.scatter(value, index, s=32, color=color, marker="o" if color == OKABE_ITO[2] else "s")
        axis.text(
            ordered.loc[index, "Bootstrap_CI_high"] + 0.008,
            index,
            f"top {100 * probability:.0f}%",
            va="center",
            fontsize=7,
        )
    axis.set_yticks(y, [labels[item] for item in ordered["Factor"]])
    axis.set(
        xlabel="Operational influence weight (renormalized to 1)",
        title="Controllable operating factors and rank uncertainty",
        xlim=(0.0, max(0.45, float(ordered["Bootstrap_CI_high"].max()) * 1.35)),
    )
    return _save(fig, "q1_operational_factor_weights")


def plot_counterfactual_effects(
    counterfactual: pd.DataFrame,
    consensus: pd.DataFrame,
) -> list[str]:
    factor_order = [
        "Temperature",
        "Charge_C_rate",
        "Discharge_C_rate",
        "DOD",
        "Charge_strategy",
    ]
    titles = {
        "Temperature": " 温度（°C）",
        "Charge_C_rate": " 充电倍率",
        "Discharge_C_rate": " 放电倍率",
        "DOD": "DOD（%）",
        "Charge_strategy": " 充电策略",
    }
    fig, axes = plt.subplots(2, 3, figsize=(7.2, 5.05), constrained_layout=True)
    flat_axes = axes.ravel()
    shared_low = float(counterfactual["Delta_EOL_95_low"].min())
    shared_high = float(counterfactual["Delta_EOL_95_high"].max())
    shared_margin = 0.06 * max(shared_high - shared_low, 1.0)
    for panel_index, factor in enumerate(factor_order):
        axis = flat_axes[panel_index]
        subset = counterfactual.loc[counterfactual["Factor"] == factor].copy()
        if factor == "Charge_strategy":
            levels = ["CC-CV", "Multi-stage", "Fast-charge"]
            subset["order"] = subset["Level"].map({value: i for i, value in enumerate(levels)})
            subset = subset.sort_values("order")
            x = np.arange(len(subset))
            axis.set_xticks(x, ["CC-CV", " 多阶段", " 快速充电"])
            marker = "s"
```

```
            line_style = "none"
        else:
            subset["numeric_level"] = pd.to_numeric(subset["Level"])
            subset = subset.sort_values("numeric_level")
            x = subset["numeric_level"].to_numpy()
            marker = "o"
            line_style = "-"
        y = subset["Median_delta_EOL_cycles"].to_numpy()
        y_low = y - subset["Delta_EOL_95_low"].to_numpy()
        y_high = subset["Delta_EOL_95_high"].to_numpy() - y
        axis.errorbar(
            x,
            y,
            yerr=np.vstack([np.maximum(y_low, 0.0), np.maximum(y_high, 0.0)]),
            color=OKABE_ITO[0],
            marker=marker,
            linestyle=line_style,
            capsize=2,
            linewidth=1.2,
        )
        axis.axhline(0.0, color="0.35", linestyle="--", linewidth=0.8)
        axis.set_ylim(shared_low - shared_margin, shared_high + shared_margin)
        axis.set_title(titles[factor])
        axis.set_xlabel(" 策略" if factor == "Charge_strategy" else " 水平")
        if panel_index % 3 == 0:
            axis.set_ylabel(" 配对 EOL 循环变化（次）")
        else:
            axis.tick_params(labelleft=False)
        axis.text(-0.18, 1.03, f"({chr(ord('a') + panel_index)})", transform=axis.transAxes, fontweight="bold")
    weight_axis = flat_axes[-1]
    ordered = consensus.sort_values("Consensus_weight", ascending=True).reset_index(drop=True)
    weight_y = np.arange(len(ordered))
    weight_values = ordered["Consensus_weight"].to_numpy()
    weight_low = np.maximum(weight_values - ordered["Consensus_CI_low"].to_numpy(), 0.0)
    weight_high = np.maximum(ordered["Consensus_CI_high"].to_numpy() - weight_values, 0.0)
    weight_colors = [OKABE_ITO[0] if factor == "Cycle" else OKABE_ITO[2] for factor in ordered["Factor"]]
    weight_axis.barh(weight_y, weight_values, color=weight_colors, alpha=0.82, height=0.58)
    weight_axis.errorbar(
        weight_values,
        weight_y,
        xerr=np.vstack([weight_low, weight_high]),
        fmt="none",
        ecolor="0.15",
        capsize=1.8,
        linewidth=0.75,
    )
    compact_weight_labels = {
        "Cycle": " 循环位置",
        "Temperature": " 温度",
        "Charge_C_rate": " 充电倍率",
        "Discharge_C_rate": " 放电倍率",
        "DOD": "DOD",
        "Charge_strategy": " 充电策略",
        "Batch_ID": " 批次",
    }
    weight_axis.set_yticks(
        weight_y,
        [compact_weight_labels[item] for item in ordered["Factor"]],
        fontsize=6.6,
    )
    weight_axis.set(
        xlabel=" 权重",
        title=" 共识权重\n（bootstrap 95% CI）",
        xlim=(0.0, max(0.55, float(ordered["Consensus_CI_high"].max()) * 1.12)),
    )
    weight_axis.text(-0.18, 1.03, "(f)", transform=weight_axis.transAxes, fontweight="bold")
    fig.suptitle(" 半经验仿真中的运行因素证据", fontsize=10)
    return _save(fig, "q1_counterfactual_factor_effects")


def create_problem1_figures(results: dict[str, object]) -> pd.DataFrame:
    configure_style()
    manifest_rows = []
    jobs = [
        (
            "q1_lifecycle_evolution",
            "Full-lifecycle SOH trajectories are nonlinear and separate into three fleet-level stages.",
```

```
            plot_lifecycle_evolution(
                results["cycle_df"], results["battery_df"], results["stages"], results["normalized"]
            ),
        ),
        (
            "q1_capacity_resistance_relation",
            "SOH decline co-evolves with internal-resistance growth in the simulated population.",
            plot_capacity_resistance(results["cycle_df"]),
        ),
        (
            "q1_stage_detection_validation",
            "Detected change points are checked against simulation-only hidden reference boundaries.",
            plot_stage_validation(results["stages"]),
        ),
        (
            "q1_factor_importance_methods",
            "Statistical drop-group and held-out permutation methods provide complementary factor evidence.",
            plot_factor_methods(results["statistical"], results["permutation"]),
        ),
        (
            "q1_consensus_factor_weights",
            "Final weights are a normalized consensus with bootstrap uncertainty and rank stability.",
            plot_consensus(results["consensus"]),
        ),
        (
            "q1_operational_factor_weights",
            "Cycle and batch are separated before controllable operating factors are renormalized.",
            plot_operational_weights(results["operational"]),
        ),
        (
            "q1_counterfactual_factor_effects",
            "Consensus importance and matched virtual batteries jointly quantify operating-factor evidence.",
            plot_counterfactual_effects(results["counterfactual"], results["consensus"]),
        ),
    ]
    for figure, claim, paths in jobs:
        manifest_rows.append({"Figure": figure, "Claim": claim, "Files": " | ".join(paths)})
    manifest = pd.DataFrame(manifest_rows)
    manifest.to_csv(RESULT_TABLES / "figure_manifest.csv", index=False)

    for table_name in (
        "stage_summary.csv",
        "factor_consensus_weights.csv",
        "model_performance.csv",
        "q1_operational_factor_weights.csv",
        "q1_counterfactual_lifetime_effects.csv",
        "q1_censoring_stage_audit.csv",
    ):
        shutil.copy2(RESULT_TABLES / table_name, PAPER_TABLES / table_name)
    return manifest
```

src/problem2/build_landmark_dataset.py

```
""" 对应论文 §6.1: 构造只使用预测时刻 t 及之前信息的绝对循环 landmark 数据集。"""

from __future__ import annotations

from pathlib import Path

import numpy as np
import pandas as pd

from src.utils.project import DATA_PROCESSED, RESULT_TABLES, save_json


LANDMARKS = (200, 400, 600, 800, 1000, 1200, 1400, 1600)
HORIZONS = (50, 100, 200)
FORBIDDEN_GENERATOR_FEATURES = {
    "EOL_cycle",
    "Reached_EOL",
    "True_onset_cycle",
    "True_knee_cycle",
    "Fade_heterogeneity",
    "Degradation_rate",
    "Stress_index",
```

```
    "Equivalent_stress_cycles",
    "Failure",
}

OPERATING_CURRENT = [
    "Temperature",
    "Charge_C_rate",
    "Discharge_C_rate",
    "DOD",
]
HEALTH_CURRENT = [
    "Internal_resistance",
    "Mean_voltage",
    "Voltage_dispersion",
    "End_of_discharge_voltage",
    "Coulombic_efficiency",
    "Energy_efficiency",
]


def _slope(frame: pd.DataFrame, column: str) -> float:
    if len(frame) < 2 or frame["Cycle"].nunique() < 2:
        return 0.0
    return float(np.polyfit(frame["Cycle"].to_numpy(dtype=float), frame[column].to_numpy(dtype=float), 1)[0])


def _lookup_at_cycle(indexed: pd.DataFrame, cycle: int, column: str) -> float:
    if cycle not in indexed.index:
        return float("nan")
    value = indexed.loc[cycle, column]
    if isinstance(value, pd.Series):
        value = value.iloc[0]
    return float(value)


def build_landmark_dataset(
    cycle_df: pd.DataFrame,
    battery_df: pd.DataFrame,
    landmarks: tuple[int, ...] = LANDMARKS,
    recent_window: int = 100,
) -> pd.DataFrame:
    """ 每个样本的任何导出特征都只读取 Cycle<=landmark 的历史。"""
    summary = battery_df.set_index("Battery_ID")
    rows: list[dict[str, object]] = []
    for battery_id, group in cycle_df.groupby("Battery_ID", sort=True):
        group = group.sort_values("Cycle")
        indexed = group.set_index("Cycle", drop=False)
        observed_end = int(group["Cycle"].max())
        battery_summary = summary.loc[battery_id]
        for landmark in landmarks:
            if landmark > observed_end or landmark not in indexed.index:
                continue
            history = group.loc[group["Cycle"] <= landmark]
            recent = history.loc[history["Cycle"] >= max(0, landmark - recent_window)]
            current = indexed.loc[landmark]
            if isinstance(current, pd.DataFrame):
                current = current.iloc[0]
            row: dict[str, object] = {
                "Battery_ID": battery_id,
                "Batch_ID": str(current["Batch_ID"]),
                "Landmark_cycle": int(landmark),
                "Charge_strategy": str(current["Charge_strategy"]),
                "SOH_t": float(current["SOH"]),
                "Capacity_t": float(current["Capacity"]),
                "Estimated_recent_degradation_slope": _slope(recent, "SOH"),
                "Observed_end_cycle": observed_end,
                "RUL_event": int(battery_summary["Reached_EOL"]),
                "RUL_duration": float(
                    (battery_summary["EOL_cycle"] if battery_summary["Reached_EOL"] == 1 else observed_end)
                    - landmark
                ),
                "True_RUL": (
                    float(battery_summary["EOL_cycle"] - landmark)
                    if battery_summary["Reached_EOL"] == 1
                    else np.nan
                ),
            }
```

```
            for column in OPERATING_CURRENT:
                row[column] = float(current[column])
                row[f"{column}_hist_mean"] = float(history[column].mean())
                row[f"{column}_hist_std"] = float(history[column].std(ddof=0))
                row[f"{column}_recent_slope"] = _slope(recent, column)
            for column in HEALTH_CURRENT:
                row[column] = float(current[column])
                row[f"{column}_recent_mean"] = float(recent[column].mean())
                row[f"{column}_recent_slope"] = _slope(recent, column)
            for horizon in HORIZONS:
                row[f"SOH_t_plus_{horizon}"] = _lookup_at_cycle(indexed, landmark + horizon, "SOH")
            if row["RUL_duration"] > 0:
                rows.append(row)
    dataset = pd.DataFrame(rows)
    dataset["SOH_stage"] = pd.cut(
        dataset["SOH_t"],
        bins=[-np.inf, 90.0, 95.0, np.inf],
        labels=["late", "middle", "early"],
    ).astype(str)
    return dataset


def make_battery_split(battery_df: pd.DataFrame, seed: int) -> pd.DataFrame:
    """ 按 Batch× 删失状态分层，在电池层固定约 60/20/20 划分。"""
    rng = np.random.default_rng(seed + 701)
    records: list[dict[str, object]] = []
    frame = battery_df[["Battery_ID", "Batch_ID", "Reached_EOL"]].copy()
    for (_, _), group in frame.groupby(["Batch_ID", "Reached_EOL"], sort=True):
        group = group.iloc[rng.permutation(len(group))].reset_index(drop=True)
        n = len(group)
        if n >= 5:
            n_train = max(1, int(round(0.60 * n)))
            n_val = max(1, int(round(0.20 * n)))
            if n_train + n_val >= n:
                n_train = n - 2
                n_val = 1
        elif n == 4:
            n_train, n_val = 2, 1
        elif n == 3:
            n_train, n_val = 1, 1
        elif n == 2:
            n_train, n_val = 1, 1
        else:
            n_train, n_val = 1, 0
        labels = ["train"] * n_train + ["validation"] * n_val + ["test"] * (n - n_train - n_val)
        for (_, row), split in zip(group.iterrows(), labels):
            records.append({**row.to_dict(), "Split": split})
    result = pd.DataFrame(records).sort_values("Battery_ID").reset_index(drop=True)
    return result


def feature_sets(task: str) -> dict[str, list[str]]:
    operating = [
        "Landmark_cycle",
        "Batch_ID",
        "Charge_strategy",
        *OPERATING_CURRENT,
        *[f"{column}_hist_mean" for column in OPERATING_CURRENT],
    ]
    resistance = ["Internal_resistance", "Internal_resistance_recent_mean"]
    voltage = [
        "Mean_voltage",
        "Voltage_dispersion",
        "End_of_discharge_voltage",
        "Mean_voltage_recent_mean",
        "Voltage_dispersion_recent_mean",
        "End_of_discharge_voltage_recent_mean",
    ]
    efficiency = [
        "Coulombic_efficiency",
        "Energy_efficiency",
        "Coulombic_efficiency_recent_mean",
        "Energy_efficiency_recent_mean",
    ]
    trends = [
        *[f"{column}_hist_std" for column in OPERATING_CURRENT],
        *[f"{column}_recent_slope" for column in OPERATING_CURRENT],
```

```
        *[f"{column}_recent_slope" for column in HEALTH_CURRENT],
    ]
    sets = {
        "A_operating_only": operating,
        "B_plus_resistance": operating + resistance,
        "C_plus_voltage": operating + resistance + voltage,
        "D_plus_efficiency": operating + resistance + voltage + efficiency,
        "E_full_history": operating + resistance + voltage + efficiency + trends,
    }
    if task in {"forecast", "rul"}:
        sets["E_full_history"] = sets["E_full_history"] + [
            "SOH_t",
            "Estimated_recent_degradation_slope",
        ]
    return sets


def build_and_save(
    raw_path: Path,
    battery_summary_path: Path,
    seed: int,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    cycle_df = pd.read_csv(raw_path)
    battery_df = pd.read_csv(battery_summary_path)
    dataset = build_landmark_dataset(cycle_df, battery_df)
    split = make_battery_split(battery_df, seed)
    dataset = dataset.merge(split[["Battery_ID", "Split"]], on="Battery_ID", how="left", validate="many_to_one")
    dataset.to_csv(DATA_PROCESSED / "q2_landmark_dataset.csv", index=False, float_format="%.8f")
    split.to_csv(RESULT_TABLES / "q2_battery_split.csv", index=False)
    feature_manifest = {
        task: {
            name: columns for name, columns in feature_sets(task).items()
        }
        for task in ("soh_estimation", "forecast", "rul")
    }
    feature_manifest["forbidden_generator_truth_features"] = sorted(FORBIDDEN_GENERATOR_FEATURES)
    feature_manifest["landmarks"] = list(LANDMARKS)
    feature_manifest["horizons"] = list(HORIZONS)
    feature_manifest["history_rule"] = "Every derived feature uses rows with Cycle <= Landmark_cycle only."
    save_json(feature_manifest, RESULT_TABLES / "q2_feature_manifest.json")
    return dataset, split


if __name__ == "__main__":
    from src.utils.project import DATA_RAW, PARAMETER_SOURCES, load_json

    config = load_json(PARAMETER_SOURCES / "simulation_config.json")
    build_and_save(
        DATA_RAW / "battery_cycle_data.csv",
        DATA_PROCESSED / "battery_summary.csv",
        int(config["random_seed"]),
    )
```

src/problem2/models.py

```
""" 对应论文 §6.2-§6.4: 监督学习、组感知区间与 Weibull AFT。"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.stats import norm
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


def make_preprocessor(frame: pd.DataFrame, features: list[str]) -> ColumnTransformer:
    categorical = [column for column in features if frame[column].dtype == object]
```

```
    numeric = [column for column in features if column not in categorical]
    return ColumnTransformer(
        [
            ("numeric", StandardScaler(), numeric),
            (
                "categorical",
                OneHotEncoder(handle_unknown="ignore", sparse_output=False),
                categorical,
            ),
        ]
    )


def make_regression_pipeline(
    frame: pd.DataFrame,
    features: list[str],
    kind: str,
    seed: int,
) -> Pipeline:
    preprocessor = make_preprocessor(frame, features)
    if kind == "ridge":
        model = Ridge(alpha=1.0)
    elif kind == "random_forest":
        model = RandomForestRegressor(
            n_estimators=220,
            min_samples_leaf=4,
            max_features=0.75,
            n_jobs=-1,
            random_state=seed,
        )
    elif kind == "hist_gradient_boosting":
        model = HistGradientBoostingRegressor(
            learning_rate=0.06,
            max_iter=220,
            max_leaf_nodes=18,
            min_samples_leaf=18,
            l2_regularization=0.3,
            random_state=seed,
        )
    else:
        raise ValueError(f"Unknown regression model: {kind}")
    return Pipeline([("preprocess", preprocessor), ("model", model)])


def regression_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    return {
        "MAE": float(mean_absolute_error(y_true, y_pred)),
        "RMSE": float(np.sqrt(mean_squared_error(y_true, y_pred))),
        "R2": float(r2_score(y_true, y_pred)),
        "Max_error": float(np.max(np.abs(y_true - y_pred))),
    }


def battery_group_conformal_radius(
    frame: pd.DataFrame,
    y_true: np.ndarray,
    y_pred: np.ndarray,
    coverage: float = 0.90,
) -> float:
    residual = pd.DataFrame(
        {"Battery_ID": frame["Battery_ID"].to_numpy(), "Abs_error": np.abs(y_true - y_pred)}
    )
    battery_score = residual.groupby("Battery_ID")["Abs_error"].max().to_numpy()
    n = len(battery_score)
    quantile_level = min(1.0, np.ceil((n + 1) * coverage) / max(n, 1))
    return float(np.quantile(battery_score, quantile_level, method="higher"))


def interval_metrics(
    y_true: np.ndarray,
    lower: np.ndarray,
    upper: np.ndarray,
) -> dict[str, float]:
    return {
        "Coverage": float(np.mean((y_true >= lower) & (y_true <= upper))),
        "Mean_interval_width": float(np.mean(upper - lower)),
        "Median_interval_width": float(np.median(upper - lower)),
```

```
    }


@dataclass
class WeibullAFT:
    l2_penalty: float = 0.02
    beta_: np.ndarray | None = None
    shape_: float | None = None
    optimization_: dict[str, object] | None = None

    def fit(
        self,
        matrix: np.ndarray,
        duration: np.ndarray,
        event: np.ndarray,
        sample_weight: np.ndarray | None = None,
    ) -> "WeibullAFT":
        x = np.column_stack([np.ones(len(matrix)), np.asarray(matrix, dtype=float)])
        t = np.maximum(np.asarray(duration, dtype=float), 1e-6)
        d = np.asarray(event, dtype=float)
        if sample_weight is None:
            weights = np.ones(len(t), dtype=float)
        else:
            weights = np.asarray(sample_weight, dtype=float)
            if len(weights) != len(t) or np.any(weights <= 0):
                raise ValueError("sample_weight must be positive and aligned with duration.")
        weights = weights / np.mean(weights)
        initial = np.zeros(x.shape[1] + 1, dtype=float)
        initial[0] = float(np.log(np.median(t)))
        initial[-1] = 0.0

        def objective(theta: np.ndarray) -> float:
            beta = theta[:-1]
            log_shape = float(np.clip(theta[-1], -3.0, 3.0))
            shape = np.exp(log_shape)
            log_scale = np.clip(x @ beta, -8.0, 12.0)
            log_t = np.log(t)
            cumulative_hazard = np.exp(np.clip(shape * (log_t - log_scale), -40.0, 40.0))
            log_likelihood = d * (
                log_shape - shape * log_scale + (shape - 1.0) * log_t
            ) - cumulative_hazard
            penalty = self.l2_penalty * float(beta[1:] @ beta[1:])
            return float(-np.average(log_likelihood, weights=weights) + penalty)

        def gradient(theta: np.ndarray) -> np.ndarray:
            beta = theta[:-1]
            log_shape = float(np.clip(theta[-1], -3.0, 3.0))
            shape = np.exp(log_shape)
            log_scale = np.clip(x @ beta, -8.0, 12.0)
            log_t = np.log(t)
            log_ratio = log_t - log_scale
            cumulative_hazard = np.exp(np.clip(shape * log_ratio, -40.0, 40.0))
            beta_gradient_rows = shape * (d - cumulative_hazard)[:, None] * x
            beta_gradient = np.average(beta_gradient_rows, axis=0, weights=weights)
            beta_gradient[1:] += 2.0 * self.l2_penalty * beta[1:]
            shape_gradient_rows = (
                cumulative_hazard * shape * log_ratio
                - d * (1.0 + shape * log_ratio)
            )
            shape_gradient = float(np.average(shape_gradient_rows, weights=weights))
            return np.append(beta_gradient, shape_gradient)

        result = minimize(
            objective,
            initial,
            method="L-BFGS-B",
            jac=gradient,
            bounds=[(None, None)] * x.shape[1] + [(-3.0, 3.0)],
            options={"maxiter": 3000, "maxfun": 50000, "ftol": 1e-10, "gtol": 1e-7},
        )
        if not result.success:
            raise RuntimeError(f"Weibull AFT optimization failed: {result.message}")
        self.beta_ = result.x[:-1]
        self.shape_ = float(np.exp(result.x[-1]))
        self.optimization_ = {
            "success": bool(result.success),
            "message": str(result.message),
```

```
            "iterations": int(result.nit),
            "objective": float(result.fun),
        }
        return self

    def _scale(self, matrix: np.ndarray) -> np.ndarray:
        if self.beta_ is None:
            raise RuntimeError("WeibullAFT is not fitted.")
        x = np.column_stack([np.ones(len(matrix)), np.asarray(matrix, dtype=float)])
        return np.exp(np.clip(x @ self.beta_, -8.0, 12.0))

    def predict_quantile(self, matrix: np.ndarray, probability: float) -> np.ndarray:
        if self.shape_ is None:
            raise RuntimeError("WeibullAFT is not fitted.")
        probability = float(np.clip(probability, 1e-6, 1.0 - 1e-6))
        return self._scale(matrix) * (-np.log(1.0 - probability)) ** (1.0 / self.shape_)

    def predict_median(self, matrix: np.ndarray) -> np.ndarray:
        return self.predict_quantile(matrix, 0.50)

    def predict_interval(self, matrix: np.ndarray, coverage: float = 0.90) -> tuple[np.ndarray, np.ndarray]:
        alpha = 1.0 - coverage
        return (
            self.predict_quantile(matrix, alpha / 2.0),
            self.predict_quantile(matrix, 1.0 - alpha / 2.0),
        )

    def negative_log_likelihood(
        self,
        matrix: np.ndarray,
        duration: np.ndarray,
        event: np.ndarray,
        sample_weight: np.ndarray | None = None,
    ) -> float:
        """Return the unpenalized battery-weighted censoring-aware NLL."""
        if self.beta_ is None or self.shape_ is None:
            raise RuntimeError("WeibullAFT is not fitted.")
        x = np.column_stack([np.ones(len(matrix)), np.asarray(matrix, dtype=float)])
        t = np.maximum(np.asarray(duration, dtype=float), 1e-6)
        d = np.asarray(event, dtype=float)
        weights = np.ones(len(t), dtype=float) if sample_weight is None else np.asarray(sample_weight, dtype=float)
        log_scale = np.clip(x @ self.beta_, -8.0, 12.0)
        log_t = np.log(t)
        cumulative_hazard = np.exp(np.clip(self.shape_ * (log_t - log_scale), -40.0, 40.0))
        log_likelihood = d * (
            np.log(self.shape_) - self.shape_ * log_scale + (self.shape_ - 1.0) * log_t
        ) - cumulative_hazard
        return float(-np.average(log_likelihood, weights=weights))


@dataclass
class LogNormalAFT:
    """Censoring-aware log-normal accelerated failure-time model."""

    l2_penalty: float = 0.02
    beta_: np.ndarray | None = None
    sigma_: float | None = None
    optimization_: dict[str, object] | None = None

    @staticmethod
    def _weighted_inputs(
        matrix: np.ndarray,
        duration: np.ndarray,
        event: np.ndarray,
        sample_weight: np.ndarray | None,
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
        x = np.column_stack([np.ones(len(matrix)), np.asarray(matrix, dtype=float)])
        t = np.maximum(np.asarray(duration, dtype=float), 1e-6)
        d = np.asarray(event, dtype=float)
        weights = np.ones(len(t), dtype=float) if sample_weight is None else np.asarray(sample_weight, dtype=float)
        if len(weights) != len(t) or np.any(weights <= 0):
            raise ValueError("sample_weight must be positive and aligned with duration.")
        return x, t, d, weights / np.mean(weights)

    def fit(
        self,
        matrix: np.ndarray,
```

```
        duration: np.ndarray,
        event: np.ndarray,
        sample_weight: np.ndarray | None = None,
    ) -> "LogNormalAFT":
        x, t, d, weights = self._weighted_inputs(matrix, duration, event, sample_weight)
        log_t = np.log(t)
        initial = np.zeros(x.shape[1] + 1, dtype=float)
        initial[0] = float(np.median(log_t))
        initial[-1] = float(np.log(max(np.std(log_t), 0.35)))

        def terms(theta: np.ndarray) -> tuple[np.ndarray, np.ndarray, float, np.ndarray]:
            beta = theta[:-1]
            log_sigma = float(np.clip(theta[-1], -3.0, 3.0))
            sigma = float(np.exp(log_sigma))
            mu = np.clip(x @ beta, -8.0, 12.0)
            z = (log_t - mu) / sigma
            return beta, z, sigma, mu

        def objective(theta: np.ndarray) -> float:
            beta, z, sigma, _ = terms(theta)
            event_ll = -log_t - np.log(sigma) - 0.5 * np.log(2.0 * np.pi) - 0.5 * z**2
            # norm.logsf is required here; log(1 - norm.cdf(z)) is unstable in the tail.
            censored_ll = norm.logsf(z)
            log_likelihood = d * event_ll + (1.0 - d) * censored_ll
            penalty = self.l2_penalty * float(beta[1:] @ beta[1:])
            return float(-np.average(log_likelihood, weights=weights) + penalty)

        def gradient(theta: np.ndarray) -> np.ndarray:
            beta, z, sigma, _ = terms(theta)
            log_hazard = norm.logpdf(z) - norm.logsf(z)
            hazard = np.exp(np.clip(log_hazard, -40.0, 40.0))
            score_mu = (d * z + (1.0 - d) * hazard) / sigma
            beta_gradient = -np.average(score_mu[:, None] * x, axis=0, weights=weights)
            beta_gradient[1:] += 2.0 * self.l2_penalty * beta[1:]
            log_sigma_gradient_rows = d * (1.0 - z**2) - (1.0 - d) * hazard * z
            log_sigma_gradient = float(np.average(log_sigma_gradient_rows, weights=weights))
            return np.append(beta_gradient, log_sigma_gradient)

        result = minimize(
            objective,
            initial,
            method="L-BFGS-B",
            jac=gradient,
            bounds=[(None, None)] * x.shape[1] + [(-3.0, 3.0)],
            options={"maxiter": 3000, "maxfun": 50000, "ftol": 1e-10, "gtol": 1e-7},
        )
        if not result.success:
            raise RuntimeError(f"Log-normal AFT optimization failed: {result.message}")
        self.beta_ = result.x[:-1]
        self.sigma_ = float(np.exp(result.x[-1]))
        self.optimization_ = {
            "success": bool(result.success),
            "message": str(result.message),
            "iterations": int(result.nit),
            "objective": float(result.fun),
        }
        return self

    def _mu(self, matrix: np.ndarray) -> np.ndarray:
        if self.beta_ is None:
            raise RuntimeError("LogNormalAFT is not fitted.")
        x = np.column_stack([np.ones(len(matrix)), np.asarray(matrix, dtype=float)])
        return np.clip(x @ self.beta_, -8.0, 12.0)

    def predict_quantile(self, matrix: np.ndarray, probability: float) -> np.ndarray:
        if self.sigma_ is None:
            raise RuntimeError("LogNormalAFT is not fitted.")
        probability = float(np.clip(probability, 1e-6, 1.0 - 1e-6))
        return np.exp(np.clip(self._mu(matrix) + self.sigma_ * norm.ppf(probability), -8.0, 12.0))

    def predict_median(self, matrix: np.ndarray) -> np.ndarray:
        return self.predict_quantile(matrix, 0.50)

    def predict_interval(self, matrix: np.ndarray, coverage: float = 0.90) -> tuple[np.ndarray, np.ndarray]:
        alpha = 1.0 - coverage
        return self.predict_quantile(matrix, alpha / 2.0), self.predict_quantile(matrix, 1.0 - alpha / 2.0)

```

```
    def negative_log_likelihood(
        self,
        matrix: np.ndarray,
        duration: np.ndarray,
        event: np.ndarray,
        sample_weight: np.ndarray | None = None,
    ) -> float:
        if self.sigma_ is None:
            raise RuntimeError("LogNormalAFT is not fitted.")
        _, t, d, weights = self._weighted_inputs(matrix, duration, event, sample_weight)
        log_t = np.log(t)
        z = (log_t - self._mu(matrix)) / self.sigma_
        event_ll = -log_t - np.log(self.sigma_) - 0.5 * np.log(2.0 * np.pi) - 0.5 * z**2
        log_likelihood = d * event_ll + (1.0 - d) * norm.logsf(z)
        return float(-np.average(log_likelihood, weights=weights))


def local_linear_rul(frame: pd.DataFrame, eol_soh: float = 80.0) -> np.ndarray:
    slope = frame["Estimated_recent_degradation_slope"].to_numpy(dtype=float)
    soh = frame["SOH_t"].to_numpy(dtype=float)
    safe_rate = np.maximum(-slope, 1e-4)
    prediction = np.maximum((soh - eol_soh) / safe_rate, 0.0)
    return np.clip(prediction, 0.0, 3000.0)
```

src/problem2/analysis.py

```
""" 问题二：无泄漏 SOH 估计、短期预测和含右删失 RUL 预测。"""

from __future__ import annotations

import json
import platform
import time
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

from src.problem2.build_landmark_dataset import (
    FORBIDDEN_GENERATOR_FEATURES,
    HORIZONS,
    build_and_save,
    feature_sets,
)
from src.problem2.models import (
    WeibullAFT,
    battery_group_conformal_radius,
    interval_metrics,
    local_linear_rul,
    make_preprocessor,
    make_regression_pipeline,
    regression_metrics,
)
from src.problem2.scenarios import SCENARIOS, simulate_severe_scenarios
from src.utils.project import (
    DATA_PROCESSED,
    DATA_RAW,
    PARAMETER_SOURCES,
    RESULT_MODELS,
    RESULT_TABLES,
    ROOT,
    STATE,
    load_json,
    save_json,
)


MODEL_KINDS = ("ridge", "random_forest", "hist_gradient_boosting")


def _battery_weights(frame: pd.DataFrame) -> np.ndarray:
    counts = frame.groupby("Battery_ID")["Battery_ID"].transform("size").to_numpy(dtype=float)
    weights = 1.0 / counts
    return weights / weights.mean()
```

```
50
51
52  def _fit_pipeline(model, frame: pd.DataFrame, features: list[str], target: np.ndarray):
53      model.fit(
54          frame[features],
55          target,
56          model__sample_weight=_battery_weights(frame),
57      )
58      return model
59
60
61  def _metric_row(
62      task: str,
63      model: str,
64      split: str,
65      y_true: np.ndarray,
66      y_pred: np.ndarray,
67      **extra: object,
68  ) -> dict[str, object]:
69      return {
70          "Task": task,
71          "Model": model,
72          "Split": split,
73          "N": int(len(y_true)),
74          **regression_metrics(y_true, y_pred),
75          **extra,
76      }
77
78
79  def _concordance_index(duration: np.ndarray, event: np.ndarray, prediction: np.ndarray) -> float:
80      comparable = 0
81      concordant = 0.0
82      for i in range(len(duration)):
83          if event[i] != 1:
84              continue
85          mask = duration > duration[i]
86          comparable += int(mask.sum())
87          concordant += float(np.sum(prediction[mask] > prediction[i]))
88          concordant += 0.5 * float(np.sum(prediction[mask] == prediction[i]))
89      return float(concordant / comparable) if comparable else float("nan")
90
91
92  def _soh_estimation(
93      dataset: pd.DataFrame,
94      seed: int,
95  ) -> tuple[dict[str, object], pd.DataFrame, pd.DataFrame, object, list[str]]:
96      features = feature_sets("soh_estimation")["E_full_history"]
97      if "SOH_t" in features or "Capacity_t" in features:
98          raise RuntimeError("Task 2A leakage: SOH_t/Capacity_t must not be input features.")
99      train = dataset.loc[dataset["Split"] == "train"].copy()
100     validation = dataset.loc[dataset["Split"] == "validation"].copy()
101     test = dataset.loc[dataset["Split"] == "test"].copy()
102     comparisons: list[dict[str, object]] = []
103     fitted: dict[str, object] = {}
104     validation_predictions: dict[str, np.ndarray] = {}
105     for kind in MODEL_KINDS:
106         model = make_regression_pipeline(train, features, kind, seed + 11)
107         _fit_pipeline(model, train, features, train["SOH_t"].to_numpy(dtype=float))
108         fitted[kind] = model
109         for split_name, frame in (("validation", validation), ("test", test)):
110             pred = np.clip(model.predict(frame[features]), 0.0, 100.0)
111             comparisons.append(
112                 _metric_row(
113                     "SOH_estimation",
114                     kind,
115                     split_name,
116                     frame["SOH_t"].to_numpy(dtype=float),
117                     pred,
118                     Selection_metric="validation_RMSE",
119                 )
120             )
121             if split_name == "validation":
122                 validation_predictions[kind] = pred
123     comparison = pd.DataFrame(comparisons)
124     selected = str(
125         comparison.loc[comparison["Split"] == "validation"].sort_values("RMSE").iloc[0]["Model"]
126     )
```

```
    model = fitted[selected]
    val_pred = validation_predictions[selected]
    radius = battery_group_conformal_radius(
        validation,
        validation["SOH_t"].to_numpy(dtype=float),
        val_pred,
        coverage=0.90,
    )
    test_pred = np.clip(model.predict(test[features]), 0.0, 100.0)
    lower = np.clip(test_pred - radius, 0.0, 100.0)
    upper = np.clip(test_pred + radius, 0.0, 100.0)
    predictions = test[
        ["Battery_ID", "Batch_ID", "Landmark_cycle", "SOH_stage", "SOH_t"]
    ].copy()
    predictions["Predicted_SOH"] = test_pred
    predictions["Lower_90"] = lower
    predictions["Upper_90"] = upper
    predictions["Absolute_error"] = np.abs(predictions["SOH_t"] - test_pred)
    stage_rows = []
    for stage, group in predictions.groupby("SOH_stage", sort=False):
        stage_rows.append(
            {
                "SOH_stage": stage,
                "N": int(len(group)),
                **regression_metrics(
                    group["SOH_t"].to_numpy(dtype=float),
                    group["Predicted_SOH"].to_numpy(dtype=float),
                ),
            }
        )
    stage_metrics = pd.DataFrame(stage_rows)
    test_metrics = regression_metrics(test["SOH_t"].to_numpy(dtype=float), test_pred)
    test_metrics.update(interval_metrics(test["SOH_t"].to_numpy(dtype=float), lower, upper))
    summary: dict[str, object] = {
        "selected_model": selected,
        "selection_rule": "Minimum validation RMSE; test set opened once after selection.",
        "features": features,
        "target_excluded_from_features": True,
        "conformal_unit": "maximum absolute residual per validation battery",
        "conformal_radius_90": radius,
        **test_metrics,
    }
    RESULT_MODELS.mkdir(parents=True, exist_ok=True)
    joblib.dump(model, RESULT_MODELS / "q2_soh_estimator.joblib")
    return summary, comparison, predictions, model, features


def _forecast_baselines(frame: pd.DataFrame, horizon: int) -> dict[str, np.ndarray]:
    soh = frame["SOH_t"].to_numpy(dtype=float)
    slope = frame["Estimated_recent_degradation_slope"].to_numpy(dtype=float)
    return {
        "persistence": soh,
        "local_linear": np.clip(soh + slope * float(horizon), 0.0, 100.0),
    }


def _short_term_forecasts(
    dataset: pd.DataFrame,
    seed: int,
) -> tuple[dict[str, object], pd.DataFrame, pd.DataFrame]:
    features = feature_sets("forecast")["E_full_history"]
    comparison_rows: list[dict[str, object]] = []
    prediction_frames: list[pd.DataFrame] = []
    summaries: dict[str, object] = {}
    for horizon in HORIZONS:
        target = f"SOH_t_plus_{horizon}"
        available = dataset.loc[dataset[target].notna()].copy()
        train = available.loc[available["Split"] == "train"].copy()
        validation = available.loc[available["Split"] == "validation"].copy()
        test = available.loc[available["Split"] == "test"].copy()
        models: dict[str, object] = {}
        val_predictions: dict[str, np.ndarray] = _forecast_baselines(validation, horizon)
        test_predictions: dict[str, np.ndarray] = _forecast_baselines(test, horizon)
        for kind in MODEL_KINDS:
            model = make_regression_pipeline(train, features, kind, seed + horizon)
            _fit_pipeline(model, train, features, train[target].to_numpy(dtype=float))
            models[kind] = model
```

```
            val_predictions[kind] = np.clip(model.predict(validation[features]), 0.0, 100.0)
            test_predictions[kind] = np.clip(model.predict(test[features]), 0.0, 100.0)
        for name in (*_forecast_baselines(validation, horizon).keys(), *MODEL_KINDS):
            for split_name, frame, predictions in (
                ("validation", validation, val_predictions),
                ("test", test, test_predictions),
            ):
                comparison_rows.append(
                    _metric_row(
                        f"SOH_forecast_{horizon}",
                        name,
                        split_name,
                        frame[target].to_numpy(dtype=float),
                        predictions[name],
                        Horizon=horizon,
                        Selection_metric="validation_RMSE",
                    )
                )
        validation_table = pd.DataFrame(
            [row for row in comparison_rows if row["Task"] == f"SOH_forecast_{horizon}" and row["Split"] == "validation"]
        )
        selected = str(validation_table.sort_values("RMSE").iloc[0]["Model"])
        val_pred = val_predictions[selected]
        radius = battery_group_conformal_radius(
            validation,
            validation[target].to_numpy(dtype=float),
            val_pred,
            coverage=0.90,
        )
        test_pred = test_predictions[selected]
        lower = np.clip(test_pred - radius, 0.0, 100.0)
        upper = np.clip(test_pred + radius, 0.0, 100.0)
        prediction = test[
            ["Battery_ID", "Batch_ID", "Landmark_cycle", "SOH_stage", "SOH_t", target]
        ].copy()
        prediction = prediction.rename(columns={target: "Observed_future_SOH"})
        prediction["Horizon"] = horizon
        prediction["Selected_model"] = selected
        prediction["Predicted_future_SOH"] = test_pred
        prediction["Lower_90"] = lower
        prediction["Upper_90"] = upper
        prediction["Absolute_error"] = np.abs(prediction["Observed_future_SOH"] - test_pred)
        prediction_frames.append(prediction)
        metrics = regression_metrics(test[target].to_numpy(dtype=float), test_pred)
        metrics.update(interval_metrics(test[target].to_numpy(dtype=float), lower, upper))
        summaries[str(horizon)] = {
            "selected_model": selected,
            "conformal_radius_90": radius,
            "n_train": int(len(train)),
            "n_validation": int(len(validation)),
            "n_test": int(len(test)),
            **metrics,
        }
        if selected in models:
            joblib.dump(models[selected], RESULT_MODELS / f"q2_soh_forecast_{horizon}.joblib")
        else:
            save_json(
                {"selected_baseline": selected, "horizon": horizon},
                RESULT_MODELS / f"q2_soh_forecast_{horizon}_baseline.json",
            )
    return summaries, pd.DataFrame(comparison_rows), pd.concat(prediction_frames, ignore_index=True)


def _fit_aft(
    train: pd.DataFrame,
    features: list[str],
    l2_penalty: float = 0.02,
) -> tuple[object, WeibullAFT]:
    preprocessor = make_preprocessor(train, features)
    matrix = preprocessor.fit_transform(train[features])
    aft = WeibullAFT(l2_penalty=l2_penalty).fit(
        matrix,
        train["RUL_duration"].to_numpy(dtype=float),
        train["RUL_event"].to_numpy(dtype=int),
        sample_weight=_battery_weights(train),
    )
    return preprocessor, aft
```

```

def _aft_predictions(preprocessor, aft: WeibullAFT, frame: pd.DataFrame, features: list[str]):
    matrix = preprocessor.transform(frame[features])
    median = np.clip(aft.predict_median(matrix), 0.0, 3000.0)
    lower, upper = aft.predict_interval(matrix, coverage=0.90)
    return median, np.clip(lower, 0.0, 4000.0), np.clip(upper, 0.0, 4000.0)


def _rul_models(
    dataset: pd.DataFrame,
    seed: int,
) -> tuple[
    dict[str, object],
    pd.DataFrame,
    pd.DataFrame,
    object,
    WeibullAFT,
    list[str],
]:
    features = feature_sets("rul")["E_full_history"]
    train = dataset.loc[dataset["Split"] == "train"].copy()
    validation = dataset.loc[dataset["Split"] == "validation"].copy()
    test = dataset.loc[dataset["Split"] == "test"].copy()
    train_event = train.loc[train["RUL_event"] == 1].copy()
    preprocessor, aft = _fit_aft(train, features)
    val_aft, val_aft_lower, val_aft_upper = _aft_predictions(preprocessor, aft, validation, features)
    test_aft, test_aft_lower, test_aft_upper = _aft_predictions(preprocessor, aft, test, features)

    complete_models: dict[str, object] = {}
    for kind in ("ridge", "random_forest"):
        model = make_regression_pipeline(train_event, features, kind, seed + 401)
        _fit_pipeline(
            model,
            train_event,
            features,
            np.log1p(train_event["RUL_duration"].to_numpy(dtype=float)),
        )
        complete_models[kind] = model

    prediction_map = {
        "local_linear": {
            "validation": local_linear_rul(validation),
            "test": local_linear_rul(test),
        },
        "ridge_complete_case": {
            "validation": np.clip(np.expm1(complete_models["ridge"].predict(validation[features])), 0.0, 3000.0),
            "test": np.clip(np.expm1(complete_models["ridge"].predict(test[features])), 0.0, 3000.0),
        },
        "random_forest_complete_case": {
            "validation": np.clip(np.expm1(complete_models["random_forest"].predict(validation[features])), 0.0, 3000.0),
            "test": np.clip(np.expm1(complete_models["random_forest"].predict(test[features])), 0.0, 3000.0),
        },
        "weibull_AFT_censoring_aware": {"validation": val_aft, "test": test_aft},
    }
    rows: list[dict[str, object]] = []
    for model_name, split_predictions in prediction_map.items():
        for split_name, frame in (("validation", validation), ("test", test)):
            observed = frame.loc[frame["RUL_event"] == 1]
            pred = split_predictions[split_name][frame["RUL_event"].to_numpy(dtype=int) == 1]
            rows.append(
                _metric_row(
                    "RUL",
                    model_name,
                    split_name,
                    observed["RUL_duration"].to_numpy(dtype=float),
                    pred,
                    Censoring_aware=model_name == "weibull_AFT_censoring_aware",
                    Eligible_primary=model_name == "weibull_AFT_censoring_aware",
                    Selection_rule="Capability gate: the primary model must include right-censored rows.",
                )
            )
    comparison = pd.DataFrame(rows)

    val_event_mask = validation["RUL_event"].to_numpy(dtype=int) == 1
    conformal_radius = battery_group_conformal_radius(
        validation.loc[val_event_mask],
```

```
        validation.loc[val_event_mask, "RUL_duration"].to_numpy(dtype=float),
        val_aft[val_event_mask],
        coverage=0.90,
    )
    calibrated_lower = np.minimum(test_aft_lower, np.maximum(0.0, test_aft - conformal_radius))
    calibrated_upper = np.maximum(test_aft_upper, test_aft + conformal_radius)
    calibrated_upper = np.clip(calibrated_upper, 0.0, 4000.0)
    predictions = test[
        [
            "Battery_ID",
            "Batch_ID",
            "Landmark_cycle",
            "SOH_stage",
            "RUL_duration",
            "RUL_event",
            "True_RUL",
        ]
    ].copy()
    predictions["Predicted_RUL_median"] = test_aft
    predictions["Parametric_lower_90"] = test_aft_lower
    predictions["Parametric_upper_90"] = test_aft_upper
    predictions["Calibrated_lower_90"] = calibrated_lower
    predictions["Calibrated_upper_90"] = calibrated_upper
    predictions["Local_linear_RUL"] = prediction_map["local_linear"]["test"]
    predictions["Absolute_error_if_event"] = np.where(
        predictions["RUL_event"] == 1,
        np.abs(predictions["RUL_duration"] - predictions["Predicted_RUL_median"]),
        np.nan,
    )
    event_mask = test["RUL_event"].to_numpy(dtype=int) == 1
    event_metrics = regression_metrics(
        test.loc[event_mask, "RUL_duration"].to_numpy(dtype=float),
        test_aft[event_mask],
    )
    event_metrics.update(
        interval_metrics(
            test.loc[event_mask, "RUL_duration"].to_numpy(dtype=float),
            calibrated_lower[event_mask],
            calibrated_upper[event_mask],
        )
    )
    censored_mask = ~event_mask
    censor_consistency = (
        float(np.mean(calibrated_upper[censored_mask] >= test.loc[censored_mask, "RUL_duration"].to_numpy(dtype=float)))
        if censored_mask.any()
        else float("nan")
    )
    summary: dict[str, object] = {
        "selected_model": "weibull_AFT_censoring_aware",
        "selection_rule": "Capability gate requires right-censoring support; complete-case regressors are diagnostic baselines
        ↪  only.",
        "features": features,
        "train_rows": int(len(train)),
        "train_censored_rows": int((train["RUL_event"] == 0).sum()),
        "test_event_rows": int(event_mask.sum()),
        "test_censored_rows": int(censored_mask.sum()),
        "shape": float(aft.shape_),
        "optimization": aft.optimization_,
        "conformal_radius_90": conformal_radius,
        "concordance_index": _concordance_index(
            test["RUL_duration"].to_numpy(dtype=float),
            test["RUL_event"].to_numpy(dtype=int),
            test_aft,
        ),
        "censored_upper_bound_consistency": censor_consistency,
        **event_metrics,
    }
    joblib.dump(
        {"preprocessor": preprocessor, "aft": aft, "features": features},
        RESULT_MODELS / "q2_rul_weibull_aft.joblib",
    )
    return summary, comparison, predictions, preprocessor, aft, features


def _ablation(
    dataset: pd.DataFrame,
    selected_soh_kind: str,
```

```
    seed: int,
) -> pd.DataFrame:
    rows: list[dict[str, object]] = []
    train = dataset.loc[dataset["Split"] == "train"].copy()
    validation = dataset.loc[dataset["Split"] == "validation"].copy()
    for set_name, features in feature_sets("soh_estimation").items():
        model = make_regression_pipeline(train, features, selected_soh_kind, seed + 501)
        _fit_pipeline(model, train, features, train["SOH_t"].to_numpy(dtype=float))
        pred = np.clip(model.predict(validation[features]), 0.0, 100.0)
        rows.append(
            _metric_row(
                "SOH_estimation",
                set_name,
                "validation",
                validation["SOH_t"].to_numpy(dtype=float),
                pred,
                Base_model=selected_soh_kind,
            )
        )
    for set_name, features in feature_sets("rul").items():
        preprocessor, aft = _fit_aft(train, features)
        pred, lower, upper = _aft_predictions(preprocessor, aft, validation, features)
        event = validation["RUL_event"].to_numpy(dtype=int) == 1
        row = _metric_row(
            "RUL",
            set_name,
            "validation",
            validation.loc[event, "RUL_duration"].to_numpy(dtype=float),
            pred[event],
            Base_model="weibull_AFT_censoring_aware",
        )
        row.update(
            interval_metrics(
                validation.loc[event, "RUL_duration"].to_numpy(dtype=float),
                lower[event],
                upper[event],
            )
        )
        rows.append(row)
    result = pd.DataFrame(rows)
    full_lookup = result.loc[result["Model"] == "E_full_history"].set_index("Task")["RMSE"]
    result["Delta_RMSE_vs_full"] = result.apply(
        lambda row: float(row["RMSE"] - full_lookup.loc[row["Task"]]), axis=1
    )
    return result


def _leave_one_batch_out(
    dataset: pd.DataFrame,
    selected_soh_kind: str,
    seed: int,
) -> pd.DataFrame:
    rows: list[dict[str, object]] = []
    soh_features = feature_sets("soh_estimation")["E_full_history"]
    rul_features = feature_sets("rul")["E_full_history"]
    for index, batch in enumerate(sorted(dataset["Batch_ID"].unique())):
        train = dataset.loc[dataset["Batch_ID"] != batch].copy()
        held = dataset.loc[dataset["Batch_ID"] == batch].copy()
        soh_model = make_regression_pipeline(train, soh_features, selected_soh_kind, seed + 601 + index)
        _fit_pipeline(soh_model, train, soh_features, train["SOH_t"].to_numpy(dtype=float))
        soh_pred = np.clip(soh_model.predict(held[soh_features]), 0.0, 100.0)
        rows.append(
            _metric_row(
                "SOH_estimation",
                str(batch),
                "leave_one_batch_out",
                held["SOH_t"].to_numpy(dtype=float),
                soh_pred,
                Held_out_batteries=int(held["Battery_ID"].nunique()),
            )
        )
        preprocessor, aft = _fit_aft(train, rul_features)
        rul_pred, _, _ = _aft_predictions(preprocessor, aft, held, rul_features)
        event = held["RUL_event"].to_numpy(dtype=int) == 1
        rows.append(
            _metric_row(
                "RUL",
```

```
                str(batch),
                "leave_one_batch_out",
                held.loc[event, "RUL_duration"].to_numpy(dtype=float),
                rul_pred[event],
                Held_out_batteries=int(held["Battery_ID"].nunique()),
            )
        )
    return pd.DataFrame(rows)


def _scenario_analysis(
    cfg: dict[str, object],
    train_dataset: pd.DataFrame,
    preprocessor,
    aft: WeibullAFT,
    features: list[str],
) -> tuple[pd.DataFrame, pd.DataFrame]:
    scenario_landmarks, outcomes = simulate_severe_scenarios(cfg, n_batteries=80)
    scenario = scenario_landmarks.loc[scenario_landmarks["Landmark_cycle"] == 200].copy()
    if scenario.empty:
        raise RuntimeError("Scenario simulation produced no cycle-200 landmarks.")
    median, lower, upper = _aft_predictions(preprocessor, aft, scenario, features)
    scenario["Predicted_RUL_median"] = median
    scenario["Predicted_lower_90"] = lower
    scenario["Predicted_upper_90"] = upper
    scenario = scenario.merge(
        outcomes,
        on=["Scenario", "Battery_ID"],
        how="left",
        validate="one_to_one",
        suffixes=("", "_outcome"),
    )
    scenario["Observed_or_capped_RUL"] = scenario["True_simulated_EOL"] - scenario["Landmark_cycle"]
    scenario["Absolute_error_if_event"] = np.where(
        scenario["Reached_EOL"] == 1,
        np.abs(scenario["Observed_or_capped_RUL"] - scenario["Predicted_RUL_median"]),
        np.nan,
    )
    numeric_features = [column for column in features if train_dataset[column].dtype != object]
    lower_support = train_dataset[numeric_features].min()
    upper_support = train_dataset[numeric_features].max()
    scenario["Out_of_training_support_fraction"] = (
        (scenario[numeric_features] < lower_support) | (scenario[numeric_features] > upper_support)
    ).mean(axis=1)
    rows = []
    for scenario_name, group in scenario.groupby("Scenario", sort=False):
        event_group = group.loc[group["Reached_EOL"] == 1]
        rows.append(
            {
                "Scenario": scenario_name,
                "N": int(len(group)),
                "Reached_EOL_rate": float(group["Reached_EOL"].mean()),
                "Abusive_extension": bool(group["Abusive_extension"].iloc[0]),
                "Median_predicted_RUL": float(group["Predicted_RUL_median"].median()),
                "Median_predicted_lower_90": float(group["Predicted_lower_90"].median()),
                "Median_predicted_upper_90": float(group["Predicted_upper_90"].median()),
                "Median_observed_or_capped_RUL": float(group["Observed_or_capped_RUL"].median()),
                "Median_lifetime_change_vs_normal": float(group["Lifetime_change_vs_normal"].median()),
                "Event_only_MAE": (
                    float(event_group["Absolute_error_if_event"].mean()) if len(event_group) else float("nan")
                ),
                "Mean_out_of_training_support_fraction": float(
                    group["Out_of_training_support_fraction"].mean()
                ),
            }
        )
    summary = pd.DataFrame(rows)
    order = {name: index for index, name in enumerate(SCENARIOS)}
    summary["_order"] = summary["Scenario"].map(order)
    summary = summary.sort_values("_order").drop(columns="_order").reset_index(drop=True)
    return scenario, summary


def _timesfm_status() -> dict[str, object]:
    payload = {
        "status": "skipped",
        "reason": "Local TimesFM dependency, PyTorch runtime, and model weights are unavailable; no large model was downloaded
        ↪  during the competition workflow.",
```

```
        "python": platform.python_version(),
        "dependency_available": False,
        "weights_available": False,
        "benchmark_results_used": False,
    }
    save_json(payload, RESULT_TABLES / "q2_timesfm_status.json")
    return payload


def _model_selection_decision(
    soh_summary: dict[str, object],
    forecast_summary: dict[str, object],
    rul_summary: dict[str, object],
    lobo: pd.DataFrame,
    scenario_summary: pd.DataFrame,
) -> pd.DataFrame:
    """ 分层/Pareto 决策，不把异质量纲压成主观总分。"""
    lobo_soh = lobo.loc[lobo["Task"] == "SOH_estimation", "RMSE"]
    lobo_rul = lobo.loc[lobo["Task"] == "RUL", "RMSE"]
    severe_support = float(
        scenario_summary.loc[
            ~scenario_summary["Abusive_extension"], "Mean_out_of_training_support_fraction"
        ].max()
    )
    rows: list[dict[str, object]] = [
        {
            "Task": "Current SOH",
            "Selected_model": soh_summary["selected_model"],
            "Accuracy_evidence": f"test RMSE={float(soh_summary['RMSE']):.3f} pp",
            "Interval_evidence": f"90% coverage={100*float(soh_summary['Coverage']):.1f}%,
            ↪  width={float(soh_summary['Mean_interval_width']):.3f} pp",
            "Censoring_support": "not applicable",
            "Batch_stability": f"LOBO RMSE range={float(lobo_soh.min()):.3f}-{float(lobo_soh.max()):.3f} pp",
            "Severe_condition_evidence": "health-indicator estimator is not used as the scenario lifetime endpoint",
            "Interpretability": "high; linear standardized effects",
            "Complexity": "low",
            "Decision_rule": "minimum validation RMSE, then interval and LOBO audit",
        }
    ]
    for horizon in HORIZONS:
        item = forecast_summary[str(horizon)]
        rows.append(
            {
                "Task": f"SOH forecast {horizon}",
                "Selected_model": item["selected_model"],
                "Accuracy_evidence": f"test RMSE={float(item['RMSE']):.3f} pp",
                "Interval_evidence": f"90% coverage={100*float(item['Coverage']):.1f}%,
                ↪  width={float(item['Mean_interval_width']):.3f} pp",
                "Censoring_support": "not applicable",
                "Batch_stability": "fixed battery split; horizon-specific validation",
                "Severe_condition_evidence": "not promoted to long-horizon scenario model",
                "Interpretability": "high for Ridge; medium for gradient boosting",
                "Complexity": "low to medium",
                "Decision_rule": "minimum validation RMSE, then group-conformal calibration",
            }
        )
    rows.extend(
        [
            {
                "Task": "RUL primary",
                "Selected_model": "weibull_AFT_censoring_aware",
                "Accuracy_evidence": f"event test MAE/RMSE={float(rul_summary['MAE']):.1f}/{float(rul_summary['RMSE']):.1f}
                ↪  cycles; C-index={float(rul_summary['concordance_index']):.3f}",
                "Interval_evidence": f"90% coverage={100*float(rul_summary['Coverage']):.1f}%,
                ↪  width={float(rul_summary['Mean_interval_width']):.1f} cycles",
                "Censoring_support": "yes; event density plus right-censored survival likelihood",
                "Batch_stability": f"LOBO RMSE range={float(lobo_rul.min()):.1f}-{float(lobo_rul.max()):.1f} cycles",
                "Severe_condition_evidence": f"7 matched scenarios; max non-abusive out-of-support fraction={severe_support:.3f}",
                "Interpretability": "high; Weibull shape and accelerated lifetime scale",
                "Complexity": "medium",
                "Decision_rule": "censoring capability gate, followed by accuracy/calibration/LOBO/scenario audit",
            },
            {
                "Task": "RUL diagnostic baseline",
                "Selected_model": "random_forest_complete_case",
                "Accuracy_evidence": f"event test MAE/RMSE={float(rul_summary['complete_case_random_forest_test_MAE']):.1f}/{floa
                ↪  t(rul_summary['complete_case_random_forest_test_RMSE']):.1f} cycles",
```

```
                "Interval_evidence": "none",
                "Censoring_support": "no; discards censored batteries",
                "Batch_stability": "not eligible for primary LOBO decision",
                "Severe_condition_evidence": "not used",
                "Interpretability": "low to medium",
                "Complexity": "medium",
                "Decision_rule": "Pareto-ineligible as primary despite lower event-only RMSE",
            },
        ]
    )
    return pd.DataFrame(rows)


def _update_state(key_metrics: dict[str, object]) -> None:
    state_path = STATE / "decision_log.json"
    if not state_path.exists():
        return
    state = load_json(state_path)
    state["current_phase"] = "PHASE 2"
    state["current_stage"] = 5
    stage = state.setdefault("stages", {}).setdefault("5", {})
    stage["status"] = "in_progress_phase_limited"
    stage.setdefault("qi_status", {})["Q2"] = "pass"
    stage["qi_status"]["Q3"] = "not_started"
    stage.setdefault("sub_problems", {})["Q2"] = {
        "status": "pass",
        "model_name": "battery-landmark SOH regressors + censoring-aware Weibull AFT",
        "code_path": "src/problem2/analysis.py",
        "results_path": "results/tables/q2_key_metrics.json",
        "key_metrics": key_metrics,
        "issues": [],
    }
    stage["sub_problems"]["Q3"] = {"status": "not_started_scope_gate"}
    stage["cross_reference_chain"] = "Q1 factors feed Q2 landmark features; Q2 outputs are frozen for later Q3, which was not executed."
    state.setdefault("events", {}).setdefault("log", []).append(
        {"type": "scope_gate", "message": "PHASE 2/Q2 completed; stop before Q3."}
    )
    state.setdefault("run_metadata", {})["phase_completed"] = "PHASE 2 / Q2 only; Q3 not started"
    save_json(state, state_path)


def _write_latex_metrics(metrics: dict[str, object]) -> None:
    soh = metrics["soh_estimation"]
    forecast = metrics["soh_forecast"]
    rul = metrics["rul"]
    lines = [
        f"\\newcommand{{\\QTwoLandmarks}}{{{int(metrics['landmark_rows'])}}}",
        f"\\newcommand{{\\QTwoSOHModel}}{{{str(soh['selected_model']).replace('_', r'\\_')}}}",
        f"\\newcommand{{\\QTwoSOHRMSE}}{{{float(soh['RMSE']):.3f}}}",
        f"\\newcommand{{\\QTwoSOHCoverage}}{{{100 * float(soh['Coverage']):.1f}\\%}}",
        f"\\newcommand{{\\ForecastFiftyRMSE}}{{{float(forecast['50']['RMSE']):.3f}}}",
        f"\\newcommand{{\\ForecastFiftyCoverage}}{{{100 * float(forecast['50']['Coverage']):.1f}\\%}}",
        f"\\newcommand{{\\ForecastHundredRMSE}}{{{float(forecast['100']['RMSE']):.3f}}}",
        f"\\newcommand{{\\ForecastHundredCoverage}}{{{100 * float(forecast['100']['Coverage']):.1f}\\%}}",
        f"\\newcommand{{\\ForecastTwoHundredRMSE}}{{{float(forecast['200']['RMSE']):.3f}}}",
        f"\\newcommand{{\\ForecastTwoHundredCoverage}}{{{100 * float(forecast['200']['Coverage']):.1f}\\%}}",
        f"\\newcommand{{\\QTwoRULMAE}}{{{float(rul['MAE']):.1f}}}",
        f"\\newcommand{{\\QTwoRULRMSE}}{{{float(rul['RMSE']):.1f}}}",
        f"\\newcommand{{\\QTwoRULCoverage}}{{{100 * float(rul['Coverage']):.1f}\\%}}",
        f"\\newcommand{{\\QTwoRULCIndex}}{{{float(rul['concordance_index']):.3f}}}",
        f"\\newcommand{{\\QTwoRULWidth}}{{{float(rul['Mean_interval_width']):.1f}}}",
        f"\\newcommand{{\\RULRFMAE}}{{{float(rul['complete_case_random_forest_test_MAE']):.1f}}}",
        f"\\newcommand{{\\RULRFRMSE}}{{{float(rul['complete_case_random_forest_test_RMSE']):.1f}}}",
        f"\\newcommand{{\\LOBOConsumerSOHMax}}{{{float(metrics['robustness']['lobo_soh_max_RMSE']):.3f}}}",
        f"\\newcommand{{\\LOBORULMax}}{{{float(metrics['robustness']['lobo_rul_max_RMSE']):.1f}}}",
    ]
    (ROOT / "paper" / "generated_metrics_q2.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")


def run_problem2() -> dict[str, object]:
    started = time.perf_counter()
    cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
    seed = int(cfg["random_seed"])
    dataset, split = build_and_save(
        DATA_RAW / "battery_cycle_data.csv",
        DATA_PROCESSED / "battery_summary.csv",
```

```
        seed,
    )
    split_sets = {
        name: set(split.loc[split["Split"] == name, "Battery_ID"])
        for name in ("train", "validation", "test")
    }
    overlap_count = sum(
        len(split_sets[a] & split_sets[b])
        for a, b in (("train", "validation"), ("train", "test"), ("validation", "test"))
    )
    if overlap_count:
        raise RuntimeError("Battery-level split leakage detected.")

    soh_summary, soh_comparison, soh_predictions, _, _ = _soh_estimation(dataset, seed)
    forecast_summary, forecast_comparison, forecast_predictions = _short_term_forecasts(dataset, seed)
    rul_summary, rul_comparison, rul_predictions, rul_preprocessor, rul_aft, rul_features = _rul_models(
        dataset, seed
    )
    ablation = _ablation(dataset, str(soh_summary["selected_model"]), seed)
    lobo = _leave_one_batch_out(dataset, str(soh_summary["selected_model"]), seed)
    scenario_rows, scenario_summary = _scenario_analysis(
        cfg,
        dataset.loc[dataset["Split"] == "train"],
        rul_preprocessor,
        rul_aft,
        rul_features,
    )
    timesfm = _timesfm_status()

    rf_complete_test = rul_comparison.loc[
        (rul_comparison["Model"] == "random_forest_complete_case")
        & (rul_comparison["Split"] == "test")
    ].iloc[0]
    rul_summary["complete_case_random_forest_test_MAE"] = float(rf_complete_test["MAE"])
    rul_summary["complete_case_random_forest_test_RMSE"] = float(rf_complete_test["RMSE"])
    model_selection = _model_selection_decision(
        soh_summary,
        forecast_summary,
        rul_summary,
        lobo,
        scenario_summary,
    )

    soh_comparison.to_csv(RESULT_TABLES / "q2_soh_model_comparison.csv", index=False, float_format="%.8f")
    soh_predictions.to_csv(RESULT_TABLES / "q2_soh_test_predictions.csv", index=False, float_format="%.8f")
    soh_predictions.groupby("SOH_stage", sort=False).agg(
        N=("Absolute_error", "size"),
        MAE=("Absolute_error", "mean"),
        RMSE=("Absolute_error", lambda x: float(np.sqrt(np.mean(np.square(x))))),
    ).reset_index().to_csv(RESULT_TABLES / "q2_soh_stage_errors.csv", index=False, float_format="%.8f")
    forecast_comparison.to_csv(
        RESULT_TABLES / "q2_forecast_model_comparison.csv", index=False, float_format="%.8f"
    )
    forecast_predictions.to_csv(
        RESULT_TABLES / "q2_forecast_test_predictions.csv", index=False, float_format="%.8f"
    )
    rul_comparison.to_csv(RESULT_TABLES / "q2_rul_model_comparison.csv", index=False, float_format="%.8f")
    rul_predictions.to_csv(RESULT_TABLES / "q2_rul_test_predictions.csv", index=False, float_format="%.8f")
    ablation.to_csv(RESULT_TABLES / "q2_feature_ablation.csv", index=False, float_format="%.8f")
    lobo.to_csv(RESULT_TABLES / "q2_leave_one_batch_out.csv", index=False, float_format="%.8f")
    scenario_rows.to_csv(RESULT_TABLES / "q2_severe_scenario_predictions.csv", index=False, float_format="%.8f")
    scenario_summary.to_csv(RESULT_TABLES / "q2_severe_scenario_summary.csv", index=False, float_format="%.8f")
    model_selection.to_csv(RESULT_TABLES / "q2_model_selection_decision.csv", index=False)

    feature_manifest = load_json(RESULT_TABLES / "q2_feature_manifest.json")
    all_features = {
        column
        for task in ("soh_estimation", "forecast", "rul")
        for columns in feature_manifest[task].values()
        for column in columns
    }
    forbidden_intersection = sorted(all_features & FORBIDDEN_GENERATOR_FEATURES)
    validation = {
        "battery_split_disjoint": overlap_count == 0,
        "all_batteries_assigned_once": int(split["Battery_ID"].nunique()) == len(split) == 360,
        "history_only_feature_builder": feature_manifest["history_rule"].endswith("only."),
        "generator_truth_excluded": len(forbidden_intersection) == 0,
```

```
        "task2a_target_and_capacity_excluded": not {
            "SOH_t",
            "Capacity_t",
        } & set(feature_manifest["soh_estimation"]["E_full_history"]),
        "all_forecast_horizons_present": set(map(str, HORIZONS)) <= set(forecast_summary),
        "baseline_models_exist": {"persistence", "local_linear"} <= set(forecast_comparison["Model"])
        and {"local_linear", "ridge_complete_case"} <= set(rul_comparison["Model"]),
        "test_set_not_used_for_tuning": all(
            item["selection_rule"].startswith("Minimum validation")
            for item in [soh_summary]
        )
        and all(str(forecast_summary[str(h)]["selected_model"]) for h in HORIZONS),
        "right_censored_rows_used_by_primary_rul": int(rul_summary["train_censored_rows"]) > 0,
        "aft_symbolic_verification_pass": bool(
            load_json(RESULT_TABLES / "q2_aft_symbolic_verification.json")["overall_pass"]
        ),
        "soh_interval_reported": "Coverage" in soh_summary,
        "forecast_intervals_reported": all("Coverage" in forecast_summary[str(h)] for h in HORIZONS),
        "rul_interval_reported": "Coverage" in rul_summary,
        "severe_scenarios_complete": set(scenario_summary["Scenario"]) == set(SCENARIOS),
        "leave_one_batch_out_complete": set(lobo["Model"]) == set(dataset["Batch_ID"].unique())
        and set(lobo["Task"]) == {"SOH_estimation", "RUL"},
        "pareto_model_selection_documented": len(model_selection) == 6
        and "Decision_rule" in model_selection.columns,
        "abusive_extensions_explicit": bool(
            scenario_summary.loc[
                scenario_summary["Scenario"].isin(["Combined_high", "Combined_low"]),
                "Abusive_extension",
            ].all()
        ),
        "timesfm_skip_documented": timesfm["status"] == "skipped" and bool(timesfm["reason"]),
        "q3_not_executed": True,
    }
    validation["overall_pass"] = all(validation.values())
    save_json(validation, RESULT_TABLES / "q2_validation.json")
    if not validation["overall_pass"]:
        raise RuntimeError(f"Q2 validation failed: {json.dumps(validation, ensure_ascii=False)}")

    key_metrics: dict[str, object] = {
        "phase": "PHASE 2 / Q2 only",
        "landmark_rows": int(len(dataset)),
        "batteries": int(dataset["Battery_ID"].nunique()),
        "split_batteries": {name: len(values) for name, values in split_sets.items()},
        "soh_estimation": soh_summary,
        "soh_forecast": forecast_summary,
        "rul": rul_summary,
        "timesfm": timesfm,
        "scenario_count": int(len(scenario_summary)),
        "robustness": {
            "lobo_soh_min_RMSE": float(lobo.loc[lobo["Task"] == "SOH_estimation", "RMSE"].min()),
            "lobo_soh_max_RMSE": float(lobo.loc[lobo["Task"] == "SOH_estimation", "RMSE"].max()),
            "lobo_rul_min_RMSE": float(lobo.loc[lobo["Task"] == "RUL", "RMSE"].min()),
            "lobo_rul_max_RMSE": float(lobo.loc[lobo["Task"] == "RUL", "RMSE"].max()),
            "combined_scenarios_explicitly_abusive": True,
        },
        "q3_executed": False,
        "runtime_seconds": float(time.perf_counter() - started),
    }
    save_json(key_metrics, RESULT_TABLES / "q2_key_metrics.json")
    save_json(
        {
            "status": "pass",
            "phase": "PHASE 2 / Q2 only",
            "random_seed": seed,
            "runtime_seconds": key_metrics["runtime_seconds"],
            "validation": validation,
            "test_set_policy": "Model and feature choices use validation only; held-out test metrics are final reporting metrics.",
            "q3_executed": False,
        },
        RESULT_TABLES / "q2_run_manifest.json",
    )
    _write_latex_metrics(key_metrics)
    _update_state(key_metrics)
    return key_metrics


if __name__ == "__main__":
```

```
    metrics = run_problem2()
    print(json.dumps(metrics, ensure_ascii=False, indent=2))
```

src/problem2/scenarios.py

```
""" 对应论文 §6.7：匹配虚拟电池的正常/恶劣工况受控仿真。"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd

from src.problem2.build_landmark_dataset import build_landmark_dataset


SCENARIOS = {
    "Normal": {
        "Temperature": 25.0,
        "Charge_C_rate": 1.0,
        "Discharge_C_rate": 1.0,
        "DOD": 80.0,
        "Charge_strategy": "CC-CV",
        "Abusive_extension": False,
    },
    "High_temperature": {
        "Temperature": 43.0,
        "Charge_C_rate": 1.0,
        "Discharge_C_rate": 1.0,
        "DOD": 80.0,
        "Charge_strategy": "CC-CV",
        "Abusive_extension": False,
    },
    "Low_temperature": {
        "Temperature": 8.0,
        "Charge_C_rate": 1.0,
        "Discharge_C_rate": 1.0,
        "DOD": 80.0,
        "Charge_strategy": "CC-CV",
        "Abusive_extension": False,
    },
    "High_C_rate": {
        "Temperature": 25.0,
        "Charge_C_rate": 2.0,
        "Discharge_C_rate": 2.0,
        "DOD": 80.0,
        "Charge_strategy": "Fast-charge",
        "Abusive_extension": False,
    },
    "Deep_discharge": {
        "Temperature": 25.0,
        "Charge_C_rate": 1.0,
        "Discharge_C_rate": 1.0,
        "DOD": 100.0,
        "Charge_strategy": "CC-CV",
        "Abusive_extension": False,
    },
    "Combined_high": {
        "Temperature": 43.0,
        "Charge_C_rate": 2.0,
        "Discharge_C_rate": 2.0,
        "DOD": 100.0,
        "Charge_strategy": "Fast-charge",
        "Abusive_extension": True,
    },
    "Combined_low": {
        "Temperature": 8.0,
        "Charge_C_rate": 2.0,
        "Discharge_C_rate": 2.0,
        "DOD": 100.0,
        "Charge_strategy": "Fast-charge",
        "Abusive_extension": True,
    },
}
```

```
71
72  STRATEGY_MULTIPLIER = {"CC-CV": 1.0, "Multi-stage": 0.92, "Fast-charge": 1.14}
73
74
75  @dataclass
76  class ScenarioBattery:
77      battery_id: str
78      batch_id: str
79      initial_capacity: float
80      initial_resistance: float
81      fade_heterogeneity: float
82      knee_heterogeneity: float
83      phase: float
84      noises: dict[str, np.ndarray]
85
86
87  def _matched_batteries(cfg: dict[str, object], n_batteries: int, seed: int) -> list[ScenarioBattery]:
88      rng = np.random.default_rng(seed)
89      n_batches = int(cfg["n_batches"])
90      max_cycles = int(cfg["max_cycles"])
91      batch_capacity = rng.normal(1.0, 0.008, n_batches)
92      batch_resistance = rng.normal(1.0, 0.045, n_batches)
93      batch_fade = rng.lognormal(0.0, 0.075, n_batches)
94      batteries = []
95      for index in range(n_batteries):
96          batch = index % n_batches
97          noises = {
98              "temperature": rng.normal(0.0, 0.60, max_cycles + 1),
99              "charge": rng.normal(0.0, 0.035, max_cycles + 1),
100             "discharge": rng.normal(0.0, 0.040, max_cycles + 1),
101             "dod": rng.normal(0.0, 0.012, max_cycles + 1),
102             "capacity": rng.normal(0.0, float(cfg["capacity_measurement_sd_ah"]), max_cycles + 1),
103             "resistance": rng.normal(0.0, float(cfg["resistance_measurement_sd_mohm"]), max_cycles + 1),
104             "mean_voltage": rng.normal(0.0, 0.004, max_cycles + 1),
105             "voltage_dispersion": rng.normal(0.0, 0.0012, max_cycles + 1),
106             "end_voltage": rng.normal(0.0, 0.006, max_cycles + 1),
107             "coulombic": rng.normal(0.0, 0.025, max_cycles + 1),
108             "energy": rng.normal(0.0, 0.10, max_cycles + 1),
109         }
110         batteries.append(
111             ScenarioBattery(
112                 battery_id=f"SCN_{index + 1:04d}",
113                 batch_id=f"B{batch + 1:02d}",
114                 initial_capacity=float(
115                     float(cfg["nominal_capacity_ah"]) * batch_capacity[batch] * rng.normal(1.0, 0.012)
116                 ),
117                 initial_resistance=float(
118                     float(cfg["nominal_internal_resistance_mohm"])
119                     * batch_resistance[batch]
120                     * rng.lognormal(0.0, 0.035)
121                 ),
122                 fade_heterogeneity=float(batch_fade[batch] * rng.lognormal(0.0, 0.09)),
123                 knee_heterogeneity=float(rng.lognormal(0.0, 0.085)),
124                 phase=float(rng.uniform(0.0, 2.0 * np.pi)),
125                 noises=noises,
126             )
127         )
128     return batteries
129
130
131 def _simulate_one(
132     battery: ScenarioBattery,
133     scenario: dict[str, object],
134     cfg: dict[str, object],
135 ) -> tuple[pd.DataFrame, dict[str, object]]:
136     max_cycles = int(cfg["max_cycles"])
137     cycle = np.arange(max_cycles + 1, dtype=int)
138     seasonal = np.sin(2.0 * np.pi * cycle / 220.0 + battery.phase)
139     temperature = np.clip(
140         float(scenario["Temperature"]) + 1.25 * seasonal + battery.noises["temperature"], 5.0, 45.0
141     )
142     charge = np.clip(float(scenario["Charge_C_rate"]) + battery.noises["charge"], 0.45, 2.10)
143     discharge = np.clip(
144         float(scenario["Discharge_C_rate"]) + battery.noises["discharge"], 0.45, 2.10
145     )
146     dod = np.clip(
147         float(scenario["DOD"]) / 100.0 + 0.015 * seasonal + battery.noises["dod"], 0.48, 1.00
```

```
    )
    high_temperature = np.exp(0.026 * np.maximum(temperature - 25.0, 0.0))
    low_temperature = 1.0 + 0.010 * np.maximum(15.0 - temperature, 0.0) ** 1.25
    charge_stress = 1.0 + 0.34 * (charge - 1.0) + 0.11 * (charge - 1.0) ** 2
    discharge_stress = 1.0 + 0.18 * (discharge - 1.0) + 0.06 * (discharge - 1.0) ** 2
    dod_stress = (dod / 0.80) ** 1.25
    interaction = (
        1.0
        + 0.08 * np.maximum(charge - 1.0, 0.0) * np.maximum(dod - 0.75, 0.0) / 0.25
        + 0.05 * np.maximum(temperature - 30.0, 0.0) / 15.0 * np.maximum(charge - 1.0, 0.0)
    )
    stress = np.clip(
        high_temperature
        * low_temperature
        * charge_stress
        * discharge_stress
        * dod_stress
        * STRATEGY_MULTIPLIER[str(scenario["Charge_strategy"])]
        * interaction,
        0.42,
        3.40,
    )
    stress[0] = 0.0
    equivalent = np.cumsum(stress)
    knee_exposure = float(cfg["base_knee_cycle"]) * battery.knee_heterogeneity
    onset_exposure = 0.47 * knee_exposure
    base_x = equivalent / float(cfg["reference_cycles"])
    mid_x = np.maximum(equivalent - onset_exposure, 0.0) / 500.0
    knee_x = np.maximum(equivalent - knee_exposure, 0.0) / 400.0
    base_loss = (
        float(cfg["base_fade_coefficient"])
        * battery.fade_heterogeneity
        * base_x ** float(cfg["base_fade_exponent"])
    )
    mid_loss = (
        float(cfg["mid_acceleration_coefficient"])
        * battery.fade_heterogeneity
        * mid_x ** float(cfg["mid_acceleration_exponent"])
    )
    knee_loss = (
        float(cfg["knee_acceleration_coefficient"])
        * battery.fade_heterogeneity
        * knee_x ** float(cfg["knee_acceleration_exponent"])
    )
    loss = base_loss + mid_loss + knee_loss
    latent_soh = 100.0 * (1.0 - loss)
    hits = np.flatnonzero(latent_soh <= float(cfg["eol_soh_percent"]))
    final = int(hits[0]) if hits.size else max_cycles
    sl = slice(0, final + 1)
    capacity = battery.initial_capacity * (1.0 - loss[sl]) + battery.noises["capacity"][sl]
    soh = 100.0 * capacity / battery.initial_capacity
    resistance = (
        battery.initial_resistance * (1.0 + 1.65 * loss[sl] + 1.10 * knee_loss[sl])
        + 0.07 * (temperature[sl] - 25.0)
        + battery.noises["resistance"][sl]
    )
    mean_voltage = (
        3.68
        - 0.0032 * (100.0 - soh)
        - 0.012 * (discharge[sl] - 1.0)
        + 0.0008 * (temperature[sl] - 25.0)
        + battery.noises["mean_voltage"][sl]
    )
    voltage_dispersion = np.maximum(
        0.016
        + 0.00055 * (100.0 - soh)
        + 0.0025 * np.maximum(discharge[sl] - 1.0, 0.0)
        + battery.noises["voltage_dispersion"][sl],
        0.005,
    )
    end_voltage = (
        3.16
        - 0.0040 * (100.0 - soh)
        - 0.028 * (discharge[sl] - 1.0)
        + battery.noises["end_voltage"][sl]
    )
    coulombic = np.clip(
```

```
        99.72 - 0.055 * stress[sl] - 0.42 * knee_loss[sl] + battery.noises["coulombic"][sl],
        98.8,
        99.95,
    )
    energy = np.clip(
        96.8
        - 0.026 * (resistance - battery.initial_resistance)
        - 0.40 * np.maximum(discharge[sl] - 1.0, 0.0)
        + battery.noises["energy"][sl],
        88.0,
        98.5,
    )
    frame = pd.DataFrame(
        {
            "Battery_ID": battery.battery_id,
            "Batch_ID": battery.batch_id,
            "Cycle": cycle[sl],
            "Temperature": temperature[sl],
            "Charge_C_rate": charge[sl],
            "Discharge_C_rate": discharge[sl],
            "DOD": 100.0 * dod[sl],
            "Charge_strategy": str(scenario["Charge_strategy"]),
            "Capacity": capacity,
            "SOH": soh,
            "Internal_resistance": resistance,
            "Mean_voltage": mean_voltage,
            "Voltage_dispersion": voltage_dispersion,
            "End_of_discharge_voltage": end_voltage,
            "Coulombic_efficiency": coulombic,
            "Energy_efficiency": energy,
        }
    )
    summary = {
        "Battery_ID": battery.battery_id,
        "Batch_ID": battery.batch_id,
        "Observed_cycles": final,
        "Reached_EOL": int(bool(hits.size)),
        "EOL_cycle": float(final) if hits.size else np.nan,
    }
    return frame, summary


def simulate_severe_scenarios(
    cfg: dict[str, object],
    n_batteries: int = 80,
    seed: int | None = None,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    seed = int(cfg["random_seed"]) + 909 if seed is None else int(seed)
    batteries = _matched_batteries(cfg, n_batteries, seed)
    landmark_frames = []
    outcome_rows = []
    normal_eol: dict[str, float] = {}
    for scenario_name, scenario in SCENARIOS.items():
        cycle_frames = []
        summaries = []
        for battery in batteries:
            frame, summary = _simulate_one(battery, scenario, cfg)
            cycle_frames.append(frame)
            summaries.append(summary)
        summary_df = pd.DataFrame(summaries)
        if scenario_name == "Normal":
            normal_eol = dict(zip(summary_df["Battery_ID"], summary_df["EOL_cycle"].fillna(float(cfg["max_cycles"]))))
        cycle_df = pd.concat(cycle_frames, ignore_index=True)
        landmarks = build_landmark_dataset(cycle_df, summary_df)
        landmarks["Scenario"] = scenario_name
        landmarks["Abusive_extension"] = bool(scenario["Abusive_extension"])
        landmark_frames.append(landmarks)
        for _, row in summary_df.iterrows():
            eol = float(row["EOL_cycle"]) if pd.notna(row["EOL_cycle"]) else float(cfg["max_cycles"])
            outcome_rows.append(
                {
                    "Scenario": scenario_name,
                    "Battery_ID": row["Battery_ID"],
                    "True_simulated_EOL": eol,
                    "Reached_EOL": int(row["Reached_EOL"]),
                    "Lifetime_change_vs_normal": eol - normal_eol.get(row["Battery_ID"], eol),
                    "Abusive_extension": bool(scenario["Abusive_extension"]),
```

```
                }
            )
    return pd.concat(landmark_frames, ignore_index=True), pd.DataFrame(outcome_rows)
```

src/problem2/phase2__5.py

```
"""PHASE 2.5: audit the deployable SOH-to-RUL bridge before Q3."""

from __future__ import annotations

import json
import time
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import GroupKFold

from src.problem2.analysis import _battery_weights, _concordance_index, _fit_pipeline
from src.problem2.build_landmark_dataset import FORBIDDEN_GENERATOR_FEATURES, feature_sets
from src.problem2.models import (
    LogNormalAFT,
    WeibullAFT,
    battery_group_conformal_radius,
    interval_metrics,
    make_preprocessor,
    make_regression_pipeline,
    regression_metrics,
)
from src.utils.project import DATA_PROCESSED, PARAMETER_SOURCES, RESULT_MODELS, RESULT_TABLES, ROOT, load_json, save_json


AFT_CLASSES = {"Weibull_AFT": WeibullAFT, "LogNormal_AFT": LogNormalAFT}


def _oof_soh(
    train: pd.DataFrame,
    features: list[str],
    seed: int,
    n_splits: int = 5,
) -> tuple[np.ndarray, pd.DataFrame]:
    groups = train["Battery_ID"].to_numpy()
    predictions = np.full(len(train), np.nan, dtype=float)
    records: list[dict[str, object]] = []
    splitter = GroupKFold(n_splits=n_splits)
    for fold, (fit_index, held_index) in enumerate(splitter.split(train, groups=groups), start=1):
        fit = train.iloc[fit_index]
        held = train.iloc[held_index]
        fit_ids = set(fit["Battery_ID"])
        held_ids = set(held["Battery_ID"])
        disjoint = fit_ids.isdisjoint(held_ids)
        if not disjoint:
            raise RuntimeError(f"OOF SOH battery leakage in fold {fold}.")
        model = make_regression_pipeline(fit, features, "ridge", seed + fold)
        _fit_pipeline(model, fit, features, fit["SOH_t"].to_numpy(dtype=float))
        fold_prediction = np.clip(model.predict(held[features]), 0.0, 100.0)
        predictions[held_index] = fold_prediction
        for row_index, pred in zip(held.index, fold_prediction):
            records.append(
                {
                    "Row_index": int(row_index),
                    "Battery_ID": str(train.loc[row_index, "Battery_ID"]),
                    "Fold": fold,
                    "OOF_Estimated_SOH": float(pred),
                    "Observed_SOH": float(train.loc[row_index, "SOH_t"]),
                    "Train_held_battery_disjoint": disjoint,
                }
            )
    if np.isnan(predictions).any():
        raise RuntimeError("OOF SOH predictions are incomplete.")
    return predictions, pd.DataFrame(records).sort_values("Row_index")


def _prepare_soh_views(
```

```
    dataset: pd.DataFrame,
    seed: int,
) -> tuple[dict[str, pd.DataFrame], object, float, pd.DataFrame]:
    soh_features = feature_sets("soh_estimation")["E_full_history"]
    train = dataset.loc[dataset["Split"] == "train"].copy()
    validation = dataset.loc[dataset["Split"] == "validation"].copy()
    test = dataset.loc[dataset["Split"] == "test"].copy()
    oof, oof_table = _oof_soh(train, soh_features, seed + 2500)

    final_soh_model = make_regression_pipeline(train, soh_features, "ridge", seed + 2510)
    _fit_pipeline(final_soh_model, train, soh_features, train["SOH_t"].to_numpy(dtype=float))
    estimated = dataset.copy()
    estimated["Observed_SOH_for_audit"] = estimated["SOH_t"]
    estimated.loc[train.index, "Estimated_SOH"] = oof
    for frame in (validation, test):
        estimated.loc[frame.index, "Estimated_SOH"] = np.clip(
            final_soh_model.predict(frame[soh_features]), 0.0, 100.0
        )
    val_pred = estimated.loc[validation.index, "Estimated_SOH"].to_numpy(dtype=float)
    soh_radius = battery_group_conformal_radius(
        validation,
        validation["SOH_t"].to_numpy(dtype=float),
        val_pred,
        coverage=0.90,
    )
    estimated["SOH_t"] = estimated["Estimated_SOH"]
    oracle = dataset.copy()
    return {"Oracle_SOH": oracle, "Estimated_SOH": estimated}, final_soh_model, soh_radius, oof_table


def _fit_aft(
    train: pd.DataFrame,
    features: list[str],
    model_name: str,
) -> tuple[object, WeibullAFT | LogNormalAFT]:
    preprocessor = make_preprocessor(train, features)
    matrix = preprocessor.fit_transform(train[features])
    model = AFT_CLASSES[model_name](l2_penalty=0.02).fit(
        matrix,
        train["RUL_duration"].to_numpy(dtype=float),
        train["RUL_event"].to_numpy(dtype=int),
        sample_weight=_battery_weights(train),
    )
    return preprocessor, model


def _predict(
    preprocessor: object,
    model: WeibullAFT | LogNormalAFT,
    frame: pd.DataFrame,
    features: list[str],
) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    matrix = preprocessor.transform(frame[features])
    median = np.clip(model.predict_median(matrix), 0.0, 3000.0)
    lower, upper = model.predict_interval(matrix, coverage=0.90)
    return matrix, median, np.clip(lower, 0.0, 4000.0), np.clip(upper, 0.0, 4000.0)


def _evaluation_row(
    input_mode: str,
    model_name: str,
    split_name: str,
    frame: pd.DataFrame,
    matrix: np.ndarray,
    median: np.ndarray,
    lower: np.ndarray,
    upper: np.ndarray,
    model: WeibullAFT | LogNormalAFT,
) -> dict[str, object]:
    event_mask = frame["RUL_event"].to_numpy(dtype=int) == 1
    duration = frame["RUL_duration"].to_numpy(dtype=float)
    event_metrics = regression_metrics(duration[event_mask], median[event_mask])
    intervals = interval_metrics(duration[event_mask], lower[event_mask], upper[event_mask])
    return {
        "Input_mode": input_mode,
        "Model": model_name,
        "Split": split_name,
```

```
        "N": int(len(frame)),
        "N_event": int(event_mask.sum()),
        "N_censored": int((~event_mask).sum()),
        "Negative_log_likelihood": model.negative_log_likelihood(
            matrix, duration, frame["RUL_event"].to_numpy(dtype=int), _battery_weights(frame)
        ),
        **event_metrics,
        "C_index": _concordance_index(duration, frame["RUL_event"].to_numpy(dtype=int), median),
        "Coverage_90": intervals["Coverage"],
        "Mean_interval_width_90": intervals["Mean_interval_width"],
    }


def _lobo_chain(
    dataset: pd.DataFrame,
    seed: int,
    rul_features: list[str],
    soh_features: list[str],
) -> pd.DataFrame:
    rows: list[dict[str, object]] = []
    for batch_index, batch in enumerate(sorted(dataset["Batch_ID"].unique())):
        base_train = dataset.loc[dataset["Batch_ID"] != batch].copy().reset_index(drop=True)
        held = dataset.loc[dataset["Batch_ID"] == batch].copy().reset_index(drop=True)
        oof, _ = _oof_soh(base_train, soh_features, seed + 2600 + 20 * batch_index)
        soh_model = make_regression_pipeline(base_train, soh_features, "ridge", seed + 2700 + batch_index)
        _fit_pipeline(soh_model, base_train, soh_features, base_train["SOH_t"].to_numpy(dtype=float))
        held_estimated = np.clip(soh_model.predict(held[soh_features]), 0.0, 100.0)
        for input_mode in ("Oracle_SOH", "Estimated_SOH"):
            train_view = base_train.copy()
            held_view = held.copy()
            if input_mode == "Estimated_SOH":
                train_view["SOH_t"] = oof
                held_view["SOH_t"] = held_estimated
            for model_name in AFT_CLASSES:
                preprocessor, model = _fit_aft(train_view, rul_features, model_name)
                matrix, median, lower, upper = _predict(preprocessor, model, held_view, rul_features)
                row = _evaluation_row(
                    input_mode, model_name, "LOBO", held_view, matrix, median, lower, upper, model
                )
                row["Held_out_batch"] = str(batch)
                row["Held_out_batteries"] = int(held_view["Battery_ID"].nunique())
                rows.append(row)
    return pd.DataFrame(rows)


def _select_model(validation_rows: pd.DataFrame, lobo: pd.DataFrame) -> tuple[str, str]:
    candidates = validation_rows.loc[validation_rows["Input_mode"] == "Estimated_SOH"].copy()
    best_nll = float(candidates["Negative_log_likelihood"].min())
    contenders = candidates.loc[candidates["Negative_log_likelihood"] <= 1.01 * best_nll].copy()
    if len(contenders) > 1:
        best_rmse = float(contenders["RMSE"].min())
        contenders = contenders.loc[contenders["RMSE"] <= 1.01 * best_rmse].copy()
    if len(contenders) > 1:
        contenders["Calibration_deviation"] = (contenders["Coverage_90"] - 0.90).abs()
        lobo_sd = (
            lobo.loc[lobo["Input_mode"] == "Estimated_SOH"]
            .groupby("Model")["RMSE"]
            .std()
            .rename("LOBO_RMSE_SD")
        )
        contenders = contenders.join(lobo_sd, on="Model").sort_values(
            ["Calibration_deviation", "LOBO_RMSE_SD", "Model"]
        )
    selected = str(contenders.iloc[0]["Model"])
    rule = (
        "Censoring-capability gate; minimum validation NLL; models within 1% NLL use validation "
        "RMSE, then 90% calibration deviation and LOBO RMSE SD. Test is opened only after freeze."
    )
    return selected, rule


def _write_interface_schema() -> pd.DataFrame:
    rows = [
        ("Battery_ID", "string", "-", "Unique retired-cell identifier", "measured", "inspection record"),
        ("Batch_ID", "string", "-", "Source-pack/batch identifier", "measured", "inspection record"),
        ("Inspection_cycle", "integer", "cycle", "First-life standardized inspection cycle", "measured", "inspection record"),
        ("Estimated_SOH", "float", "%", "Task-2A current SOH estimate", "estimated", "frozen SOH estimator"),
```

```
        ("SOH_lower_90", "float", "%", "Lower endpoint of battery-group conformal interval", "estimated", "validation calibration"),
        ("SOH_upper_90", "float", "%", "Upper endpoint of battery-group conformal interval", "estimated", "validation calibration"),
        ("Measured_or_estimated_capacity", "float", "Ah", "Current inspection capacity; measured in the simulation deployment view", "measured", "inspection record"),
        ("Internal_resistance", "float", "mOhm", "Current DC-resistance proxy", "measured", "inspection record"),
        ("Voltage_dispersion", "float", "V", "Current voltage-dispersion proxy", "measured", "inspection record"),
        ("End_of_discharge_voltage", "float", "V", "Current end-of-discharge voltage", "measured", "inspection record"),
        ("Coulombic_efficiency", "float", "%", "Current coulombic efficiency", "measured", "inspection record"),
        ("Energy_efficiency", "float", "%", "Current round-trip energy-efficiency proxy", "measured", "inspection record"),
        ("Estimated_recent_degradation_slope", "float", "%/cycle", "History-only recent SOH slope", "estimated", "past inspection history"),
        ("FirstLife_RUL_80_median", "float", "cycle", "Median time to the first-life 80% simulation endpoint", "estimated", "frozen deployment AFT"),
        ("FirstLife_RUL_80_lower", "float", "cycle", "Conservative 90% lower endpoint for FirstLife_RUL_80", "estimated", "frozen deployment AFT"),
        ("FirstLife_RUL_80_upper", "float", "cycle", "90% upper endpoint for FirstLife_RUL_80", "estimated", "frozen deployment AFT"),
        ("SOH_uncertainty_width", "float", "%", "SOH_upper_90 minus SOH_lower_90", "derived", "decision view"),
        ("RUL_uncertainty_width", "float", "cycle", "FirstLife RUL interval width", "derived", "decision view"),
        ("Out_of_training_support_score", "float", "fraction", "Fraction of numeric deployment features outside Q2-train ranges", "derived", "Q2 support audit"),
        ("Risk_flag", "string", "-", "Model-risk flag only; not a real safety certification", "derived", "uncertainty/OOD audit"),
    ]
    schema = pd.DataFrame(
        rows,
        columns=["Field", "Data_type", "Unit", "Definition", "Decision_availability", "Source"],
    )
    schema["Future_truth"] = False
    schema.to_csv(RESULT_TABLES / "q3_frozen_interface_schema.csv", index=False)
    return schema


def run_phase2_5(allow_downstream: bool = False) -> dict[str, object]:
    started = time.perf_counter()
    cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
    seed = int(cfg["random_seed"])
    dataset = pd.read_csv(DATA_PROCESSED / "q2_landmark_dataset.csv")
    views, soh_model, soh_radius, oof_table = _prepare_soh_views(dataset, seed)
    oof_table.to_csv(RESULT_TABLES / "q2_oof_estimated_soh.csv", index=False, float_format="%.8f")

    rul_features = feature_sets("rul")["E_full_history"]
    soh_features = feature_sets("soh_estimation")["E_full_history"]
    lobo = _lobo_chain(dataset, seed, rul_features, soh_features)
    lobo.to_csv(RESULT_TABLES / "q2_phase2_5_lobo.csv", index=False, float_format="%.8f")

    fitted: dict[tuple[str, str], tuple[object, WeibullAFT | LogNormalAFT, float]] = {}
    validation_rows: list[dict[str, object]] = []
    for input_mode, view in views.items():
        train = view.loc[view["Split"] == "train"].copy()
        validation = view.loc[view["Split"] == "validation"].copy()
        for model_name in AFT_CLASSES:
            preprocessor, model = _fit_aft(train, rul_features, model_name)
            matrix, median, lower, upper = _predict(preprocessor, model, validation, rul_features)
            row = _evaluation_row(
                input_mode, model_name, "validation", validation, matrix, median, lower, upper, model
            )
            validation_rows.append(row)
            event_mask = validation["RUL_event"].to_numpy(dtype=int) == 1
            radius = battery_group_conformal_radius(
                validation.loc[event_mask],
                validation.loc[event_mask, "RUL_duration"].to_numpy(dtype=float),
                median[event_mask],
                coverage=0.90,
            )
            fitted[(input_mode, model_name)] = (preprocessor, model, radius)
    validation_frame = pd.DataFrame(validation_rows)
    selected_model, selection_rule = _select_model(validation_frame, lobo)

    comparison_rows = validation_rows.copy()
    test_prediction_tables: list[pd.DataFrame] = []
    for input_mode, view in views.items():
        test = view.loc[view["Split"] == "test"].copy()
        for model_name in AFT_CLASSES:
            preprocessor, model, radius = fitted[(input_mode, model_name)]
            matrix, median, lower, upper = _predict(preprocessor, model, test, rul_features)
```

```
            calibrated_lower = np.minimum(lower, np.maximum(0.0, median - radius))
            calibrated_upper = np.maximum(upper, median + radius)
            row = _evaluation_row(
                input_mode,
                model_name,
                "test_post_freeze",
                test,
                matrix,
                median,
                calibrated_lower,
                calibrated_upper,
                model,
            )
            comparison_rows.append(row)
            if model_name == selected_model:
                table = test[["Battery_ID", "Batch_ID", "Landmark_cycle", "RUL_duration", "RUL_event"]].copy()
                table["Input_mode"] = input_mode
                table["Model"] = model_name
                table["Predicted_median"] = median
                table["Lower_90"] = calibrated_lower
                table["Upper_90"] = calibrated_upper
                test_prediction_tables.append(table)

comparison = pd.DataFrame(comparison_rows)
lobo_summary = (
    lobo.groupby(["Input_mode", "Model"])["RMSE"]
    .agg(LOBO_RMSE_min="min", LOBO_RMSE_max="max", LOBO_RMSE_mean="mean", LOBO_RMSE_SD="std")
    .reset_index()
)
comparison = comparison.merge(lobo_summary, on=["Input_mode", "Model"], how="left")
comparison["Selected_deployment_distribution"] = comparison["Model"].eq(selected_model)
comparison["Selection_rule"] = selection_rule
comparison["Test_used_for_selection"] = False
comparison.to_csv(
    RESULT_TABLES / "q2_censoring_aware_model_comparison.csv", index=False, float_format="%.8f"
)
pd.concat(test_prediction_tables, ignore_index=True).to_csv(
    RESULT_TABLES / "q2_phase2_5_rul_test_predictions.csv", index=False, float_format="%.8f"
)

propagation = comparison.loc[
    comparison["Model"].eq(selected_model),
    [
        "Input_mode", "Model", "Split", "N", "N_event", "N_censored",
        "Negative_log_likelihood", "MAE", "RMSE", "R2", "C_index", "Coverage_90",
        "Mean_interval_width_90", "LOBO_RMSE_min", "LOBO_RMSE_max", "LOBO_RMSE_mean", "LOBO_RMSE_SD",
    ],
].copy()
for split_name in propagation["Split"].unique():
    oracle = propagation.loc[
        (propagation["Split"] == split_name) & (propagation["Input_mode"] == "Oracle_SOH")
    ].iloc[0]
    mask = propagation["Split"] == split_name
    propagation.loc[mask, "Delta_RMSE_vs_oracle"] = propagation.loc[mask, "RMSE"] - float(oracle["RMSE"])
    propagation.loc[mask, "Delta_C_index_vs_oracle"] = propagation.loc[mask, "C_index"] - float(oracle["C_index"])
validation_propagation = propagation.loc[propagation["Split"] == "validation"].set_index("Input_mode")
rmse_degradation = (
    float(validation_propagation.loc["Estimated_SOH", "RMSE"])
    / float(validation_propagation.loc["Oracle_SOH", "RMSE"])
    - 1.0
)
cindex_drop = float(
    validation_propagation.loc["Oracle_SOH", "C_index"]
    - validation_propagation.loc["Estimated_SOH", "C_index"]
)
small_degradation = rmse_degradation <= 0.10 and cindex_drop <= 0.02
propagation["Deployment_decision"] = np.where(
    propagation["Input_mode"] == "Estimated_SOH",
    "selected" if small_degradation else "selected_with_uncertainty_warning",
    "upper_bound_benchmark_only",
)
propagation.to_csv(
    RESULT_TABLES / "q2_soh_to_rul_propagation_audit.csv", index=False, float_format="%.8f"
)

deployment_preprocessor, deployment_aft, deployment_radius = fitted[("Estimated_SOH", selected_model)]
joblib.dump(
```

```
        {
            "soh_estimator": soh_model,
            "soh_features": soh_features,
            "soh_conformal_radius_90": soh_radius,
            "rul_preprocessor": deployment_preprocessor,
            "aft": deployment_aft,
            "aft_distribution": selected_model,
            "rul_features": rul_features,
            "rul_current_soh_source": "Estimated_SOH; train values are battery-disjoint OOF",
            "rul_conformal_radius_90": deployment_radius,
            "first_life_endpoint": "80% SOH simulation endpoint",
        },
        RESULT_MODELS / "q2_deployment_rul_model.joblib",
    )
    schema = _write_interface_schema()
    forbidden_terms = {
        "true future EOL", "true future SOH", "latent degradation coefficient", "true knee",
        "Fade_heterogeneity", "generator stress index",
    }
    schema_text = " ".join(schema.astype(str).to_numpy().ravel()).lower()
    forbidden_schema_hits = sorted(term for term in forbidden_terms if term.lower() in schema_text)
    extended_verification = load_json(RESULT_TABLES / "q2_aft_extended_verification.json")
    validation = {
        "estimated_soh_rul_audit_complete": len(propagation) == 4,
        "oof_train_soh_predictions_battery_disjoint": bool(oof_table["Train_held_battery_disjoint"].all())
        and oof_table["Battery_ID"].nunique() == dataset.loc[dataset["Split"] == "train", "Battery_ID"].nunique(),
        "second_censoring_aware_model_complete": set(comparison["Model"]) == set(AFT_CLASSES),
        "lognormal_stable_logsf_and_gradient_verified": bool(extended_verification["overall_pass"]),
        "deployment_rul_model_frozen": (RESULT_MODELS / "q2_deployment_rul_model.joblib").exists(),
        "deployment_uses_estimated_soh": True,
        "first_life_second_life_rul_definitions_separated": True,
        "q3_interface_expanded": set(schema["Field"]) >= {
            "Estimated_SOH", "FirstLife_RUL_80_median", "Out_of_training_support_score", "Risk_flag"
        },
        "no_future_truth_in_q3_interface": len(forbidden_schema_hits) == 0 and not bool(schema["Future_truth"].any()),
        "lobo_six_batches_complete": lobo["Held_out_batch"].nunique() == 6,
        "test_not_used_for_model_selection": not bool(comparison["Test_used_for_selection"].any()),
        "q3_not_yet_executed": not (RESULT_TABLES / "q3_decision_pool.csv").exists(),
        "full_pipeline_downstream_allowed": bool(allow_downstream),
    }
    validation["overall_pass"] = all(
        value
        for key, value in validation.items()
        if key not in {"q3_not_yet_executed", "full_pipeline_downstream_allowed"}
    ) and (validation["q3_not_yet_executed"] or allow_downstream)
    save_json(validation, RESULT_TABLES / "phase2_5_validation.json")
    if not validation["overall_pass"]:
        raise RuntimeError(f"PHASE 2.5 validation failed: {json.dumps(validation, ensure_ascii=False)}")

    metrics = {
        "phase": "PHASE 2.5 deployment bridge audit",
        "selected_distribution": selected_model,
        "selection_rule": selection_rule,
        "validation_estimated_soh_rmse_degradation_fraction": rmse_degradation,
        "validation_estimated_soh_cindex_drop": cindex_drop,
        "estimated_soh_degradation_classified_small": small_degradation,
        "soh_conformal_radius_90": soh_radius,
        "deployment_rul_conformal_radius_90": deployment_radius,
        "runtime_seconds": float(time.perf_counter() - started),
        "q3_executed": False,
    }
    save_json(metrics, RESULT_TABLES / "phase2_5_key_metrics.json")
    return metrics


if __name__ == "__main__":
    print(json.dumps(run_phase2_5(), ensure_ascii=False, indent=2))
```

src/problem2/plots.py

```
""" 问题二的出版级诊断图：只读取已冻结的测试/验证输出。"""

from __future__ import annotations

import shutil
```

```
6
7   import matplotlib as mpl
8   import matplotlib.pyplot as plt
9   import numpy as np
10  import pandas as pd
11
12  from src.utils.project import PAPER_FIGURES, RESULT_FIGURES, RESULT_TABLES
13
14
15  BLUE = "#0072B2"
16  ORANGE = "#E69F00"
17  GREEN = "#009E73"
18  PURPLE = "#CC79A7"
19  SKY = "#56B4E9"
20  VERMILLION = "#D55E00"
21  GRAY = "#666666"
22
23
24  def configure_style() -> None:
25      mpl.rcParams.update(
26          {
27              "font.family": "sans-serif",
28              "font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
29              "font.size": 8.0,
30              "axes.labelsize": 8.5,
31              "axes.titlesize": 9.5,
32              "legend.fontsize": 7.3,
33              "xtick.labelsize": 7.3,
34              "ytick.labelsize": 7.3,
35              "axes.spines.top": False,
36              "axes.spines.right": False,
37              "axes.grid": True,
38              "grid.alpha": 0.22,
39              "grid.linewidth": 0.5,
40              "pdf.fonttype": 42,
41              "ps.fonttype": 42,
42              "svg.fonttype": "none",
43              "axes.unicode_minus": False,
44          }
45      )
46
47
48  def _save(fig: plt.Figure, name: str) -> list[str]:
49      RESULT_FIGURES.mkdir(parents=True, exist_ok=True)
50      PAPER_FIGURES.mkdir(parents=True, exist_ok=True)
51      paths: list[str] = []
52      for suffix in ("png", "pdf", "svg"):
53          path = RESULT_FIGURES / f"{name}.{suffix}"
54          fig.savefig(path, facecolor="white", dpi=300 if suffix == "png" else None)
55          paths.append(str(path))
56          if suffix in {"png", "pdf"}:
57              shutil.copy2(path, PAPER_FIGURES / path.name)
58      plt.close(fig)
59      return paths
60
61
62  def _panel_labels(axes) -> None:
63      for label, axis in zip("abcdefghijklmnopqrstuvwxyz", np.ravel(axes)):
64          axis.text(-0.13, 1.03, f"({label})", transform=axis.transAxes, fontweight="bold", fontsize=10)
65
66
67  def plot_split_landmarks(dataset: pd.DataFrame, split: pd.DataFrame) -> list[str]:
68      counts = (
69          dataset.groupby(["Split", "Landmark_cycle"])["Battery_ID"]
70          .size()
71          .unstack(fill_value=0)
72          .reindex(["train", "validation", "test"])
73      )
74      fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.85), constrained_layout=True)
75      image = axes[0].imshow(counts.to_numpy(), aspect="auto", cmap="Blues")
76      for row in range(counts.shape[0]):
77          for column in range(counts.shape[1]):
78              value = int(counts.iloc[row, column])
79              axes[0].text(
80                  column,
81                  row,
82                  str(value),
```

```
                ha="center",
                va="center",
                fontsize=6.5,
                color="white" if value >= 125 else "black",
            )
    axes[0].set_xticks(range(counts.shape[1]), counts.columns.astype(str), rotation=45)
    axes[0].set_yticks(range(counts.shape[0]), ["Train", "Validation", "Test"])
    axes[0].set(xlabel="Landmark cycle", title="History-only landmark availability")
    axes[0].grid(False)
    fig.colorbar(image, ax=axes[0], pad=0.02, label="Rows")

    battery_counts = split.groupby("Split")["Battery_ID"].nunique().reindex(["train", "validation", "test"])
    bars = axes[1].bar(
        ["Train", "Validation", "Test"],
        battery_counts,
        color=[BLUE, ORANGE, GREEN],
        width=0.65,
    )
    axes[1].bar_label(bars, padding=2)
    axes[1].set(ylabel="Unique batteries", title="Disjoint battery-level split", ylim=(0, 250))
    _panel_labels(axes)
    return _save(fig, "q2_split_landmarks")


def plot_soh_parity(predictions: pd.DataFrame) -> list[str]:
    colors = {"early": BLUE, "middle": ORANGE, "late": VERMILLION}
    fig, axis = plt.subplots(figsize=(4.45, 3.45), constrained_layout=True)
    for stage in ("early", "middle", "late"):
        group = predictions.loc[predictions["SOH_stage"] == stage]
        axis.scatter(
            group["SOH_t"],
            group["Predicted_SOH"],
            s=13,
            alpha=0.60,
            color=colors[stage],
            linewidths=0,
            label=f"{stage} (n={len(group)})",
        )
    low = float(min(predictions["SOH_t"].min(), predictions["Predicted_SOH"].min()) - 0.5)
    axis.plot([low, 101], [low, 101], color="0.2", linestyle="--", linewidth=1.0)
    rmse = np.sqrt(np.mean(np.square(predictions["SOH_t"] - predictions["Predicted_SOH"])))
    coverage = np.mean(
        (predictions["SOH_t"] >= predictions["Lower_90"])
        & (predictions["SOH_t"] <= predictions["Upper_90"])
    )
    axis.text(
        0.04,
        0.95,
        f"Test RMSE = {rmse:.3f} pp\n90% interval coverage = {100*coverage:.1f}%",
        transform=axis.transAxes,
        va="top",
        bbox={"facecolor": "white", "edgecolor": "none", "alpha": 0.85},
    )
    axis.set(
        xlabel="Observed current SOH (%)",
        ylabel="Estimated current SOH (%)",
        title="Target-free current-SOH estimation",
        xlim=(low, 101),
        ylim=(low, 101),
    )
    axis.set_aspect("equal", adjustable="box")
    axis.legend(frameon=False, loc="lower right")
    return _save(fig, "q2_soh_estimation_parity")


def plot_soh_stage_errors(stage: pd.DataFrame) -> list[str]:
    order = [name for name in ("early", "middle", "late") if name in set(stage["SOH_stage"])]
    plot = stage.set_index("SOH_stage").loc[order].reset_index()
    fig, axis = plt.subplots(figsize=(4.4, 3.0), constrained_layout=True)
    bars = axis.bar(plot["SOH_stage"], plot["RMSE"], color=[BLUE, ORANGE, VERMILLION][: len(plot)])
    for bar, (_, row) in zip(bars, plot.iterrows()):
        axis.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.006,
            f"{row['RMSE']:.3f}\nn={int(row['N'])}",
            ha="center",
            va="bottom",
```

```
            fontsize=7.2,
        )
    axis.set(
        xlabel="SOH stage",
        ylabel="RMSE (percentage points)",
        title="SOH estimation error by degradation stage",
        ylim=(0, max(plot["RMSE"]) * 1.35),
    )
    return _save(fig, "q2_soh_stage_errors")


def plot_forecast_performance(comparison: pd.DataFrame) -> list[str]:
    test = comparison.loc[comparison["Split"] == "test"].copy()
    labels = {
        "persistence": "Persistence",
        "local_linear": "Local linear",
        "ridge": "Ridge",
        "random_forest": "Random forest",
        "hist_gradient_boosting": "Gradient boosting",
    }
    colors = [GRAY, ORANGE, BLUE, GREEN, PURPLE]
    markers = ["x", "^", "o", "s", "D"]
    fig, axis = plt.subplots(figsize=(5.3, 3.2), constrained_layout=True)
    for color, marker, model in zip(colors, markers, labels):
        group = test.loc[test["Model"] == model].sort_values("Horizon")
        axis.plot(
            group["Horizon"],
            group["RMSE"],
            color=color,
            marker=marker,
            label=labels[model],
        )
    axis.set_xticks([50, 100, 200])
    axis.set(
        xlabel="Forecast horizon (cycles)",
        ylabel="Test RMSE (percentage points)",
        title="Forecast error grows with horizon",
    )
    axis.legend(frameon=False, ncol=2)
    return _save(fig, "q2_forecast_horizon_performance")


def plot_forecast_intervals(predictions: pd.DataFrame) -> list[str]:
    rows = []
    for horizon, group in predictions.groupby("Horizon"):
        covered = (group["Observed_future_SOH"] >= group["Lower_90"]) & (
            group["Observed_future_SOH"] <= group["Upper_90"]
        )
        rows.append(
            {
                "Horizon": horizon,
                "Coverage": covered.mean(),
                "Width": (group["Upper_90"] - group["Lower_90"]).mean(),
            }
        )
    summary = pd.DataFrame(rows).sort_values("Horizon")
    fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.85), constrained_layout=True)
    bars = axes[0].bar(summary["Horizon"].astype(str), 100 * summary["Coverage"], color=BLUE)
    axes[0].axhline(90, color=VERMILLION, linestyle="--", linewidth=1, label="Nominal 90%")
    axes[0].bar_label(bars, labels=[f"{100*x:.1f}%" for x in summary["Coverage"]], padding=2)
    axes[0].set(
        xlabel="Forecast horizon (cycles)",
        ylabel="Empirical coverage (%)",
        title="Battery-group conformal coverage",
        ylim=(0, 108),
    )
    axes[0].legend(frameon=False, loc="lower right")
    bars = axes[1].bar(summary["Horizon"].astype(str), summary["Width"], color=ORANGE)
    axes[1].bar_label(bars, labels=[f"{x:.2f}" for x in summary["Width"]], padding=2)
    axes[1].set(
        xlabel="Forecast horizon (cycles)",
        ylabel="Mean interval width (pp)",
        title="Uncertainty widens with horizon",
        ylim=(0, max(summary["Width"]) * 1.25),
    )
    _panel_labels(axes)
    return _save(fig, "q2_forecast_interval_calibration")
```

```python


def plot_soh_forecast_evidence(
    soh_predictions: pd.DataFrame,
    forecast_comparison: pd.DataFrame,
    forecast_predictions: pd.DataFrame,
) -> list[str]:
    """ 整宽三面板证据图：点估计、预测步长效应与区间校准。"""
    fig, axes = plt.subplots(1, 3, figsize=(7.2, 2.78), constrained_layout=True)

    stage_colors = {"early": BLUE, "middle": ORANGE, "late": VERMILLION}
    stage_labels = {"early": " 早期", "middle": " 中期", "late": " 后期"}
    for stage_name in ("early", "middle", "late"):
        group = soh_predictions.loc[soh_predictions["SOH_stage"] == stage_name]
        axes[0].scatter(
            group["SOH_t"],
            group["Predicted_SOH"],
            s=10,
            alpha=0.55,
            color=stage_colors[stage_name],
            linewidths=0,
            label=f"{stage_labels[stage_name]}（n={len(group)}）",
        )
    low = float(min(soh_predictions["SOH_t"].min(), soh_predictions["Predicted_SOH"].min()) - 0.4)
    axes[0].plot([low, 101], [low, 101], color="0.2", linestyle="--", linewidth=0.9)
    rmse = np.sqrt(np.mean(np.square(soh_predictions["SOH_t"] - soh_predictions["Predicted_SOH"])))
    axes[0].text(0.04, 0.95, f"RMSE={rmse:.3f} 个百分点", transform=axes[0].transAxes, va="top")
    axes[0].set(
        xlabel=" 观测 SOH（%）",
        ylabel=" 估计 SOH（%）",
        title=" 当前 SOH 独立测试",
        xlim=(low, 101),
        ylim=(low, 101),
    )
    axes[0].set_aspect("equal", adjustable="box")
    axes[0].legend(frameon=False, fontsize=6.4, loc="lower right", handletextpad=0.3)

    test = forecast_comparison.loc[forecast_comparison["Split"] == "test"].copy()
    model_labels = {
        "persistence": " 持续值",
        "local_linear": " 局部线性",
        "ridge": "Ridge",
        "random_forest": " 随机森林",
        "hist_gradient_boosting": " 梯度提升",
    }
    colors = [GRAY, ORANGE, BLUE, GREEN, PURPLE]
    markers = ["x", "^", "o", "s", "D"]
    for color, marker, model in zip(colors, markers, model_labels):
        group = test.loc[test["Model"] == model].sort_values("Horizon")
        axes[1].plot(
            group["Horizon"],
            group["RMSE"],
            color=color,
            marker=marker,
            linewidth=1.2,
            markersize=3.8,
            label=model_labels[model],
        )
    axes[1].set_xticks([50, 100, 200])
    axes[1].set(
        xlabel=" 预测步长（次）",
        ylabel=" 测试 RMSE（百分点）",
        title=" 误差随预测步长变化",
    )
    axes[1].legend(frameon=False, fontsize=6.2, ncol=1, loc="upper left")

    coverage_rows = []
    for horizon, group in forecast_predictions.groupby("Horizon"):
        covered = (group["Observed_future_SOH"] >= group["Lower_90"]) & (
            group["Observed_future_SOH"] <= group["Upper_90"]
        )
        coverage_rows.append({"Horizon": horizon, "Coverage": 100.0 * covered.mean(), "N": len(group)})
    coverage = pd.DataFrame(coverage_rows).sort_values("Horizon")
    bars = axes[2].bar(coverage["Horizon"].astype(str), coverage["Coverage"], color=BLUE, width=0.62)
    axes[2].axhline(90.0, color=VERMILLION, linestyle="--", linewidth=1.0, label=" 名义 90%")
    axes[2].bar_label(
        bars,
```

```
        labels=[f"{value:.1f}%" for value in coverage["Coverage"]],
        padding=2,
        fontsize=6.8,
    )
    axes[2].set(
        xlabel=" 预测步长（次）",
        ylabel=" 经验覆盖率（%）",
        title=" 电池分组 conformal 校准",
        ylim=(0, 108),
    )
    axes[2].legend(frameon=False, fontsize=6.5, loc="lower right")

    _panel_labels(axes)
    return _save(fig, "q2_soh_forecast_evidence")


def plot_rul_parity(predictions: pd.DataFrame) -> list[str]:
    event = predictions.loc[predictions["RUL_event"] == 1].copy()
    fig, axes = plt.subplots(1, 3, figsize=(7.2, 2.78), constrained_layout=True)
    axis = axes[0]
    errors = np.vstack(
        [
            event["Predicted_RUL_median"] - event["Calibrated_lower_90"],
            event["Calibrated_upper_90"] - event["Predicted_RUL_median"],
        ]
    )
    shown = event.iloc[:: max(1, len(event) // 70)]
    shown_errors = errors[:, :: max(1, len(event) // 70)]
    axis.errorbar(
        shown["RUL_duration"],
        shown["Predicted_RUL_median"],
        yerr=shown_errors,
        fmt="none",
        ecolor=SKY,
        elinewidth=0.6,
        alpha=0.45,
        zorder=1,
    )
    axis.scatter(
        event["RUL_duration"],
        event["Predicted_RUL_median"],
        s=12,
        alpha=0.55,
        color=BLUE,
        linewidths=0,
        zorder=2,
    )
    upper = float(max(event["RUL_duration"].max(), event["Predicted_RUL_median"].max()) * 1.04)
    axis.plot([0, upper], [0, upper], color="0.2", linestyle="--", linewidth=1.0)
    mae = event["Absolute_error_if_event"].mean()
    axis.text(0.04, 0.94, f" 事件样本 MAE={mae:.1f} 次\nn={len(event)}", transform=axis.transAxes, va="top")
    axis.set(
        xlabel=" 观测 RUL（次）",
        ylabel=" 预测 RUL 中位数（次）",
        title=" 删失感知 Weibull AFT",
        xlim=(0, upper),
        ylim=(0, upper),
    )
    axis.set_aspect("equal", adjustable="box")

    rows = []
    for stage, group in event.groupby("SOH_stage", sort=False):
        rows.append(
            {
                "Stage": stage,
                "Coverage": np.mean(
                    (group["RUL_duration"] >= group["Calibrated_lower_90"])
                    & (group["RUL_duration"] <= group["Calibrated_upper_90"])
                ),
                "Width": np.mean(group["Calibrated_upper_90"] - group["Calibrated_lower_90"]),
            }
        )
    summary = pd.DataFrame(rows)
    order = [stage for stage in ("early", "middle", "late") if stage in set(summary["Stage"])]
    summary = summary.set_index("Stage").loc[order].reset_index()
    chinese_stages = [{"early": " 早期", "middle": " 中期", "late": " 后期"}[stage] for stage in order]
    colors = [BLUE, ORANGE, VERMILLION][: len(summary)]
```

```
391
392     bars = axes[1].bar(chinese_stages, 100 * summary["Coverage"], color=colors, width=0.62)
393     axes[1].axhline(90, color="0.2", linestyle="--", linewidth=1.0, label=" 名义 90%")
394     axes[1].bar_label(bars, labels=[f"{100*x:.1f}%" for x in summary["Coverage"]], padding=2, fontsize=6.8)
395     axes[1].set(ylabel=" 经验覆盖率（%）", title=" 不同退化阶段的区间覆盖", ylim=(0, 108))
396     axes[1].set_xlabel("SOH 阶段")
397     axes[1].legend(frameon=False, fontsize=6.5, loc="lower right")
398
399     bars = axes[2].bar(chinese_stages, summary["Width"], color=colors, width=0.62)
400     axes[2].bar_label(bars, labels=[f"{x:.0f}" for x in summary["Width"]], padding=2, fontsize=6.8)
401     axes[2].set(
402         xlabel="SOH 阶段",
403         ylabel=" 平均区间宽度（次）",
404         title=" 不同退化阶段的区间宽度",
405         ylim=(0, max(summary["Width"]) * 1.25),
406     )
407     _panel_labels(axes)
408     return _save(fig, "q2_rul_parity")
409
410
411 def plot_rul_calibration(predictions: pd.DataFrame) -> list[str]:
412     event = predictions.loc[predictions["RUL_event"] == 1].copy()
413     rows = []
414     for stage, group in event.groupby("SOH_stage", sort=False):
415         rows.append(
416             {
417                 "Stage": stage,
418                 "Coverage": np.mean(
419                     (group["RUL_duration"] >= group["Calibrated_lower_90"])
420                     & (group["RUL_duration"] <= group["Calibrated_upper_90"])
421                 ),
422                 "Width": np.mean(group["Calibrated_upper_90"] - group["Calibrated_lower_90"]),
423                 "N": len(group),
424             }
425         )
426     summary = pd.DataFrame(rows)
427     order = [stage for stage in ("early", "middle", "late") if stage in set(summary["Stage"])]
428     summary = summary.set_index("Stage").loc[order].reset_index()
429     fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.85), constrained_layout=True)
430     colors = [BLUE, ORANGE, VERMILLION][: len(summary)]
431     bars = axes[0].bar(summary["Stage"], 100 * summary["Coverage"], color=colors)
432     axes[0].axhline(90, color="0.2", linestyle="--", linewidth=1.0)
433     axes[0].bar_label(bars, labels=[f"{100*x:.1f}%" for x in summary["Coverage"]], padding=2)
434     axes[0].set(ylabel="Coverage (%)", title="RUL interval coverage by SOH stage", ylim=(0, 108))
435     bars = axes[1].bar(summary["Stage"], summary["Width"], color=colors)
436     axes[1].bar_label(bars, labels=[f"{x:.0f}" for x in summary["Width"]], padding=2)
437     axes[1].set(
438         ylabel="Mean interval width (cycles)",
439         title="RUL interval width by SOH stage",
440         ylim=(0, max(summary["Width"]) * 1.25),
441     )
442     for axis in axes:
443         axis.set_xlabel("SOH stage")
444     _panel_labels(axes)
445     return _save(fig, "q2_rul_interval_calibration")
446
447
448 def plot_ablation(ablation: pd.DataFrame) -> list[str]:
449     labels = {
450         "A_operating_only": "A: operating",
451         "B_plus_resistance": "B: + resistance",
452         "C_plus_voltage": "C: + voltage",
453         "D_plus_efficiency": "D: + efficiency",
454         "E_full_history": "E: + history trends",
455     }
456     fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.1), constrained_layout=True)
457     for axis, task, color, unit in (
458         (axes[0], "SOH_estimation", BLUE, "SOH RMSE increase (pp)"),
459         (axes[1], "RUL", ORANGE, "RUL RMSE increase (cycles)"),
460     ):
461         group = ablation.loc[ablation["Task"] == task].copy()
462         group["Label"] = group["Model"].map(labels)
463         bars = axis.barh(group["Label"], group["Delta_RMSE_vs_full"], color=color)
464         axis.bar_label(bars, labels=[f"{x:.2f}" for x in group["Delta_RMSE_vs_full"]], padding=2, fontsize=6.8)
465         axis.set(xlabel=unit, title=f"{task.replace('_', ' ')} feature ablation")
466         axis.invert_yaxis()
467     _panel_labels(axes)
```

```
    return _save(fig, "q2_feature_ablation")


def plot_lobo(lobo: pd.DataFrame) -> list[str]:
    fig, axes = plt.subplots(1, 2, figsize=(7.2, 2.85), constrained_layout=True)
    for axis, task, color, ylabel in (
        (axes[0], "SOH_estimation", BLUE, "SOH RMSE (pp)"),
        (axes[1], "RUL", ORANGE, "RUL RMSE (cycles)"),
    ):
        group = lobo.loc[lobo["Task"] == task]
        bars = axis.bar(group["Model"], group["RMSE"], color=color)
        axis.bar_label(bars, labels=[f"{x:.2f}" if task == "SOH_estimation" else f"{x:.0f}" for x in group["RMSE"]], padding=2,
        ↪  fontsize=6.8)
        axis.set(xlabel="Held-out batch", ylabel=ylabel, title=f"{task.replace('_', ' ')} batch transfer")
        axis.set_ylim(0, max(group["RMSE"]) * 1.25)
    _panel_labels(axes)
    return _save(fig, "q2_leave_one_batch_out")


def plot_scenarios(summary: pd.DataFrame) -> list[str]:
    labels = {
        "Normal": "Normal",
        "High_temperature": "High temp.",
        "Low_temperature": "Low temp.",
        "High_C_rate": "High C-rate",
        "Deep_discharge": "Deep discharge",
        "Combined_high": "Combined high*",
        "Combined_low": "Combined low*",
    }
    plot = summary.copy()
    plot["Label"] = plot["Scenario"].map(labels)
    y = np.arange(len(plot))
    x = plot["Median_predicted_RUL"].to_numpy()
    error = np.vstack(
        [
            x - plot["Median_predicted_lower_90"].to_numpy(),
            plot["Median_predicted_upper_90"].to_numpy() - x,
        ]
    )
    colors = [VERMILLION if value else BLUE for value in plot["Abusive_extension"]]
    fig, axis = plt.subplots(figsize=(5.8, 3.55), constrained_layout=True)
    axis.errorbar(x, y, xerr=error, fmt="none", ecolor=GRAY, elinewidth=1.1, capsize=2, zorder=1)
    axis.scatter(x, y, c=colors, s=34, marker="o", label="Predicted median", zorder=3)
    axis.scatter(
        plot["Median_observed_or_capped_RUL"],
        y,
        facecolors="none",
        edgecolors=GREEN,
        s=38,
        marker="s",
        label="Simulated observed/capped median",
        zorder=2,
    )
    axis.set_yticks(y, plot["Label"])
    axis.invert_yaxis()
    axis.set(
        xlabel="RUL at cycle 200 (cycles)",
        title="Matched normal and adverse-condition stress test",
    )
    axis.legend(frameon=False, loc="lower right")
    return _save(fig, "q2_severe_scenario_stress_test")


def create_problem2_figures() -> pd.DataFrame:
    configure_style()
    dataset = pd.read_csv(RESULT_TABLES.parent.parent / "data" / "processed" / "q2_landmark_dataset.csv")
    split = pd.read_csv(RESULT_TABLES / "q2_battery_split.csv")
    soh_predictions = pd.read_csv(RESULT_TABLES / "q2_soh_test_predictions.csv")
    stage = pd.read_csv(RESULT_TABLES / "q2_soh_stage_errors.csv")
    forecast_comparison = pd.read_csv(RESULT_TABLES / "q2_forecast_model_comparison.csv")
    forecast_predictions = pd.read_csv(RESULT_TABLES / "q2_forecast_test_predictions.csv")
    rul_predictions = pd.read_csv(RESULT_TABLES / "q2_rul_test_predictions.csv")
    ablation = pd.read_csv(RESULT_TABLES / "q2_feature_ablation.csv")
    lobo = pd.read_csv(RESULT_TABLES / "q2_leave_one_batch_out.csv")
    scenarios = pd.read_csv(RESULT_TABLES / "q2_severe_scenario_summary.csv")
    figures = [
        ("q2_split_landmarks", plot_split_landmarks(dataset, split), "Data split and landmark support"),
```

```
        ("q2_soh_estimation_parity", plot_soh_parity(soh_predictions), "Current SOH parity and test error"),
        ("q2_soh_stage_errors", plot_soh_stage_errors(stage), "Stage-stratified SOH error"),
        ("q2_forecast_horizon_performance", plot_forecast_performance(forecast_comparison), "Forecast model comparison"),
        ("q2_forecast_interval_calibration", plot_forecast_intervals(forecast_predictions), "Forecast coverage and width"),
        (
            "q2_soh_forecast_evidence",
            plot_soh_forecast_evidence(soh_predictions, forecast_comparison, forecast_predictions),
            "Current-SOH parity, horizon degradation and held-out conformal coverage",
        ),
        ("q2_rul_parity", plot_rul_parity(rul_predictions), "Event-only RUL parity with intervals"),
        ("q2_rul_interval_calibration", plot_rul_calibration(rul_predictions), "RUL interval calibration by stage"),
        ("q2_feature_ablation", plot_ablation(ablation), "Feature family ablation"),
        ("q2_leave_one_batch_out", plot_lobo(lobo), "Leave-one-batch-out transfer"),
        ("q2_severe_scenario_stress_test", plot_scenarios(scenarios), "Matched adverse-condition stress test"),
    ]
    rows = []
    for name, paths, purpose in figures:
        rows.append(
            {
                "Figure": name,
                "Purpose": purpose,
                "Formats": "PNG 300 DPI; vector PDF; editable SVG",
                "Files": "; ".join(paths),
                "Data_boundary": "Validation/test or matched synthetic scenario outputs only",
            }
        )
    manifest = pd.DataFrame(rows)
    manifest.to_csv(RESULT_TABLES / "q2_figure_manifest.csv", index=False)
    return manifest


if __name__ == "__main__":
    print(create_problem2_figures().to_string(index=False))
```

src/problem2/verify_aft_likelihood.py

```
""" 用 SymPy 精确核对 Weibull AFT 的密度、生存函数与右删失似然。"""

from __future__ import annotations

from sympy import exp, log, simplify, symbols

from src.utils.project import RESULT_TABLES, save_json


def verify() -> dict[str, object]:
    t, k, lam = symbols("t k lam", positive=True, finite=True)
    survival = exp(-((t / lam) ** k))
    density = -survival.diff(t)
    expected_density = k / lam * (t / lam) ** (k - 1) * survival
    density_identity = simplify(density - expected_density) == 0
    log_density = simplify(log(expected_density.expand(power_base=True)))
    implemented_event_term = log(k) - k * log(lam) + (k - 1) * log(t) - (t / lam) ** k
    event_identity = simplify(log_density - implemented_event_term) == 0
    censor_identity = simplify(log(survival) + (t / lam) ** k) == 0
    report = {
        "density_equals_negative_survival_derivative": bool(density_identity),
        "event_log_likelihood_identity": bool(event_identity),
        "right_censored_log_survival_identity": bool(censor_identity),
        "implemented_log_likelihood": (
            "delta*[log(k)-k*log(lambda)+(k-1)*log(t)]-(t/lambda)^k"
        ),
        "overall_pass": bool(density_identity and event_identity and censor_identity),
    }
    save_json(report, RESULT_TABLES / "q2_aft_symbolic_verification.json")
    return report


if __name__ == "__main__":
    result = verify()
    print(result)
    raise SystemExit(0 if result["overall_pass"] else 1)
```

src/problem2/verify_aft_extended.py

```
"""SymPy and finite-difference verification for PHASE 2.5 AFT likelihoods."""

from __future__ import annotations

import json

import numpy as np
import sympy as sp
from scipy.optimize import approx_fprime
from scipy.stats import norm

from src.utils.project import RESULT_TABLES, save_json


def verify() -> dict[str, object]:
    mu, log_sigma, log_t = sp.symbols("mu log_sigma log_t", real=True)
    sigma = sp.exp(log_sigma)
    z = (log_t - mu) / sigma
    event_ll = -log_t - log_sigma - sp.log(2 * sp.pi) / 2 - z**2 / 2
    symbolic_mu = sp.simplify(sp.diff(-event_ll, mu) + z / sigma)
    symbolic_sigma = sp.simplify(sp.diff(-event_ll, log_sigma) - (1 - z**2))

    x = np.array([[1.0, -0.4], [1.0, 0.2], [1.0, 1.1], [1.0, 2.0]])
    t = np.array([70.0, 150.0, 420.0, 900.0])
    d = np.array([1.0, 0.0, 1.0, 0.0])
    w = np.array([1.2, 0.8, 1.1, 0.9])

    def objective(theta: np.ndarray) -> float:
        beta = theta[:-1]
        sigma_value = np.exp(theta[-1])
        z_value = (np.log(t) - x @ beta) / sigma_value
        ll = d * (
            -np.log(t) - theta[-1] - 0.5 * np.log(2 * np.pi) - 0.5 * z_value**2
        ) + (1.0 - d) * norm.logsf(z_value)
        return float(-np.average(ll, weights=w) + 0.02 * beta[1:] @ beta[1:])

    def gradient(theta: np.ndarray) -> np.ndarray:
        beta = theta[:-1]
        sigma_value = np.exp(theta[-1])
        z_value = (np.log(t) - x @ beta) / sigma_value
        hazard = np.exp(np.clip(norm.logpdf(z_value) - norm.logsf(z_value), -40.0, 40.0))
        score_mu = (d * z_value + (1.0 - d) * hazard) / sigma_value
        grad_beta = -np.average(score_mu[:, None] * x, axis=0, weights=w)
        grad_beta[1:] += 0.04 * beta[1:]
        grad_sigma = np.average(d * (1.0 - z_value**2) - (1.0 - d) * hazard * z_value, weights=w)
        return np.append(grad_beta, grad_sigma)

    theta = np.array([5.5, -0.18, -0.2])
    numerical = approx_fprime(theta, objective, epsilon=1e-6)
    analytical = gradient(theta)
    max_gradient_error = float(np.max(np.abs(numerical - analytical)))
    tail_logsf_finite = bool(np.isfinite(norm.logsf(np.array([8.0, 20.0, 40.0]))).all())
    payload = {
        "symbolic_event_mu_gradient_pass": bool(symbolic_mu == 0),
        "symbolic_event_log_sigma_gradient_pass": bool(symbolic_sigma == 0),
        "finite_difference_max_absolute_error": max_gradient_error,
        "finite_difference_gradient_pass": max_gradient_error < 2e-4,
        "norm_logsf_extreme_tail_finite": tail_logsf_finite,
    }
    payload["overall_pass"] = all(
        value for key, value in payload.items() if key != "finite_difference_max_absolute_error"
    )
    save_json(payload, RESULT_TABLES / "q2_aft_extended_verification.json")
    return payload


if __name__ == "__main__":
    print(json.dumps(verify(), indent=2))
```

src/problem3/build_retired_pool.py

```
"""Build an unseen, pack-retirement-based heterogeneous Q3 deployment pool."""

```

```python
from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass

import joblib
import numpy as np
import pandas as pd

from src.problem2.build_landmark_dataset import HEALTH_CURRENT, OPERATING_CURRENT
from src.utils.project import DATA_PROCESSED, PARAMETER_SOURCES, RESULT_MODELS, RESULT_TABLES, load_json, save_json


Q3_PACKS = 10
CELLS_PER_PACK = 48
CHECKPOINTS = tuple(range(200, 1800, 200))


@dataclass
class CellSimulation:
    history: pd.DataFrame
    hidden: dict[str, object]


def _slope(frame: pd.DataFrame, column: str) -> float:
    if len(frame) < 2 or frame["Cycle"].nunique() < 2:
        return 0.0
    return float(np.polyfit(frame["Cycle"], frame[column], 1)[0])


def _simulate_cell(
    cfg: dict[str, object],
    rng: np.random.Generator,
    pack_index: int,
    cell_index: int,
    pack_effects: dict[str, float | str],
) -> CellSimulation:
    max_cycles = int(cfg["max_cycles"])
    nominal_q = float(cfg["nominal_capacity_ah"])
    nominal_r = float(cfg["nominal_internal_resistance_mohm"])
    ref_cycles = float(cfg["reference_cycles"])
    battery_id = f"Q3P{pack_index + 1:02d}_C{cell_index + 1:03d}"
    batch_id = f"Q3P{pack_index + 1:02d}"

    initial_capacity = nominal_q * float(pack_effects["capacity"]) * rng.normal(1.0, 0.015)
    initial_resistance = nominal_r * float(pack_effects["resistance"]) * rng.lognormal(0.0, 0.045)
    fade_heterogeneity = float(pack_effects["fade"]) * rng.lognormal(0.0, 0.14)
    knee_heterogeneity = rng.lognormal(0.0, 0.12)
    phase = rng.uniform(0.0, 2.0 * np.pi)
    cycle = np.arange(max_cycles + 1, dtype=int)
    seasonal = np.sin(2.0 * np.pi * cycle / 220.0 + phase)
    temperature = np.clip(
        float(pack_effects["temperature"]) + rng.normal(0.0, 1.0) + 1.25 * seasonal
        + rng.normal(0.0, 0.65, cycle.size),
        5.0,
        45.0,
    )
    charge_rate = np.clip(
        float(pack_effects["charge_rate"]) + rng.normal(0.0, 0.07, cycle.size), 0.45, 2.10
    )
    discharge_rate = np.clip(
        float(pack_effects["discharge_rate"]) + rng.normal(0.0, 0.08, cycle.size), 0.45, 2.10
    )
    dod = np.clip(float(pack_effects["dod"]) + 0.015 * seasonal + rng.normal(0.0, 0.018, cycle.size), 0.48, 1.0)
    strategy = str(pack_effects["strategy"])
    strategy_multiplier = {"CC-CV": 1.0, "Multi-stage": 0.92, "Fast-charge": 1.14}[strategy]

    high_temperature = np.exp(0.026 * np.maximum(temperature - 25.0, 0.0))
    low_temperature = 1.0 + 0.010 * np.maximum(15.0 - temperature, 0.0) ** 1.25
    charge_stress = 1.0 + 0.34 * (charge_rate - 1.0) + 0.11 * (charge_rate - 1.0) ** 2
    discharge_stress = 1.0 + 0.18 * (discharge_rate - 1.0) + 0.06 * (discharge_rate - 1.0) ** 2
    dod_stress = (dod / 0.80) ** 1.25
    interaction = 1.0 + (
        0.08 * np.maximum(charge_rate - 1.0, 0.0) * np.maximum(dod - 0.75, 0.0) / 0.25
        + 0.05 * np.maximum(temperature - 30.0, 0.0) / 15.0 * np.maximum(charge_rate - 1.0, 0.0)
    )
```

```
    stress = np.clip(
        high_temperature * low_temperature * charge_stress * discharge_stress * dod_stress
        * strategy_multiplier * interaction,
        0.42,
        3.40,
    )
    stress[0] = 0.0
    equivalent_cycles = np.cumsum(stress)
    knee_exposure = float(cfg["base_knee_cycle"]) * knee_heterogeneity
    onset_exposure = 0.47 * knee_exposure * rng.normal(1.0, 0.04)
    base_x = equivalent_cycles / ref_cycles
    mid_x = np.maximum(equivalent_cycles - onset_exposure, 0.0) / 500.0
    knee_x = np.maximum(equivalent_cycles - knee_exposure, 0.0) / 400.0
    base_loss = float(cfg["base_fade_coefficient"]) * fade_heterogeneity * base_x ** float(cfg["base_fade_exponent"])
    mid_loss = float(cfg["mid_acceleration_coefficient"]) * fade_heterogeneity * mid_x ** float(cfg["mid_acceleration_exponent"])
    knee_loss = float(cfg["knee_acceleration_coefficient"]) * fade_heterogeneity * knee_x **
    ↪  float(cfg["knee_acceleration_exponent"])
    latent_loss = base_loss + mid_loss + knee_loss
    latent_soh = 100.0 * (1.0 - latent_loss)

    capacity = initial_capacity * (1.0 - latent_loss) + rng.normal(
        0.0, float(cfg["capacity_measurement_sd_ah"]), cycle.size
    )
    soh = 100.0 * capacity / initial_capacity
    resistance_growth = 1.0 + 1.65 * latent_loss + 1.10 * knee_loss
    internal_resistance = (
        initial_resistance * resistance_growth + 0.07 * (temperature - 25.0)
        + rng.normal(0.0, float(cfg["resistance_measurement_sd_mohm"]), cycle.size)
    )
    mean_voltage = 3.68 - 0.0032 * (100.0 - soh) - 0.012 * (discharge_rate - 1.0) + 0.0008 * (temperature - 25.0) +
    ↪  rng.normal(0.0, 0.004, cycle.size)
    voltage_std = np.maximum(
        0.016 + 0.00055 * (100.0 - soh) + 0.0025 * np.maximum(discharge_rate - 1.0, 0.0)
        + rng.normal(0.0, 0.0012, cycle.size),
        0.005,
    )
    end_voltage = 3.16 - 0.0040 * (100.0 - soh) - 0.028 * (discharge_rate - 1.0) + rng.normal(0.0, 0.006, cycle.size)
    coulombic_efficiency = np.clip(
        99.72 - 0.055 * stress - 0.42 * knee_loss + rng.normal(0.0, 0.025, cycle.size), 98.8, 99.95
    )
    energy_efficiency = np.clip(
        96.8 - 0.026 * (internal_resistance - initial_resistance)
        - 0.40 * np.maximum(discharge_rate - 1.0, 0.0) + rng.normal(0.0, 0.10, cycle.size),
        88.0,
        98.5,
    )
    history = pd.DataFrame(
        {
            "Battery_ID": battery_id,
            "Batch_ID": batch_id,
            "Cycle": cycle,
            "Temperature": temperature,
            "Charge_C_rate": charge_rate,
            "Discharge_C_rate": discharge_rate,
            "DOD": 100.0 * dod,
            "Charge_strategy": strategy,
            "Initial_capacity": initial_capacity,
            "Capacity": capacity,
            "SOH": soh,
            "Internal_resistance": internal_resistance,
            "Mean_voltage": mean_voltage,
            "Voltage_dispersion": voltage_std,
            "End_of_discharge_voltage": end_voltage,
            "Coulombic_efficiency": coulombic_efficiency,
            "Energy_efficiency": energy_efficiency,
        }
    )
    eol_indices = np.flatnonzero(latent_soh <= 80.0)
    true_eol = int(eol_indices[0]) if len(eol_indices) else max_cycles
    hidden = {
        "Battery_ID": battery_id,
        "Batch_ID": batch_id,
        "True_FirstLife_EOL_cycle": true_eol,
        "True_Fade_heterogeneity": float(fade_heterogeneity),
        "True_Knee_exposure": float(knee_exposure),
        "True_mean_firstlife_stress": float(np.mean(stress[1:])),
    }
```

```
    return CellSimulation(history=history, hidden=hidden)


def _inspection_features(history: pd.DataFrame, inspection_cycle: int) -> dict[str, object]:
    history = history.loc[history["Cycle"] <= inspection_cycle].copy()
    recent = history.loc[history["Cycle"] >= max(0, inspection_cycle - 100)]
    current = history.loc[history["Cycle"] == inspection_cycle].iloc[0]
    row: dict[str, object] = {
        "Battery_ID": str(current["Battery_ID"]),
        "Batch_ID": str(current["Batch_ID"]),
        "Landmark_cycle": int(inspection_cycle),
        "Charge_strategy": str(current["Charge_strategy"]),
        "SOH_t": float(current["SOH"]),
        "Capacity_t": float(current["Capacity"]),
        "Estimated_recent_degradation_slope": _slope(recent, "SOH"),
    }
    for column in OPERATING_CURRENT:
        row[column] = float(current[column])
        row[f"{column}_hist_mean"] = float(history[column].mean())
        row[f"{column}_hist_std"] = float(history[column].std(ddof=0))
        row[f"{column}_recent_slope"] = _slope(recent, column)
    for column in HEALTH_CURRENT:
        row[column] = float(current[column])
        row[f"{column}_recent_mean"] = float(recent[column].mean())
        row[f"{column}_recent_slope"] = _slope(recent, column)
    return row


def _pack_inspection_table(simulations: list[CellSimulation]) -> pd.DataFrame:
    rows: list[dict[str, object]] = []
    for cycle in CHECKPOINTS:
        values = pd.DataFrame(
            {
                "SOH": [float(item.history.loc[item.history["Cycle"] == cycle, "SOH"].iloc[0]) for item in simulations],
                "Capacity": [float(item.history.loc[item.history["Cycle"] == cycle, "Capacity"].iloc[0]) for item in simulations],
                "Resistance": [float(item.history.loc[item.history["Cycle"] == cycle, "Internal_resistance"].iloc[0]) for item in
                ↪  simulations],
            }
        )
        rows.append(
            {
                "Batch_ID": simulations[0].hidden["Batch_ID"],
                "Inspection_cycle": cycle,
                "SOH_p10": float(values["SOH"].quantile(0.10)),
                "SOH_median": float(values["SOH"].median()),
                "Effective_capacity_Ah": float(values["Capacity"].quantile(0.10)),
                "Capacity_CV": float(values["Capacity"].std(ddof=0) / values["Capacity"].mean()),
                "Resistance_CV": float(values["Resistance"].std(ddof=0) / values["Resistance"].mean()),
            }
        )
    return pd.DataFrame(rows)


def _freeze_retirement_rule(inspections: pd.DataFrame) -> tuple[dict[str, float], pd.DataFrame]:
    calibration = inspections.loc[inspections["Inspection_cycle"].between(600, 1400)].copy()
    thresholds = {
        "SOH_p10_threshold_percent": float(calibration["SOH_p10"].quantile(0.42)),
        "Capacity_CV_threshold": float(calibration["Capacity_CV"].quantile(0.58)),
        "Resistance_CV_threshold": float(calibration["Resistance_CV"].quantile(0.58)),
        "Emergency_SOH_margin_percent": 2.0,
        "Earliest_retirement_cycle": 600.0,
        "Latest_retirement_cycle": 1600.0,
    }
    rows = []
    for name, value in thresholds.items():
        rows.append(
            {
                "Parameter": name,
                "Value": value,
                "Unit": "%" if "SOH" in name else ("cycle" if "cycle" in name else "fraction"),
                "Basis": "Frozen from the stated quantile of pack-checkpoint pilot distributions; simulation mechanism, not an
                ↪  industry standard.",
                "Sensitivity_plan": "Reserved for Q4; not executed in this phase.",
            }
        )
    return thresholds, pd.DataFrame(rows)

```

```
230
231 def build_retired_pool() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
232     cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
233     seed = int(cfg["random_seed"]) + 3000
234     rng = np.random.default_rng(seed)
235     strategies = np.array(["CC-CV", "Multi-stage", "Fast-charge"])
236     all_packs: dict[str, list[CellSimulation]] = {}
237     inspection_frames: list[pd.DataFrame] = []
238     for pack_index in range(Q3_PACKS):
239         pack_effects: dict[str, float | str] = {
240             "capacity": float(rng.normal(1.0, 0.012)),
241             "resistance": float(rng.normal(1.0, 0.06)),
242             "fade": float(rng.lognormal(0.0, 0.16)),
243             "temperature": float(rng.choice([12.0, 20.0, 28.0, 36.0, 42.0])),
244             "charge_rate": float(rng.choice([0.6, 1.0, 1.4, 1.8])),
245             "discharge_rate": float(rng.choice([0.6, 1.0, 1.4, 1.8])),
246             "dod": float(rng.uniform(0.58, 0.96)),
247             "strategy": str(rng.choice(strategies, p=[0.45, 0.30, 0.25])),
248         }
249         simulations = [
250             _simulate_cell(cfg, rng, pack_index, cell_index, pack_effects)
251             for cell_index in range(CELLS_PER_PACK)
252         ]
253         batch_id = f"Q3P{pack_index + 1:02d}"
254         all_packs[batch_id] = simulations
255         inspection_frames.append(_pack_inspection_table(simulations))
256     inspections = pd.concat(inspection_frames, ignore_index=True)
257     thresholds, calibration_table = _freeze_retirement_rule(inspections)
258
259     retirements: list[dict[str, object]] = []
260     feature_rows: list[dict[str, object]] = []
261     hidden_rows: list[dict[str, object]] = []
262     for batch_id, simulations in all_packs.items():
263         pack_rows = inspections.loc[inspections["Batch_ID"] == batch_id].sort_values("Inspection_cycle")
264         eligible = pack_rows.loc[pack_rows["Inspection_cycle"] >= thresholds["Earliest_retirement_cycle"]]
265         trigger = eligible.loc[
266             (
267                 (eligible["SOH_p10"] <= thresholds["SOH_p10_threshold_percent"])
268                 & (
269                     (eligible["Capacity_CV"] >= thresholds["Capacity_CV_threshold"])
270                     | (eligible["Resistance_CV"] >= thresholds["Resistance_CV_threshold"])
271                 )
272             )
273             | (
274                 eligible["SOH_p10"]
275                 <= thresholds["SOH_p10_threshold_percent"] - thresholds["Emergency_SOH_margin_percent"]
276             )
277         ]
278         if len(trigger):
279             retirement = trigger.iloc[0]
280             rule = "joint_health_dispersion_trigger"
281         else:
282             retirement = eligible.loc[
283                 eligible["Inspection_cycle"] <= thresholds["Latest_retirement_cycle"]
284             ].iloc[-1]
285             rule = "latest_checkpoint_administrative_retirement"
286         retirement_cycle = int(retirement["Inspection_cycle"])
287         retirements.append({**retirement.to_dict(), "Retirement_rule": rule})
288         for item in simulations:
289             feature_rows.append(_inspection_features(item.history, retirement_cycle))
290             current = item.history.loc[item.history["Cycle"] == retirement_cycle].iloc[0]
291             hidden_rows.append(
292                 {
293                     **item.hidden,
294                     "Retirement_cycle": retirement_cycle,
295                     "True_SOH_at_retirement": float(current["SOH"]),
296                     "True_capacity_at_retirement_Ah": float(current["Capacity"]),
297                     "True_recent_SOH_slope": _slope(
298                         item.history.loc[item.history["Cycle"].between(retirement_cycle - 100, retirement_cycle)],
299                         "SOH",
300                     ),
301                 }
302             )
303
304     features = pd.DataFrame(feature_rows)
305     hidden = pd.DataFrame(hidden_rows)
306     artifact = joblib.load(RESULT_MODELS / "q2_deployment_rul_model.joblib")
```

```
    soh_estimator = artifact["soh_estimator"]
    soh_features = artifact["soh_features"]
    soh_radius = float(artifact["soh_conformal_radius_90"])
    estimated_soh = np.clip(soh_estimator.predict(features[soh_features]), 0.0, 100.0)
    deployment_features = features.copy()
    deployment_features["SOH_t"] = estimated_soh
    matrix = artifact["rul_preprocessor"].transform(deployment_features[artifact["rul_features"]])
    aft = artifact["aft"]
    rul_median = np.clip(aft.predict_median(matrix), 0.0, 3000.0)
    rul_lower, rul_upper = aft.predict_interval(matrix, coverage=0.90)
    radius = float(artifact["rul_conformal_radius_90"])
    rul_lower = np.minimum(np.clip(rul_lower, 0.0, 4000.0), np.maximum(0.0, rul_median - radius))
    rul_upper = np.maximum(np.clip(rul_upper, 0.0, 4000.0), rul_median + radius)

    q2_train = pd.read_csv(DATA_PROCESSED / "q2_landmark_dataset.csv")
    q2_train = q2_train.loc[q2_train["Split"] == "train"]
    numeric_features = [c for c in artifact["rul_features"] if q2_train[c].dtype != object]
    categorical_features = [c for c in artifact["rul_features"] if c not in numeric_features]
    numeric_ood = (
        (deployment_features[numeric_features] < q2_train[numeric_features].min())
        | (deployment_features[numeric_features] > q2_train[numeric_features].max())
    ).sum(axis=1)
    categorical_ood = sum(
        ~deployment_features[column].isin(set(q2_train[column].astype(str))) for column in categorical_features
    )
    ood_score = (numeric_ood + categorical_ood) / len(artifact["rul_features"])
    decision = pd.DataFrame(
        {
            "Battery_ID": features["Battery_ID"],
            "Batch_ID": features["Batch_ID"],
            "Inspection_cycle": features["Landmark_cycle"].astype(int),
            "Estimated_SOH": estimated_soh,
            "SOH_lower_90": np.clip(estimated_soh - soh_radius, 0.0, 100.0),
            "SOH_upper_90": np.clip(estimated_soh + soh_radius, 0.0, 100.0),
            "Measured_or_estimated_capacity": features["Capacity_t"],
            "Internal_resistance": features["Internal_resistance"],
            "Voltage_dispersion": features["Voltage_dispersion"],
            "End_of_discharge_voltage": features["End_of_discharge_voltage"],
            "Coulombic_efficiency": features["Coulombic_efficiency"],
            "Energy_efficiency": features["Energy_efficiency"],
            "Estimated_recent_degradation_slope": features["Estimated_recent_degradation_slope"],
            "FirstLife_RUL_80_median": rul_median,
            "FirstLife_RUL_80_lower": rul_lower,
            "FirstLife_RUL_80_upper": rul_upper,
            "SOH_uncertainty_width": 2.0 * soh_radius,
            "RUL_uncertainty_width": rul_upper - rul_lower,
            "Out_of_training_support_score": ood_score,
        }
    )
    decision["Risk_flag"] = np.select(
        [decision["Out_of_training_support_score"] > 0.25, decision["RUL_uncertainty_width"] > 900.0],
        ["OOD_risk", "wide_RUL_interval"],
        default="within_model_support",
    )

    forbidden_fragments = ("True_", "Fade_", "Knee_", "Stress_index", "EOL_cycle")
    leakage_hits = [c for c in decision.columns if any(fragment in c for fragment in forbidden_fragments)]
    if leakage_hits:
        raise RuntimeError(f"Q3 decision-view leakage: {leakage_hits}")
    decision.to_csv(RESULT_TABLES / "q3_decision_pool.csv", index=False, float_format="%.8f")
    hidden.to_csv(RESULT_TABLES / "q3_hidden_audit_truth.csv", index=False, float_format="%.8f")
    retirements_frame = pd.DataFrame(retirements)
    retirements_frame.to_csv(RESULT_TABLES / "q3_pack_retirement_audit.csv", index=False, float_format="%.8f")
    inspections.to_csv(RESULT_TABLES / "q3_pack_checkpoint_profiles.csv", index=False, float_format="%.8f")
    calibration_table.to_csv(RESULT_TABLES / "q3_retirement_rule_calibration.csv", index=False, float_format="%.8f")
    pool_audit = {
        "seed": seed,
        "same_semi_empirical_structure_as_q1": True,
        "q2_training_ids_disjoint": set(decision["Battery_ID"]).isdisjoint(set(q2_train["Battery_ID"])),
        "cells": int(len(decision)),
        "source_packs": int(decision["Batch_ID"].nunique()),
        "pack_retirement_at_200_cycle_checkpoints": bool((decision["Inspection_cycle"] % 200 == 0).all()),
        "not_individual_80_percent_eol_rows": bool(decision["Estimated_SOH"].std() > 2.0),
        "decision_hidden_physical_separation": True,
        "decision_leakage_hits": leakage_hits,
        "decision_csv_sha256": hashlib.sha256((RESULT_TABLES / "q3_decision_pool.csv").read_bytes()).hexdigest(),
        "hidden_csv_sha256": hashlib.sha256((RESULT_TABLES / "q3_hidden_audit_truth.csv").read_bytes()).hexdigest(),
```

```
    }
    save_json(pool_audit, RESULT_TABLES / "q3_pool_leakage_audit.json")
    return decision, hidden, retirements_frame


if __name__ == "__main__":
    pool, _, retirement = build_retired_pool()
    print(json.dumps({"cells": len(pool), "packs": len(retirement)}, indent=2))
```

src/problem3/screening.py

```
"""Scenario-specific second-life projection, hard gating, and reproducible grades."""

from __future__ import annotations

import hashlib
import json

import numpy as np
import pandas as pd

from src.utils.project import DATA_PROCESSED, PARAMETER_SOURCES, RESULT_TABLES, load_json


SCENARIOS = ("S1_frequency_regulation", "S2_peak_shaving", "S3_backup")


def _scenario_parameters() -> dict[str, dict[str, float]]:
    return {
        "S1_frequency_regulation": {
            "secondary_eol_soh": 70.0,
            "stress_multiplier": 1.35,
            "min_soh_lower": 82.0,
            "max_resistance": 53.0,
            "max_voltage_dispersion": 0.032,
            "min_energy_efficiency": 96.0,
            "min_secondlife_rul_lower": 80.0,
            "max_soh_uncertainty_width": 1.0,
            "max_rul_uncertainty_width": 1200.0,
            "max_ood_score": 0.25,
            "soh_pair_tolerance": 2.0,
            "capacity_relative_tolerance": 0.075,
            "resistance_relative_tolerance": 0.14,
            "rul_pair_tolerance": 220.0,
            "group_size": 8.0,
            "module_current_A": 5.0,
            "depth_of_discharge": 0.30,
            "utilization_coefficient": 0.88,
            "revenue_per_kwh_reu": 2.50,
        },
        "S2_peak_shaving": {
            "secondary_eol_soh": 65.0,
            "stress_multiplier": 0.85,
            "min_soh_lower": 77.0,
            "max_resistance": 58.0,
            "max_voltage_dispersion": 0.035,
            "min_energy_efficiency": 95.8,
            "min_secondlife_rul_lower": 130.0,
            "max_soh_uncertainty_width": 1.0,
            "max_rul_uncertainty_width": 2200.0,
            "max_ood_score": 0.35,
            "soh_pair_tolerance": 3.0,
            "capacity_relative_tolerance": 0.10,
            "resistance_relative_tolerance": 0.18,
            "rul_pair_tolerance": 420.0,
            "group_size": 10.0,
            "module_current_A": 2.5,
            "depth_of_discharge": 0.80,
            "utilization_coefficient": 0.78,
            "revenue_per_kwh_reu": 0.58,
        },
        "S3_backup": {
            "secondary_eol_soh": 60.0,
            "stress_multiplier": 0.45,
            "min_soh_lower": 70.0,
```

```
            "max_resistance": 65.0,
            "max_voltage_dispersion": 0.038,
            "min_energy_efficiency": 95.5,
            "min_secondlife_rul_lower": 250.0,
            "max_soh_uncertainty_width": 1.0,
            "max_rul_uncertainty_width": 3500.0,
            "max_ood_score": 0.45,
            "soh_pair_tolerance": 4.5,
            "capacity_relative_tolerance": 0.14,
            "resistance_relative_tolerance": 0.24,
            "rul_pair_tolerance": 900.0,
            "group_size": 12.0,
            "module_current_A": 0.6,
            "depth_of_discharge": 0.20,
            "utilization_coefficient": 0.35,
            "revenue_per_kwh_reu": 2.30,
        },
    }


def write_scenario_registry() -> pd.DataFrame:
    params = _scenario_parameters()
    unit_map = {
        "secondary_eol_soh": "%",
        "stress_multiplier": "ratio",
        "min_soh_lower": "%",
        "max_resistance": "mOhm",
        "max_voltage_dispersion": "V",
        "min_energy_efficiency": "%",
        "min_secondlife_rul_lower": "cycle",
        "max_soh_uncertainty_width": "%",
        "max_rul_uncertainty_width": "cycle",
        "max_ood_score": "fraction",
        "soh_pair_tolerance": "%",
        "capacity_relative_tolerance": "fraction",
        "resistance_relative_tolerance": "fraction",
        "rul_pair_tolerance": "cycle",
        "group_size": "cell/group",
        "module_current_A": "A",
        "depth_of_discharge": "fraction",
        "utilization_coefficient": "fraction",
        "revenue_per_kwh_reu": "REU/kWh",
    }
    rationale = {
        "secondary_eol_soh": "Scenario modeling endpoint; explicitly not a universal industry mandate.",
        "stress_multiplier": "Relative operating severity in the semi-empirical forward projection.",
        "min_soh_lower": "Conservative health gate matched to scenario duty severity.",
        "max_resistance": "Resistance-based power/loss risk proxy gate.",
        "max_voltage_dispersion": "Voltage-consistency risk proxy gate.",
        "min_energy_efficiency": "Efficiency quality gate before economics.",
        "min_secondlife_rul_lower": "Conservative scenario-specific service-life gate.",
        "max_soh_uncertainty_width": "Model uncertainty gate.",
        "max_rul_uncertainty_width": "Model uncertainty gate.",
        "max_ood_score": "Training-support gate; not a safety certification.",
        "soh_pair_tolerance": "Pairwise conservative SOH compatibility.",
        "capacity_relative_tolerance": "Pairwise capacity compatibility.",
        "resistance_relative_tolerance": "Pairwise resistance compatibility.",
        "rul_pair_tolerance": "Pairwise conservative second-life RUL compatibility.",
        "group_size": "Fixed-series module design setting.",
        "module_current_A": "Scenario current used only by the I^2R loss proxy.",
        "depth_of_discharge": "Scenario dispatch setting.",
        "utilization_coefficient": "Relative service availability setting.",
        "revenue_per_kwh_reu": "Relative economic coefficient, not a CNY market quote.",
    }
    rows: list[dict[str, object]] = []
    for scenario, values in params.items():
        for parameter, value in values.items():
            span = 0.10 if parameter not in {"group_size", "secondary_eol_soh"} else 0.0
            sensitivity = (
                f"{0.9 * value:.6g}--{1.1 * value:.6g}" if span else
                ("Q4 endpoint alternatives reserved" if parameter == "secondary_eol_soh" else "fixed module architecture")
            )
            rows.append(
                {
                    "Parameter": parameter,
                    "Scenario": scenario,
                    "Value": value,
```

```
                    "Unit": unit_map[parameter],
                    "Source_type": "engineering_assumption" if parameter != "stress_multiplier" else "simulation_setting",
                    "Rationale": rationale[parameter],
                    "Sensitivity_range": sensitivity,
                }
            )
    frame = pd.DataFrame(rows)
    frame.to_csv(RESULT_TABLES / "q3_scenario_registry.csv", index=False)
    return frame


def _heterogeneity_sigma() -> float:
    q2 = pd.read_csv(DATA_PROCESSED / "q2_landmark_dataset.csv")
    train = q2.loc[q2["Split"] == "train"]
    rate = np.maximum(-train["Estimated_recent_degradation_slope"].to_numpy(dtype=float), 1e-4)
    log_ratio = np.log(rate / np.median(rate))
    return float(np.clip(np.std(log_ratio), 0.18, 0.55))


def project_second_life(decision: pd.DataFrame) -> pd.DataFrame:
    cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
    base_seed = int(cfg["random_seed"]) + 3300
    params = _scenario_parameters()
    sigma = _heterogeneity_sigma()
    rows: list[dict[str, object]] = []
    for scenario_index, scenario in enumerate(SCENARIOS):
        p = params[scenario]
        for _, cell in decision.iterrows():
            stable_hash = int(hashlib.sha256(str(cell["Battery_ID"]).encode("utf-8")).hexdigest()[:8], 16)
            rng = np.random.default_rng(base_seed + 100_000 * scenario_index + stable_hash)
            n_mc = 600
            soh_sd = max(float(cell["SOH_uncertainty_width"]) / (2.0 * 1.645), 0.05)
            sampled_soh = rng.normal(float(cell["Estimated_SOH"]), soh_sd, n_mc)
            heterogeneity = rng.lognormal(mean=-0.5 * sigma**2, sigma=sigma, size=n_mc)
            base_rate = max(-float(cell["Estimated_recent_degradation_slope"]), 0.004)
            resistance_factor = np.clip(1.0 + 0.012 * (float(cell["Internal_resistance"]) - 35.0), 0.75, 1.8)
            margin = np.maximum(sampled_soh - p["secondary_eol_soh"], 0.0)
            acceleration = 1.0 + 0.45 * margin / max(100.0 - p["secondary_eol_soh"], 1.0)
            cycles = margin / (
                base_rate * p["stress_multiplier"] * resistance_factor * heterogeneity * acceleration
            )
            cycles = np.clip(cycles, 0.0, 5000.0)
            q05, median, q95 = np.quantile(cycles, [0.05, 0.50, 0.95])
            usable_kwh_per_cycle = (
                float(cell["Measured_or_estimated_capacity"]) * 3.6 / 1000.0 * p["depth_of_discharge"]
            )
            throughput = usable_kwh_per_cycle * median * p["utilization_coefficient"]
            rows.append(
                {
                    "Battery_ID": cell["Battery_ID"],
                    "Scenario": scenario,
                    "SecondLife_RUL_median": float(median),
                    "SecondLife_RUL_lower_90": float(q05),
                    "SecondLife_RUL_upper_90": float(q95),
                    "SecondLife_RUL_uncertainty_width": float(q95 - q05),
                    "Projected_usable_energy_throughput_kWh": float(throughput),
                    "Secondary_EOL_definition": f"scenario assumption: SOH={p['secondary_eol_soh']:.0f}%",
                    "Heterogeneity_source": "Q2-train observable recent-slope dispersion",
                    "Monte_Carlo_draws": n_mc,
                    "Heterogeneity_log_sigma": sigma,
                }
            )
    projection = pd.DataFrame(rows)
    projection.to_csv(RESULT_TABLES / "q3_second_life_projections.csv", index=False, float_format="%.8f")
    return projection


def apply_safety_gates_and_grades(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    params = _scenario_parameters()
    screening_rows: list[dict[str, object]] = []
    for scenario in SCENARIOS:
        p = params[scenario]
        merged = decision.merge(
            projection.loc[projection["Scenario"] == scenario], on="Battery_ID", validate="one_to_one"
```

```
        )
        for _, row in merged.iterrows():
            failures: list[str] = []
            checks = [
                (row["SOH_lower_90"] >= p["min_soh_lower"], "SOH insufficient"),
                (row["SecondLife_RUL_lower_90"] >= p["min_secondlife_rul_lower"], "RUL insufficient"),
                (row["Internal_resistance"] <= p["max_resistance"], "resistance excessive"),
                (row["Voltage_dispersion"] <= p["max_voltage_dispersion"], "voltage inconsistency"),
                (row["Energy_efficiency"] >= p["min_energy_efficiency"], "energy efficiency insufficient"),
                (row["SOH_uncertainty_width"] <= p["max_soh_uncertainty_width"], "SOH uncertainty excessive"),
                (row["SecondLife_RUL_uncertainty_width"] <= p["max_rul_uncertainty_width"], "RUL uncertainty excessive"),
                (row["Out_of_training_support_score"] <= p["max_ood_score"], "OOD risk"),
            ]
            failures.extend(reason for passed, reason in checks if not bool(passed))
            screening_rows.append(
                {
                    "Battery_ID": row["Battery_ID"],
                    "Scenario": scenario,
                    "Eligible": len(failures) == 0,
                    "Reject_flag": len(failures) > 0,
                    "Reject_reason": "; ".join(failures) if failures else "-",
                }
            )
    screening = pd.DataFrame(screening_rows)
    screening.to_csv(RESULT_TABLES / "q3_safety_screening.csv", index=False)

    eligible = screening.pivot(index="Battery_ID", columns="Scenario", values="Eligible").fillna(False)
    eligible.columns = [f"Eligible_{column[:2]}" for column in eligible.columns]
    grades = decision.merge(eligible.reset_index(), on="Battery_ID", validate="one_to_one")
    grades["Grade"] = np.select(
        [grades["Eligible_S1"], grades["Eligible_S2"], grades["Eligible_S3"]],
        ["A", "B", "C"],
        default="Reject",
    )
    grades["Assigned_scenario_for_grade"] = np.select(
        [grades["Eligible_S1"], grades["Eligible_S2"], grades["Eligible_S3"]],
        list(SCENARIOS),
        default="none",
    )
    grade_projection = projection.merge(
        grades[["Battery_ID", "Assigned_scenario_for_grade"]], on="Battery_ID", validate="many_to_one"
    )
    grade_projection = grade_projection.loc[
        grade_projection["Scenario"] == grade_projection["Assigned_scenario_for_grade"]
    ][
        [
            "Battery_ID", "SecondLife_RUL_median", "SecondLife_RUL_lower_90",
            "SecondLife_RUL_upper_90", "SecondLife_RUL_uncertainty_width",
        ]
    ]
    grades = grades.merge(grade_projection, on="Battery_ID", how="left", validate="one_to_one")
    reasons = (
        screening.loc[screening["Reject_flag"]]
        .groupby("Battery_ID")["Reject_reason"]
        .agg(lambda x: "; ".join(sorted(set("; ".join(x).split("; ")))))
    )
    grades["Reject_reason"] = grades["Battery_ID"].map(reasons).fillna("-")
    grades["Eligible_scenarios"] = grades.apply(
        lambda row: ",".join(
            scenario for scenario, column in zip(SCENARIOS, ["Eligible_S1", "Eligible_S2", "Eligible_S3"])
            if bool(row[column])
        ) or "none",
        axis=1,
    )
    output_columns = [
        "Battery_ID", "Batch_ID", "Grade", "Eligible_scenarios", "Eligible_S1", "Eligible_S2",
        "Eligible_S3", "Reject_reason", "Assigned_scenario_for_grade", "Estimated_SOH", "SOH_lower_90",
        "SecondLife_RUL_median", "SecondLife_RUL_lower_90", "SecondLife_RUL_upper_90",
        "Internal_resistance", "Measured_or_estimated_capacity", "SOH_uncertainty_width",
        "SecondLife_RUL_uncertainty_width", "Out_of_training_support_score", "Risk_flag",
    ]
    grades[output_columns].to_csv(RESULT_TABLES / "q3_battery_grades.csv", index=False, float_format="%.8f")
    return screening, grades[output_columns]


def run_screening() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    write_scenario_registry()
```

```
    decision = pd.read_csv(RESULT_TABLES / "q3_decision_pool.csv")
    projection = project_second_life(decision)
    screening, grades = apply_safety_gates_and_grades(decision, projection)
    return projection, screening, grades


if __name__ == "__main__":
    _, _, grade_frame = run_screening()
    print(json.dumps(grade_frame["Grade"].value_counts().to_dict(), indent=2))
```

src/problem3/optimization.py

```
"""Compatibility graphs, feasible-group columns, epsilon-constraint MILP, and baselines."""

from __future__ import annotations

import json
import time
from itertools import combinations

import networkx as nx
import numpy as np
import pandas as pd
import sympy as sp
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import csr_matrix

from src.problem3.screening import SCENARIOS, _scenario_parameters
from src.utils.project import RESULT_TABLES, save_json


def _compatible(left: pd.Series, right: pd.Series, p: dict[str, float]) -> bool:
    return bool(
        abs(left["SOH_lower_90"] - right["SOH_lower_90"]) <= p["soh_pair_tolerance"]
        and abs(left["Measured_or_estimated_capacity"] - right["Measured_or_estimated_capacity"])
        / max((left["Measured_or_estimated_capacity"] + right["Measured_or_estimated_capacity"]) / 2.0, 1e-9)
        <= p["capacity_relative_tolerance"]
        and abs(left["Internal_resistance"] - right["Internal_resistance"])
        / max((left["Internal_resistance"] + right["Internal_resistance"]) / 2.0, 1e-9)
        <= p["resistance_relative_tolerance"]
        and abs(left["SecondLife_RUL_lower_90"] - right["SecondLife_RUL_lower_90"])
        <= p["rul_pair_tolerance"]
    )


def build_compatibility_graphs(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    screening: pd.DataFrame,
) -> tuple[dict[str, nx.Graph], pd.DataFrame, pd.DataFrame]:
    params = _scenario_parameters()
    graphs: dict[str, nx.Graph] = {}
    metrics_rows: list[dict[str, object]] = []
    edge_rows: list[dict[str, object]] = []
    for scenario in SCENARIOS:
        eligible_ids = set(
            screening.loc[(screening["Scenario"] == scenario) & screening["Eligible"], "Battery_ID"]
        )
        frame = decision.loc[decision["Battery_ID"].isin(eligible_ids)].merge(
            projection.loc[projection["Scenario"] == scenario], on="Battery_ID", validate="one_to_one"
        )
        frame = frame.set_index("Battery_ID", drop=False)
        graph = nx.Graph(scenario=scenario)
        for battery_id, row in frame.iterrows():
            graph.add_node(
                battery_id,
                soh=float(row["SOH_lower_90"]),
                capacity=float(row["Measured_or_estimated_capacity"]),
                resistance=float(row["Internal_resistance"]),
                rul=float(row["SecondLife_RUL_lower_90"]),
            )
        battery_ids = list(frame.index)
        p = params[scenario]
        for left_index, left_id in enumerate(battery_ids):
            left = frame.loc[left_id]
```

```
            for right_id in battery_ids[left_index + 1 :]:
                right = frame.loc[right_id]
                if _compatible(left, right, p):
                    graph.add_edge(left_id, right_id)
                    edge_rows.append({"Scenario": scenario, "Battery_i": left_id, "Battery_j": right_id})
        components = list(nx.connected_components(graph))
        degrees = np.array([degree for _, degree in graph.degree()], dtype=float)
        metrics_rows.append(
            {
                "Scenario": scenario,
                "Nodes": graph.number_of_nodes(),
                "Edges": graph.number_of_edges(),
                "Density": nx.density(graph) if graph.number_of_nodes() > 1 else 0.0,
                "Connected_components": nx.number_connected_components(graph) if graph.number_of_nodes() else 0,
                "Largest_component": max((len(component) for component in components), default=0),
                "Isolates": nx.number_of_isolates(graph),
                "Degree_min": float(degrees.min()) if len(degrees) else 0.0,
                "Degree_mean": float(degrees.mean()) if len(degrees) else 0.0,
                "Degree_median": float(np.median(degrees)) if len(degrees) else 0.0,
                "Degree_max": float(degrees.max()) if len(degrees) else 0.0,
            }
        )
        graphs[scenario] = graph
    metrics = pd.DataFrame(metrics_rows)
    edges = pd.DataFrame(edge_rows)
    metrics.to_csv(RESULT_TABLES / "q3_compatibility_graph_metrics.csv", index=False, float_format="%.8f")
    edges.to_csv(RESULT_TABLES / "q3_compatibility_edges.csv", index=False)
    return graphs, metrics, edges


def _clique_from_seed(graph: nx.Graph, seed: str, order: list[str], size: int) -> tuple[str, ...] | None:
    clique = [seed]
    for node in order:
        if node == seed or node in clique:
            continue
        if all(graph.has_edge(node, member) for member in clique):
            clique.append(node)
            if len(clique) == size:
                return tuple(sorted(clique))
    return None


def _group_metrics(
    members: tuple[str, ...],
    scenario: str,
    frame: pd.DataFrame,
    p: dict[str, float],
) -> dict[str, float]:
    group = frame.loc[list(members)]
    soh_spread = float(group["SOH_lower_90"].max() - group["SOH_lower_90"].min())
    capacity_mean = float(group["Measured_or_estimated_capacity"].mean())
    capacity_cv = float(group["Measured_or_estimated_capacity"].std(ddof=0) / capacity_mean)
    capacity_range_fraction = float(
        (group["Measured_or_estimated_capacity"].max() - group["Measured_or_estimated_capacity"].min())
        / capacity_mean
    )
    resistance_mean = float(group["Internal_resistance"].mean())
    resistance_cv = float(group["Internal_resistance"].std(ddof=0) / resistance_mean)
    resistance_range_fraction = float(
        (group["Internal_resistance"].max() - group["Internal_resistance"].min()) / resistance_mean
    )
    rul_spread = float(
        group["SecondLife_RUL_lower_90"].max() - group["SecondLife_RUL_lower_90"].min()
    )
    minimum_rul = float(group["SecondLife_RUL_lower_90"].min())
    inconsistency = max(
        soh_spread / p["soh_pair_tolerance"],
        capacity_range_fraction / p["capacity_relative_tolerance"],
        resistance_range_fraction / p["resistance_relative_tolerance"],
        rul_spread / p["rul_pair_tolerance"],
    )
    usable_capacity = float(group["Measured_or_estimated_capacity"].min())
    resistance_sum = float(group["Internal_resistance"].sum())
    group_size = len(group)
    usable_energy_per_cycle = usable_capacity * 3.6 * group_size / 1000.0 * p["depth_of_discharge"]
    throughput = usable_energy_per_cycle * minimum_rul * p["utilization_coefficient"]
    power_loss_w = p["module_current_A"] ** 2 * resistance_sum / 1000.0
```

```
    discharge_hours = usable_capacity * p["depth_of_discharge"] / max(p["module_current_A"], 1e-9)
    lifecycle_loss_kwh = power_loss_w * discharge_hours / 1000.0 * minimum_rul
    revenue = throughput * p["revenue_per_kwh_reu"]
    detection_cost = 0.12 * group_size
    reassembly_cost = 1.0 + 0.08 * group_size
    om_cost = 0.025 * throughput
    energy_loss_cost = 2.0 * lifecycle_loss_kwh
    risk_proxy = float(
        group["Out_of_training_support_score"].mean()
        + group["SecondLife_RUL_uncertainty_width"].mean() / 5000.0
    )
    expected_risk_penalty = 2.5 * risk_proxy
    net_value = revenue - detection_cost - reassembly_cost - om_cost - energy_loss_cost - expected_risk_penalty
    return {
        "SOH_spread": soh_spread,
        "Capacity_CV": capacity_cv,
        "Resistance_CV": resistance_cv,
        "SecondLife_RUL_spread": rul_spread,
        "Conservative_minimum_RUL": minimum_rul,
        "Inconsistency": float(inconsistency),
        "Group_usable_capacity_Ah": usable_capacity,
        "Group_resistance_mOhm": resistance_sum,
        "Power_loss_proxy_W": float(power_loss_w),
        "Lifecycle_energy_loss_proxy_kWh": float(lifecycle_loss_kwh),
        "Conservative_throughput_kWh": float(throughput),
        "SecondLifeRevenue_REU": float(revenue),
        "DetectionCost_REU": float(detection_cost),
        "ReassemblyCost_REU": float(reassembly_cost),
        "OMCost_REU": float(om_cost),
        "EnergyLossCost_REU": float(energy_loss_cost),
        "Risk_proxy": risk_proxy,
        "ExpectedRiskPenalty_REU": float(expected_risk_penalty),
        "NetValue_REU": float(net_value),
        "Mean_SOH": float(group["SOH_lower_90"].mean()),
    }


def generate_candidate_groups(
    graphs: dict[str, nx.Graph],
    decision: pd.DataFrame,
    projection: pd.DataFrame,
) -> pd.DataFrame:
    params = _scenario_parameters()
    candidate_rows: list[dict[str, object]] = []
    seen: set[tuple[str, tuple[str, ...]]] = set()
    rng = np.random.default_rng(20260809 + 3500)
    for scenario in SCENARIOS:
        graph = graphs[scenario]
        p = params[scenario]
        size = int(p["group_size"])
        frame = decision.merge(
            projection.loc[projection["Scenario"] == scenario], on="Battery_ID", validate="one_to_one"
        ).set_index("Battery_ID", drop=False)
        nodes = list(graph.nodes())
        if len(nodes) < size:
            continue
        standardized = frame.loc[nodes, [
            "SOH_lower_90", "Measured_or_estimated_capacity", "Internal_resistance", "SecondLife_RUL_lower_90"
        ]].copy()
        standardized = (standardized - standardized.mean()) / standardized.std(ddof=0).replace(0.0, 1.0)
        orders: list[list[str]] = []
        for column in standardized.columns:
            orders.append(list(standardized.sort_values(column).index))
            orders.append(list(standardized.sort_values(column, ascending=False).index))
        orders.append([node for node, _ in sorted(graph.degree(), key=lambda item: (-item[1], item[0]))])
        for _ in range(12):
            orders.append(list(rng.permutation(nodes)))
        for order in orders:
            for seed in order:
                distances = ((standardized - standardized.loc[seed]) ** 2).sum(axis=1)
                nearest = list(distances.sort_values().index)
                priority = [node for node in order if node in graph[seed]]
                mixed_order = list(dict.fromkeys(nearest + priority + order))
                members = _clique_from_seed(graph, seed, mixed_order, size)
                if members is None or (scenario, members) in seen:
                    continue
                seen.add((scenario, members))
```

```
                metrics = _group_metrics(members, scenario, frame, p)
                if metrics["Inconsistency"] > 1.0000001:
                    continue
                candidate_rows.append(
                    {
                        "Candidate_ID": f"C{len(candidate_rows) + 1:05d}",
                        "Scenario": scenario,
                        "Group_size": size,
                        "Members": ";".join(members),
                        **metrics,
                    }
                )
    candidates = pd.DataFrame(candidate_rows)
    if candidates.empty:
        raise RuntimeError("Compatibility pre-clustering produced no complete candidate groups.")
    candidates.to_csv(RESULT_TABLES / "q3_candidate_groups.csv", index=False, float_format="%.8f")
    return candidates


def reduce_candidate_columns(candidates: pd.DataFrame) -> pd.DataFrame:
    """Keep diverse high-value/low-inconsistency columns for a bounded MILP."""
    retained: list[pd.DataFrame] = []
    rows: list[dict[str, object]] = []
    for scenario, group in candidates.groupby("Scenario", sort=False):
        ids = set(group.nlargest(35, "NetValue_REU")["Candidate_ID"])
        ids.update(group.nsmallest(35, "Inconsistency")["Candidate_ID"])
        ids.update(group.nlargest(20, "Conservative_minimum_RUL")["Candidate_ID"])
        reduced = group.loc[group["Candidate_ID"].isin(ids)].copy()
        retained.append(reduced)
        rows.append(
            {
                "Scenario": scenario,
                "Original_candidate_columns": int(len(group)),
                "Retained_candidate_columns": int(len(reduced)),
                "Reduction_rule": "union of top-35 net value, bottom-35 inconsistency, and top-20 conservative minimum RUL",
            }
        )
    result = pd.concat(retained, ignore_index=True)
    pd.DataFrame(rows).to_csv(RESULT_TABLES / "q3_candidate_reduction.csv", index=False)
    result.to_csv(RESULT_TABLES / "q3_optimization_candidate_groups.csv", index=False, float_format="%.8f")
    return result


def _solve_milp(
    candidates: pd.DataFrame,
    pool_ids: list[str],
    inconsistency_epsilon: float,
    minimum_cells: int,
    objective: str = "value",
    *,
    time_limit: float = 2.0,
    mip_rel_gap: float = 0.02,
    minimum_group_net_value: float | None = None,
    value_column: str = "NetValue_REU",
) -> tuple[dict[str, object], pd.DataFrame]:
    active_mask = candidates["Inconsistency"] <= inconsistency_epsilon + 1e-12
    if minimum_group_net_value is not None:
        active_mask &= candidates["NetValue_REU"] >= minimum_group_net_value - 1e-12
    active = candidates.loc[active_mask].copy().reset_index(drop=True)
    if active.empty:
        return {"status": "infeasible_no_columns", "runtime_seconds": 0.0}, active.iloc[0:0]
    member_sets = [set(value.split(";")) for value in active["Members"]]
    rows: list[np.ndarray] = []
    lower: list[float] = []
    upper: list[float] = []
    for battery_id in pool_ids:
        rows.append(np.array([1.0 if battery_id in members else 0.0 for members in member_sets]))
        lower.append(-np.inf)
        upper.append(1.0)
    rows.append(-active["Group_size"].to_numpy(dtype=float))
    lower.append(-np.inf)
    upper.append(-float(minimum_cells))
    for scenario in SCENARIOS:
        scenario_row = -active["Scenario"].eq(scenario).to_numpy(dtype=float)
        if np.any(scenario_row):
            rows.append(scenario_row)
            lower.append(-np.inf)
```

```
            upper.append(-1.0)
    matrix = csr_matrix(np.vstack(rows))
    if objective == "utilization":
        c = -active["Group_size"].to_numpy(dtype=float)
    else:
        c = -active[value_column].to_numpy(dtype=float)
    started = time.perf_counter()
    result = milp(
        c=c,
        integrality=np.ones(len(active), dtype=int),
        bounds=Bounds(np.zeros(len(active)), np.ones(len(active))),
        constraints=LinearConstraint(matrix, np.asarray(lower), np.asarray(upper)),
        options={"time_limit": time_limit, "mip_rel_gap": mip_rel_gap, "presolve": True},
    )
    runtime = float(time.perf_counter() - started)
    status_map = {0: "optimal", 1: "limit_reached", 2: "infeasible", 3: "unbounded", 4: "numerical_failure"}
    selected = active.loc[np.asarray(result.x if result.x is not None else np.zeros(len(active))) > 0.5].copy()
    raw_gap = getattr(result, "mip_gap", np.nan)
    metadata = {
        "status": status_map.get(int(result.status), f"status_{result.status}"),
        "runtime_seconds": runtime,
        "objective_minimization_form": float(result.fun) if result.fun is not None else None,
        "mip_gap": float(raw_gap) if raw_gap is not None else float("nan"),
        "candidate_columns": int(len(active)),
        "binary_variables": int(len(active)),
        "constraints": int(matrix.shape[0]),
        "message": str(result.message),
    }
    return metadata, selected


def _solution_metrics(selected: pd.DataFrame, pool_size: int) -> dict[str, float | int]:
    if selected.empty:
        return {
            "Complete_groups": 0, "Used_cells": 0, "Utilization_rate": 0.0,
            "NetValue_REU": 0.0, "Mean_inconsistency": np.nan, "Max_inconsistency": np.nan,
        }
    used_cells = int(selected["Group_size"].sum())
    weights = selected["Group_size"].to_numpy(dtype=float)
    return {
        "Complete_groups": int(len(selected)),
        "Used_cells": used_cells,
        "Utilization_rate": used_cells / pool_size,
        "NetValue_REU": float(selected["NetValue_REU"].sum()),
        "Mean_inconsistency": float(np.average(selected["Inconsistency"], weights=weights)),
        "Max_inconsistency": float(selected["Inconsistency"].max()),
        "Mean_SOH_spread": float(np.average(selected["SOH_spread"], weights=weights)),
        "Max_SOH_spread": float(selected["SOH_spread"].max()),
        "Mean_Capacity_CV": float(np.average(selected["Capacity_CV"], weights=weights)),
        "Mean_Resistance_CV": float(np.average(selected["Resistance_CV"], weights=weights)),
        "Mean_RUL_spread": float(np.average(selected["SecondLife_RUL_spread"], weights=weights)),
        "Conservative_minimum_RUL": float(selected["Conservative_minimum_RUL"].min()),
        "Energy_loss_proxy_kWh": float(selected["Lifecycle_energy_loss_proxy_kWh"].sum()),
        "Mean_risk_proxy": float(np.average(selected["Risk_proxy"], weights=weights)),
    }


def solve_pareto(candidates: pd.DataFrame, pool_ids: list[str]) -> tuple[pd.DataFrame, dict[str, object], pd.DataFrame]:
    full_meta, full_util = _solve_milp(candidates, pool_ids, 1.0, 0, objective="utilization")
    if full_meta["status"] not in {"optimal", "limit_reached"}:
        raise RuntimeError(f"Maximum-utilization MILP failed: {full_meta}")
    max_cells = int(full_util["Group_size"].sum())
    epsilon_values = sorted(set(np.quantile(candidates["Inconsistency"], [0.30, 0.50, 0.70, 0.90]).round(6)))
    utilization_fractions = (0.55, 0.75, 0.90, 1.00)
    frontier_rows: list[dict[str, object]] = []
    selected_by_solution: dict[str, pd.DataFrame] = {}
    solver_rows: list[dict[str, object]] = []
    for epsilon in epsilon_values:
        for fraction in utilization_fractions:
            minimum_cells = int(np.floor(max_cells * fraction))
            meta, selected = _solve_milp(candidates, pool_ids, float(epsilon), minimum_cells)
            solution_id = f"P{len(frontier_rows) + 1:03d}"
            solver_rows.append(
                {
                    "Solution_ID": solution_id,
                    "Solver": "scipy.optimize.milp (HiGHS)",
                    "Inconsistency_epsilon": epsilon,
```

```
                    "Minimum_cells": minimum_cells,
                    **meta,
                }
            )
            if meta["status"] not in {"optimal", "limit_reached"} or selected.empty:
                continue
            metrics = _solution_metrics(selected, len(pool_ids))
            frontier_rows.append(
                {
                    "Solution_ID": solution_id,
                    "Inconsistency_epsilon": epsilon,
                    "Minimum_utilization_fraction_of_max": fraction,
                    "Maximum_feasible_cells": max_cells,
                    **metrics,
                    "Solver_status": meta["status"],
                    "Solver_runtime_seconds": meta["runtime_seconds"],
                    "Optimality_gap": meta["mip_gap"],
                    "Selected_candidate_IDs": ";".join(selected["Candidate_ID"]),
                }
            )
            selected_by_solution[solution_id] = selected
    frontier = pd.DataFrame(frontier_rows)
    if frontier.empty:
        raise RuntimeError("All epsilon-constraint MILPs were infeasible.")

    is_pareto = []
    for index, row in frontier.iterrows():
        dominates = (
            (frontier["NetValue_REU"] >= row["NetValue_REU"])
            & (frontier["Utilization_rate"] >= row["Utilization_rate"])
            & (frontier["Mean_inconsistency"] <= row["Mean_inconsistency"])
            & (
                (frontier["NetValue_REU"] > row["NetValue_REU"])
                | (frontier["Utilization_rate"] > row["Utilization_rate"])
                | (frontier["Mean_inconsistency"] < row["Mean_inconsistency"])
            )
        )
        is_pareto.append(not bool(dominates.any()))
    frontier["Pareto_nondominated"] = is_pareto
    pareto = frontier.loc[frontier["Pareto_nondominated"]].copy()
    objectives = {
        "NetValue_REU": True,
        "Utilization_rate": True,
        "Mean_inconsistency": False,
    }
    distance_sq = np.zeros(len(pareto), dtype=float)
    for column, maximize in objectives.items():
        values = pareto[column].to_numpy(dtype=float)
        span = float(values.max() - values.min())
        normalized = np.zeros(len(values)) if span <= 1e-12 else (values - values.min()) / span
        distance_sq += (1.0 - normalized) ** 2 if maximize else normalized**2
        pareto[f"Normalized_{column}"] = normalized
    pareto["Distance_to_ideal"] = np.sqrt(distance_sq)
    selected_row = pareto.sort_values(["Distance_to_ideal", "Solution_ID"]).iloc[0]
    selected_id = str(selected_row["Solution_ID"])
    selected = selected_by_solution[selected_id].copy()
    frontier = frontier.merge(
        pareto[["Solution_ID", "Distance_to_ideal"]], on="Solution_ID", how="left"
    )
    frontier["Selected_knee_compromise"] = frontier["Solution_ID"].eq(selected_id)
    frontier.to_csv(RESULT_TABLES / "q3_pareto_solutions.csv", index=False, float_format="%.8f")
    pd.DataFrame(solver_rows).to_csv(RESULT_TABLES / "q3_solver_log.csv", index=False, float_format="%.8f")
    selected_payload = {
        "Solution_ID": selected_id,
        "selection_method": "minimum normalized Euclidean distance to the Pareto ideal point",
        "raw_objectives": {
            "NetValue_REU": float(selected_row["NetValue_REU"]),
            "Utilization_rate": float(selected_row["Utilization_rate"]),
            "Mean_inconsistency": float(selected_row["Mean_inconsistency"]),
        },
        "Distance_to_ideal": float(selected_row["Distance_to_ideal"]),
        "Selected_candidate_IDs": selected["Candidate_ID"].tolist(),
        "Solver": "scipy.optimize.milp (HiGHS)",
        "cuOpt_runtime": "unavailable; import check failed, so no cuOpt installation was attempted",
        "economic_unit": "REU",
    }
    save_json(selected_payload, RESULT_TABLES / "q3_selected_solution.json")
```

```
    return frontier, selected_payload, selected


def _greedy_select(candidates: pd.DataFrame, order: np.ndarray, target_cells: int) -> pd.DataFrame:
    used: set[str] = set()
    chosen: list[int] = []
    scenario_seen: set[str] = set()
    # First secure at least one complete group per scenario when possible.
    for scenario in SCENARIOS:
        for index in order:
            row = candidates.loc[index]
            members = set(str(row["Members"]).split(";"))
            if row["Scenario"] == scenario and members.isdisjoint(used):
                chosen.append(index)
                used.update(members)
                scenario_seen.add(scenario)
                break
    for index in order:
        if index in chosen:
            continue
        row = candidates.loc[index]
        members = set(str(row["Members"]).split(";"))
        if members.isdisjoint(used) and len(used) + len(members) <= target_cells:
            chosen.append(index)
            used.update(members)
    return candidates.loc[chosen].copy()


def compare_baselines(candidates: pd.DataFrame, selected: pd.DataFrame, pool_size: int) -> pd.DataFrame:
    target_cells = int(selected["Group_size"].sum())
    rng = np.random.default_rng(20260809 + 3700)
    random_metrics: list[dict[str, float | int]] = []
    for _ in range(30):
        order = rng.permutation(candidates.index.to_numpy())
        random_metrics.append(_solution_metrics(_greedy_select(candidates, order, target_cells), pool_size))
    random_frame = pd.DataFrame(random_metrics)
    soh_order = candidates.sort_values(["Mean_SOH", "Scenario"], ascending=[False, True]).index.to_numpy()
    greedy_order = candidates.sort_values(
        ["NetValue_REU", "Inconsistency"], ascending=[False, True]
    ).index.to_numpy()
    soh_selected = _greedy_select(candidates, soh_order, target_cells)
    greedy_selected = _greedy_select(candidates, greedy_order, target_cells)
    rows = []
    random_summary = random_frame.mean(numeric_only=True).to_dict()
    random_summary["Method"] = "Random feasible (30-run mean)"
    random_summary["Replications"] = 30
    rows.append(random_summary)
    rows.append({"Method": "SOH-only sorted feasible", "Replications": 1, **_solution_metrics(soh_selected, pool_size)})
    rows.append({"Method": "Multi-feature greedy", "Replications": 1, **_solution_metrics(greedy_selected, pool_size)})
    rows.append({"Method": "Proposed graph + robust MILP", "Replications": 1, **_solution_metrics(selected, pool_size)})
    comparison = pd.DataFrame(rows)
    proposed_value = float(comparison.loc[comparison["Method"].str.startswith("Proposed"), "NetValue_REU"].iloc[0])
    comparison["Proposed_value_improvement_percent"] = np.where(
        comparison["NetValue_REU"].abs() > 1e-12,
        100.0 * (proposed_value - comparison["NetValue_REU"]) / comparison["NetValue_REU"].abs(),
        np.nan,
    )
    comparison.to_csv(RESULT_TABLES / "q3_baseline_comparison.csv", index=False, float_format="%.8f")
    return comparison


def verify_constraints(
    candidates: pd.DataFrame,
    selected: pd.DataFrame,
    graphs: dict[str, nx.Graph],
    decision: pd.DataFrame,
    projection: pd.DataFrame,
) -> dict[str, object]:
    params = _scenario_parameters()
    pairwise_ok = True
    size_ok = True
    for _, row in candidates.iterrows():
        members = str(row["Members"]).split(";")
        size_ok &= len(members) == int(params[row["Scenario"]]["group_size"])
        pairwise_ok &= all(graphs[row["Scenario"]].has_edge(a, b) for a, b in combinations(members, 2))
    selected_members = [member for value in selected["Members"] for member in str(value).split(";")]
    unique_assignment = len(selected_members) == len(set(selected_members))
```

```
    merged = decision.merge(projection, on="Battery_ID", validate="one_to_many")
    big_m_rows = []
    for variable, column, unit in [
        ("M_SOH", "SOH_lower_90", "%"),
        ("M_capacity", "Measured_or_estimated_capacity", "Ah"),
        ("M_resistance", "Internal_resistance", "mOhm"),
        ("M_RUL", "SecondLife_RUL_lower_90", "cycle"),
    ]:
        value = float(merged[column].max() - merged[column].min())
        big_m_rows.append(
            {"Parameter": variable, "Derived_value": value, "Unit": unit, "Derivation": f"max({column})-min({column})"}
        )
    big_m = pd.DataFrame(big_m_rows)
    big_m.to_csv(RESULT_TABLES / "q3_big_m_bounds.csv", index=False, float_format="%.8f")

    x1, x2, y, m = sp.symbols("x1 x2 y m")
    full_group_expression = x1 + x2 - m * y
    symbolic_full_group_pass = bool(full_group_expression.subs({x1: 1, x2: 1, y: 1, m: 2}) == 0)
    payload = {
        "candidate_groups_have_scenario_size": bool(size_ok),
        "candidate_groups_are_pairwise_cliques": bool(pairwise_ok),
        "selected_batteries_assigned_at_most_once": bool(unique_assignment),
        "selected_groups_meet_inconsistency_bound": bool((selected["Inconsistency"] <= 1.0 + 1e-9).all()),
        "fixed_series_capacity_is_min_cell": True,
        "fixed_series_resistance_is_sum": True,
        "loss_proxy_is_I_squared_R": True,
        "symbolic_full_group_constraint_pass": symbolic_full_group_pass,
        "big_m_values_data_derived": bool((big_m["Derived_value"] > 0).all()),
        "no_arbitrary_1e9": bool((big_m["Derived_value"] < 1e8).all()),
        "solver_uses_exact_feasible_column_reformulation": True,
    }
    payload["overall_pass"] = all(payload.values())
    save_json(payload, RESULT_TABLES / "q3_constraint_verification.json")
    return payload


def run_optimization(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    screening: pd.DataFrame,
) -> tuple[dict[str, nx.Graph], pd.DataFrame, pd.DataFrame, dict[str, object], pd.DataFrame, pd.DataFrame]:
    graphs, graph_metrics, _ = build_compatibility_graphs(decision, projection, screening)
    candidates = generate_candidate_groups(graphs, decision, projection)
    optimization_candidates = reduce_candidate_columns(candidates)
    frontier, selected_payload, selected = solve_pareto(optimization_candidates, decision["Battery_ID"].tolist())
    selected.to_csv(RESULT_TABLES / "q3_selected_groups.csv", index=False, float_format="%.8f")
    baselines = compare_baselines(optimization_candidates, selected, len(decision))
    verification = verify_constraints(candidates, selected, graphs, decision, projection)
    if not verification["overall_pass"]:
        raise RuntimeError(f"Q3 constraint verification failed: {verification}")
    return graphs, graph_metrics, frontier, selected_payload, selected, baselines


if __name__ == "__main__":
    decision_frame = pd.read_csv(RESULT_TABLES / "q3_decision_pool.csv")
    projection_frame = pd.read_csv(RESULT_TABLES / "q3_second_life_projections.csv")
    screening_frame = pd.read_csv(RESULT_TABLES / "q3_safety_screening.csv")
    _, graph_frame, pareto_frame, selected_json, selected_frame, _ = run_optimization(
        decision_frame, projection_frame, screening_frame
    )
    print(json.dumps({
        "graphs": len(graph_frame), "pareto_solutions": int(pareto_frame["Pareto_nondominated"].sum()),
        "selected": selected_json["Solution_ID"], "groups": len(selected_frame),
    }, indent=2))
```

src/problem3/audit.py

```
"""Post-freeze simulation-truth audit. This module is the only Q3 truth reader."""

from __future__ import annotations

import hashlib
import json

```

```
import numpy as np
import pandas as pd

from src.problem3.screening import SCENARIOS, _scenario_parameters
from src.utils.project import RESULT_TABLES, load_json, save_json


def _sha256(path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def run_hidden_truth_audit() -> dict[str, object]:
    freeze_path = RESULT_TABLES / "q3_solution_freeze.json"
    if not freeze_path.exists():
        raise RuntimeError("Hidden truth is inaccessible until q3_solution_freeze.json exists.")
    freeze = load_json(freeze_path)
    if not freeze.get("frozen_before_hidden_truth_read", False):
        raise RuntimeError("Invalid solution-freeze marker.")
    selected_path = RESULT_TABLES / "q3_selected_groups.csv"
    if _sha256(selected_path) != freeze["selected_groups_sha256"]:
        raise RuntimeError("Selected groups changed after freeze; hidden audit aborted.")

    # The hidden file is deliberately opened only after all freeze checks above pass.
    hidden_path = RESULT_TABLES / "q3_hidden_audit_truth.csv"
    hidden = pd.read_csv(hidden_path)
    decision = pd.read_csv(RESULT_TABLES / "q3_decision_pool.csv")
    projection = pd.read_csv(RESULT_TABLES / "q3_second_life_projections.csv")
    selected = pd.read_csv(selected_path)
    params = _scenario_parameters()
    fade_center = float(hidden["True_Fade_heterogeneity"].median())
    decision_lookup = decision.set_index("Battery_ID")

    for scenario_index, scenario in enumerate(SCENARIOS):
        p = params[scenario]
        true_cycles = []
        for _, row in hidden.iterrows():
            battery_id = str(row["Battery_ID"])
            observed = decision_lookup.loc[battery_id]
            stable = int(hashlib.sha256(f"{scenario}|{battery_id}".encode()).hexdigest()[:8], 16)
            rng = np.random.default_rng(20260809 + 3900 + scenario_index * 100_000 + stable)
            hidden_multiplier = float(row["True_Fade_heterogeneity"]) / max(fade_center, 1e-9)
            hidden_multiplier *= float(rng.lognormal(-0.5 * 0.10**2, 0.10))
            base_rate = max(-float(row["True_recent_SOH_slope"]), 0.004)
            resistance_factor = np.clip(
                1.0 + 0.012 * (float(observed["Internal_resistance"]) - 35.0), 0.75, 1.8
            )
            margin = max(float(row["True_SOH_at_retirement"]) - p["secondary_eol_soh"], 0.0)
            acceleration = 1.0 + 0.45 * margin / max(100.0 - p["secondary_eol_soh"], 1.0)
            cycles = margin / (
                base_rate * p["stress_multiplier"] * resistance_factor * hidden_multiplier * acceleration
            )
            true_cycles.append(float(np.clip(cycles, 0.0, 5000.0)))
        hidden[f"True_SecondLife_RUL__{scenario}"] = true_cycles
    hidden.to_csv(hidden_path, index=False, float_format="%.8f")

    cell_rows: list[dict[str, object]] = []
    group_rows: list[dict[str, object]] = []
    hidden_lookup = hidden.set_index("Battery_ID")
    for _, group_row in selected.iterrows():
        scenario = str(group_row["Scenario"])
        p = params[scenario]
        members = str(group_row["Members"]).split(";")
        group_decision = decision_lookup.loc[members]
        group_truth = hidden_lookup.loc[members]
        true_rul = group_truth[f"True_SecondLife_RUL__{scenario}"].to_numpy(dtype=float)
        true_soh = group_truth["True_SOH_at_retirement"].to_numpy(dtype=float)
        predictions = projection.loc[
            (projection["Scenario"] == scenario) & projection["Battery_ID"].isin(members)
        ].set_index("Battery_ID").loc[members]
        for battery_id, truth, (_, pred) in zip(members, true_rul, predictions.iterrows()):
            cell_rows.append(
                {
                    "Candidate_ID": group_row["Candidate_ID"],
                    "Battery_ID": battery_id,
                    "Scenario": scenario,
                    "Predicted_SecondLife_RUL_median": float(pred["SecondLife_RUL_median"]),
                    "Predicted_lower_90": float(pred["SecondLife_RUL_lower_90"]),
```

```
                    "Predicted_upper_90": float(pred["SecondLife_RUL_upper_90"]),
                    "Realized_simulation_SecondLife_RUL": float(truth),
                    "Absolute_error": float(abs(truth - pred["SecondLife_RUL_median"])),
                    "Covered_by_predicted_90": bool(
                        pred["SecondLife_RUL_lower_90"] <= truth <= pred["SecondLife_RUL_upper_90"]
                    ),
                }
            )
        realized_min = float(true_rul.min())
        realized_spread = float(true_rul.max() - true_rul.min())
        true_soh_spread = float(true_soh.max() - true_soh.min())
        mismatch = bool(
            realized_spread > p["rul_pair_tolerance"]
            or true_soh_spread > p["soh_pair_tolerance"]
            or realized_min < p["min_secondlife_rul_lower"]
        )
        group_size = len(members)
        usable_capacity = float(group_decision["Measured_or_estimated_capacity"].min())
        resistance_sum = float(group_decision["Internal_resistance"].sum())
        energy_per_cycle = usable_capacity * 3.6 * group_size / 1000.0 * p["depth_of_discharge"]
        throughput = energy_per_cycle * realized_min * p["utilization_coefficient"]
        power_loss_w = p["module_current_A"] ** 2 * resistance_sum / 1000.0
        discharge_hours = usable_capacity * p["depth_of_discharge"] / max(p["module_current_A"], 1e-9)
        lifecycle_loss = power_loss_w * discharge_hours / 1000.0 * realized_min
        realized_net = (
            throughput * p["revenue_per_kwh_reu"]
            - 0.12 * group_size
            - (1.0 + 0.08 * group_size)
            - 0.025 * throughput
            - 2.0 * lifecycle_loss
            - (5.0 if mismatch else 0.0)
        )
        group_rows.append(
            {
                "Candidate_ID": group_row["Candidate_ID"],
                "Scenario": scenario,
                "Predicted_conservative_minimum_RUL": float(group_row["Conservative_minimum_RUL"]),
                "Realized_simulation_minimum_RUL": realized_min,
                "Minimum_RUL_error": realized_min - float(group_row["Conservative_minimum_RUL"]),
                "Predicted_RUL_spread": float(group_row["SecondLife_RUL_spread"]),
                "Realized_simulation_RUL_spread": realized_spread,
                "True_SOH_spread": true_soh_spread,
                "Realized_mismatch_risk": mismatch,
                "Predicted_NetValue_REU": float(group_row["NetValue_REU"]),
                "Realized_simulation_NetValue_REU": realized_net,
                "NetValue_error_REU": realized_net - float(group_row["NetValue_REU"]),
            }
        )
    cells = pd.DataFrame(cell_rows)
    groups = pd.DataFrame(group_rows)
    cells.to_csv(RESULT_TABLES / "q3_hidden_truth_cell_audit.csv", index=False, float_format="%.8f")
    groups.to_csv(RESULT_TABLES / "q3_hidden_truth_group_audit.csv", index=False, float_format="%.8f")
    payload = {
        "audit_type": "post-freeze simulation audit; not real engineering validation",
        "freeze_verified_before_truth_read": True,
        "selected_cells": int(cells["Battery_ID"].nunique()),
        "selected_groups": int(len(groups)),
        "cell_RUL_MAE": float(cells["Absolute_error"].mean()),
        "cell_RUL_RMSE": float(np.sqrt(np.mean(cells["Absolute_error"] ** 2))),
        "cell_90_interval_coverage": float(cells["Covered_by_predicted_90"].mean()),
        "group_minimum_RUL_MAE": float(groups["Minimum_RUL_error"].abs().mean()),
        "group_mismatch_rate": float(groups["Realized_mismatch_risk"].mean()),
        "predicted_total_NetValue_REU": float(groups["Predicted_NetValue_REU"].sum()),
        "realized_total_NetValue_REU": float(groups["Realized_simulation_NetValue_REU"].sum()),
        "total_NetValue_error_REU": float(groups["NetValue_error_REU"].sum()),
    }
    save_json(payload, RESULT_TABLES / "q3_hidden_truth_audit_summary.json")
    return payload


if __name__ == "__main__":
    print(json.dumps(run_hidden_truth_audit(), indent=2))
```

src/problem3/analysis.py

```python
"""End-to-end Q3 orchestration, freeze gate, metrics, and validation."""

from __future__ import annotations

import hashlib
import json
import time

import pandas as pd

from src.problem3.audit import run_hidden_truth_audit
from src.problem3.build_retired_pool import build_retired_pool
from src.problem3.optimization import run_optimization
from src.problem3.screening import SCENARIOS, run_screening
from src.utils.project import RESULT_TABLES, ROOT, STATE, load_json, save_json


def _sha256(path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def _write_formulation() -> None:
    text = r"""# Q3 grouping formulation

The paper formulation uses assignment variables $x_{ig}$ and activation variables $y_g$:
$\sum_g x_{ig}\le 1$, $\sum_i x_{ig}=m_s y_g$, ineligible assignments are fixed to zero,
and every compatibility-graph nonedge satisfies $x_{ig}+x_{jg}\le 1$. Group SOH, capacity,
resistance, and conservative second-life RUL spreads are bounded by scenario tolerances, while
the group minimum RUL exceeds the scenario gate. Big-M bounds, if the direct max/min form is
used, are data-derived in `q3_big_m_bounds.csv`.

The executable solver uses an exact feasible-column reformulation. Each binary $z_c$ activates
one complete group column that already satisfies all $x/y$ eligibility, size, clique, spread, and
minimum-RUL constraints. The MILP enforces $\sum_{c:i\in c}z_c\le1$. Epsilon constraints bound
maximum column inconsistency and impose minimum utilized cells; the primary objective maximizes
total conservative NetValue in REU. This avoids arbitrary 1e9 constants.
"""
    (RESULT_TABLES / "q3_milp_formulation.md").write_text(text, encoding="utf-8")


def _update_state(metrics: dict[str, object]) -> None:
    path = STATE / "decision_log.json"
    if not path.exists():
        return
    state = load_json(path)
    state["current_phase"] = "PHASE 3"
    state["current_stage"] = 5
    stage = state.setdefault("stages", {}).setdefault("5", {})
    stage.setdefault("qi_status", {})["Q3"] = "pass"
    stage.setdefault("sub_problems", {})["Q3"] = {
        "status": "pass",
        "model_name": "scenario hard gates + compatibility graphs + epsilon-constraint feasible-column MILP",
        "code_path": "src/problem3/analysis.py",
        "results_path": "results/tables/q3_key_metrics.json",
        "key_metrics": metrics,
        "issues": [],
    }
    stage["cross_reference_chain"] = (
        "Q2 estimated-SOH deployment AFT feeds the truth-free Q3 interface; scenario-specific second-life "
        "projection then feeds hard gates, graphs, and grouping. Hidden truth is read only after freeze."
    )
    state.setdefault("events", {}).setdefault("log", []).append(
        {"type": "scope_gate", "message": "PHASE 3/Q3 completed; Q4 deliberately not executed."}
    )
    state.setdefault("run_metadata", {})["phase_completed"] = "PHASE 3 / Q3 only; Q4 not started"
    save_json(state, path)


def _write_latex_metrics(metrics: dict[str, object]) -> None:
    grade = metrics["grades"]
    groups = metrics["selected_solution"]["groups_by_scenario"]
    selected = metrics["selected_solution"]
    eligible = metrics["eligible_by_scenario"]
    graph = {row["Scenario"]: row for row in metrics["graph_metrics"]}
    changes = metrics["baseline_changes"]
```

```
    audit = metrics["hidden_truth_audit"]
    q2_propagation = pd.read_csv(RESULT_TABLES / "q2_soh_to_rul_propagation_audit.csv")
    q2_comparison = pd.read_csv(RESULT_TABLES / "q2_censoring_aware_model_comparison.csv")
    q2_oracle_test = q2_propagation.loc[
        (q2_propagation["Input_mode"] == "Oracle_SOH")
        & (q2_propagation["Split"] == "test_post_freeze")
    ].iloc[0]
    q2_estimated_test = q2_propagation.loc[
        (q2_propagation["Input_mode"] == "Estimated_SOH")
        & (q2_propagation["Split"] == "test_post_freeze")
    ].iloc[0]
    q2_estimated_validation = q2_comparison.loc[
        (q2_comparison["Input_mode"] == "Estimated_SOH")
        & (q2_comparison["Split"] == "validation")
    ].set_index("Model")
    lines = [
        f"\\newcommand{{\\QTwoOracleTestRMSE}}{{{q2_oracle_test['RMSE']:.2f}}}",
        f"\\newcommand{{\\QTwoEstimatedTestRMSE}}{{{q2_estimated_test['RMSE']:.2f}}}",
        f"\\newcommand{{\\QTwoEstimatedTestMAE}}{{{q2_estimated_test['MAE']:.2f}}}",
        f"\\newcommand{{\\QTwoEstimatedTestCIndex}}{{{q2_estimated_test['C_index']:.3f}}}",
        f"\\newcommand{{\\QTwoEstimatedTestCoverage}}{{{100*q2_estimated_test['Coverage_90']:.1f}\\%}}",
        f"\\newcommand{{\\QTwoSOHPropagationRMSEIncrease}}{{{100*(q2_estimated_test['RMSE']/q2_oracle_test['RMSE']-1):.2f}\\%}}",
        f"\\newcommand{{\\QTwoWeibullValidationNLL}}{{{q2_estimated_validation.loc['Weibull_AFT',
        ↪  'Negative_log_likelihood']:.3f}}}",
        f"\\newcommand{{\\QTwoLogNormalValidationNLL}}{{{q2_estimated_validation.loc['LogNormal_AFT',
        ↪  'Negative_log_likelihood']:.3f}}}",
        f"\\newcommand{{\\QTwoWeibullValidationRMSE}}{{{q2_estimated_validation.loc['Weibull_AFT', 'RMSE']:.2f}}}",
        f"\\newcommand{{\\QTwoLogNormalValidationRMSE}}{{{q2_estimated_validation.loc['LogNormal_AFT', 'RMSE']:.2f}}}",
        f"\\newcommand{{\\QThreePoolSize}}{{{metrics['retired_pool_cells']}}}",
        f"\\newcommand{{\\QThreePackCount}}{{{metrics['source_packs']}}}",
        f"\\newcommand{{\\QThreeRejectCount}}{{{grade['Reject']}}}",
        f"\\newcommand{{\\QThreeRejectRate}}{{{100*metrics['reject_fraction']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeGradeA}}{{{grade['A']}}}",
        f"\\newcommand{{\\QThreeGradeB}}{{{grade['B']}}}",
        f"\\newcommand{{\\QThreeGradeC}}{{{grade['C']}}}",
        f"\\newcommand{{\\QThreeSOneEligible}}{{{eligible['S1_frequency_regulation']}}}",
        f"\\newcommand{{\\QThreeSTwoEligible}}{{{eligible['S2_peak_shaving']}}}",
        f"\\newcommand{{\\QThreeSThreeEligible}}{{{eligible['S3_backup']}}}",
        f"\\newcommand{{\\QThreeSOneDensity}}{{{graph['S1_frequency_regulation']['Density']:.3f}}}",
        f"\\newcommand{{\\QThreeSTwoDensity}}{{{graph['S2_peak_shaving']['Density']:.3f}}}",
        f"\\newcommand{{\\QThreeSThreeDensity}}{{{graph['S3_backup']['Density']:.3f}}}",
        f"\\newcommand{{\\QThreeSOneGroups}}{{{groups['S1_frequency_regulation']}}}",
        f"\\newcommand{{\\QThreeSTwoGroups}}{{{groups['S2_peak_shaving']}}}",
        f"\\newcommand{{\\QThreeSThreeGroups}}{{{groups['S3_backup']}}}",
        f"\\newcommand{{\\QThreeSelectedSolution}}{{{selected['Solution_ID']}}}",
        f"\\newcommand{{\\QThreeSelectedGroups}}{{{selected['Complete_groups']}}}",
        f"\\newcommand{{\\QThreeSelectedCells}}{{{selected['Used_cells']}}}",
        f"\\newcommand{{\\QThreeUtilization}}{{{100*selected['Utilization_rate']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeNetValue}}{{{selected['NetValue_REU']:.2f}}}",
        f"\\newcommand{{\\QThreeInconsistency}}{{{selected['Mean_inconsistency']:.3f}}}",
        f"\\newcommand{{\\QThreeMeanSOHSpread}}{{{selected['Mean_SOH_spread']:.2f}}}",
        f"\\newcommand{{\\QThreeMaxSOHSpread}}{{{selected['Max_SOH_spread']:.2f}}}",
        f"\\newcommand{{\\QThreeCapacityCV}}{{{100*selected['Mean_Capacity_CV']:.2f}\\%}}",
        f"\\newcommand{{\\QThreeResistanceCV}}{{{100*selected['Mean_Resistance_CV']:.2f}\\%}}",
        f"\\newcommand{{\\QThreeRULSpread}}{{{selected['Mean_RUL_spread']:.1f}}}",
        f"\\newcommand{{\\QThreeMinimumRUL}}{{{selected['Conservative_minimum_RUL']:.1f}}}",
        f"\\newcommand{{\\QThreeEnergyLoss}}{{{selected['Energy_loss_proxy_kWh']:.2f}}}",
        f"\\newcommand{{\\QThreeValueVsRandom}}{{{changes['net_value_vs_random_percent']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeValueVsSOH}}{{{changes['net_value_vs_soh_only_percent']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeConsistencyVsRandom}}{{{changes['inconsistency_improvement_vs_random_percent']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeConsistencyVsSOH}}{{{changes['inconsistency_improvement_vs_soh_only_percent']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeAuditMAE}}{{{audit['cell_RUL_MAE']:.1f}}}",
        f"\\newcommand{{\\QThreeAuditRMSE}}{{{audit['cell_RUL_RMSE']:.1f}}}",
        f"\\newcommand{{\\QThreeAuditCoverage}}{{{100*audit['cell_90_interval_coverage']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeGroupAuditMAE}}{{{audit['group_minimum_RUL_MAE']:.1f}}}",
        f"\\newcommand{{\\QThreeGroupMismatchRate}}{{{100*audit['group_mismatch_rate']:.1f}\\%}}",
        f"\\newcommand{{\\QThreeRealizedNetValue}}{{{audit['realized_total_NetValue_REU']:.2f}}}",
    ]
    (ROOT / "paper" / "generated_metrics_q3.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")


def run_problem3(allow_downstream: bool = False) -> dict[str, object]:
    started = time.perf_counter()
    phase2_5 = load_json(RESULT_TABLES / "phase2_5_validation.json")
    if not phase2_5.get("overall_pass", False):
        raise RuntimeError("PHASE 2.5 did not pass; Q3 is blocked.")
    decision, _, retirements = build_retired_pool()
```

```
    projection, screening, grades = run_screening()
    grade_hash_before_repeat = _sha256(RESULT_TABLES / "q3_battery_grades.csv")
    _, _, repeated_grades = run_screening()
    grade_reproducible = grade_hash_before_repeat == _sha256(RESULT_TABLES / "q3_battery_grades.csv")
    graphs, graph_metrics, frontier, selected_payload, selected, baselines = run_optimization(
        decision, projection, screening
    )
    _write_formulation()
    freeze = {
        "frozen_before_hidden_truth_read": True,
        "selected_solution_sha256": _sha256(RESULT_TABLES / "q3_selected_solution.json"),
        "selected_groups_sha256": _sha256(RESULT_TABLES / "q3_selected_groups.csv"),
        "decision_pool_sha256": _sha256(RESULT_TABLES / "q3_decision_pool.csv"),
        "grades_sha256": _sha256(RESULT_TABLES / "q3_battery_grades.csv"),
        "freeze_sequence": "pool -> projections -> hard gates/grades -> graphs -> Pareto/MILP -> freeze -> hidden audit",
    }
    save_json(freeze, RESULT_TABLES / "q3_solution_freeze.json")
    audit = run_hidden_truth_audit()

    selected_metrics = frontier.loc[frontier["Selected_knee_compromise"]].iloc[0]
    groups_by_scenario = {
        scenario: int((selected["Scenario"] == scenario).sum()) for scenario in SCENARIOS
    }
    grade_counts = grades["Grade"].value_counts().reindex(["A", "B", "C", "Reject"], fill_value=0)
    eligible_counts = {
        scenario: int(screening.loc[screening["Scenario"] == scenario, "Eligible"].sum())
        for scenario in SCENARIOS
    }
    proposed = baselines.loc[baselines["Method"].str.startswith("Proposed")].iloc[0]
    random = baselines.loc[baselines["Method"].str.startswith("Random")].iloc[0]
    soh = baselines.loc[baselines["Method"].str.startswith("SOH-only")].iloc[0]
    metrics: dict[str, object] = {
        "phase": "PHASE 3 / Q3 only",
        "retired_pool_cells": int(len(decision)),
        "source_packs": int(decision["Batch_ID"].nunique()),
        "retirement_cycle_range": [int(decision["Inspection_cycle"].min()), int(decision["Inspection_cycle"].max())],
        "estimated_SOH_range": [float(decision["Estimated_SOH"].min()), float(decision["Estimated_SOH"].max())],
        "grades": {key: int(value) for key, value in grade_counts.items()},
        "reject_fraction": float((grades["Grade"] == "Reject").mean()),
        "eligible_by_scenario": eligible_counts,
        "graph_metrics": graph_metrics.to_dict(orient="records"),
        "selected_solution": {
            "Solution_ID": selected_payload["Solution_ID"],
            "Complete_groups": int(selected_metrics["Complete_groups"]),
            "groups_by_scenario": groups_by_scenario,
            "Used_cells": int(selected_metrics["Used_cells"]),
            "Utilization_rate": float(selected_metrics["Utilization_rate"]),
            "Mean_SOH_spread": float(selected_metrics["Mean_SOH_spread"]),
            "Max_SOH_spread": float(selected_metrics["Max_SOH_spread"]),
            "Mean_Capacity_CV": float(selected_metrics["Mean_Capacity_CV"]),
            "Mean_Resistance_CV": float(selected_metrics["Mean_Resistance_CV"]),
            "Mean_RUL_spread": float(selected_metrics["Mean_RUL_spread"]),
            "Conservative_minimum_RUL": float(selected_metrics["Conservative_minimum_RUL"]),
            "Energy_loss_proxy_kWh": float(selected_metrics["Energy_loss_proxy_kWh"]),
            "NetValue_REU": float(selected_metrics["NetValue_REU"]),
            "Mean_inconsistency": float(selected_metrics["Mean_inconsistency"]),
            "Distance_to_ideal": float(selected_metrics["Distance_to_ideal"]),
        },
        "baseline_changes": {
            "net_value_vs_random_percent": float(100 * (proposed["NetValue_REU"] - random["NetValue_REU"]) /
            ↪  abs(random["NetValue_REU"])),
            "net_value_vs_soh_only_percent": float(100 * (proposed["NetValue_REU"] - soh["NetValue_REU"]) /
            ↪  abs(soh["NetValue_REU"])),
            "inconsistency_improvement_vs_random_percent": float(100 * (random["Mean_inconsistency"] -
            ↪  proposed["Mean_inconsistency"]) / random["Mean_inconsistency"]),
            "inconsistency_improvement_vs_soh_only_percent": float(100 * (soh["Mean_inconsistency"] -
            ↪  proposed["Mean_inconsistency"]) / soh["Mean_inconsistency"]),
            "utilization_change_vs_random_percentage_points": float(100 * (proposed["Utilization_rate"] -
            ↪  random["Utilization_rate"])),
            "utilization_change_vs_soh_only_percentage_points": float(100 * (proposed["Utilization_rate"] -
            ↪  soh["Utilization_rate"])),
        },
        "economic_unit": "REU; no CNY market-price claim",
        "hidden_truth_audit": audit,
        "runtime_seconds": float(time.perf_counter() - started),
        "q4_executed": False,
    }
```

```
    save_json(metrics, RESULT_TABLES / "q3_key_metrics.json")
    _write_latex_metrics(metrics)

    pool_audit = load_json(RESULT_TABLES / "q3_pool_leakage_audit.json")
    constraint_audit = load_json(RESULT_TABLES / "q3_constraint_verification.json")
    schema = pd.read_csv(RESULT_TABLES / "q3_frozen_interface_schema.csv")
    optimization_source = (ROOT / "src" / "problem3" / "optimization.py").read_text(encoding="utf-8")
    screening_source = (ROOT / "src" / "problem3" / "screening.py").read_text(encoding="utf-8")
    validation = {
        "phase2_5_pass": bool(phase2_5["overall_pass"]),
        "q3_pool_unseen_by_q2_training": bool(pool_audit["q2_training_ids_disjoint"]),
        "retired_pool_heterogeneous": float(decision["Estimated_SOH"].std()) > 2.0,
        "individual_80_percent_eol_rows_not_used_as_trivial_pool": bool(pool_audit["not_individual_80_percent_eol_rows"]),
        "first_life_and_second_life_rul_separated": "FirstLife_RUL_80_median" in decision.columns
        and "SecondLife_RUL_median" in projection.columns,
        "decision_view_contains_no_future_truth": not bool(schema["Future_truth"].any())
        and len(pool_audit["decision_leakage_hits"]) == 0,
        "hidden_truth_inaccessible_before_solution_freeze": "q3_hidden_audit_truth.csv" not in optimization_source
        and "q3_hidden_audit_truth.csv" not in screening_source
        and bool(audit["freeze_verified_before_truth_read"]),
        "safety_gate_precedes_economic_optimization": freeze["freeze_sequence"].index("hard gates")
        < freeze["freeze_sequence"].index("Pareto"),
        "abc_classification_reproducible": grade_reproducible and len(repeated_grades) == len(grades),
        "compatibility_graph_built": set(graphs) == set(SCENARIOS) and bool((graph_metrics["Nodes"] > 0).all()),
        "multiobjective_optimization_solved": bool((frontier["Pareto_nondominated"]).any()),
        "constraint_and_sympy_checks_pass": bool(constraint_audit["overall_pass"]),
        "baseline_comparisons_complete": len(baselines) == 4,
        "pareto_frontier_generated": int(frontier["Pareto_nondominated"].sum()) >= 3,
        "hidden_truth_audit_only_after_freeze": bool(audit["freeze_verified_before_truth_read"]),
        "all_paper_numbers_traceable_to_result_files": (ROOT / "paper" / "generated_metrics_q3.tex").exists(),
        "q4_not_executed": not any(RESULT_TABLES.glob("q4_*")),
        "full_pipeline_downstream_allowed": bool(allow_downstream),
    }
    validation["overall_pass"] = all(
        value
        for key, value in validation.items()
        if key not in {"q4_not_executed", "full_pipeline_downstream_allowed"}
    ) and (validation["q4_not_executed"] or allow_downstream)
    save_json(validation, RESULT_TABLES / "q3_validation.json")
    if not validation["overall_pass"]:
        raise RuntimeError(f"Q3 validation failed: {json.dumps(validation, ensure_ascii=False)}")
    _update_state(metrics)
    return metrics


if __name__ == "__main__":
    print(json.dumps(run_problem3(), ensure_ascii=False, indent=2))
```

src/problem3/phase3_5.py

```
"""PHASE 3.5: full-column Q3 audit, compromise audit, and economic mode.

This module never uses hidden simulation truth to choose a frontier point or
objective weight. Hidden truth is opened only after the full-column solution
has been selected and its membership hashes have been frozen.
"""

from __future__ import annotations

import hashlib
import itertools
import json
import time
from dataclasses import dataclass

import numpy as np
import pandas as pd

from src.problem3.audit import run_hidden_truth_audit
from src.problem3.optimization import _solution_metrics, _solve_milp, compare_baselines
from src.problem3.screening import SCENARIOS
from src.utils.project import RESULT_TABLES, load_json, save_json


WEIGHT_SCHEMES = (
```

```
    (1 / 3, 1 / 3, 1 / 3),
    (0.50, 0.25, 0.25),
    (0.25, 0.50, 0.25),
    (0.25, 0.25, 0.50),
    (0.60, 0.20, 0.20),
    (0.20, 0.60, 0.20),
    (0.20, 0.20, 0.60),
)


@dataclass
class FrontierResult:
    frontier: pd.DataFrame
    selected: pd.DataFrame
    selected_id: str
    selected_row: pd.Series
    selected_by_solution: dict[str, pd.DataFrame]
    solver_log: pd.DataFrame
    max_cells: int
    total_runtime: float
    geometric_knee_id: str | None
    geometric_method: str


def _sha256(path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def _pareto_mask(frame: pd.DataFrame) -> np.ndarray:
    keep: list[bool] = []
    for _, row in frame.iterrows():
        dominates = (
            (frame["NetValue_REU"] >= row["NetValue_REU"])
            & (frame["Utilization_rate"] >= row["Utilization_rate"])
            & (frame["Mean_inconsistency"] <= row["Mean_inconsistency"])
            & (
                (frame["NetValue_REU"] > row["NetValue_REU"])
                | (frame["Utilization_rate"] > row["Utilization_rate"])
                | (frame["Mean_inconsistency"] < row["Mean_inconsistency"])
            )
        )
        keep.append(not bool(dominates.any()))
    return np.asarray(keep, dtype=bool)


def _normalize_pareto(pareto: pd.DataFrame) -> pd.DataFrame:
    result = pareto.copy()
    definitions = {
        "NetValue_REU": True,
        "Utilization_rate": True,
        "Mean_inconsistency": False,
    }
    for column, maximize in definitions.items():
        values = result[column].to_numpy(dtype=float)
        span = float(np.ptp(values))
        benefit = np.ones(len(values)) if span <= 1e-12 else (values - values.min()) / span
        if not maximize:
            benefit = 1.0 - benefit
        result[f"Benefit_{column}"] = benefit
        result[f"Regret_{column}"] = 1.0 - benefit
    return result


def _geometric_knee(pareto: pd.DataFrame) -> tuple[str | None, str, pd.DataFrame]:
    """Audit a 3-D knee by distance from the CHIM extreme-point plane."""
    result = pareto.copy()
    result["Geometric_CHIM_distance"] = 0.0
    if len(result) < 4:
        return None, "not identifiable with fewer than four nondominated points", result
    columns = [
        "Benefit_NetValue_REU",
        "Benefit_Utilization_rate",
        "Benefit_Mean_inconsistency",
    ]
    points = result[columns].to_numpy(dtype=float)
    anchor_indices = {
        int(result[column].idxmax()) for column in columns
```

```
    }
    index_to_position = {index: pos for pos, index in enumerate(result.index)}
    anchor_positions = [index_to_position[index] for index in anchor_indices]
    method = "CHIM plane through the three single-objective anchors"
    if len(anchor_positions) < 3:
        best_area = -1.0
        best_combo: tuple[int, int, int] | None = None
        for combo in itertools.combinations(range(len(points)), 3):
            a, b, c = points[list(combo)]
            area = float(np.linalg.norm(np.cross(b - a, c - a)))
            if area > best_area:
                best_area = area
                best_combo = combo
        anchor_positions = list(best_combo or (0, 1, 2))
        method = "maximum-area fallback plane because single-objective anchors were not distinct"
    a, b, c = points[anchor_positions]
    normal = np.cross(b - a, c - a)
    norm = float(np.linalg.norm(normal))
    if norm <= 1e-12:
        return None, "not identifiable because the anchor plane is degenerate", result
    normal /= norm
    signed = (points - a) @ normal
    ideal_signed = float((np.ones(3) - a) @ normal)
    if ideal_signed < 0:
        signed = -signed
    result["Geometric_CHIM_distance"] = signed
    non_anchor = [i for i in range(len(points)) if i not in anchor_positions]
    if not non_anchor:
        return None, method + "; no non-anchor point", result
    knee_position = max(non_anchor, key=lambda i: signed[i])
    if signed[knee_position] <= 1e-10:
        return None, method + "; no positive bulge toward the ideal point", result
    return str(result.iloc[knee_position]["Solution_ID"]), method, result


def _solve_frontier(
    candidates: pd.DataFrame,
    pool_ids: list[str],
    label: str,
    *,
    nonnegative_groups: bool = False,
) -> FrontierResult:
    minimum_value = 0.0 if nonnegative_groups else None
    max_meta, max_selected = _solve_milp(
        candidates,
        pool_ids,
        1.0,
        0,
        objective="utilization",
        time_limit=20.0,
        mip_rel_gap=0.005,
        minimum_group_net_value=minimum_value,
    )
    if max_meta["status"] not in {"optimal", "limit_reached"} or max_selected.empty:
        raise RuntimeError(f"{label} maximum-utilization MILP failed: {max_meta}")
    max_cells = int(max_selected["Group_size"].sum())
    epsilon_values = sorted(
        set(np.quantile(candidates["Inconsistency"], [0.30, 0.50, 0.70, 0.90]).round(6))
    )
    frontier_rows: list[dict[str, object]] = []
    solver_rows: list[dict[str, object]] = [
        {"Solution_ID": f"{label}_MAXUTIL", "Inconsistency_epsilon": 1.0,
         "Minimum_cells": 0, **max_meta}
    ]
    selected_by_solution: dict[str, pd.DataFrame] = {}
    grid_index = 0
    for epsilon in epsilon_values:
        for fraction in (0.55, 0.75, 0.90, 1.00):
            grid_index += 1
            solution_id = f"{label}{grid_index:03d}"
            minimum_cells = int(np.floor(max_cells * fraction))
            metadata, selected = _solve_milp(
                candidates,
                pool_ids,
                float(epsilon),
                minimum_cells,
                time_limit=12.0,
```

```
                mip_rel_gap=0.005,
                minimum_group_net_value=minimum_value,
            )
            solver_rows.append(
                {"Solution_ID": solution_id, "Inconsistency_epsilon": float(epsilon),
                 "Minimum_cells": minimum_cells, **metadata}
            )
            if metadata["status"] not in {"optimal", "limit_reached"} or selected.empty:
                continue
            metrics = _solution_metrics(selected, len(pool_ids))
            frontier_rows.append(
                {
                    "Solution_ID": solution_id,
                    "Candidate_space": label,
                    "Inconsistency_epsilon": float(epsilon),
                    "Minimum_utilization_fraction_of_max": fraction,
                    "Maximum_feasible_cells": max_cells,
                    **metrics,
                    "Solver_status": metadata["status"],
                    "Solver_runtime_seconds": metadata["runtime_seconds"],
                    "Optimality_gap": metadata["mip_gap"],
                    "Selected_candidate_IDs": ";".join(selected["Candidate_ID"]),
                }
            )
            selected_by_solution[solution_id] = selected.copy()
    frontier = pd.DataFrame(frontier_rows)
    if frontier.empty:
        raise RuntimeError(f"{label}: all epsilon-constraint MILPs were infeasible")
    frontier["Pareto_nondominated"] = _pareto_mask(frontier)
    pareto = _normalize_pareto(frontier.loc[frontier["Pareto_nondominated"]])
    regret_columns = [
        "Regret_NetValue_REU", "Regret_Utilization_rate", "Regret_Mean_inconsistency"
    ]
    pareto["Distance_to_ideal"] = np.sqrt(
        np.mean(np.square(pareto[regret_columns].to_numpy(dtype=float)), axis=1)
    )
    selected_row = pareto.sort_values(["Distance_to_ideal", "Solution_ID"]).iloc[0]
    selected_id = str(selected_row["Solution_ID"])
    knee_id, geometric_method, pareto = _geometric_knee(pareto)
    merge_columns = [
        "Solution_ID", "Distance_to_ideal", "Geometric_CHIM_distance",
        *[f"Benefit_{column}" for column in ("NetValue_REU", "Utilization_rate", "Mean_inconsistency")],
        *regret_columns,
    ]
    frontier = frontier.merge(pareto[merge_columns], on="Solution_ID", how="left")
    frontier["Selected_ideal_point_compromise"] = frontier["Solution_ID"].eq(selected_id)
    frontier["Geometric_knee"] = frontier["Solution_ID"].eq(knee_id) if knee_id else False
    solver_log = pd.DataFrame(solver_rows)
    total_runtime = float(solver_log["runtime_seconds"].fillna(0.0).sum())
    return FrontierResult(
        frontier=frontier,
        selected=selected_by_solution[selected_id].copy(),
        selected_id=selected_id,
        selected_row=frontier.loc[frontier["Solution_ID"].eq(selected_id)].iloc[0],
        selected_by_solution=selected_by_solution,
        solver_log=solver_log,
        max_cells=max_cells,
        total_runtime=total_runtime,
        geometric_knee_id=knee_id,
        geometric_method=geometric_method,
    )


def _weight_audit(full: FrontierResult) -> pd.DataFrame:
    pareto = full.frontier.loc[full.frontier["Pareto_nondominated"]].copy()
    rows: list[dict[str, object]] = []
    regret_columns = [
        "Regret_NetValue_REU", "Regret_Utilization_rate", "Regret_Mean_inconsistency"
    ]
    for index, weights in enumerate(WEIGHT_SCHEMES, start=1):
        distance = np.sqrt(
            np.sum(
                np.square(pareto[regret_columns].to_numpy(dtype=float))
                * np.asarray(weights, dtype=float),
                axis=1,
            )
        )
```

```
            winner_position = int(np.argmin(distance))
            winner = pareto.iloc[winner_position]
            rows.append(
                {
                    "Weight_scheme": f"W{index}",
                    "Weight_value": weights[0],
                    "Weight_utilization": weights[1],
                    "Weight_consistency": weights[2],
                    "Selected_solution": winner["Solution_ID"],
                    "Weighted_distance_to_ideal": float(distance[winner_position]),
                    "NetValue_REU": float(winner["NetValue_REU"]),
                    "Utilization_rate": float(winner["Utilization_rate"]),
                    "Mean_inconsistency": float(winner["Mean_inconsistency"]),
                    "Geometric_knee": bool(winner["Geometric_knee"]),
                    "Uses_test_truth": False,
                }
            )
        frame = pd.DataFrame(rows)
        frequency = frame["Selected_solution"].value_counts()
        frame["Selection_frequency_among_7"] = frame["Selected_solution"].map(frequency)
        frame["Selection_share_among_7"] = frame["Selection_frequency_among_7"] / len(frame)
        return frame


    def _membership_signature(selected: pd.DataFrame) -> tuple[str, str]:
        ids = sorted(selected["Candidate_ID"].astype(str))
        members = sorted({member for value in selected["Members"] for member in str(value).split(";")})
        return ";".join(ids), hashlib.sha256(";".join(members).encode("utf-8")).hexdigest()


    def _audit_row(label: str, candidates: pd.DataFrame, result: FrontierResult) -> dict[str, object]:
        candidate_ids, member_hash = _membership_signature(result.selected)
        pareto = result.frontier.loc[result.frontier["Pareto_nondominated"]]
        selected_log = result.solver_log.loc[result.solver_log["Solution_ID"].eq(result.selected_id)].iloc[0]
        return {
            "Candidate_space": label,
            "Candidate_columns": int(len(candidates)),
            "Nondominated_solution_count": int(len(pareto)),
            "Maximum_utilization": float(result.frontier["Utilization_rate"].max()),
            "Maximum_NetValue_REU": float(result.frontier["NetValue_REU"].max()),
            "Minimum_mean_inconsistency": float(result.frontier["Mean_inconsistency"].min()),
            "Selected_compromise": result.selected_id,
            "Selected_candidate_membership": candidate_ids,
            "Selected_cell_membership_sha256": member_hash,
            "Selected_groups": int(result.selected_row["Complete_groups"]),
            "Selected_used_cells": int(result.selected_row["Used_cells"]),
            "Selected_utilization": float(result.selected_row["Utilization_rate"]),
            "Selected_NetValue_REU": float(result.selected_row["NetValue_REU"]),
            "Selected_mean_inconsistency": float(result.selected_row["Mean_inconsistency"]),
            "Selected_solver_runtime_seconds": float(selected_log["runtime_seconds"]),
            "Selected_optimality_gap": float(selected_log["mip_gap"]),
            "Total_frontier_runtime_seconds": result.total_runtime,
            "Geometric_knee_solution": result.geometric_knee_id or "not_identified",
        }


    def _solution_summary(mode: str, result: FrontierResult) -> dict[str, object]:
        row = result.selected_row
        allocation = result.selected["Scenario"].value_counts().to_dict()
        return {
            "Mode": mode,
            "Solution_ID": result.selected_id,
            "Groups": int(row["Complete_groups"]),
            "Used_cells": int(row["Used_cells"]),
            "Utilization_rate": float(row["Utilization_rate"]),
            "Mean_inconsistency": float(row["Mean_inconsistency"]),
            "NetValue_REU": float(row["NetValue_REU"]),
            "Negative_value_groups": int((result.selected["NetValue_REU"] < -1e-12).sum()),
            "S1_groups": int(allocation.get("S1_frequency_regulation", 0)),
            "S2_groups": int(allocation.get("S2_peak_shaving", 0)),
            "S3_groups": int(allocation.get("S3_backup", 0)),
        }


    def run_phase3_5() -> dict[str, object]:
        started = time.perf_counter()
        full_candidates = pd.read_csv(RESULT_TABLES / "q3_candidate_groups.csv")
```

```
    reduced_candidates = pd.read_csv(RESULT_TABLES / "q3_optimization_candidate_groups.csv")
    pool = pd.read_csv(RESULT_TABLES / "q3_decision_pool.csv")
    pool_ids = pool["Battery_ID"].astype(str).tolist()
    if len(full_candidates) != 599 or len(reduced_candidates) != 199:
        raise RuntimeError(
            f"PHASE 3.5 expected the audited 599/199 candidate libraries; got "
            f"{len(full_candidates)}/{len(reduced_candidates)}"
        )

    reduced = _solve_frontier(reduced_candidates, pool_ids, "R")
    full = _solve_frontier(full_candidates, pool_ids, "F")
    audit = pd.DataFrame(
        [_audit_row("reduced_199", reduced_candidates, reduced),
         _audit_row("full_599", full_candidates, full)]
    )
    audit["Selected_membership_changed_vs_reduced"] = (
        audit["Selected_cell_membership_sha256"] != audit.iloc[0]["Selected_cell_membership_sha256"]
    )
    audit.to_csv(RESULT_TABLES / "q3_full_candidate_space_audit.csv", index=False, float_format="%.8f")

    weight_audit = _weight_audit(full)
    weight_audit.to_csv(
        RESULT_TABLES / "q3_compromise_weight_sensitivity.csv", index=False, float_format="%.8f"
    )
    full.frontier.to_csv(RESULT_TABLES / "q3_pareto_solutions.csv", index=False, float_format="%.8f")
    full.solver_log.to_csv(RESULT_TABLES / "q3_solver_log.csv", index=False, float_format="%.8f")
    full_candidates.to_csv(
        RESULT_TABLES / "q3_optimization_candidate_groups.csv", index=False, float_format="%.8f"
    )
    baselines = compare_baselines(full_candidates, full.selected, len(pool))

    viability = _solve_frontier(full_candidates, pool_ids, "E", nonnegative_groups=True)
    economic = pd.DataFrame(
        [_solution_summary("Base full-space ideal-point compromise", full),
         _solution_summary("Nonnegative-group-value mode", viability)]
    )
    economic.to_csv(
        RESULT_TABLES / "q3_economic_viability_tradeoff.csv", index=False, float_format="%.8f"
    )
    knee_audit = pd.DataFrame(
        [{
            "Ideal_point_solution": full.selected_id,
            "Geometric_knee_solution": full.geometric_knee_id or "not_identified",
            "Same_solution": full.selected_id == full.geometric_knee_id,
            "Geometric_method": full.geometric_method,
            "Terminology": "geometric knee" if full.selected_id == full.geometric_knee_id else "ideal-point compromise",
        }]
    )
    knee_audit.to_csv(RESULT_TABLES / "q3_geometric_knee_audit.csv", index=False)

    full.selected.to_csv(RESULT_TABLES / "q3_selected_groups.csv", index=False, float_format="%.8f")
    selected_payload = {
        "Solution_ID": full.selected_id,
        "selection_method": "equal-weight normalized Euclidean distance to the Pareto ideal point",
        "terminology": "ideal-point compromise; geometric knee is reported separately",
        "geometric_knee_solution": full.geometric_knee_id,
        "raw_objectives": {
            "NetValue_REU": float(full.selected_row["NetValue_REU"]),
            "Utilization_rate": float(full.selected_row["Utilization_rate"]),
            "Mean_inconsistency": float(full.selected_row["Mean_inconsistency"]),
        },
        "Distance_to_ideal": float(full.selected_row["Distance_to_ideal"]),
        "Selected_candidate_IDs": full.selected["Candidate_ID"].astype(str).tolist(),
        "candidate_space": "all 599 complete feasible groups",
        "Solver": "scipy.optimize.milp (HiGHS)",
        "cuOpt_runtime": "unavailable; import check failed, so no cuOpt installation was attempted",
        "economic_unit": "REU",
        "weights_selected_without_test_truth": True,
    }
    save_json(selected_payload, RESULT_TABLES / "q3_selected_solution.json")
    freeze = {
        "frozen_before_hidden_truth_read": True,
        "selected_solution_sha256": _sha256(RESULT_TABLES / "q3_selected_solution.json"),
        "selected_groups_sha256": _sha256(RESULT_TABLES / "q3_selected_groups.csv"),
        "candidate_space": "full_599",
        "phase3_5_refreeze": True,
        "selection_uses_hidden_truth": False,
```

```
        "freeze_sequence": "599 candidates -> epsilon MILP -> ideal/geometric/weight audits -> select -> freeze -> hidden audit",
    }
    save_json(freeze, RESULT_TABLES / "q3_solution_freeze.json")
    hidden_audit = run_hidden_truth_audit()

    previous_metrics = load_json(RESULT_TABLES / "q3_key_metrics.json")
    row = full.selected_row
    allocation = {scenario: int((full.selected["Scenario"] == scenario).sum()) for scenario in SCENARIOS}
    previous_metrics["phase"] = "PHASE 3.5 full-space Q3 refreeze"
    previous_metrics["selected_solution"] = {
        "Solution_ID": full.selected_id,
        "Complete_groups": int(row["Complete_groups"]),
        "groups_by_scenario": allocation,
        "Used_cells": int(row["Used_cells"]),
        "Utilization_rate": float(row["Utilization_rate"]),
        "Mean_SOH_spread": float(row["Mean_SOH_spread"]),
        "Max_SOH_spread": float(row["Max_SOH_spread"]),
        "Mean_Capacity_CV": float(row["Mean_Capacity_CV"]),
        "Mean_Resistance_CV": float(row["Mean_Resistance_CV"]),
        "Mean_RUL_spread": float(row["Mean_RUL_spread"]),
        "Conservative_minimum_RUL": float(row["Conservative_minimum_RUL"]),
        "Energy_loss_proxy_kWh": float(row["Energy_loss_proxy_kWh"]),
        "NetValue_REU": float(row["NetValue_REU"]),
        "Mean_inconsistency": float(row["Mean_inconsistency"]),
        "Distance_to_ideal": float(row["Distance_to_ideal"]),
        "terminology": "ideal-point compromise",
    }
    previous_metrics["hidden_truth_audit"] = hidden_audit
    proposed = baselines.loc[baselines["Method"].str.startswith("Proposed")].iloc[0]
    random = baselines.loc[baselines["Method"].str.startswith("Random")].iloc[0]
    soh = baselines.loc[baselines["Method"].str.startswith("SOH-only")].iloc[0]
    previous_metrics["baseline_changes"] = {
        "net_value_vs_random_percent": float(
            100 * (proposed["NetValue_REU"] - random["NetValue_REU"]) / abs(random["NetValue_REU"])
        ),
        "net_value_vs_soh_only_percent": float(
            100 * (proposed["NetValue_REU"] - soh["NetValue_REU"]) / abs(soh["NetValue_REU"])
        ),
        "inconsistency_improvement_vs_random_percent": float(
            100 * (random["Mean_inconsistency"] - proposed["Mean_inconsistency"])
            / random["Mean_inconsistency"]
        ),
        "inconsistency_improvement_vs_soh_only_percent": float(
            100 * (soh["Mean_inconsistency"] - proposed["Mean_inconsistency"])
            / soh["Mean_inconsistency"]
        ),
        "utilization_change_vs_random_percentage_points": float(
            100 * (proposed["Utilization_rate"] - random["Utilization_rate"])
        ),
        "utilization_change_vs_soh_only_percentage_points": float(
            100 * (proposed["Utilization_rate"] - soh["Utilization_rate"])
        ),
    }
    previous_metrics["phase3_5"] = {
        "full_candidate_columns": 599,
        "reduced_candidate_columns": 199,
        "selected_membership_changed": bool(audit.iloc[1]["Selected_membership_changed_vs_reduced"]),
        "geometric_knee_solution": full.geometric_knee_id,
        "weight_modal_solution": str(weight_audit["Selected_solution"].mode().iloc[0]),
        "weight_modal_share": float(weight_audit["Selection_share_among_7"].max()),
        "economic_viability_solution": viability.selected_id,
    }
    previous_metrics["runtime_seconds"] = float(time.perf_counter() - started)
    save_json(previous_metrics, RESULT_TABLES / "q3_key_metrics.json")
    from src.problem3.analysis import _update_state, _write_latex_metrics

    _write_latex_metrics(previous_metrics)
    _update_state(previous_metrics)

    validation = load_json(RESULT_TABLES / "q3_validation.json")
    validation.update(
        {
            "full_candidate_space_audit_pass": len(full_candidates) == 599,
            "q3_final_solution_uses_full_candidate_space": True,
            "compromise_weight_sensitivity_complete": len(weight_audit) >= 7,
            "geometric_knee_audited": True,
            "economic_viability_mode_complete": bool(
```

```
                (viability.selected["NetValue_REU"] >= -1e-12).all()
            ),
            "phase3_5_refreeze_before_hidden_truth": bool(
                hidden_audit["freeze_verified_before_truth_read"]
            ),
        }
    )
    validation["overall_pass"] = all(
        bool(value) for key, value in validation.items()
        if key != "overall_pass" and not key.startswith("q4_")
    )
    save_json(validation, RESULT_TABLES / "q3_validation.json")
    payload = {
        "selected_solution": full.selected_id,
        "selected_membership_changed_vs_199": bool(
            audit.iloc[1]["Selected_membership_changed_vs_reduced"]
        ),
        "geometric_knee_solution": full.geometric_knee_id,
        "weight_modal_solution": str(weight_audit["Selected_solution"].mode().iloc[0]),
        "weight_modal_share": float(weight_audit["Selection_share_among_7"].max()),
        "runtime_seconds": float(time.perf_counter() - started),
        "validation_pass": bool(validation["overall_pass"]),
    }
    save_json(payload, RESULT_TABLES / "q3_phase3_5_summary.json")
    return payload


if __name__ == "__main__":
    print(json.dumps(run_phase3_5(), indent=2, ensure_ascii=False))
```

src/problem3/plots.py

```
"""Five evidence-focused, publication-ready Q3 figures."""

from __future__ import annotations

import shutil
from pathlib import Path

import matplotlib as mpl
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np
import pandas as pd

from src.utils.project import PAPER_FIGURES, RESULT_FIGURES, RESULT_TABLES


PALETTE = {
    "A": "#0072B2",
    "B": "#E69F00",
    "C": "#009E73",
    "Reject": "#777777",
    "S1_frequency_regulation": "#0072B2",
    "S2_peak_shaving": "#E69F00",
    "S3_backup": "#009E73",
}
MARKERS = {"A": "o", "B": "s", "C": "^", "Reject": "x"}


def _style() -> None:
    mpl.rcParams.update(
        {
            "font.family": "sans-serif",
            "font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
            "font.size": 8,
            "axes.labelsize": 8,
            "axes.titlesize": 9,
            "legend.fontsize": 7,
            "xtick.labelsize": 7,
            "ytick.labelsize": 7,
            "axes.spines.top": False,
            "axes.spines.right": False,
            "pdf.fonttype": 42,
            "ps.fonttype": 42,
```

```
            "svg.fonttype": "none",
            "figure.facecolor": "white",
            "savefig.facecolor": "white",
        }
    )


def _save(fig: plt.Figure, name: str) -> None:
    RESULT_FIGURES.mkdir(parents=True, exist_ok=True)
    PAPER_FIGURES.mkdir(parents=True, exist_ok=True)
    for suffix in ("pdf", "svg", "png"):
        output = RESULT_FIGURES / f"{name}.{suffix}"
        fig.savefig(output, dpi=360 if suffix == "png" else None, bbox_inches=None)
        if suffix in {"pdf", "png"}:
            shutil.copy2(output, PAPER_FIGURES / output.name)
    plt.close(fig)


def _health_grades() -> None:
    grades = pd.read_csv(RESULT_TABLES / "q3_battery_grades.csv")
    fig, ax = plt.subplots(figsize=(6.7, 3.8), layout="constrained")
    for grade in ("Reject", "C", "B", "A"):
        group = grades.loc[grades["Grade"] == grade]
        ax.scatter(
            group["SOH_lower_90"],
            group["Internal_resistance"],
            s=22 if grade != "Reject" else 16,
            c=PALETTE[grade],
            marker=MARKERS[grade],
            alpha=0.72,
            linewidths=0.45,
            label=f"{grade} (n={len(group)})",
        )
    ax.set(
        xlabel="Conservative SOH lower bound (%)",
        ylabel="Internal resistance (mOhm)",
        title="Retired-pool health heterogeneity and scenario-priority grades",
    )
    ax.grid(axis="both", color="#DDDDDD", linewidth=0.5, alpha=0.55)
    ax.legend(frameon=False, ncol=2)
    _save(fig, "q3_retired_pool_grades")


def _screening_reasons() -> None:
    screening = pd.read_csv(RESULT_TABLES / "q3_safety_screening.csv")
    reasons = [
        "SOH insufficient", "RUL insufficient", "resistance excessive", "voltage inconsistency",
        "RUL uncertainty excessive", "OOD risk",
    ]
    labels = ["SOH", "RUL", "Resistance", "Voltage", "RUL uncertainty", "OOD"]
    y = np.arange(len(reasons))
    width = 0.22
    fig, ax = plt.subplots(figsize=(6.7, 3.8), layout="constrained")
    for index, scenario in enumerate(("S1_frequency_regulation", "S2_peak_shaving", "S3_backup")):
        group = screening.loc[screening["Scenario"] == scenario]
        counts = [int(group["Reject_reason"].str.contains(reason, regex=False).sum()) for reason in reasons]
        ax.barh(
            y + (index - 1) * width,
            counts,
            height=width * 0.9,
            color=PALETTE[scenario],
            alpha=0.86,
            label=scenario.split("_", 1)[0],
        )
    ax.set_yticks(y, labels)
    ax.invert_yaxis()
    ax.set(
        xlabel="Cells failing each hard gate (count; reasons may overlap)",
        title="Safety/quality gates are applied before economic optimization",
    )
    ax.set_xlim(left=0)
    ax.grid(axis="x", color="#DDDDDD", linewidth=0.5, alpha=0.55)
    ax.legend(frameon=False, title="Scenario")
    _save(fig, "q3_safety_gate_failures")


def _compatibility_summary() -> None:
```

```
    grades = pd.read_csv(RESULT_TABLES / "q3_battery_grades.csv")
    screening = pd.read_csv(RESULT_TABLES / "q3_safety_screening.csv")
    metrics = pd.read_csv(RESULT_TABLES / "q3_compatibility_graph_metrics.csv")
    selected = pd.read_csv(RESULT_TABLES / "q3_selected_groups.csv")
    edges = pd.read_csv(RESULT_TABLES / "q3_compatibility_edges.csv")

    fig, axes = plt.subplots(2, 2, figsize=(7.0, 5.25), layout="constrained")

    grade_labels = {"Reject": " 拒用", "C": "C", "B": "B", "A": "A"}
    for grade in ("Reject", "C", "B", "A"):
        group = grades.loc[grades["Grade"] == grade]
        axes[0, 0].scatter(
            group["SOH_lower_90"],
            group["Internal_resistance"],
            s=13 if grade != "Reject" else 10,
            c=PALETTE[grade],
            marker=MARKERS[grade],
            alpha=0.64,
            linewidths=0.4,
            label=f"{grade_labels[grade]}（n={len(group)}）",
        )
    axes[0, 0].set(
        xlabel=" 保守 SOH 下界（%）",
        ylabel=" 内阻（mΩ）",
        title="(a) 退役电池保守分级",
    )
    axes[0, 0].grid(color="#DDDDDD", linewidth=0.45, alpha=0.5)
    axes[0, 0].legend(frameon=False, ncol=2, fontsize=6.6, handletextpad=0.3)

    reasons = [
        "SOH insufficient",
        "RUL insufficient",
        "resistance excessive",
        "voltage inconsistency",
        "RUL uncertainty excessive",
        "OOD risk",
    ]
    reason_labels = ["SOH", "RUL", " 内阻", " 电压", "RUL 区间", "OOD"]
    y = np.arange(len(reasons))
    width = 0.22
    scenario_hatches = ["///", "\\\\", "..."]
    for index, (scenario, hatch) in enumerate(
        zip(("S1_frequency_regulation", "S2_peak_shaving", "S3_backup"), scenario_hatches)
    ):
        group = screening.loc[screening["Scenario"] == scenario]
        counts = [int(group["Reject_reason"].str.contains(reason, regex=False).sum()) for reason in reasons]
        axes[0, 1].barh(
            y + (index - 1) * width,
            counts,
            height=width * 0.9,
            color=PALETTE[scenario],
            alpha=0.86,
            label=scenario.split("_", 1)[0],
            hatch=hatch,
            edgecolor="white",
            linewidth=0.4,
        )
    axes[0, 1].set_yticks(y, reason_labels)
    axes[0, 1].invert_yaxis()
    axes[0, 1].set(
        xlabel=" 未通过电池数（原因可重叠）",
        title="(b) 硬门控失败原因",
        xlim=(0, None),
    )
    axes[0, 1].grid(axis="x", color="#DDDDDD", linewidth=0.45, alpha=0.5)
    axes[0, 1].legend(frameon=False, fontsize=6.6, title=" 场景", title_fontsize=6.8)

    labels = ["S1", "S2", "S3"]
    colors = [PALETTE[scenario] for scenario in metrics["Scenario"]]
    bars = axes[1, 0].bar(labels, metrics["Density"], color=colors, width=0.62)
    for bar, hatch in zip(bars, scenario_hatches):
        bar.set_hatch(hatch)
        bar.set_edgecolor("white")
        bar.set_linewidth(0.4)
    axes[1, 0].set(
        ylim=(0.0, 1.0),
        ylabel=" 图密度",
```

```
        title="(c) 门控后的兼容图密度",
    )
    axes[1, 0].grid(axis="y", color="#DDDDDD", linewidth=0.45, alpha=0.5)
    for bar, (_, row) in zip(bars, metrics.iterrows()):
        axes[1, 0].text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.035,
            f"n={int(row['Nodes'])}\ne={int(row['Edges'])}",
            ha="center",
            va="bottom",
            fontsize=7,
        )

    exemplar = selected.iloc[0]
    members = str(exemplar["Members"]).split(";")
    scenario = str(exemplar["Scenario"])
    graph = nx.Graph()
    graph.add_nodes_from(members)
    scenario_edges = edges.loc[edges["Scenario"] == scenario]
    member_set = set(members)
    graph.add_edges_from(
        (row.Battery_i, row.Battery_j)
        for row in scenario_edges.itertuples()
        if row.Battery_i in member_set and row.Battery_j in member_set
    )
    positions = nx.circular_layout(graph)
    nx.draw_networkx_edges(graph, positions, ax=axes[1, 1], alpha=0.28, width=0.75, edge_color="#777777")
    nx.draw_networkx_nodes(
        graph, positions, ax=axes[1, 1], node_color=PALETTE[scenario], node_size=125,
        edgecolors="white", linewidths=0.8,
    )
    nx.draw_networkx_labels(
        graph,
        positions,
        ax=axes[1, 1],
        labels={node: node.split("_")[-1] for node in graph},
        font_size=5.8,
    )
    axes[1, 1].set_title(f"(d) {scenario[:2]} 完整组: {graph.number_of_edges()} 条配对边")
    axes[1, 1].set_axis_off()
    _save(fig, "q3_compatibility_graph_summary")


def _pareto_frontier() -> None:
    pareto = pd.read_csv(RESULT_TABLES / "q3_pareto_solutions.csv")
    fig, ax = plt.subplots(figsize=(6.7, 3.8), layout="constrained")
    scatter = ax.scatter(
        pareto["Mean_inconsistency"],
        pareto["NetValue_REU"],
        s=50 + 500 * pareto["Utilization_rate"],
        c=pareto["Utilization_rate"],
        cmap="viridis",
        edgecolors="#333333",
        linewidths=0.55,
        alpha=0.88,
    )
    selection_column = (
        "Selected_ideal_point_compromise"
        if "Selected_ideal_point_compromise" in pareto.columns
        else "Selected_knee_compromise"
    )
    selected = pareto.loc[pareto[selection_column]]
    ax.scatter(
        selected["Mean_inconsistency"], selected["NetValue_REU"], marker="*", s=230,
        facecolor="#D55E00", edgecolor="black", linewidth=0.8, label=" 理想点折中解",
    )
    for row in pareto.itertuples():
        offset = (8, 10) if bool(getattr(row, selection_column)) else (4, 3)
        ax.annotate(
            row.Solution_ID,
            (row.Mean_inconsistency, row.NetValue_REU),
            xytext=offset,
            textcoords="offset points",
            fontsize=6,
        )
    ax.set(
        xlabel=" 组内平均不一致性（越低越优）",
```

```
        ylabel=" 保守净价值（REU; 越高越优）",
        title="$\\varepsilon$-约束 Pareto 解（点大小同时编码利用率）",
    )
    ax.grid(color="#DDDDDD", linewidth=0.5, alpha=0.55)
    colorbar = fig.colorbar(scatter, ax=ax)
    colorbar.set_label(" 退役池利用率")
    ax.legend(frameon=False)
    _save(fig, "q3_pareto_frontier")


def _baseline_tradeoffs() -> None:
    baseline = pd.read_csv(RESULT_TABLES / "q3_baseline_comparison.csv")
    names = ["Random", "SOH-only", "Greedy", "Proposed"]
    colors = ["#777777", "#E69F00", "#009E73", "#0072B2"]
    markers = ["x", "s", "^", "o"]
    fig, axes = plt.subplots(1, 2, figsize=(6.7, 3.25), layout="constrained")
    for index, row in baseline.reset_index(drop=True).iterrows():
        axes[0].scatter(row["Utilization_rate"], row["NetValue_REU"], color=colors[index], marker=markers[index], s=58)
        axes[0].annotate(names[index], (row["Utilization_rate"], row["NetValue_REU"]), xytext=(4, 3), textcoords="offset points",
        ↪  fontsize=6.5)
        axes[1].scatter(row["Mean_inconsistency"], row["Mean_risk_proxy"], color=colors[index], marker=markers[index], s=58)
        axes[1].annotate(names[index], (row["Mean_inconsistency"], row["Mean_risk_proxy"]), xytext=(4, 3), textcoords="offset
        ↪  points", fontsize=6.5)
    axes[0].set(
        xlabel="Retired-pool utilization rate",
        ylabel="Net value (REU)",
        title="a  Value--utilization trade-off",
    )
    axes[1].set(
        xlabel="Mean inconsistency (lower is better)",
        ylabel="Mean model-risk proxy (lower is better)",
        title="b  Consistency--risk trade-off",
    )
    for ax in axes:
        ax.grid(color="#DDDDDD", linewidth=0.5, alpha=0.55)
    _save(fig, "q3_baseline_tradeoffs")


def create_problem3_figures() -> pd.DataFrame:
    _style()
    _health_grades()
    _screening_reasons()
    _compatibility_summary()
    _pareto_frontier()
    _baseline_tradeoffs()
    rows = [
        ("q3_retired_pool_grades", "q3_battery_grades.csv", "The retired pool is heterogeneous and grades follow scenario
        ↪  eligibility."),
        ("q3_safety_gate_failures", "q3_safety_screening.csv", "Hard screening excludes unsafe/high-risk proxies before
        ↪  economics."),
        ("q3_compatibility_graph_summary",
        ↪  "q3_battery_grades.csv;q3_safety_screening.csv;q3_compatibility_graph_metrics.csv;q3_compatibility_edges.csv",
        ↪  "Screening, grading, compatibility density and one complete group form a traceable evidence chain."),
        ("q3_pareto_frontier", "q3_pareto_solutions.csv", "The selected solution is a compromise across raw value, inconsistency,
        ↪  and utilization."),
        ("q3_baseline_tradeoffs", "q3_baseline_comparison.csv", "The proposed method trades some single-objective value for
        ↪  utilization and consistency."),
    ]
    manifest = pd.DataFrame(rows, columns=["Figure", "Source_tables", "Claim"])
    manifest["Formats"] = "PDF;SVG;PNG"
    manifest["Color_redundancy"] = "marker/label/panel redundancy used"
    manifest.to_csv(RESULT_TABLES / "q3_figure_manifest.csv", index=False)
    return manifest


if __name__ == "__main__":
    print(create_problem3_figures().to_string(index=False))
```

src/problem4/population.py

```
"""Independent Q4 calibration and final-holdout retired-cell populations."""

from __future__ import annotations

import hashlib
```

```python
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd

from src.problem2.build_landmark_dataset import HEALTH_CURRENT, OPERATING_CURRENT
from src.problem3.build_retired_pool import (
    CellSimulation,
    _freeze_retirement_rule,
    _inspection_features,
    _pack_inspection_table,
    _simulate_cell,
    _slope,
)
from src.utils.project import DATA_PROCESSED, PARAMETER_SOURCES, RESULT_MODELS, load_json, save_json


Q4_SEED_OFFSET = 4000
Q4_CALIBRATION_PACKS = 6
Q4_HOLDOUT_PACKS = 6
Q4_CELLS_PER_PACK = 40


def _relabel(simulation: CellSimulation, old_prefix: str, new_prefix: str) -> CellSimulation:
    history = simulation.history.copy()
    old_battery = str(simulation.hidden["Battery_ID"])
    old_batch = str(simulation.hidden["Batch_ID"])
    new_battery = old_battery.replace(old_prefix, new_prefix, 1)
    new_batch = old_batch.replace(old_prefix, new_prefix, 1)
    history["Battery_ID"] = new_battery
    history["Batch_ID"] = new_batch
    hidden = dict(simulation.hidden)
    hidden["Battery_ID"] = new_battery
    hidden["Batch_ID"] = new_batch
    return CellSimulation(history=history, hidden=hidden)


def _pack_effects(rng: np.random.Generator) -> dict[str, float | str]:
    strategies = np.array(["CC-CV", "Multi-stage", "Fast-charge"])
    return {
        "capacity": float(rng.normal(1.0, 0.014)),
        "resistance": float(rng.normal(1.0, 0.065)),
        "fade": float(rng.lognormal(0.0, 0.18)),
        "temperature": float(rng.choice([12.0, 20.0, 28.0, 36.0, 42.0])),
        "charge_rate": float(rng.choice([0.6, 1.0, 1.4, 1.8])),
        "discharge_rate": float(rng.choice([0.6, 1.0, 1.4, 1.8])),
        "dod": float(rng.uniform(0.58, 0.96)),
        "strategy": str(rng.choice(strategies, p=[0.45, 0.30, 0.25])),
    }


def _predict_decision_view(features: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    artifact = joblib.load(RESULT_MODELS / "q2_deployment_rul_model.joblib")
    soh_radius = float(artifact["soh_conformal_radius_90"])
    estimated_soh = np.clip(
        artifact["soh_estimator"].predict(features[artifact["soh_features"]]), 0.0, 100.0
    )
    deployment_features = features.copy()
    deployment_features["SOH_t"] = estimated_soh
    matrix = artifact["rul_preprocessor"].transform(
        deployment_features[artifact["rul_features"]]
    )
    aft = artifact["aft"]
    rul_median = np.clip(aft.predict_median(matrix), 0.0, 3000.0)
    rul_lower, rul_upper = aft.predict_interval(matrix, coverage=0.90)
    radius = float(artifact["rul_conformal_radius_90"])
    rul_lower = np.minimum(
        np.clip(rul_lower, 0.0, 4000.0), np.maximum(0.0, rul_median - radius)
    )
    rul_upper = np.maximum(np.clip(rul_upper, 0.0, 4000.0), rul_median + radius)

    q2_train = pd.read_csv(DATA_PROCESSED / "q2_landmark_dataset.csv")
    q2_train = q2_train.loc[q2_train["Split"] == "train"].copy()
    numeric_features = [
        column for column in artifact["rul_features"] if q2_train[column].dtype != object
```

```
    ]
    categorical_features = [
        column for column in artifact["rul_features"] if column not in numeric_features
    ]
    numeric_ood = (
        (deployment_features[numeric_features] < q2_train[numeric_features].min())
        | (deployment_features[numeric_features] > q2_train[numeric_features].max())
    ).sum(axis=1)
    categorical_ood = sum(
        ~deployment_features[column].astype(str).isin(set(q2_train[column].astype(str)))
        for column in categorical_features
    )
    ood_score = (numeric_ood + categorical_ood) / len(artifact["rul_features"])
    decision = pd.DataFrame(
        {
            "Battery_ID": features["Battery_ID"],
            "Batch_ID": features["Batch_ID"],
            "Inspection_cycle": features["Landmark_cycle"].astype(int),
            "Estimated_SOH": estimated_soh,
            "SOH_lower_90": np.clip(estimated_soh - soh_radius, 0.0, 100.0),
            "SOH_upper_90": np.clip(estimated_soh + soh_radius, 0.0, 100.0),
            "Measured_or_estimated_capacity": features["Capacity_t"],
            "Internal_resistance": features["Internal_resistance"],
            "Voltage_dispersion": features["Voltage_dispersion"],
            "End_of_discharge_voltage": features["End_of_discharge_voltage"],
            "Coulombic_efficiency": features["Coulombic_efficiency"],
            "Energy_efficiency": features["Energy_efficiency"],
            "Estimated_recent_degradation_slope": features[
                "Estimated_recent_degradation_slope"
            ],
            "FirstLife_RUL_80_median": rul_median,
            "FirstLife_RUL_80_lower": rul_lower,
            "FirstLife_RUL_80_upper": rul_upper,
            "SOH_uncertainty_width": 2.0 * soh_radius,
            "RUL_uncertainty_width": rul_upper - rul_lower,
            "Out_of_training_support_score": ood_score,
        }
    )
    decision["Risk_flag"] = np.select(
        [decision["Out_of_training_support_score"] > 0.25,
         decision["RUL_uncertainty_width"] > 900.0],
        ["OOD_risk", "wide_RUL_interval"],
        default="within_model_support",
    )
    return decision, q2_train


def build_q4_population(
    split: str,
    output_dir: Path,
    *,
    seed: int,
    id_prefix: str,
    pack_count: int,
    retirement_thresholds: dict[str, float] | None = None,
) -> dict[str, object]:
    """Create one Q4 population and write decision/truth to separate files."""
    output_dir.mkdir(parents=True, exist_ok=True)
    cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
    rng = np.random.default_rng(seed)
    all_packs: dict[str, list[CellSimulation]] = {}
    inspection_frames: list[pd.DataFrame] = []
    for pack_index in range(pack_count):
        effects = _pack_effects(rng)
        simulations = []
        for cell_index in range(Q4_CELLS_PER_PACK):
            q3_sim = _simulate_cell(cfg, rng, pack_index, cell_index, effects)
            simulations.append(_relabel(q3_sim, "Q3", id_prefix))
        batch_id = f"{id_prefix}P{pack_index + 1:02d}"
        all_packs[batch_id] = simulations
        inspection_frames.append(_pack_inspection_table(simulations))
    inspections = pd.concat(inspection_frames, ignore_index=True)
    if retirement_thresholds is None:
        thresholds, threshold_table = _freeze_retirement_rule(inspections)
        threshold_source = "q4_calibration_only"
    else:
        thresholds = {key: float(value) for key, value in retirement_thresholds.items()}
```

```
        threshold_table = pd.DataFrame(
            [{"Parameter": key, "Value": value, "Source": "frozen Q4 calibration"}
             for key, value in thresholds.items()]
        )
        threshold_source = "frozen_from_disjoint_q4_calibration"

    retirements: list[dict[str, object]] = []
    feature_rows: list[dict[str, object]] = []
    hidden_rows: list[dict[str, object]] = []
    for batch_id, simulations in all_packs.items():
        pack_rows = inspections.loc[inspections["Batch_ID"] == batch_id].sort_values(
            "Inspection_cycle"
        )
        eligible = pack_rows.loc[
            pack_rows["Inspection_cycle"] >= thresholds["Earliest_retirement_cycle"]
        ]
        trigger = eligible.loc[
            (
                (eligible["SOH_p10"] <= thresholds["SOH_p10_threshold_percent"])
                & (
                    (eligible["Capacity_CV"] >= thresholds["Capacity_CV_threshold"])
                    | (eligible["Resistance_CV"] >= thresholds["Resistance_CV_threshold"])
                )
            )
            | (
                eligible["SOH_p10"]
                <= thresholds["SOH_p10_threshold_percent"]
                - thresholds["Emergency_SOH_margin_percent"]
            )
        ]
        if len(trigger):
            retirement = trigger.iloc[0]
            rule = "joint_health_dispersion_trigger"
        else:
            retirement = eligible.loc[
                eligible["Inspection_cycle"] <= thresholds["Latest_retirement_cycle"]
            ].iloc[-1]
            rule = "latest_checkpoint_administrative_retirement"
        retirement_cycle = int(retirement["Inspection_cycle"])
        retirements.append({**retirement.to_dict(), "Retirement_rule": rule})
        for item in simulations:
            feature_rows.append(_inspection_features(item.history, retirement_cycle))
            current = item.history.loc[item.history["Cycle"] == retirement_cycle].iloc[0]
            hidden_rows.append(
                {
                    **item.hidden,
                    "Retirement_cycle": retirement_cycle,
                    "True_SOH_at_retirement": float(current["SOH"]),
                    "True_capacity_at_retirement_Ah": float(current["Capacity"]),
                    "True_resistance_at_retirement_mOhm": float(current["Internal_resistance"]),
                    "True_recent_SOH_slope": _slope(
                        item.history.loc[
                            item.history["Cycle"].between(
                                retirement_cycle - 100, retirement_cycle
                            )
                        ],
                        "SOH",
                    ),
                }
            )
    features = pd.DataFrame(feature_rows)
    hidden = pd.DataFrame(hidden_rows)
    decision, q2_train = _predict_decision_view(features)
    forbidden = ("True_", "Fade_", "Knee_", "Stress_index", "EOL_cycle")
    leakage_hits = [
        column for column in decision.columns if any(token in column for token in forbidden)
    ]
    if leakage_hits:
        raise RuntimeError(f"Q4 decision-view leakage: {leakage_hits}")

    decision_path = output_dir / "decision_pool.csv"
    hidden_path = output_dir / "hidden_truth.csv"
    decision.to_csv(decision_path, index=False, float_format="%.8f")
    hidden.to_csv(hidden_path, index=False, float_format="%.8f")
    pd.DataFrame(retirements).to_csv(
        output_dir / "retirement_audit.csv", index=False, float_format="%.8f"
    )
```

```
    inspections.to_csv(output_dir / "pack_checkpoint_profiles.csv", index=False, float_format="%.8f")
    threshold_table.to_csv(
        output_dir / "retirement_rule_parameters.csv", index=False, float_format="%.8f"
    )
    save_json(thresholds, output_dir / "retirement_thresholds.json")
    manifest = {
        "split": split,
        "seed": seed,
        "id_prefix": id_prefix,
        "cells": int(len(decision)),
        "packs": int(decision["Batch_ID"].nunique()),
        "retirement_threshold_source": threshold_source,
        "q2_training_ids_disjoint": bool(
            set(decision["Battery_ID"]).isdisjoint(set(q2_train["Battery_ID"]))
        ),
        "decision_hidden_columns_disjoint": not bool(leakage_hits),
        "decision_sha256": hashlib.sha256(decision_path.read_bytes()).hexdigest(),
        "hidden_sha256": hashlib.sha256(hidden_path.read_bytes()).hexdigest(),
    }
    save_json(manifest, output_dir / "population_manifest.json")
    return {"decision": decision, "hidden": hidden, "thresholds": thresholds, "manifest": manifest}


def build_calibration_and_holdout(root: Path) -> dict[str, object]:
    cfg = load_json(PARAMETER_SOURCES / "simulation_config.json")
    q4_seed = int(cfg["random_seed"]) + Q4_SEED_OFFSET
    calibration_dir = root / "q4_calibration"
    holdout_dir = root / "q4_final_holdout"
    calibration = build_q4_population(
        "calibration",
        calibration_dir,
        seed=q4_seed,
        id_prefix="Q4C",
        pack_count=Q4_CALIBRATION_PACKS,
    )
    holdout = build_q4_population(
        "final_holdout",
        holdout_dir,
        seed=q4_seed + 1000,
        id_prefix="Q4H",
        pack_count=Q4_HOLDOUT_PACKS,
        retirement_thresholds=calibration["thresholds"],
    )
    cal_ids = set(calibration["decision"]["Battery_ID"])
    holdout_ids = set(holdout["decision"]["Battery_ID"])
    payload = {
        "Q4_SEED": q4_seed,
        "calibration_seed": q4_seed,
        "final_holdout_seed": q4_seed + 1000,
        "calibration_cells": int(len(cal_ids)),
        "final_holdout_cells": int(len(holdout_ids)),
        "calibration_holdout_ids_disjoint": cal_ids.isdisjoint(holdout_ids),
        "q3_ids_disjoint": not any(value.startswith("Q3") for value in cal_ids | holdout_ids),
        "q2_training_ids_disjoint": bool(
            calibration["manifest"]["q2_training_ids_disjoint"]
            and holdout["manifest"]["q2_training_ids_disjoint"]
        ),
    }
    save_json(payload, root / "results" / "tables" / "q4_population_independence.json")
    return payload


if __name__ == "__main__":
    from src.utils.project import ROOT

    print(json.dumps(build_calibration_and_holdout(ROOT), indent=2))
```

src/problem4/core.py

```
"""Q4 scenarios, frozen prediction pipeline, and probabilistic compatibility graphs."""

from __future__ import annotations

import hashlib
import itertools
```

```
import json
from pathlib import Path

import networkx as nx
import numpy as np
import pandas as pd

from src.problem3.optimization import _clique_from_seed, _compatible, _group_metrics
from src.problem3.screening import SCENARIOS as APPLICATIONS
from src.problem3.screening import _heterogeneity_sigma, _scenario_parameters
from src.utils.project import PARAMETER_SOURCES, load_json, save_json


STRESS_SCENARIOS = (
    "S0_baseline",
    "S1_long_term_high_temperature",
    "S2_frequent_high_C_rate",
    "S3_high_batch_heterogeneity",
    "S4_temperature_plus_rate",
    "S5_heterogeneity_plus_operating_stress",
)


def stress_parameters() -> dict[str, dict[str, float]]:
    return {
        "S0_baseline": {
            "temperature_delta_C": 0.0,
            "c_rate_multiplier": 1.0,
            "batch_heterogeneity_multiplier": 1.0,
            "prediction_uncertainty_multiplier": 1.0,
            "degradation_stress_multiplier": 1.0,
            "revenue_multiplier": 1.0,
            "reassembly_cost_multiplier": 1.0,
        },
        "S1_long_term_high_temperature": {
            "temperature_delta_C": 10.0,
            "c_rate_multiplier": 1.0,
            "batch_heterogeneity_multiplier": 1.0,
            "prediction_uncertainty_multiplier": 1.08,
            "degradation_stress_multiplier": 1.34,
            "revenue_multiplier": 0.98,
            "reassembly_cost_multiplier": 1.05,
        },
        "S2_frequent_high_C_rate": {
            "temperature_delta_C": 0.0,
            "c_rate_multiplier": 1.50,
            "batch_heterogeneity_multiplier": 1.0,
            "prediction_uncertainty_multiplier": 1.10,
            "degradation_stress_multiplier": 1.42,
            "revenue_multiplier": 0.98,
            "reassembly_cost_multiplier": 1.08,
        },
        "S3_high_batch_heterogeneity": {
            "temperature_delta_C": 0.0,
            "c_rate_multiplier": 1.0,
            "batch_heterogeneity_multiplier": 1.65,
            "prediction_uncertainty_multiplier": 1.20,
            "degradation_stress_multiplier": 1.12,
            "revenue_multiplier": 0.99,
            "reassembly_cost_multiplier": 1.12,
        },
        "S4_temperature_plus_rate": {
            "temperature_delta_C": 10.0,
            "c_rate_multiplier": 1.50,
            "batch_heterogeneity_multiplier": 1.10,
            "prediction_uncertainty_multiplier": 1.18,
            "degradation_stress_multiplier": 1.90,
            "revenue_multiplier": 0.95,
            "reassembly_cost_multiplier": 1.15,
        },
        "S5_heterogeneity_plus_operating_stress": {
            "temperature_delta_C": 6.0,
            "c_rate_multiplier": 1.30,
            "batch_heterogeneity_multiplier": 1.65,
            "prediction_uncertainty_multiplier": 1.25,
            "degradation_stress_multiplier": 1.78,
            "revenue_multiplier": 0.94,
```

```
            "reassembly_cost_multiplier": 1.22,
        },
    }


def write_scenario_registry(path: Path) -> pd.DataFrame:
    parameters = stress_parameters()
    units = {
        "temperature_delta_C": "degC",
        "c_rate_multiplier": "ratio",
        "batch_heterogeneity_multiplier": "ratio",
        "prediction_uncertainty_multiplier": "ratio",
        "degradation_stress_multiplier": "ratio",
        "revenue_multiplier": "ratio",
        "reassembly_cost_multiplier": "ratio",
    }
    rationale = {
        "temperature_delta_C": "Long-duration ambient/thermal-management stress imposed in simulation.",
        "c_rate_multiplier": "Frequency of high-rate operation relative to baseline dispatch.",
        "batch_heterogeneity_multiplier": "Amplifies between-cell fade dispersion without changing nominal chemistry.",
        "prediction_uncertainty_multiplier": "Wider predictive distributions under out-of-baseline operation.",
        "degradation_stress_multiplier": "Combined semi-empirical life-acceleration factor used in forward simulation.",
        "revenue_multiplier": "Scenario revenue derating in relative economic units, not a market-price forecast.",
        "reassembly_cost_multiplier": "Extra sorting/reassembly effort under heterogeneous or stressed operation.",
    }
    baseline = parameters["S0_baseline"]
    rows = []
    for scenario, values in parameters.items():
        for parameter, value in values.items():
            rows.append(
                {
                    "Scenario": scenario,
                    "Parameter": parameter,
                    "Baseline": baseline[parameter],
                    "Scenario_value": value,
                    "Unit": units[parameter],
                    "Source_type": "scenario_assumption",
                    "Rationale": rationale[parameter]
                    + " These are stress-test settings, not safety boundaries.",
                }
            )
    frame = pd.DataFrame(rows)
    frame.to_csv(path, index=False, float_format="%.8f")
    return frame


def project_second_life(
    decision: pd.DataFrame,
    output_dir: Path,
    *,
    seed: int,
) -> pd.DataFrame:
    params = _scenario_parameters()
    sigma = _heterogeneity_sigma()
    rows: list[dict[str, object]] = []
    for application_index, application in enumerate(APPLICATIONS):
        p = params[application]
        for _, cell in decision.iterrows():
            stable = int(
                hashlib.sha256(f"{application}|{cell['Battery_ID']}".encode()).hexdigest()[:8], 16
            )
            rng = np.random.default_rng(seed + application_index * 100_000 + stable)
            n_mc = 600
            soh_sd = max(float(cell["SOH_uncertainty_width"]) / (2.0 * 1.645), 0.05)
            sampled_soh = rng.normal(float(cell["Estimated_SOH"]), soh_sd, n_mc)
            heterogeneity = rng.lognormal(-0.5 * sigma**2, sigma, n_mc)
            base_rate = max(-float(cell["Estimated_recent_degradation_slope"]), 0.004)
            resistance_factor = np.clip(
                1.0 + 0.012 * (float(cell["Internal_resistance"]) - 35.0), 0.75, 1.8
            )
            margin = np.maximum(sampled_soh - p["secondary_eol_soh"], 0.0)
            acceleration = 1.0 + 0.45 * margin / max(100.0 - p["secondary_eol_soh"], 1.0)
            cycles = margin / (
                base_rate * p["stress_multiplier"] * resistance_factor * heterogeneity * acceleration
            )
            cycles = np.clip(cycles, 0.0, 5000.0)
            q05, median, q95 = np.quantile(cycles, [0.05, 0.50, 0.95])
```

```
            usable_kwh = (
                float(cell["Measured_or_estimated_capacity"]) * 3.6 / 1000.0
                * p["depth_of_discharge"]
            )
            rows.append(
                {
                    "Battery_ID": cell["Battery_ID"],
                    "Scenario": application,
                    "SecondLife_RUL_median": float(median),
                    "SecondLife_RUL_lower_90": float(q05),
                    "SecondLife_RUL_upper_90": float(q95),
                    "SecondLife_RUL_uncertainty_width": float(q95 - q05),
                    "Projected_usable_energy_throughput_kWh": float(
                        usable_kwh * median * p["utilization_coefficient"]
                    ),
                    "Monte_Carlo_draws": n_mc,
                    "Projection_seed": seed,
                }
            )
    projection = pd.DataFrame(rows)
    projection.to_csv(
        output_dir / "second_life_projections.csv", index=False, float_format="%.8f"
    )
    return projection


def screen_and_grade(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    output_dir: Path,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    params = _scenario_parameters()
    rows = []
    for application in APPLICATIONS:
        p = params[application]
        merged = decision.merge(
            projection.loc[projection["Scenario"] == application],
            on="Battery_ID",
            validate="one_to_one",
        )
        for _, row in merged.iterrows():
            checks = [
                (row["SOH_lower_90"] >= p["min_soh_lower"], "SOH insufficient"),
                (row["SecondLife_RUL_lower_90"] >= p["min_secondlife_rul_lower"], "RUL insufficient"),
                (row["Internal_resistance"] <= p["max_resistance"], "resistance excessive"),
                (row["Voltage_dispersion"] <= p["max_voltage_dispersion"], "voltage inconsistency"),
                (row["Energy_efficiency"] >= p["min_energy_efficiency"], "energy efficiency insufficient"),
                (row["SOH_uncertainty_width"] <= p["max_soh_uncertainty_width"], "SOH uncertainty excessive"),
                (row["SecondLife_RUL_uncertainty_width"] <= p["max_rul_uncertainty_width"], "RUL uncertainty excessive"),
                (row["Out_of_training_support_score"] <= p["max_ood_score"], "OOD risk"),
            ]
            failures = [reason for passed, reason in checks if not bool(passed)]
            rows.append(
                {
                    "Battery_ID": row["Battery_ID"],
                    "Scenario": application,
                    "Eligible": not failures,
                    "Reject_reason": "; ".join(failures) if failures else "-",
                }
            )
    screening = pd.DataFrame(rows)
    screening.to_csv(output_dir / "safety_screening.csv", index=False)
    eligibility = screening.pivot(index="Battery_ID", columns="Scenario", values="Eligible")
    eligibility = eligibility.reindex(columns=APPLICATIONS, fill_value=False)
    eligibility.columns = ["Eligible_S1", "Eligible_S2", "Eligible_S3"]
    grades = decision.merge(eligibility.reset_index(), on="Battery_ID", validate="one_to_one")
    grades["Grade"] = np.select(
        [grades["Eligible_S1"], grades["Eligible_S2"], grades["Eligible_S3"]],
        ["A", "B", "C"],
        default="Reject",
    )
    grades.to_csv(output_dir / "battery_grades.csv", index=False, float_format="%.8f")
    return screening, grades


def true_second_life(
    hidden: pd.DataFrame,
```

```
    decision: pd.DataFrame,
    *,
    seed: int,
) -> pd.DataFrame:
    params = _scenario_parameters()
    fade_center = float(hidden["True_Fade_heterogeneity"].median())
    observed = decision.set_index("Battery_ID")
    truth = hidden.copy()
    for application_index, application in enumerate(APPLICATIONS):
        p = params[application]
        values = []
        for _, row in truth.iterrows():
            battery_id = str(row["Battery_ID"])
            stable = int(hashlib.sha256(f"truth|{application}|{battery_id}".encode()).hexdigest()[:8], 16)
            rng = np.random.default_rng(seed + application_index * 100_000 + stable)
            hidden_multiplier = float(row["True_Fade_heterogeneity"]) / max(fade_center, 1e-9)
            hidden_multiplier *= float(rng.lognormal(-0.5 * 0.10**2, 0.10))
            cell = observed.loc[battery_id]
            base_rate = max(-float(row["True_recent_SOH_slope"]), 0.004)
            resistance_factor = np.clip(
                1.0 + 0.012 * (float(cell["Internal_resistance"]) - 35.0), 0.75, 1.8
            )
            margin = max(float(row["True_SOH_at_retirement"]) - p["secondary_eol_soh"], 0.0)
            acceleration = 1.0 + 0.45 * margin / max(100.0 - p["secondary_eol_soh"], 1.0)
            cycles = margin / (
                base_rate * p["stress_multiplier"] * resistance_factor * hidden_multiplier * acceleration
            )
            values.append(float(np.clip(cycles, 0.0, 5000.0)))
        truth[f"True_SecondLife_RUL__{application}"] = values
    return truth


def _pair_probabilities(
    left: pd.Series,
    right: pd.Series,
    p: dict[str, float],
    *,
    seed: int,
    draws: int = 256,
) -> dict[str, float]:
    rng = np.random.default_rng(seed)
    # Most predictive error is common-mode model/calibration error. Compatibility
    # depends on differential error, so correlated draws avoid treating the two
    # marginal conformal intervals as independent physical deviations.
    common_soh = rng.normal(size=draws)
    independent_left = rng.normal(size=draws)
    independent_right = rng.normal(size=draws)
    rho_soh = 0.75
    soh_left_sd = max(left["SOH_uncertainty_width"] / (2 * 1.645), 0.05)
    soh_right_sd = max(right["SOH_uncertainty_width"] / (2 * 1.645), 0.05)
    soh_left = left["Estimated_SOH"] + soh_left_sd * (
        np.sqrt(rho_soh) * common_soh + np.sqrt(1 - rho_soh) * independent_left
    )
    soh_right = right["Estimated_SOH"] + soh_right_sd * (
        np.sqrt(rho_soh) * common_soh + np.sqrt(1 - rho_soh) * independent_right
    )
    cap_left = rng.normal(left["Measured_or_estimated_capacity"], max(0.015 * left["Measured_or_estimated_capacity"], 0.005),
    ↪  draws)
    cap_right = rng.normal(right["Measured_or_estimated_capacity"], max(0.015 * right["Measured_or_estimated_capacity"], 0.005),
    ↪  draws)
    res_left = rng.normal(left["Internal_resistance"], max(0.03 * left["Internal_resistance"], 0.15), draws)
    res_right = rng.normal(right["Internal_resistance"], max(0.03 * right["Internal_resistance"], 0.15), draws)
    rul_left_sd = max((left["SecondLife_RUL_upper_90"] - left["SecondLife_RUL_lower_90"]) / (2 * 1.645), 1.0)
    rul_right_sd = max((right["SecondLife_RUL_upper_90"] - right["SecondLife_RUL_lower_90"]) / (2 * 1.645), 1.0)
    common_rul = rng.normal(size=draws)
    independent_left = rng.normal(size=draws)
    independent_right = rng.normal(size=draws)
    rho_rul = 0.90
    rul_left = np.maximum(
        left["SecondLife_RUL_median"] + rul_left_sd * (
            np.sqrt(rho_rul) * common_rul + np.sqrt(1 - rho_rul) * independent_left
        ),
        0.0,
    )
    rul_right = np.maximum(
        right["SecondLife_RUL_median"] + rul_right_sd * (
            np.sqrt(rho_rul) * common_rul + np.sqrt(1 - rho_rul) * independent_right
```

```
        ),
        0.0,
    )
    soh_ok = np.abs(soh_left - soh_right) <= p["soh_pair_tolerance"]
    cap_ok = np.abs(cap_left - cap_right) / np.maximum((cap_left + cap_right) / 2.0, 1e-9) <= p["capacity_relative_tolerance"]
    res_ok = np.abs(res_left - res_right) / np.maximum((res_left + res_right) / 2.0, 1e-9) <= p["resistance_relative_tolerance"]
    rul_ok = np.abs(rul_left - rul_right) <= p["rul_pair_tolerance"]
    return {
        "P_SOH": float(soh_ok.mean()),
        "P_capacity": float(cap_ok.mean()),
        "P_resistance": float(res_ok.mean()),
        "P_RUL": float(rul_ok.mean()),
        "P_compatible": float((soh_ok & cap_ok & res_ok & rul_ok).mean()),
    }


def probability_edge_table(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    screening: pd.DataFrame,
    *,
    seed: int,
    truth: pd.DataFrame | None = None,
) -> pd.DataFrame:
    params = _scenario_parameters()
    truth_lookup = truth.set_index("Battery_ID") if truth is not None else None
    rows: list[dict[str, object]] = []
    for app_index, application in enumerate(APPLICATIONS):
        eligible_ids = screening.loc[
            (screening["Scenario"] == application) & screening["Eligible"], "Battery_ID"
        ].astype(str).tolist()
        frame = decision.loc[decision["Battery_ID"].isin(eligible_ids)].merge(
            projection.loc[projection["Scenario"] == application],
            on="Battery_ID",
            validate="one_to_one",
        ).set_index("Battery_ID", drop=False)
        p = params[application]
        for left_id, right_id in itertools.combinations(eligible_ids, 2):
            stable = int(hashlib.sha256(f"{application}|{left_id}|{right_id}".encode()).hexdigest()[:8], 16)
            probabilities = _pair_probabilities(
                frame.loc[left_id], frame.loc[right_id], p,
                seed=seed + app_index * 1_000_000 + stable,
            )
            row: dict[str, object] = {
                "Scenario": application,
                "Battery_i": left_id,
                "Battery_j": right_id,
                **probabilities,
            }
            if truth_lookup is not None:
                left_truth = truth_lookup.loc[left_id]
                right_truth = truth_lookup.loc[right_id]
                true_cap_rel = abs(left_truth["True_capacity_at_retirement_Ah"] - right_truth["True_capacity_at_retirement_Ah"]) /
                ↪  max(
                    (left_truth["True_capacity_at_retirement_Ah"] + right_truth["True_capacity_at_retirement_Ah"]) / 2.0, 1e-9
                )
                true_res_rel = abs(left_truth["True_resistance_at_retirement_mOhm"] -
                ↪  right_truth["True_resistance_at_retirement_mOhm"]) / max(
                    (left_truth["True_resistance_at_retirement_mOhm"] + right_truth["True_resistance_at_retirement_mOhm"]) / 2.0,
                    ↪  1e-9
                )
                row["True_compatible"] = bool(
                    abs(left_truth["True_SOH_at_retirement"] - right_truth["True_SOH_at_retirement"]) <= p["soh_pair_tolerance"]
                    and true_cap_rel <= p["capacity_relative_tolerance"]
                    and true_res_rel <= p["resistance_relative_tolerance"]
                    and abs(left_truth[f"True_SecondLife_RUL__{application}"] -
                    ↪  right_truth[f"True_SecondLife_RUL__{application}"]) <= p["rul_pair_tolerance"]
                )
            rows.append(row)
    return pd.DataFrame(rows)


def calibrate_probability_threshold(edges: pd.DataFrame, output_dir: Path) -> dict[str, object]:
    """Freeze p0 on calibration truth only using a precision-prioritized rule."""
    if "True_compatible" not in edges.columns:
        raise ValueError("Calibration edge table requires calibration truth")
    rows = []
```

```
    all_nodes = set(edges["Battery_i"]) | set(edges["Battery_j"])
    for p0 in (0.80, 0.85, 0.90, 0.95):
        predicted = edges["P_compatible"] >= p0
        truth = edges["True_compatible"].astype(bool)
        tp = int((predicted & truth).sum())
        fp = int((predicted & ~truth).sum())
        fn = int((~predicted & truth).sum())
        precision = tp / max(tp + fp, 1)
        recall = tp / max(tp + fn, 1)
        beta2 = 0.25
        f_half = (1 + beta2) * precision * recall / max(beta2 * precision + recall, 1e-12)
        selected_edges = edges.loc[predicted]
        covered_nodes = set(selected_edges["Battery_i"]) | set(selected_edges["Battery_j"])
        rows.append(
            {
                "p0": p0,
                "Edges": int(predicted.sum()),
                "Precision": precision,
                "Recall": recall,
                "F0_5": f_half,
                "Node_coverage": len(covered_nodes) / max(len(all_nodes), 1),
                "Truth_source": "Q4 calibration only",
                "Final_holdout_truth_used": False,
            }
        )
    table = pd.DataFrame(rows)
    admissible = table.loc[(table["Precision"] >= 0.90) & (table["Node_coverage"] >= 0.50)]
    candidates = admissible if len(admissible) else table
    selected = candidates.sort_values(["F0_5", "Precision", "p0"], ascending=[False, False, False]).iloc[0]
    table["Selected"] = table["p0"].eq(selected["p0"])
    table.to_csv(output_dir / "probability_threshold_calibration.csv", index=False, float_format="%.8f")
    payload = {
        "p0": float(selected["p0"]),
        "selection_rule": "maximize calibration F0.5 subject to precision>=0.90 and node coverage>=0.50; fallback to F0.5",
        "calibration_truth_used": True,
        "final_holdout_truth_used": False,
        "candidate_thresholds": [0.80, 0.85, 0.90, 0.95],
    }
    save_json(payload, output_dir / "frozen_probability_threshold.json")
    return payload


def graphs_from_edges(
    edges: pd.DataFrame,
    screening: pd.DataFrame,
    *,
    p0: float,
) -> dict[str, nx.Graph]:
    graphs: dict[str, nx.Graph] = {}
    for application in APPLICATIONS:
        nodes = screening.loc[
            (screening["Scenario"] == application) & screening["Eligible"], "Battery_ID"
        ].astype(str).tolist()
        graph = nx.Graph(application=application, p0=p0)
        graph.add_nodes_from(nodes)
        subset = edges.loc[(edges["Scenario"] == application) & (edges["P_compatible"] >= p0)]
        for row in subset.itertuples():
            graph.add_edge(row.Battery_i, row.Battery_j, p_compatible=float(row.P_compatible))
        graphs[application] = graph
    return graphs


def nominal_graphs(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    screening: pd.DataFrame,
) -> dict[str, nx.Graph]:
    params = _scenario_parameters()
    graphs = {}
    for application in APPLICATIONS:
        nodes = screening.loc[
            (screening["Scenario"] == application) & screening["Eligible"], "Battery_ID"
        ].astype(str).tolist()
        frame = decision.loc[decision["Battery_ID"].isin(nodes)].merge(
            projection.loc[projection["Scenario"] == application], on="Battery_ID", validate="one_to_one"
        ).set_index("Battery_ID", drop=False)
        graph = nx.Graph(application=application)
```

```
        graph.add_nodes_from(nodes)
        for left_id, right_id in itertools.combinations(nodes, 2):
            if _compatible(frame.loc[left_id], frame.loc[right_id], params[application]):
                graph.add_edge(left_id, right_id)
        graphs[application] = graph
    return graphs


def generate_groups(
    graphs: dict[str, nx.Graph],
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    *,
    prefix: str,
    seed: int,
) -> pd.DataFrame:
    params = _scenario_parameters()
    rows: list[dict[str, object]] = []
    seen: set[tuple[str, tuple[str, ...]]] = set()
    rng = np.random.default_rng(seed)
    for application in APPLICATIONS:
        graph = graphs[application]
        p = params[application]
        size = int(p["group_size"])
        frame = decision.merge(
            projection.loc[projection["Scenario"] == application], on="Battery_ID", validate="one_to_one"
        ).set_index("Battery_ID", drop=False)
        nodes = list(graph.nodes())
        if len(nodes) < size:
            continue
        standardized = frame.loc[nodes, [
            "SOH_lower_90", "Measured_or_estimated_capacity", "Internal_resistance",
            "SecondLife_RUL_lower_90",
        ]].copy()
        standardized = (standardized - standardized.mean()) / standardized.std(ddof=0).replace(0.0, 1.0)
        orders: list[list[str]] = []
        for column in standardized.columns:
            orders.append(list(standardized.sort_values(column).index))
            orders.append(list(standardized.sort_values(column, ascending=False).index))
        orders.append([node for node, _ in sorted(graph.degree(), key=lambda item: (-item[1], item[0]))])
        for _ in range(20):
            orders.append(list(rng.permutation(nodes)))
        for order in orders:
            for start in order:
                distances = ((standardized - standardized.loc[start]) ** 2).sum(axis=1)
                nearest = list(distances.sort_values().index)
                members = _clique_from_seed(
                    graph, start, list(dict.fromkeys(nearest + order)), size
                )
                if members is None or (application, members) in seen:
                    continue
                seen.add((application, members))
                metrics = _group_metrics(members, application, frame, p)
                if metrics["Inconsistency"] > 1.0000001:
                    continue
                edge_probabilities = [
                    graph[a][b].get("p_compatible", 1.0)
                    for a, b in itertools.combinations(members, 2)
                ]
                rows.append(
                    {
                        "Candidate_ID": f"{prefix}{len(rows) + 1:05d}",
                        "Scenario": application,
                        "Group_size": size,
                        "Members": ";".join(members),
                        "Group_probability_floor": float(min(edge_probabilities, default=1.0)),
                        **metrics,
                    }
                )
    return pd.DataFrame(rows)


def graph_metrics(graphs: dict[str, nx.Graph], graph_type: str) -> pd.DataFrame:
    rows = []
    for application, graph in graphs.items():
        components = list(nx.connected_components(graph))
        rows.append(
```

```
            {
                "Graph_type": graph_type,
                "Scenario": application,
                "Nodes": graph.number_of_nodes(),
                "Edges": graph.number_of_edges(),
                "Density": nx.density(graph) if graph.number_of_nodes() > 1 else 0.0,
                "Components": nx.number_connected_components(graph) if graph.number_of_nodes() else 0,
                "Largest_component": max((len(item) for item in components), default=0),
                "Isolates": nx.number_of_isolates(graph),
            }
        )
    return pd.DataFrame(rows)
```

src/problem4/policy.py

```
"""Q4 stress evaluation and nominal/adaptive/robust feasible-column policies."""

from __future__ import annotations

import hashlib
import json
from pathlib import Path

import numpy as np
import pandas as pd

from src.problem3.optimization import _solve_milp
from src.problem3.screening import SCENARIOS as APPLICATIONS
from src.problem3.screening import _scenario_parameters
from src.problem4.core import STRESS_SCENARIOS, stress_parameters
from src.utils.project import save_json


def _stable_seed(base_seed: int, *parts: str) -> int:
    value = "|".join(parts)
    return base_seed + int(hashlib.sha256(value.encode()).hexdigest()[:8], 16)


def _cvar_lower(values: np.ndarray, alpha: float = 0.20) -> float:
    ordered = np.sort(np.asarray(values, dtype=float))
    count = max(1, int(np.ceil(alpha * len(ordered))))
    return float(ordered[:count].mean())


def evaluate_candidate_stress(
    candidates: pd.DataFrame,
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    *,
    seed: int,
    draws: int = 400,
) -> tuple[pd.DataFrame, pd.DataFrame]:
    """Evaluate each candidate over six stress scenarios without hidden truth."""
    application_params = _scenario_parameters()
    stress = stress_parameters()
    decision_lookup = decision.set_index("Battery_ID")
    projection_lookup = projection.set_index(["Scenario", "Battery_ID"])
    rows: list[dict[str, object]] = []
    for candidate in candidates.itertuples(index=False):
        application = str(candidate.Scenario)
        p = application_params[application]
        members = str(candidate.Members).split(";")
        cells = decision_lookup.loc[members]
        projected = projection_lookup.loc[(application, members), :].reset_index(level=0, drop=True)
        means = np.maximum(projected["SecondLife_RUL_median"].to_numpy(dtype=float), 1.0)
        lower = np.maximum(projected["SecondLife_RUL_lower_90"].to_numpy(dtype=float), 1.0)
        upper = np.maximum(projected["SecondLife_RUL_upper_90"].to_numpy(dtype=float), lower + 1.0)
        log_sigma = np.maximum((np.log(upper) - np.log(lower)) / (2 * 1.645), 0.02)
        log_mu = np.log(means)
        group_size = len(members)
        usable_capacity = float(cells["Measured_or_estimated_capacity"].min())
        resistance_sum = float(cells["Internal_resistance"].sum())
        usable_energy_per_cycle = (
            usable_capacity * 3.6 * group_size / 1000.0 * p["depth_of_discharge"]
        )
```

```
        detection_cost = float(candidate.DetectionCost_REU)
        base_reassembly = float(candidate.ReassemblyCost_REU)
        risk_penalty = float(candidate.ExpectedRiskPenalty_REU)
        for stress_name in STRESS_SCENARIOS:
            q = stress[stress_name]
            rng = np.random.default_rng(
                _stable_seed(seed, str(candidate.Candidate_ID), stress_name)
            )
            common = rng.normal(size=(draws, 1))
            independent = rng.normal(size=(draws, group_size))
            correlated_error = np.sqrt(0.85) * common + np.sqrt(0.15) * independent
            uncertainty = q["prediction_uncertainty_multiplier"]
            rul = np.exp(log_mu[None, :] + uncertainty * log_sigma[None, :] * correlated_error)
            heterogeneity_sigma = 0.04 + 0.10 * (q["batch_heterogeneity_multiplier"] - 1.0)
            heterogeneity = rng.lognormal(
                -0.5 * heterogeneity_sigma**2, heterogeneity_sigma, size=(draws, group_size)
            )
            rul = np.maximum(
                rul / q["degradation_stress_multiplier"] / heterogeneity, 0.0
            )
            minimum_rul = rul.min(axis=1)
            rul_spread = rul.max(axis=1) - minimum_rul
            stressed_soh_spread = float(candidate.SOH_spread) * q[
                "batch_heterogeneity_multiplier"
            ]
            mismatch = (
                (rul_spread > p["rul_pair_tolerance"])
                | (minimum_rul < p["min_secondlife_rul_lower"])
                | (stressed_soh_spread > p["soh_pair_tolerance"])
            )
            throughput = (
                usable_energy_per_cycle * minimum_rul * p["utilization_coefficient"]
            )
            power_loss_w = (
                (p["module_current_A"] * q["c_rate_multiplier"]) ** 2
                * resistance_sum / 1000.0
            )
            discharge_hours = usable_capacity * p["depth_of_discharge"] / max(
                p["module_current_A"] * q["c_rate_multiplier"], 1e-9
            )
            energy_loss = power_loss_w * discharge_hours / 1000.0 * minimum_rul
            revenue_coefficient = p["revenue_per_kwh_reu"] * q["revenue_multiplier"]
            reassembly_cost = base_reassembly * q["reassembly_cost_multiplier"]
            om_cost = 0.025 * throughput
            energy_loss_cost = 2.0 * energy_loss
            total_cost = detection_cost + reassembly_cost + om_cost + energy_loss_cost + risk_penalty
            net_value = throughput * revenue_coefficient - total_cost
            inconsistency = np.maximum.reduce(
                [
                    np.full(draws, float(candidate.Inconsistency)),
                    rul_spread / p["rul_pair_tolerance"],
                    np.full(draws, stressed_soh_spread / p["soh_pair_tolerance"]),
                ]
            )
            rows.append(
                {
                    "Candidate_ID": candidate.Candidate_ID,
                    "Scenario": application,
                    "Group_size": int(candidate.Group_size),
                    "Members": candidate.Members,
                    "Stress_scenario": stress_name,
                    "Expected_NetValue_REU": float(net_value.mean()),
                    "WorstCase_NetValue_REU": float(net_value.min()),
                    "CVaR20_NetValue_REU": _cvar_lower(net_value, 0.20),
                    "Expected_minimum_RUL": float(minimum_rul.mean()),
                    "Conservative_minimum_RUL_p05": float(np.quantile(minimum_rul, 0.05)),
                    "Expected_RUL_spread": float(rul_spread.mean()),
                    "Expected_energy_loss_kWh": float(energy_loss.mean()),
                    "Expected_throughput_kWh": float(throughput.mean()),
                    "Expected_total_cost_REU": float(total_cost.mean()),
                    "Base_reassembly_cost_REU": base_reassembly,
                    "Revenue_coefficient_REU_per_kWh": revenue_coefficient,
                    "Expected_inconsistency": float(inconsistency.mean()),
                    "Base_inconsistency": float(candidate.Inconsistency),
                    "WorstCase_inconsistency": float(inconsistency.max()),
                    "Probability_feasible": float((~mismatch).mean()),
                    "Mismatch_probability": float(mismatch.mean()),
```

```
                    "Constraint_violation_probability": float(
                        ((inconsistency > 1.0) | mismatch).mean()
                    ),
                    "Monte_Carlo_draws": draws,
                }
            )
    detailed = pd.DataFrame(rows)
    robust_rows = []
    for candidate_id, group in detailed.groupby("Candidate_ID", sort=False):
        scenario_values = group["Expected_NetValue_REU"].to_numpy(dtype=float)
        first = group.iloc[0]
        robust_rows.append(
            {
                "Candidate_ID": candidate_id,
                "Scenario": first["Scenario"],
                "Group_size": int(first["Group_size"]),
                "Members": first["Members"],
                "Expected_NetValue_REU": float(scenario_values.mean()),
                "WorstCase_NetValue_REU": float(scenario_values.min()),
                "CVaR20_NetValue_REU": _cvar_lower(scenario_values, 0.20),
                "WorstCase_inconsistency": float(group["Expected_inconsistency"].max()),
                "Probability_feasible": float(group["Probability_feasible"].min()),
                "Mean_probability_feasible": float(group["Probability_feasible"].mean()),
                "Mean_mismatch_probability": float(group["Mismatch_probability"].mean()),
                "Robust_score_REU": float(
                    0.45 * scenario_values.mean()
                    + 0.30 * scenario_values.min()
                    + 0.25 * _cvar_lower(scenario_values, 0.20)
                    - 4.0 * group["Mismatch_probability"].mean()
                    - 2.0 * max(group["Expected_inconsistency"].max() - 1.0, 0.0)
                ),
            }
        )
    return detailed, pd.DataFrame(robust_rows)


def _prepare_policy_columns(
    candidates: pd.DataFrame,
    *,
    value_column: str,
    inconsistency_column: str,
) -> pd.DataFrame:
    result = candidates.copy()
    result["PolicyValue_REU"] = result[value_column].astype(float)
    result["Inconsistency"] = result[inconsistency_column].astype(float)
    if "NetValue_REU" not in result.columns:
        result["NetValue_REU"] = result["PolicyValue_REU"]
    return result


def select_policy(
    candidates: pd.DataFrame,
    pool_ids: list[str],
    *,
    value_column: str,
    inconsistency_column: str,
    minimum_probability: float | None = None,
    target_fraction_of_maximum: float = 0.70,
    maximum_inconsistency: float = 1.0,
) -> tuple[pd.DataFrame, dict[str, object]]:
    active = candidates.copy()
    if minimum_probability is not None:
        active = active.loc[active["Probability_feasible"] >= minimum_probability].copy()
    active = _prepare_policy_columns(
        active, value_column=value_column, inconsistency_column=inconsistency_column
    )
    # A policy column remains admissible only while its scenario-adjusted
    # inconsistency does not exceed the same normalized compatibility envelope.
    active = active.loc[active["Inconsistency"] <= maximum_inconsistency + 1e-12].copy()
    if active.empty:
        raise RuntimeError("No candidate columns remain for the requested Q4 policy")
    max_meta, max_selected = _solve_milp(
        active,
        pool_ids,
        maximum_inconsistency,
        0,
        objective="utilization",
```

```
        time_limit=12.0,
        mip_rel_gap=0.01,
    )
    if max_selected.empty:
        raise RuntimeError(f"Q4 maximum-utilization solve failed: {max_meta}")
    max_cells = int(max_selected["Group_size"].sum())
    minimum_cells = int(np.floor(target_fraction_of_maximum * max_cells))
    metadata, selected = _solve_milp(
        active,
        pool_ids,
        maximum_inconsistency,
        minimum_cells,
        objective="value",
        time_limit=15.0,
        mip_rel_gap=0.01,
        value_column="PolicyValue_REU",
    )
    if selected.empty:
        raise RuntimeError(f"Q4 policy solve failed: {metadata}")
    metadata = {
        **metadata,
        "maximum_feasible_cells": max_cells,
        "minimum_cells": minimum_cells,
        "target_fraction_of_maximum": target_fraction_of_maximum,
        "maximum_inconsistency": maximum_inconsistency,
        "selected_groups": int(len(selected)),
        "selected_cells": int(selected["Group_size"].sum()),
        "selected_policy_value_REU": float(selected["PolicyValue_REU"].sum()),
    }
    return selected, metadata


def _aggregate_selected(
    strategy: str,
    stress_scenario: str,
    selected_ids: set[str],
    detailed: pd.DataFrame,
    pool_size: int,
) -> dict[str, object]:
    frame = detailed.loc[
        detailed["Candidate_ID"].isin(selected_ids)
        & detailed["Stress_scenario"].eq(stress_scenario)
    ].copy()
    if frame.empty:
        raise RuntimeError(f"No evaluation rows for {strategy}/{stress_scenario}")
    used_cells = int(frame["Group_size"].sum())
    weights = frame["Group_size"].to_numpy(dtype=float)
    return {
        "Strategy": strategy,
        "Stress_scenario": stress_scenario,
        "Complete_groups": int(len(frame)),
        "Used_cells": used_cells,
        "Utilization_rate": used_cells / pool_size,
        "Expected_NetValue_REU": float(frame["Expected_NetValue_REU"].sum()),
        "WorstCase_NetValue_REU": float(frame["WorstCase_NetValue_REU"].sum()),
        "CVaR20_NetValue_REU": float(frame["CVaR20_NetValue_REU"].sum()),
        "Energy_loss_kWh": float(frame["Expected_energy_loss_kWh"].sum()),
        "Mean_inconsistency": float(
            np.average(frame["Expected_inconsistency"], weights=weights)
        ),
        "Mismatch_probability": float(
            np.average(frame["Mismatch_probability"], weights=weights)
        ),
        "Minimum_RUL_p05": float(frame["Conservative_minimum_RUL_p05"].min()),
        "Constraint_violation_rate": float(
            np.average(frame["Constraint_violation_probability"], weights=weights)
        ),
        "Group_survival_probability": float(
            np.average(frame["Probability_feasible"], weights=weights)
        ),
        "Expected_throughput_kWh": float(frame["Expected_throughput_kWh"].sum()),
        "Expected_total_cost_REU": float(frame["Expected_total_cost_REU"].sum()),
        "Base_reassembly_cost_REU": float(frame["Base_reassembly_cost_REU"].sum()),
        "Mean_revenue_coefficient_REU_per_kWh": float(
            np.average(frame["Revenue_coefficient_REU_per_kWh"], weights=weights)
        ),
    }
```

```
292
293
294 def solve_strategies(
295     nominal_candidates: pd.DataFrame,
296     nominal_detailed: pd.DataFrame,
297     robust_candidates: pd.DataFrame,
298     robust_detailed: pd.DataFrame,
299     robust_aggregate: pd.DataFrame,
300     pool_ids: list[str],
301     *,
302     output_dir: Path,
303 ) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, object]]:
304     pool_size = len(pool_ids)
305     nominal_s0 = nominal_detailed.loc[
306         nominal_detailed["Stress_scenario"] == "S0_baseline"
307     ].merge(
308         nominal_candidates[["Candidate_ID", "Inconsistency"]], on="Candidate_ID", validate="one_to_one"
309     )
310     nominal_selected, nominal_meta = select_policy(
311         nominal_s0,
312         pool_ids,
313         value_column="Expected_NetValue_REU",
314         inconsistency_column="Inconsistency",
315     )
316     nominal_ids = set(nominal_selected["Candidate_ID"])
317
318     robust_selected, robust_meta = select_policy(
319         robust_aggregate,
320         pool_ids,
321         value_column="Robust_score_REU",
322         inconsistency_column="WorstCase_inconsistency",
323         minimum_probability=None,
324         maximum_inconsistency=2.0,
325     )
326     robust_ids = set(robust_selected["Candidate_ID"])
327
328     selection_rows = []
329     for row in nominal_selected.itertuples():
330         selection_rows.append(
331             {"Strategy": "N_nominal", "Selection_for": "all_scenarios", "Candidate_ID": row.Candidate_ID,
332              "Scenario": row.Scenario, "Members": row.Members}
333         )
334     for row in robust_selected.itertuples():
335         selection_rows.append(
336             {"Strategy": "R_robust_unified", "Selection_for": "all_scenarios", "Candidate_ID": row.Candidate_ID,
337              "Scenario": row.Scenario, "Members": row.Members}
338         )
339
340     comparison_rows = []
341     for stress_scenario in STRESS_SCENARIOS:
342         comparison_rows.append(
343             _aggregate_selected(
344                 "N_nominal", stress_scenario, nominal_ids, nominal_detailed, pool_size
345             )
346         )
347         adaptive_frame = nominal_detailed.loc[
348             nominal_detailed["Stress_scenario"] == stress_scenario
349         ]
350         adaptive_selected, adaptive_meta = select_policy(
351             adaptive_frame,
352             pool_ids,
353             value_column="Expected_NetValue_REU",
354             inconsistency_column="Base_inconsistency",
355             minimum_probability=None,
356             maximum_inconsistency=1.0,
357         )
358         adaptive_ids = set(adaptive_selected["Candidate_ID"])
359         for row in adaptive_selected.itertuples():
360             selection_rows.append(
361                 {"Strategy": "A_scenario_adaptive", "Selection_for": stress_scenario,
362                  "Candidate_ID": row.Candidate_ID, "Scenario": row.Scenario, "Members": row.Members}
363             )
364         comparison_rows.append(
365             _aggregate_selected(
366                 "A_scenario_adaptive", stress_scenario, adaptive_ids, nominal_detailed, pool_size
367             )
368         )
```

```
        comparison_rows.append(
            _aggregate_selected(
                "R_robust_unified", stress_scenario, robust_ids, robust_detailed, pool_size
            )
        )
        save_json(adaptive_meta, output_dir / f"adaptive_solver_{stress_scenario}.json")
    selections = pd.DataFrame(selection_rows)
    comparison = pd.DataFrame(comparison_rows)
    selections.to_csv(output_dir / "strategy_selected_groups.csv", index=False)
    comparison.to_csv(output_dir / "strategy_comparison.csv", index=False, float_format="%.8f")
    metadata = {
        "nominal": nominal_meta,
        "robust": robust_meta,
        "robust_group_probability_requirement": "reported and penalized, not hard-gated; pair edges already satisfy frozen p0",
        "robust_score_definition": "0.45 expected + 0.30 worst-scenario expected + 0.25 CVaR20 - mismatch and excess-inconsistency
        ↪  penalties",
    }
    save_json(metadata, output_dir / "strategy_solver_metadata.json")
    return selections, comparison, metadata


def recovery_ratios(comparison: pd.DataFrame) -> pd.DataFrame:
    """Quantify how much scenario-adaptive regrouping recovers after stress.

    For a higher-is-better metric ``q``, the loss caused by stress scenario ``s``
    is ``q(N,S0)-q(N,s)`` and the adaptive recovery is
    ``q(A,s)-q(N,s)``.  For a lower-is-better metric the signs are reversed.
    A ratio is deliberately left missing when the stress loss is numerically
    zero; it is never clipped because negative recovery is an engineering
    finding (the adaptive policy made that metric worse).

    The robust policy is a single conservative plan rather than a
    scenario-adaptive repair, so it is reported separately through value
    retention and violation reduction relative to the nominal plan.
    """
    rows = []
    baseline = comparison.loc[comparison["Stress_scenario"] == "S0_baseline"].set_index("Strategy")
    nominal_baseline = baseline.loc["N_nominal"]

    def _ratio(numerator: float, denominator: float) -> float:
        return float(numerator / denominator) if abs(denominator) > 1e-12 else np.nan

    for stress_scenario, group in comparison.groupby("Stress_scenario", sort=False):
        indexed = group.set_index("Strategy")
        n = indexed.loc["N_nominal"]
        a = indexed.loc["A_scenario_adaptive"]
        r = indexed.loc["R_robust_unified"]
        value_loss = float(nominal_baseline["Expected_NetValue_REU"] - n["Expected_NetValue_REU"])
        rul_loss = float(nominal_baseline["Minimum_RUL_p05"] - n["Minimum_RUL_p05"])
        violation_loss = float(
            n["Constraint_violation_rate"] - nominal_baseline["Constraint_violation_rate"]
        )
        rows.append(
            {
                "Stress_scenario": stress_scenario,
                "Adaptive_NetValue_recovery_ratio": _ratio(
                    float(a["Expected_NetValue_REU"] - n["Expected_NetValue_REU"]), value_loss
                ),
                "Adaptive_minimum_RUL_recovery_ratio": _ratio(
                    float(a["Minimum_RUL_p05"] - n["Minimum_RUL_p05"]), rul_loss
                ),
                "Adaptive_violation_recovery_ratio": _ratio(
                    float(n["Constraint_violation_rate"] - a["Constraint_violation_rate"]), violation_loss
                ),
                "Robust_NetValue_retention_ratio": _ratio(
                    float(r["Expected_NetValue_REU"]), float(n["Expected_NetValue_REU"])
                ),
                "Robust_violation_reduction_ratio": _ratio(
                    float(n["Constraint_violation_rate"] - r["Constraint_violation_rate"]),
                    float(n["Constraint_violation_rate"]),
                ),
                "Definition": (
                    "adaptive recovery=(adaptive under stress-nominal under stress)/"
                    "(nominal S0-nominal under stress); signs reversed for lower-is-better violation. "
                    "Robust plan uses value retention and nominal-violation reduction."
                ),
            }
```

```
        )
    return pd.DataFrame(rows)


def break_even_analysis(comparison: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for row in comparison.itertuples(index=False):
        throughput = max(float(row.Expected_throughput_kWh), 1e-9)
        current_revenue = float(row.Mean_revenue_coefficient_REU_per_kWh)
        expected_net = float(row.Expected_NetValue_REU)
        cvar_net = float(row.CVaR20_NetValue_REU)
        worst_net = float(row.WorstCase_NetValue_REU)
        groups = max(int(row.Complete_groups), 1)
        reassembly = float(row.Base_reassembly_cost_REU)
        for basis, net_value in (
            ("expected", expected_net),
            ("conservative_CVaR20", cvar_net),
            ("worst_sample", worst_net),
        ):
            rows.append(
                {
                    "Strategy": row.Strategy,
                    "Stress_scenario": row.Stress_scenario,
                    "Basis": basis,
                    "Minimum_revenue_coefficient_REU_per_kWh": float(
                        current_revenue - net_value / throughput
                    ),
                    "Maximum_allowable_reassembly_cost_REU_per_group": float(
                        (reassembly + net_value) / groups
                    ),
                    "Current_revenue_coefficient_REU_per_kWh": current_revenue,
                    "Current_base_reassembly_cost_REU_per_group": reassembly / groups,
                }
            )
    return pd.DataFrame(rows)
```

src/problem4/sensitivity.py

```
"""Q4 local/global sensitivity, candidate-size audit, and weight-region map."""

from __future__ import annotations

import time
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from scipy.stats.qmc import LatinHypercube

from src.problem3.optimization import _solve_milp
from src.problem4.core import STRESS_SCENARIOS


SENSITIVITY_PARAMETERS = (
    ("temperature_delta_C", 0.0, 0.0, 12.0, "degC", "scenario_assumption"),
    ("c_rate_multiplier", 1.0, 1.0, 1.6, "ratio", "scenario_assumption"),
    ("batch_heterogeneity_multiplier", 1.0, 1.0, 1.8, "ratio", "scenario_assumption"),
    ("prediction_uncertainty_multiplier", 1.0, 0.85, 1.30, "ratio", "model_uncertainty"),
    ("revenue_multiplier", 1.0, 0.70, 1.30, "ratio", "economic_scenario"),
    ("reassembly_cost_multiplier", 1.0, 0.80, 1.40, "ratio", "economic_scenario"),
    ("detection_cost_multiplier", 1.0, 0.80, 1.30, "ratio", "economic_scenario"),
    ("om_cost_rate", 0.025, 0.018, 0.035, "REU/throughput", "economic_scenario"),
    ("energy_loss_cost_REU_per_kWh", 2.0, 1.0, 3.0, "REU/kWh", "economic_scenario"),
    ("soh_tolerance_multiplier", 1.0, 0.80, 1.25, "ratio", "engineering_range"),
    ("rul_tolerance_multiplier", 1.0, 0.80, 1.30, "ratio", "engineering_range"),
    ("capacity_tolerance_multiplier", 1.0, 0.80, 1.25, "ratio", "engineering_range"),
    ("resistance_tolerance_multiplier", 1.0, 0.80, 1.25, "ratio", "engineering_range"),
    ("p0", 0.80, 0.80, 0.95, "probability", "calibration_grid"),
    ("minimum_RUL_gate_multiplier", 1.0, 0.85, 1.15, "ratio", "engineering_range"),
    ("energy_efficiency_gate_percent", 96.0, 94.0, 98.5, "%", "physical_range<=100"),
)


def write_parameter_registry(path: Path) -> pd.DataFrame:
```

```
    rows = []
    for parameter, baseline, low, high, unit, source in SENSITIVITY_PARAMETERS:
        rows.append(
            {
                "Parameter": parameter,
                "Baseline": baseline,
                "Low": low,
                "High": high,
                "Unit": unit,
                "Source_type": source,
                "Physical_validity": "bounded; efficiency gate never exceeds 100%",
            }
        )
    frame = pd.DataFrame(rows)
    frame.to_csv(path, index=False, float_format="%.8f")
    return frame


def _greedy_policy(candidates: pd.DataFrame, values: dict[str, float], pool_size: int) -> dict[str, float]:
    tolerance = np.mean([
        values["soh_tolerance_multiplier"],
        values["rul_tolerance_multiplier"],
        values["capacity_tolerance_multiplier"],
        values["resistance_tolerance_multiplier"],
    ])
    stress = (
        np.exp(0.026 * values["temperature_delta_C"])
        * (1.0 + 0.55 * (values["c_rate_multiplier"] - 1.0))
        * (1.0 + 0.10 * (values["batch_heterogeneity_multiplier"] - 1.0))
    )
    frame = candidates.copy()
    frame["Adjusted_probability"] = np.clip(
        frame["Mean_probability_feasible"]
        * tolerance**0.8
        / (
            stress**0.35
            * values["prediction_uncertainty_multiplier"] ** 0.4
            * values["batch_heterogeneity_multiplier"] ** 0.3
            * values["minimum_RUL_gate_multiplier"] ** 0.4
        ),
        0.0,
        1.0,
    )
    frame["Adjusted_mismatch"] = np.clip(
        1.0 - frame["Adjusted_probability"], 0.0, 1.0
    )
    frame["Adjusted_inconsistency"] = (
        frame["WorstCase_inconsistency"]
        * values["batch_heterogeneity_multiplier"] ** 0.5
        * values["prediction_uncertainty_multiplier"] ** 0.4
        / max(tolerance, 1e-9)
    )
    economic_cost_delta = (
        1.6 * (values["reassembly_cost_multiplier"] - 1.0)
        + 0.7 * (values["detection_cost_multiplier"] - 1.0)
        + 24.0 * (values["om_cost_rate"] - 0.025)
        + 0.6 * (values["energy_loss_cost_REU_per_kWh"] - 2.0)
    )
    frame["Adjusted_value"] = (
        frame["Expected_NetValue_REU"] * values["revenue_multiplier"] / stress
        - economic_cost_delta
    )
    frame["Adjusted_score"] = (
        frame["Adjusted_value"]
        - 4.0 * frame["Adjusted_mismatch"]
        - 2.0 * np.maximum(frame["Adjusted_inconsistency"] - 1.0, 0.0)
    )
    frame = frame.loc[
        (frame["Group_probability_floor"] >= values["p0"] - 1e-12)
        & (frame["Adjusted_inconsistency"] <= 2.5)
    ].copy()
    used: set[str] = set()
    selected_rows: list[pd.Series] = []
    for application in ("S1_frequency_regulation", "S2_peak_shaving", "S3_backup"):
        subset = frame.loc[frame["Scenario"] == application].sort_values(
            ["Adjusted_score", "Adjusted_probability"], ascending=False
        )
```

```
        for _, row in subset.iterrows():
            members = set(str(row["Members"]).split(";"))
            if members.isdisjoint(used):
                selected_rows.append(row)
                used.update(members)
                break
    for _, row in frame.sort_values(["Adjusted_score", "Adjusted_probability"], ascending=False).iterrows():
        if row["Adjusted_score"] <= 0:
            continue
        members = set(str(row["Members"]).split(";"))
        if members.isdisjoint(used):
            selected_rows.append(row)
            used.update(members)
    selected = pd.DataFrame(selected_rows)
    if selected.empty:
        return {"NetValue_REU": 0.0, "Utilization_rate": 0.0, "Mismatch_risk": 1.0,
                "Reject_rate": 1.0, "Inconsistency": 2.5}
    efficiency_penalty = max(values["energy_efficiency_gate_percent"] - 96.0, 0.0) / 10.0
    reject_rate = np.clip(
        0.358
        + 0.10 * (values["minimum_RUL_gate_multiplier"] - 1.0)
        + 0.08 * (values["prediction_uncertainty_multiplier"] - 1.0)
        + efficiency_penalty,
        0.0,
        1.0,
    )
    weights = selected["Group_size"].to_numpy(dtype=float)
    return {
        "NetValue_REU": float(selected["Adjusted_value"].sum()),
        "Utilization_rate": float(selected["Group_size"].sum() / pool_size),
        "Mismatch_risk": float(np.average(selected["Adjusted_mismatch"], weights=weights)),
        "Reject_rate": float(reject_rate),
        "Inconsistency": float(np.average(selected["Adjusted_inconsistency"], weights=weights)),
    }


def run_local_and_global_sensitivity(
    robust_candidates: pd.DataFrame,
    robust_aggregate: pd.DataFrame,
    *,
    pool_size: int,
    output_dir: Path,
    seed: int,
    lhs_samples: int = 384,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    registry = write_parameter_registry(output_dir / "q4_sensitivity_parameter_registry.csv")
    merged = robust_aggregate.merge(
        robust_candidates[["Candidate_ID", "Group_probability_floor"]],
        on="Candidate_ID",
        validate="one_to_one",
    )
    baseline_values = {row.Parameter: float(row.Baseline) for row in registry.itertuples()}
    baseline_response = _greedy_policy(merged, baseline_values, pool_size)
    local_rows = []
    for parameter in registry.itertuples():
        low_values = dict(baseline_values)
        high_values = dict(baseline_values)
        low_values[parameter.Parameter] = float(parameter.Low)
        high_values[parameter.Parameter] = float(parameter.High)
        low_response = _greedy_policy(merged, low_values, pool_size)
        high_response = _greedy_policy(merged, high_values, pool_size)
        for output in baseline_response:
            normalized = (
                high_response[output] - low_response[output]
            ) / (2.0 * max(abs(baseline_response[output]), 1e-9))
            local_rows.append(
                {
                    "Parameter": parameter.Parameter,
                    "Output": output,
                    "Low_output": low_response[output],
                    "Baseline_output": baseline_response[output],
                    "High_output": high_response[output],
                    "Normalized_local_sensitivity": normalized,
                }
            )
    local = pd.DataFrame(local_rows)
    local.to_csv(output_dir / "q4_local_oat_sensitivity.csv", index=False, float_format="%.8f")
```

```
    sampler = LatinHypercube(d=len(registry), seed=seed)
    unit = sampler.random(lhs_samples)
    lows = registry["Low"].to_numpy(dtype=float)
    highs = registry["High"].to_numpy(dtype=float)
    samples = lows + unit * (highs - lows)
    sample_rows = []
    for sample_index, sample in enumerate(samples, start=1):
        values = {registry.iloc[i]["Parameter"]: float(sample[i]) for i in range(len(registry))}
        response = _greedy_policy(merged, values, pool_size)
        sample_rows.append({"Sample": sample_index, **values, **response})
    lhs = pd.DataFrame(sample_rows)
    lhs.to_csv(output_dir / "q4_global_lhs_samples.csv", index=False, float_format="%.8f")
    global_rows = []
    outputs = list(baseline_response)
    for parameter in registry["Parameter"]:
        for output in outputs:
            rho, p_value = spearmanr(lhs[parameter], lhs[output])
            global_rows.append(
                {
                    "Parameter": parameter,
                    "Output": output,
                    "Spearman_rho": float(rho),
                    "p_value": float(p_value),
                    "Abs_rho": float(abs(rho)),
                    "Samples": lhs_samples,
                    "Method": "LHS + Spearman rank correlation on the fixed feasible-column policy emulator",
                }
            )
    global_sensitivity = pd.DataFrame(global_rows)
    global_sensitivity["Rank_within_output"] = global_sensitivity.groupby("Output")["Abs_rho"].rank(
        method="dense", ascending=False
    ).astype(int)
    global_sensitivity.to_csv(
        output_dir / "q4_global_sensitivity.csv", index=False, float_format="%.8f"
    )
    return local, global_sensitivity, lhs


def _reduced_199(full: pd.DataFrame) -> pd.DataFrame:
    retained = []
    for _, group in full.groupby("Scenario", sort=False):
        ids = set(group.nlargest(35, "NetValue_REU")["Candidate_ID"])
        ids.update(group.nsmallest(35, "Inconsistency")["Candidate_ID"])
        ids.update(group.nlargest(20, "Conservative_minimum_RUL")["Candidate_ID"])
        retained.append(group.loc[group["Candidate_ID"].isin(ids)])
    return pd.concat(retained, ignore_index=True)


def _ranked_subset(full: pd.DataFrame, size: int) -> pd.DataFrame:
    frame = full.copy()
    for column, maximize in (("NetValue_REU", True), ("Inconsistency", False), ("Conservative_minimum_RUL", True)):
        values = frame[column].to_numpy(dtype=float)
        normalized = (values - values.min()) / max(float(np.ptp(values)), 1e-12)
        frame[f"score_{column}"] = normalized if maximize else 1.0 - normalized
    frame["subset_score"] = (
        0.45 * frame["score_NetValue_REU"]
        + 0.35 * frame["score_Inconsistency"]
        + 0.20 * frame["score_Conservative_minimum_RUL"]
    )
    quotas = (frame["Scenario"].value_counts(normalize=True) * size).round().astype(int)
    while quotas.sum() < size:
        quotas.iloc[0] += 1
    while quotas.sum() > size:
        quotas.iloc[-1] -= 1
    parts = [
        frame.loc[frame["Scenario"] == scenario].nlargest(int(quota), "subset_score")
        for scenario, quota in quotas.items()
    ]
    return pd.concat(parts, ignore_index=True).head(size)


def candidate_size_sensitivity(
    full: pd.DataFrame,
    pool_ids: list[str],
    *,
    output_path: Path,
```

```
) -> pd.DataFrame:
    libraries = {
        "100": _ranked_subset(full, 100),
        "199": _reduced_199(full),
        "300": _ranked_subset(full, 300),
        "full": full.copy(),
    }
    records = []
    memberships: dict[str, set[str]] = {}
    for label, candidates in libraries.items():
        started = time.perf_counter()
        max_meta, max_selected = _solve_milp(
            candidates, pool_ids, 1.0, 0, objective="utilization",
            time_limit=15.0, mip_rel_gap=0.01,
        )
        max_cells = int(max_selected["Group_size"].sum())
        metadata, selected = _solve_milp(
            candidates, pool_ids, 1.0, int(np.floor(0.70 * max_cells)),
            time_limit=15.0, mip_rel_gap=0.01,
        )
        members = {member for value in selected["Members"] for member in str(value).split(";")}
        memberships[label] = members
        records.append(
            {
                "Candidate_size": label,
                "Candidate_columns": int(len(candidates)),
                "Runtime_seconds": float(time.perf_counter() - started),
                "Solver_status": metadata["status"],
                "Optimality_gap": metadata["mip_gap"],
                "Objective_NetValue_REU": float(selected["NetValue_REU"].sum()),
                "Used_cells": int(selected["Group_size"].sum()),
                "Utilization_rate": float(selected["Group_size"].sum() / len(pool_ids)),
                "Selected_groups": int(len(selected)),
            }
        )
    full_members = memberships["full"]
    frame = pd.DataFrame(records)
    frame["Selected_cell_Jaccard_vs_full"] = [
        len(memberships[label] & full_members) / max(len(memberships[label] | full_members), 1)
        for label in frame["Candidate_size"]
    ]
    frame.to_csv(output_path, index=False, float_format="%.8f")
    return frame


def pareto_weight_region(pareto: pd.DataFrame, output_path: Path, *, seed: int, samples: int = 1024) -> pd.DataFrame:
    points = pareto.loc[pareto["Pareto_nondominated"]].copy()
    regret_columns = ["Regret_NetValue_REU", "Regret_Utilization_rate", "Regret_Mean_inconsistency"]
    rng = np.random.default_rng(seed)
    weights = rng.dirichlet(np.ones(3), size=samples)
    winners = []
    regrets = points[regret_columns].to_numpy(dtype=float)
    for weight in weights:
        distance = np.sqrt(np.sum(np.square(regrets) * weight, axis=1))
        winners.append(str(points.iloc[int(np.argmin(distance))]["Solution_ID"]))
    sample_frame = pd.DataFrame(
        {"Weight_value": weights[:, 0], "Weight_utilization": weights[:, 1],
         "Weight_consistency": weights[:, 2], "Selected_solution": winners}
    )
    rows = []
    for solution, group in sample_frame.groupby("Selected_solution"):
        rows.append(
            {
                "Solution_ID": solution,
                "Selection_count": int(len(group)),
                "Selection_frequency": float(len(group) / samples),
                "Mean_weight_value": float(group["Weight_value"].mean()),
                "Mean_weight_utilization": float(group["Weight_utilization"].mean()),
                "Mean_weight_consistency": float(group["Weight_consistency"].mean()),
                "Min_weight_value": float(group["Weight_value"].min()),
                "Max_weight_value": float(group["Weight_value"].max()),
                "Min_weight_utilization": float(group["Weight_utilization"].min()),
                "Max_weight_utilization": float(group["Weight_utilization"].max()),
                "Min_weight_consistency": float(group["Weight_consistency"].min()),
                "Max_weight_consistency": float(group["Weight_consistency"].max()),
            }
        )
```

```
    result = pd.DataFrame(rows).sort_values("Selection_frequency", ascending=False)
    result.to_csv(output_path, index=False, float_format="%.8f")
    return result
```

src/problem4/analysis.py

```
"""End-to-end Q4 orchestration with calibration/holdout isolation."""

from __future__ import annotations

import hashlib
import json
import time
from pathlib import Path

import numpy as np
import pandas as pd

from src.problem4.core import (
    STRESS_SCENARIOS,
    calibrate_probability_threshold,
    generate_groups,
    graph_metrics,
    graphs_from_edges,
    nominal_graphs,
    probability_edge_table,
    project_second_life,
    screen_and_grade,
    true_second_life,
    write_scenario_registry,
)
from src.problem4.policy import (
    _aggregate_selected,
    break_even_analysis,
    evaluate_candidate_stress,
    recovery_ratios,
    solve_strategies,
)
from src.problem4.population import build_calibration_and_holdout
from src.problem4.sensitivity import (
    candidate_size_sensitivity,
    pareto_weight_region,
    run_local_and_global_sensitivity,
)
from src.utils.project import RESULT_TABLES, ROOT, load_json, save_json


def _sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


def _copy_csv(frame: pd.DataFrame, name: str) -> None:
    frame.to_csv(RESULT_TABLES / name, index=False, float_format="%.8f")


def _holdout_prediction_audit(
    decision: pd.DataFrame,
    projection: pd.DataFrame,
    truth: pd.DataFrame,
    selections: pd.DataFrame,
) -> tuple[dict[str, float], pd.DataFrame]:
    soh_error = decision[["Battery_ID", "Estimated_SOH"]].merge(
        truth[["Battery_ID", "True_SOH_at_retirement"]], on="Battery_ID", validate="one_to_one"
    )
    soh_error["Absolute_error"] = (
        soh_error["Estimated_SOH"] - soh_error["True_SOH_at_retirement"]
    ).abs()
    truth_long = []
    for application in (
        "S1_frequency_regulation", "S2_peak_shaving", "S3_backup"
    ):
        part = truth[["Battery_ID", f"True_SecondLife_RUL__{application}"]].copy()
        part["Scenario"] = application
        part = part.rename(
            columns={f"True_SecondLife_RUL__{application}": "True_SecondLife_RUL"}
```

```
        )
        truth_long.append(part)
    rul_audit = projection.merge(pd.concat(truth_long), on=["Battery_ID", "Scenario"], validate="one_to_one")
    rul_audit["Absolute_error"] = (
        rul_audit["SecondLife_RUL_median"] - rul_audit["True_SecondLife_RUL"]
    ).abs()
    rul_audit["Covered_by_predicted_90"] = (
        (rul_audit["True_SecondLife_RUL"] >= rul_audit["SecondLife_RUL_lower_90"])
        & (rul_audit["True_SecondLife_RUL"] <= rul_audit["SecondLife_RUL_upper_90"])
    )

    robust_groups = selections.loc[
        selections["Strategy"] == "R_robust_unified"
    ].drop_duplicates("Candidate_ID")
    truth_lookup = truth.set_index("Battery_ID")
    mismatch_flags = []
    from src.problem3.screening import _scenario_parameters

    params = _scenario_parameters()
    for row in robust_groups.itertuples():
        members = str(row.Members).split(";")
        app = str(row.Scenario)
        group = truth_lookup.loc[members]
        true_rul = group[f"True_SecondLife_RUL__{app}"].to_numpy(dtype=float)
        mismatch_flags.append(
            bool(
                np.ptp(true_rul) > params[app]["rul_pair_tolerance"]
                or np.ptp(group["True_SOH_at_retirement"].to_numpy(dtype=float))
                > params[app]["soh_pair_tolerance"]
                or true_rul.min() < params[app]["min_secondlife_rul_lower"]
            )
        )
    payload = {
        "SOH_MAE_percentage_points": float(soh_error["Absolute_error"].mean()),
        "SOH_RMSE_percentage_points": float(
            np.sqrt(np.mean(np.square(
                soh_error["Estimated_SOH"] - soh_error["True_SOH_at_retirement"]
            )))
        ),
        "SecondLife_RUL_MAE_cycles": float(rul_audit["Absolute_error"].mean()),
        "SecondLife_RUL_RMSE_cycles": float(
            np.sqrt(np.mean(np.square(
                rul_audit["SecondLife_RUL_median"] - rul_audit["True_SecondLife_RUL"]
            )))
        ),
        "SecondLife_RUL_interval_coverage": float(
            rul_audit["Covered_by_predicted_90"].mean()
        ),
        "Robust_selected_realized_base_mismatch_rate": float(
            np.mean(mismatch_flags) if mismatch_flags else np.nan
        ),
    }
    return payload, rul_audit


def _engineering_recommendations(
    strategy: pd.DataFrame,
    threshold: dict[str, object],
    break_even: pd.DataFrame,
) -> pd.DataFrame:
    robust = strategy.loc[strategy["Strategy"] == "R_robust_unified"].set_index("Stress_scenario")
    baseline = robust.loc["S0_baseline"]
    severe = robust.loc["S4_temperature_plus_rate"]
    economic = break_even.loc[
        (break_even["Strategy"] == "R_robust_unified")
        & (break_even["Basis"] == "conservative_CVaR20")
    ]
    worst_break_even = economic.loc[
        economic["Minimum_revenue_coefficient_REU_per_kWh"].idxmax()
    ]
    rows = [
        {
            "Dimension": "operation",
            "Recommendation": "Avoid sustained +10 degC and 1.5x C-rate operation; trigger thermal/rate derating before the
            ↪  combined S4 envelope.",
            "Numeric_evidence": (
                f"Robust minimum-RUL p05 changes {baseline['Minimum_RUL_p05']:.1f} -> "
```

```
                f"{severe['Minimum_RUL_p05']:.1f} cycles; violation "
                f"{baseline['Constraint_violation_rate']:.1%} -> {severe['Constraint_violation_rate']:.1%}."
            ),
        },
        {
            "Dimension": "screening",
            "Recommendation": "Use the frozen probabilistic edge threshold and re-calibrate it for each chemistry/test system.",
            "Numeric_evidence": f"Calibration-only p0={threshold['p0']:.2f}; final-holdout truth was not used for threshold
            ↪  selection.",
        },
        {
            "Dimension": "grouping",
            "Recommendation": "Use the unified robust groups when scenario identity is unavailable; keep the adaptive plan for
            ↪  known duty cycles.",
            "Numeric_evidence": (
                f"Robust unified utilization={baseline['Utilization_rate']:.1%}, groups={int(baseline['Complete_groups'])}; "
                f"S4 violation={severe['Constraint_violation_rate']:.1%}."
            ),
        },
        {
            "Dimension": "economics",
            "Recommendation": "Reject deployments whose conservative revenue coefficient is below the scenario break-even value.",
            "Numeric_evidence": (
                f"Maximum robust CVaR20 break-even={worst_break_even['Minimum_revenue_coefficient_REU_per_kWh']:.3f} "
                f"REU/kWh in {worst_break_even['Stress_scenario']}."
            ),
        },
    ]
    return pd.DataFrame(rows)


def run_problem4() -> dict[str, object]:
    started = time.perf_counter()
    population = build_calibration_and_holdout(ROOT)
    q4_seed = int(population["Q4_SEED"])
    calibration_dir = ROOT / "q4_calibration"
    holdout_dir = ROOT / "q4_final_holdout"
    scenario_registry = write_scenario_registry(RESULT_TABLES / "q4_scenario_registry.csv")

    calibration_decision = pd.read_csv(calibration_dir / "decision_pool.csv")
    calibration_hidden = pd.read_csv(calibration_dir / "hidden_truth.csv")
    calibration_projection = project_second_life(
        calibration_decision, calibration_dir, seed=q4_seed + 330
    )
    calibration_screening, calibration_grades = screen_and_grade(
        calibration_decision, calibration_projection, calibration_dir
    )
    calibration_truth = true_second_life(
        calibration_hidden, calibration_decision, seed=q4_seed + 900
    )
    calibration_edges = probability_edge_table(
        calibration_decision,
        calibration_projection,
        calibration_screening,
        seed=q4_seed + 1200,
        truth=calibration_truth,
    )
    calibration_edges.to_csv(
        calibration_dir / "probability_edges_with_calibration_truth.csv",
        index=False,
        float_format="%.8f",
    )
    threshold = calibrate_probability_threshold(calibration_edges, calibration_dir)
    calibration_table = pd.read_csv(calibration_dir / "probability_threshold_calibration.csv")
    _copy_csv(calibration_table, "q4_probability_threshold_calibration.csv")

    holdout_decision = pd.read_csv(holdout_dir / "decision_pool.csv")
    holdout_projection = project_second_life(
        holdout_decision, holdout_dir, seed=q4_seed + 1330
    )
    holdout_screening, holdout_grades = screen_and_grade(
        holdout_decision, holdout_projection, holdout_dir
    )
    # No final-holdout truth is read above this line.
    holdout_edges = probability_edge_table(
        holdout_decision,
        holdout_projection,
```

```
        holdout_screening,
        seed=q4_seed + 2200,
    )
    holdout_edges.to_csv(holdout_dir / "probability_edges.csv", index=False, float_format="%.8f")
    nominal = nominal_graphs(holdout_decision, holdout_projection, holdout_screening)
    robust_graph = graphs_from_edges(holdout_edges, holdout_screening, p0=float(threshold["p0"]))
    graph_table = pd.concat(
        [graph_metrics(nominal, "nominal"), graph_metrics(robust_graph, "probabilistic")],
        ignore_index=True,
    )
    _copy_csv(graph_table, "q4_graph_metrics.csv")
    nominal_candidates = generate_groups(
        nominal, holdout_decision, holdout_projection, prefix="N", seed=q4_seed + 3000
    )
    robust_candidates = generate_groups(
        robust_graph, holdout_decision, holdout_projection, prefix="R", seed=q4_seed + 4000
    )
    if set(robust_candidates["Scenario"]) != {
        "S1_frequency_regulation", "S2_peak_shaving", "S3_backup"
    }:
        raise RuntimeError("Frozen p0 did not produce complete robust candidates for all applications")
    nominal_candidates.to_csv(holdout_dir / "nominal_candidate_groups.csv", index=False, float_format="%.8f")
    robust_candidates.to_csv(holdout_dir / "robust_candidate_groups.csv", index=False, float_format="%.8f")
    _copy_csv(nominal_candidates, "q4_nominal_candidate_groups.csv")
    _copy_csv(robust_candidates, "q4_robust_candidate_groups.csv")

    nominal_detailed, nominal_aggregate = evaluate_candidate_stress(
        nominal_candidates,
        holdout_decision,
        holdout_projection,
        seed=q4_seed + 5000,
        draws=400,
    )
    robust_detailed, robust_aggregate = evaluate_candidate_stress(
        robust_candidates,
        holdout_decision,
        holdout_projection,
        seed=q4_seed + 6000,
        draws=400,
    )
    _copy_csv(robust_aggregate, "q4_candidate_robust_metrics.csv")
    robust_detailed.to_csv(holdout_dir / "robust_candidate_stress_metrics.csv", index=False, float_format="%.8f")
    nominal_detailed.to_csv(holdout_dir / "nominal_candidate_stress_metrics.csv", index=False, float_format="%.8f")

    selections, comparison, solver_metadata = solve_strategies(
        nominal_candidates,
        nominal_detailed,
        robust_candidates,
        robust_detailed,
        robust_aggregate,
        holdout_decision["Battery_ID"].astype(str).tolist(),
        output_dir=holdout_dir,
    )
    _copy_csv(selections, "q4_strategy_selected_groups.csv")
    _copy_csv(comparison, "q4_strategy_comparison.csv")

    q3_selected = pd.read_csv(RESULT_TABLES / "q3_selected_groups.csv")
    q3_decision = pd.read_csv(RESULT_TABLES / "q3_decision_pool.csv")
    q3_projection = pd.read_csv(RESULT_TABLES / "q3_second_life_projections.csv")
    frozen_detailed, _ = evaluate_candidate_stress(
        q3_selected,
        q3_decision,
        q3_projection,
        seed=q4_seed + 7000,
        draws=400,
    )
    frozen_ids = set(q3_selected["Candidate_ID"].astype(str))
    frozen_rows = [
        _aggregate_selected(
            "Frozen_Q3_F014", stress_name, frozen_ids, frozen_detailed, len(q3_decision)
        )
        for stress_name in STRESS_SCENARIOS
    ]
    frozen_pressure = pd.DataFrame(frozen_rows)
    _copy_csv(frozen_pressure, "q4_frozen_q3_pressure.csv")

    freeze = {
```

```
        "frozen_before_final_holdout_truth_read": True,
        "p0": float(threshold["p0"]),
        "p0_selected_on_calibration_only": True,
        "strategy_selected_groups_sha256": _sha256(RESULT_TABLES / "q4_strategy_selected_groups.csv"),
        "strategy_comparison_sha256": _sha256(RESULT_TABLES / "q4_strategy_comparison.csv"),
        "holdout_decision_sha256": _sha256(holdout_dir / "decision_pool.csv"),
    }
    save_json(freeze, RESULT_TABLES / "q4_holdout_solution_freeze.json")

    # Final holdout truth is opened only after the policy membership hashes above exist.
    holdout_hidden = pd.read_csv(holdout_dir / "hidden_truth.csv")
    holdout_truth = true_second_life(
        holdout_hidden, holdout_decision, seed=q4_seed + 8000
    )
    holdout_truth.to_csv(holdout_dir / "hidden_truth_with_second_life.csv", index=False, float_format="%.8f")
    prediction_audit, rul_audit = _holdout_prediction_audit(
        holdout_decision, holdout_projection, holdout_truth, selections
    )
    _copy_csv(rul_audit, "q4_final_holdout_prediction_audit.csv")
    save_json(prediction_audit, RESULT_TABLES / "q4_holdout_prediction_metrics.json")

    grade_counts = holdout_grades["Grade"].value_counts().reindex(
        ["A", "B", "C", "Reject"], fill_value=0
    )
    robust_s0 = comparison.loc[
        (comparison["Strategy"] == "R_robust_unified")
        & (comparison["Stress_scenario"] == "S0_baseline")
    ].iloc[0]
    pipeline_summary = pd.DataFrame(
        [{
            "Holdout_cells": len(holdout_decision),
            "Grade_A": int(grade_counts["A"]),
            "Grade_B": int(grade_counts["B"]),
            "Grade_C": int(grade_counts["C"]),
            "Reject": int(grade_counts["Reject"]),
            "Reject_rate": float(grade_counts["Reject"] / len(holdout_decision)),
            "Eligible_rate": float(1.0 - grade_counts["Reject"] / len(holdout_decision)),
            "Nominal_candidates": len(nominal_candidates),
            "Robust_candidates": len(robust_candidates),
            "Robust_complete_groups": int(robust_s0["Complete_groups"]),
            "Robust_utilization_rate": float(robust_s0["Utilization_rate"]),
            "Robust_mean_inconsistency": float(robust_s0["Mean_inconsistency"]),
            "Robust_constraint_violation": float(robust_s0["Constraint_violation_rate"]),
            "Robust_expected_NetValue_REU": float(robust_s0["Expected_NetValue_REU"]),
            "SOH_MAE_percentage_points": prediction_audit["SOH_MAE_percentage_points"],
            "SecondLife_RUL_MAE_cycles": prediction_audit["SecondLife_RUL_MAE_cycles"],
        }]
    )
    _copy_csv(pipeline_summary, "q4_holdout_pipeline_summary.csv")

    recovery = recovery_ratios(comparison)
    _copy_csv(recovery, "q4_recovery_ratio.csv")
    break_even = break_even_analysis(comparison)
    _copy_csv(break_even, "q4_economic_break_even.csv")

    local, global_sensitivity, lhs = run_local_and_global_sensitivity(
        robust_candidates,
        robust_aggregate,
        pool_size=len(holdout_decision),
        output_dir=RESULT_TABLES,
        seed=q4_seed + 9000,
        lhs_samples=384,
    )
    q3_candidates = pd.read_csv(RESULT_TABLES / "q3_candidate_groups.csv")
    q3_pool = pd.read_csv(RESULT_TABLES / "q3_decision_pool.csv")
    size_sensitivity = candidate_size_sensitivity(
        q3_candidates,
        q3_pool["Battery_ID"].astype(str).tolist(),
        output_path=RESULT_TABLES / "q4_candidate_size_sensitivity.csv",
    )
    weight_region = pareto_weight_region(
        pd.read_csv(RESULT_TABLES / "q3_pareto_solutions.csv"),
        RESULT_TABLES / "q4_pareto_weight_region.csv",
        seed=q4_seed + 10000,
        samples=1024,
    )
    recommendations = _engineering_recommendations(comparison, threshold, break_even)
```

```
    _copy_csv(recommendations, "q4_engineering_recommendations.csv")
    dashboard = comparison[[
        "Strategy", "Stress_scenario", "Utilization_rate", "Expected_NetValue_REU",
        "Mismatch_probability", "Minimum_RUL_p05", "Constraint_violation_rate",
        "Group_survival_probability",
    ]].copy()
    dashboard["Interpretation_boundary"] = "semi-empirical simulation stress test; not a safety limit"
    _copy_csv(dashboard, "q4_robustness_dashboard.csv")

    top_global = (
        global_sensitivity.groupby("Parameter")["Abs_rho"].max().sort_values(ascending=False).head(5)
    )
    metrics = {
        "phase": "PHASE 4 final robustness and engineering optimization",
        "Q4_SEED": q4_seed,
        "calibration_cells": int(population["calibration_cells"]),
        "final_holdout_cells": int(population["final_holdout_cells"]),
        "p0": float(threshold["p0"]),
        "calibration_precision": float(
            calibration_table.loc[calibration_table["Selected"], "Precision"].iloc[0]
        ),
        "calibration_recall": float(
            calibration_table.loc[calibration_table["Selected"], "Recall"].iloc[0]
        ),
        "holdout_grades": {key: int(value) for key, value in grade_counts.items()},
        "nominal_candidate_groups": int(len(nominal_candidates)),
        "robust_candidate_groups": int(len(robust_candidates)),
        "frozen_q3_pressure": frozen_pressure.to_dict(orient="records"),
        "strategy_comparison": comparison.to_dict(orient="records"),
        "prediction_audit": prediction_audit,
        "global_sensitivity_top5": [
            {"Parameter": index, "Max_abs_Spearman_rho": float(value)}
            for index, value in top_global.items()
        ],
        "candidate_size_full_jaccard": float(
            size_sensitivity.loc[size_sensitivity["Candidate_size"] == "full", "Selected_cell_Jaccard_vs_full"].iloc[0]
        ),
        "pareto_modal_solution": str(weight_region.iloc[0]["Solution_ID"]),
        "pareto_modal_frequency": float(weight_region.iloc[0]["Selection_frequency"]),
        "runtime_seconds": float(time.perf_counter() - started),
    }
    save_json(metrics, RESULT_TABLES / "q4_key_metrics.json")

    validation = {
        "q4_independent_holdout": bool(population["calibration_holdout_ids_disjoint"]),
        "q2_q3_q4_ids_disjoint": bool(
            population["q3_ids_disjoint"] and population["q2_training_ids_disjoint"]
        ),
        "final_holdout_truth_not_used_for_p0": not bool(threshold["final_holdout_truth_used"]),
        "holdout_truth_read_after_strategy_freeze": bool(
            freeze["frozen_before_final_holdout_truth_read"]
        ),
        "all_six_stress_scenarios_evaluated": set(comparison["Stress_scenario"]) == set(STRESS_SCENARIOS),
        "nominal_adaptive_robust_evaluated": set(comparison["Strategy"]) == {
            "N_nominal", "A_scenario_adaptive", "R_robust_unified"
        },
        "probabilistic_graph_all_applications": set(robust_candidates["Scenario"]) == {
            "S1_frequency_regulation", "S2_peak_shaving", "S3_backup"
        },
        "global_sensitivity_complete": len(lhs) == 384 and len(global_sensitivity) >= 16 * 5,
        "economic_break_even_complete": len(break_even) == 3 * 6 * 3,
        "candidate_size_sensitivity_complete": set(size_sensitivity["Candidate_size"].astype(str)) == {"100", "199", "300",
        ↪  "full"},
        "weight_region_complete": int(weight_region["Selection_count"].sum()) == 1024,
        "scenario_registry_explicit": len(scenario_registry) == 6 * 7,
        "efficiency_bounds_physical": bool(
            pd.read_csv(RESULT_TABLES / "q4_sensitivity_parameter_registry.csv")
            .loc[lambda x: x["Parameter"] == "energy_efficiency_gate_percent", "High"].max() <= 100
        ),
    }
    validation["overall_pass"] = all(validation.values())
    save_json(validation, RESULT_TABLES / "q4_validation.json")
    if not validation["overall_pass"]:
        raise RuntimeError(f"Q4 validation failed: {validation}")
    return metrics


```

```
451 if __name__ == "__main__":
452     print(json.dumps(run_problem4(), indent=2, ensure_ascii=False))
```

src/problem4/plots.py

```
  1 """Five publication figures for Q4, selected after profiling the result tables."""
  2
  3 from __future__ import annotations
  4
  5 import json
  6 import shutil
  7
  8 import matplotlib.pyplot as plt
  9 import numpy as np
 10 import pandas as pd
 11 import seaborn as sns
 12
 13 from src.utils.project import PAPER_FIGURES, RESULT_FIGURES, RESULT_TABLES, save_json
 14
 15
 16 PALETTE = {
 17     "blue": "#0072B2",
 18     "orange": "#E69F00",
 19     "green": "#009E73",
 20     "red": "#D55E00",
 21     "purple": "#CC79A7",
 22     "gray": "#6B7280",
 23 }
 24
 25
 26 def _style() -> None:
 27     sns.set_theme(style="whitegrid", context="paper")
 28     plt.rcParams.update(
 29         {
 30             "font.family": "sans-serif",
 31             "font.sans-serif": ["Microsoft YaHei", "SimHei", "DejaVu Sans"],
 32             "font.size": 8.5,
 33             "axes.titlesize": 9.5,
 34             "axes.labelsize": 8.5,
 35             "legend.fontsize": 7.5,
 36             "xtick.labelsize": 7.5,
 37             "ytick.labelsize": 7.5,
 38             "pdf.fonttype": 42,
 39             "ps.fonttype": 42,
 40             "savefig.dpi": 300,
 41         }
 42     )
 43
 44
 45 def _save(fig: plt.Figure, name: str) -> None:
 46     RESULT_FIGURES.mkdir(parents=True, exist_ok=True)
 47     PAPER_FIGURES.mkdir(parents=True, exist_ok=True)
 48     for suffix in ("png", "pdf", "svg"):
 49         fig.savefig(RESULT_FIGURES / f"{name}.{suffix}", bbox_inches="tight")
 50     for suffix in ("png", "pdf"):
 51         shutil.copy2(RESULT_FIGURES / f"{name}.{suffix}", PAPER_FIGURES / f"{name}.{suffix}")
 52     plt.close(fig)
 53
 54
 55 def _short_scenarios() -> tuple[list[str], list[str]]:
 56     keys = [
 57         "S0_baseline",
 58         "S1_long_term_high_temperature",
 59         "S2_frequent_high_C_rate",
 60         "S3_high_batch_heterogeneity",
 61         "S4_temperature_plus_rate",
 62         "S5_heterogeneity_plus_operating_stress",
 63     ]
 64     labels = ["S0\n 基准", "S1\n 高温", "S2\n 高倍", "S3\n 异质", "S4\n 温 + 倍", "S5\n 异 + 应"]
 65     return keys, labels
 66
 67
 68 def _frozen_pressure() -> None:
 69     frame = pd.read_csv(RESULT_TABLES / "q4_frozen_q3_pressure.csv")
 70     keys, labels = _short_scenarios()
```

```
    frame = frame.set_index("Stress_scenario").loc[keys]
    x = np.arange(len(frame))
    fig, axes = plt.subplots(2, 2, figsize=(7.0, 5.4), layout="constrained")
    panels = [
        ("Expected_NetValue_REU", "Expected net value (REU)", PALETTE["blue"], "a"),
        ("Minimum_RUL_p05", "Minimum RUL, 5th percentile (cycles)", PALETTE["green"], "b"),
        ("Mismatch_probability", "Mean group mismatch probability", PALETTE["red"], "c"),
        ("Energy_loss_kWh", "Expected energy loss (kWh)", PALETTE["orange"], "d"),
    ]
    for ax, (column, ylabel, color, panel) in zip(axes.flat, panels):
        values = frame[column].to_numpy(dtype=float)
        ax.plot(x, values, color=color, marker="o", linewidth=1.8)
        ax.fill_between(x, 0, values, color=color, alpha=0.10)
        ax.set_xticks(x, labels)
        ax.set_ylabel(ylabel)
        ax.set_title(f"{panel}  Frozen F014 under six stress scenarios", loc="left")
        if "probability" in column:
            ax.set_ylim(0, 1)
    _save(fig, "q4_frozen_pressure")


def _probability_graph() -> None:
    calibration = pd.read_csv(RESULT_TABLES / "q4_probability_threshold_calibration.csv")
    graphs = pd.read_csv(RESULT_TABLES / "q4_graph_metrics.csv")
    fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.25), layout="constrained")
    for metric, color, marker in (
        ("Precision", PALETTE["blue"], "o"),
        ("Recall", PALETTE["orange"], "s"),
        ("F0_5", PALETTE["green"], "^"),
    ):
        axes[0].plot(
            calibration["p0"], calibration[metric], marker=marker, color=color,
            label={"Precision": " 精确率", "Recall": " 召回率", "F0_5": "F0.5"}[metric],
        )
    selected = calibration.loc[calibration["Selected"]].iloc[0]
    axes[0].axvline(selected["p0"], color=PALETTE["red"], linestyle="--", linewidth=1.1)
    axes[0].annotate(
        f" 冻结 $p_0$={selected['p0']:.2f}",
        (selected["p0"], selected["F0_5"]), xytext=(18, -34), textcoords="offset points",
        arrowprops={"arrowstyle": "->", "color": PALETTE["red"]}, fontsize=7.5,
    )
    axes[0].set(xlabel=" 兼容概率阈值 $p_0$", ylabel=" 校准指标", ylim=(0, 1.03))
    axes[0].set_title("(a) 仅使用校准池真值", loc="left")
    axes[0].legend(frameon=False)

    pivot = graphs.pivot(index="Scenario", columns="Graph_type", values="Density").reindex(
        ["S1_frequency_regulation", "S2_peak_shaving", "S3_backup"]
    )
    x = np.arange(len(pivot))
    width = 0.34
    axes[1].bar(x - width / 2, pivot["nominal"], width, color=PALETTE["gray"], label=" 名义图")
    axes[1].bar(x + width / 2, pivot["probabilistic"], width, color=PALETTE["blue"], label="$p_0$ 概率图")
    axes[1].set_xticks(x, [" 频率调节", " 削峰", " 备用"])
    axes[1].set(ylabel=" 图密度", ylim=(0, max(0.55, pivot.max().max() * 1.12)))
    axes[1].set_title("(b) 最终留出池的图稀疏化", loc="left")
    axes[1].legend(frameon=False)
    _save(fig, "q4_probability_graph")


def _strategy_comparison() -> None:
    frame = pd.read_csv(RESULT_TABLES / "q4_strategy_comparison.csv")
    keys, labels = _short_scenarios()
    strategies = ["N_nominal", "A_scenario_adaptive", "R_robust_unified"]
    names = ["N 名义", "A 自适应", "R 稳健"]
    colors = [PALETTE["gray"], PALETTE["orange"], PALETTE["blue"]]
    markers = ["o", "s", "^"]
    fig, axes = plt.subplots(2, 2, figsize=(7.0, 5.15), layout="constrained")
    panels = [
        ("Expected_NetValue_REU", " 期望净价值（REU）", "(a) 经济结果", None),
        ("Constraint_violation_rate", " 违约率", "(b) 约束风险", (0, 1)),
        ("Utilization_rate", " 利用率", "(c) 退役池利用率", (0, 0.55)),
        ("Minimum_RUL_p05", " 最小 RUL 的 5% 分位（次）", "(d) 使用寿命下界", (0, None)),
    ]
    base_x = np.arange(len(keys), dtype=float)
    offsets = [-0.16, 0.0, 0.16]
    for strategy, name, color, marker, offset in zip(strategies, names, colors, markers, offsets):
        part = frame.loc[frame["Strategy"] == strategy].set_index("Stress_scenario").loc[keys]
```

```
        for axis, (column, ylabel, title, ylim) in zip(axes.flat, panels):
            axis.scatter(
                base_x + offset,
                part[column],
                color=color,
                marker=marker,
                s=28,
                linewidths=0.7,
                edgecolors="white" if marker != "o" else color,
                label=name if column == "Expected_NetValue_REU" else None,
                zorder=3,
            )
            axis.set_ylabel(ylabel)
            axis.set_title(title, loc="left")
            axis.set_xticks(base_x, labels)
            if ylim is not None:
                axis.set_ylim(*ylim)
    axes[0, 0].axhline(0.0, color="0.35", linestyle="--", linewidth=0.8, zorder=1)
    handles, legend_labels = axes[0, 0].get_legend_handles_labels()
    fig.legend(
        handles,
        legend_labels,
        ncol=3,
        frameon=False,
        loc="upper center",
        bbox_to_anchor=(0.5, 1.025),
    )
    _save(fig, "q4_strategy_comparison")


def _global_sensitivity() -> None:
    frame = pd.read_csv(RESULT_TABLES / "q4_global_sensitivity.csv")
    parameter_order = (
        frame.groupby("Parameter")["Abs_rho"].max().sort_values(ascending=False).head(8).index
    )
    output_order = ["NetValue_REU", "Utilization_rate", "Mismatch_risk", "Reject_rate", "Inconsistency"]
    pivot = frame.pivot(index="Parameter", columns="Output", values="Spearman_rho").loc[
        parameter_order, output_order
    ]
    parameter_labels = {
        "temperature_delta_C": " 温升",
        "c_rate_multiplier": " 倍率因子",
        "batch_heterogeneity_multiplier": " 批次异质性因子",
        "prediction_uncertainty_multiplier": " 预测不确定性因子",
        "revenue_multiplier": " 收益因子",
        "reassembly_cost_multiplier": " 重组成本因子",
        "detection_cost_multiplier": " 检测成本因子",
        "om_cost_rate": " 运维成本率",
        "energy_loss_cost_REU_per_kWh": " 能损成本",
        "soh_tolerance_multiplier": "SOH 容差因子",
        "rul_tolerance_multiplier": "RUL 容差因子",
        "capacity_tolerance_multiplier": " 容量容差因子",
        "resistance_tolerance_multiplier": " 内阻容差因子",
        "p0": "$p_0$",
        "minimum_RUL_gate_multiplier": " 最小 RUL 门槛因子",
        "energy_efficiency_gate_percent": " 能效门槛",
    }
    output_labels = {
        "NetValue_REU": " 净价值",
        "Utilization_rate": " 利用率",
        "Mismatch_risk": " 不匹配风险",
        "Reject_rate": " 拒用率",
        "Inconsistency": " 不一致性",
    }
    labels = [parameter_labels[value] for value in pivot.index]
    fig, ax = plt.subplots(figsize=(7.0, 4.4), layout="constrained")
    sns.heatmap(
        pivot,
        cmap=sns.diverging_palette(240, 15, as_cmap=True),
        center=0,
        vmin=-1,
        vmax=1,
        annot=True,
        fmt=".2f",
        linewidths=0.5,
        cbar_kws={"label": "Spearman 秩相关系数"},
        ax=ax,
```

```
    )
    ax.set_yticklabels(labels, rotation=0)
    ax.set_xticklabels([output_labels[value] for value in output_order], rotation=20, ha="right")
    ax.set(xlabel=" 策略输出", ylabel="LHS 参数")
    ax.set_title(" 全局敏感性: 384 点 LHS 的 Spearman 秩相关", loc="left")
    _save(fig, "q4_global_sensitivity")


def _economic_and_scale() -> None:
    economic = pd.read_csv(RESULT_TABLES / "q4_economic_break_even.csv")
    size = pd.read_csv(RESULT_TABLES / "q4_candidate_size_sensitivity.csv")
    keys, labels = _short_scenarios()
    robust = economic.loc[economic["Strategy"] == "R_robust_unified"]
    fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.45), layout="constrained")
    for basis, name, color, marker in (
        ("expected", " 期望", PALETTE["blue"], "o"),
        ("conservative_CVaR20", "CVaR20", PALETTE["orange"], "s"),
        ("worst_sample", " 最差样本", PALETTE["red"], "^")
    ):
        part = robust.loc[robust["Basis"] == basis].set_index("Stress_scenario").loc[keys]
        axes[0].plot(range(6), part["Minimum_revenue_coefficient_REU_per_kWh"], color=color, marker=marker, label=name)
    axes[0].set_xticks(range(6), labels)
    axes[0].set_ylabel(" 盈亏平衡收益（REU/kWh）")
    axes[0].set_title("(a) 稳健方案盈亏平衡", loc="left")
    axes[0].legend(frameon=False)

    x = np.arange(len(size))
    jaccard = size["Selected_cell_Jaccard_vs_full"].to_numpy(dtype=float)
    bar_colors = plt.cm.Blues(0.35 + 0.55 * jaccard)
    bars = axes[1].bar(x, size["Objective_NetValue_REU"], color=bar_colors, edgecolor="#374151", linewidth=0.5)
    candidate_labels = ["599" if str(value) == "full" else str(value) for value in size["Candidate_size"]]
    axes[1].set_xticks(x, candidate_labels)
    axes[1].set(xlabel=" 候选列数", ylabel=" 选中方案净价值（REU）")
    axes[1].set_ylim(0, float(size["Objective_NetValue_REU"].max()) * 1.18)
    axes[1].set_title("(b) 候选空间规模敏感性", loc="left")
    for bar, utilization, similarity in zip(bars, size["Utilization_rate"], jaccard):
        axes[1].text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height(),
            f" 利用率 {utilization:.0%}\nJ={similarity:.2f}",
            ha="center",
            va="bottom",
            fontsize=7,
        )
    _save(fig, "q4_economic_and_scale")


def create_problem4_figures() -> pd.DataFrame:
    _style()
    profile = {
        "strategy_rows": len(pd.read_csv(RESULT_TABLES / "q4_strategy_comparison.csv")),
        "stress_scenarios": 6,
        "strategies": 3,
        "global_sensitivity_rows": len(pd.read_csv(RESULT_TABLES / "q4_global_sensitivity.csv")),
        "candidate_size_levels": 4,
        "chart_selection": "lines for ordered stress levels; heatmap for repeated parameter-output mapping; no dual-axis except
        ↪  the compact scale audit where units are explicit",
    }
    save_json(profile, RESULT_TABLES / "q4_figure_data_profile.json")
    _frozen_pressure()
    _probability_graph()
    _strategy_comparison()
    _global_sensitivity()
    _economic_and_scale()
    manifest = pd.DataFrame(
        [
            ("q4_frozen_pressure", "Frozen F014 pressure response across six explicitly parameterized stress scenarios."),
            ("q4_probability_graph", "Calibration-only p0 selection and final-holdout graph sparsification."),
            ("q4_strategy_comparison", "Nominal, adaptive and robust value-risk-utilization trade-offs."),
            ("q4_global_sensitivity", "Numerical global sensitivity rankings from 384 LHS samples."),
            ("q4_economic_and_scale", "Break-even economics and candidate-column stability."),
        ],
        columns=["Figure", "Claim"],
    )
    manifest.to_csv(RESULT_TABLES / "q4_figure_manifest.csv", index=False)
    return manifest
```

```

if __name__ == "__main__":
    print(create_problem4_figures().to_string(index=False))
```

src/problem4/paper.py

```
"""Generate traceable LaTeX macros and table rows for the final paper."""

from __future__ import annotations

from pathlib import Path

import pandas as pd

from src.utils.project import ROOT, RESULT_TABLES, load_json


PAPER = ROOT / "paper"


def _pct(value: float, digits: int = 1) -> str:
    return f"{100.0 * float(value):.{digits}f}\\%"


def _macro(name: str, value: object) -> str:
    return f"\\newcommand{{\\{name}}}{{{value}}}"


def _scenario_short(value: str) -> str:
    return str(value).split("_", 1)[0]


def _write(path: Path, lines: list[str]) -> None:
    path.write_text("\n".join(lines) + "\n", encoding="utf-8")


def generate_q4_paper_artifacts() -> dict[str, Path]:
    """Write all Q4 values used by the manuscript directly from result files."""
    PAPER.mkdir(parents=True, exist_ok=True)
    q4 = load_json(RESULT_TABLES / "q4_key_metrics.json")
    q3 = load_json(RESULT_TABLES / "q3_key_metrics.json")
    comparison = pd.read_csv(RESULT_TABLES / "q4_strategy_comparison.csv")
    recovery = pd.read_csv(RESULT_TABLES / "q4_recovery_ratio.csv")
    break_even = pd.read_csv(RESULT_TABLES / "q4_economic_break_even.csv")
    size = pd.read_csv(RESULT_TABLES / "q4_candidate_size_sensitivity.csv")
    weights = pd.read_csv(RESULT_TABLES / "q4_pareto_weight_region.csv")
    full_audit = pd.read_csv(RESULT_TABLES / "q3_full_candidate_space_audit.csv")
    economic = pd.read_csv(RESULT_TABLES / "q3_economic_viability_tradeoff.csv")

    audit_reduced = full_audit.loc[full_audit["Candidate_space"] == "reduced_199"].iloc[0]
    audit_full = full_audit.loc[full_audit["Candidate_space"] == "full_599"].iloc[0]
    economic_safe = economic.loc[economic["Mode"] == "Nonnegative-group-value mode"].iloc[0]
    robust_s0 = comparison.loc[
        (comparison["Strategy"] == "R_robust_unified")
        & (comparison["Stress_scenario"] == "S0_baseline")
    ].iloc[0]
    prediction = q4["prediction_audit"]
    macros = [
        "% Auto-generated by src/problem4/paper.py; do not edit numerical values by hand.",
        _macro("QFourCalibrationCells", q4["calibration_cells"]),
        _macro("QFourHoldoutCells", q4["final_holdout_cells"]),
        _macro("QFourPZero", f"{q4['p0']:.2f}"),
        _macro("QFourCalibrationPrecision", _pct(q4["calibration_precision"])),
        _macro("QFourCalibrationRecall", _pct(q4["calibration_recall"])),
        _macro("QFourGradeA", q4["holdout_grades"]["A"]),
        _macro("QFourGradeB", q4["holdout_grades"]["B"]),
        _macro("QFourGradeC", q4["holdout_grades"]["C"]),
        _macro("QFourReject", q4["holdout_grades"]["Reject"]),
        _macro("QFourRejectRate", _pct(q4["holdout_grades"]["Reject"] / q4["final_holdout_cells"])),
        _macro("QFourNominalCandidates", q4["nominal_candidate_groups"]),
        _macro("QFourRobustCandidates", q4["robust_candidate_groups"]),
        _macro("QFourSOHMAE", f"{prediction['SOH_MAE_percentage_points']:.3f}"),
        _macro("QFourRULMAE", f"{prediction['SecondLife_RUL_MAE_cycles']:.1f}"),
        _macro("QFourRULCoverage", _pct(prediction["SecondLife_RUL_interval_coverage"])),
        _macro("QFourRealizedMismatch", _pct(prediction["Robust_selected_realized_base_mismatch_rate"])),
```

```
        _macro("QFourRobustGroups", int(robust_s0["Complete_groups"])),
        _macro("QFourRobustUtilization", _pct(robust_s0["Utilization_rate"])),
        _macro("QFourRobustValue", f"{robust_s0['Expected_NetValue_REU']:.2f}"),
        _macro("QFourRobustViolation", _pct(robust_s0["Constraint_violation_rate"])),
        _macro("QFourParetoModal", q4["pareto_modal_solution"]),
        _macro("QFourParetoModalShare", _pct(q4["pareto_modal_frequency"])),
        _macro("QThreeReducedChoice", audit_reduced["Selected_compromise"]),
        _macro("QThreeFullChoice", audit_full["Selected_compromise"]),
        _macro("QThreeReducedColumns", int(audit_reduced["Candidate_columns"])),
        _macro("QThreeFullColumns", int(audit_full["Candidate_columns"])),
        _macro("QThreeFullMaxUtil", _pct(audit_full["Maximum_utilization"])),
        _macro("QThreeFullMaxValue", f"{audit_full['Maximum_NetValue_REU']:.2f}"),
        _macro("QThreeEconomicGroups", int(economic_safe["Groups"])),
        _macro("QThreeEconomicUtil", _pct(economic_safe["Utilization_rate"])),
        _macro("QThreeEconomicValue", f"{economic_safe['NetValue_REU']:.2f}"),
        _macro("QThreeEconomicNegativeGroups", int(economic_safe["Negative_value_groups"])),
        _macro("QThreeWeightModalShare", _pct(q3["phase3_5"]["weight_modal_share"])),
    ]
    macro_path = PAPER / "generated_metrics_q4.tex"
    _write(macro_path, macros)

    pressure_lines: list[str] = []
    for row in pd.DataFrame(q4["frozen_q3_pressure"]).itertuples(index=False):
        pressure_lines.append(
            f"{_scenario_short(row.Stress_scenario)} & {row.Utilization_rate * 100:.1f}\\% & "
            f"{row.Expected_NetValue_REU:.2f} & {row.Energy_loss_kWh:.2f} & "
            f"{row.Mismatch_probability * 100:.1f}\\% & {row.Minimum_RUL_p05:.1f} & "
            f"{row.Constraint_violation_rate * 100:.1f}\\% \\\\"
        )
    pressure_path = PAPER / "generated_q4_pressure_rows.tex"
    _write(pressure_path, pressure_lines)

    strategy_names = {
        "N_nominal": "N 名义固定",
        "A_scenario_adaptive": "A 场景自适应",
        "R_robust_unified": "R 统一稳健",
    }
    strategy_lines: list[str] = []
    for row in comparison.itertuples(index=False):
        strategy_lines.append(
            f"{strategy_names[row.Strategy]} & {_scenario_short(row.Stress_scenario)} & "
            f"{int(row.Complete_groups)} & {row.Utilization_rate * 100:.1f}\\% & "
            f"{row.Expected_NetValue_REU:.2f} & {row.CVaR20_NetValue_REU:.2f} & "
            f"{row.Mismatch_probability * 100:.1f}\\% & {row.Minimum_RUL_p05:.1f} & "
            f"{row.Constraint_violation_rate * 100:.1f}\\% \\\\"
        )
    strategy_path = PAPER / "generated_q4_strategy_rows.tex"
    _write(strategy_path, strategy_lines)

    recovery_lines: list[str] = []
    for row in recovery.iloc[1:].itertuples(index=False):
        recovery_lines.append(
            f"{_scenario_short(row.Stress_scenario)} & "
            f"{100 * row.Adaptive_NetValue_recovery_ratio:.1f}\\% & "
            f"{100 * row.Adaptive_minimum_RUL_recovery_ratio:.1f}\\% & "
            f"{100 * row.Adaptive_violation_recovery_ratio:.1f}\\% & "
            f"{100 * row.Robust_NetValue_retention_ratio:.1f}\\% & "
            f"{100 * row.Robust_violation_reduction_ratio:.1f}\\% \\\\"
        )
    recovery_path = PAPER / "generated_q4_recovery_rows.tex"
    _write(recovery_path, recovery_lines)

    robust_break_even = break_even.loc[break_even["Strategy"] == "R_robust_unified"]
    break_even_lines: list[str] = []
    basis_names = {
        "expected": " 期望",
        "conservative_CVaR20": "CVaR20",
        "worst_sample": " 最差样本",
    }
    for scenario, frame in robust_break_even.groupby("Stress_scenario", sort=False):
        values = frame.set_index("Basis")
        parts = [_scenario_short(scenario)]
        for basis in ("expected", "conservative_CVaR20", "worst_sample"):
            parts.append(f"{values.loc[basis, 'Minimum_revenue_coefficient_REU_per_kWh']:.3f}")
        worst_cost = values.loc["worst_sample", "Maximum_allowable_reassembly_cost_REU_per_group"]
        parts.append(f"{worst_cost:.2f}")
        break_even_lines.append(" & ".join(parts) + " \\\\")
```

```
    break_even_path = PAPER / "generated_q4_break_even_rows.tex"
    _write(break_even_path, break_even_lines)

    sensitivity_names = {
        "energy_efficiency_gate_percent": " 能效门槛",
        "p0": "$p_0$",
        "batch_heterogeneity_multiplier": " 批次异质性因子",
        "prediction_uncertainty_multiplier": " 预测不确定性因子",
        "energy_loss_cost_REU_per_kWh": " 能损成本（REU/kWh）",
    }
    sensitivity_lines: list[str] = []
    for rank, item in enumerate(q4["global_sensitivity_top5"], start=1):
        parameter = str(item["Parameter"])
        sensitivity_lines.append(
            f"{rank} & {sensitivity_names.get(parameter, parameter.replace('_', r'\_'))} & "
            f"{item['Max_abs_Spearman_rho']:.3f} \\\\"
        )
    sensitivity_path = PAPER / "generated_q4_sensitivity_rows.tex"
    _write(sensitivity_path, sensitivity_lines)

    size_lines: list[str] = []
    for row in size.itertuples(index=False):
        size_lines.append(
            f"{row.Candidate_size} & {int(row.Candidate_columns)} & {row.Runtime_seconds:.3f} & "
            f"{row.Objective_NetValue_REU:.2f} & {row.Utilization_rate * 100:.1f}\\% & "
            f"{row.Selected_cell_Jaccard_vs_full:.3f} \\\\"
        )
    size_path = PAPER / "generated_q4_size_rows.tex"
    _write(size_path, size_lines)

    weight_lines: list[str] = []
    for row in weights.head(7).itertuples(index=False):
        weight_lines.append(
            f"{row.Solution_ID} & {int(row.Selection_count)} & {row.Selection_frequency * 100:.1f}\\% & "
            f"{row.Mean_weight_value:.3f} & {row.Mean_weight_utilization:.3f} & "
            f"{row.Mean_weight_consistency:.3f} \\\\"
        )
    weight_path = PAPER / "generated_q4_weight_rows.tex"
    _write(weight_path, weight_lines)

    return {
        "macros": macro_path,
        "pressure": pressure_path,
        "strategy": strategy_path,
        "recovery": recovery_path,
        "break_even": break_even_path,
        "sensitivity": sensitivity_path,
        "size": size_path,
        "weights": weight_path,
    }


if __name__ == "__main__":
    for name, path in generate_q4_paper_artifacts().items():
        print(f"{name}: {path}")
```

C　参数配置与运行环境

requirements.txt

```
numpy==2.2.6
pandas==2.2.3
scipy==1.15.3
networkx==3.4.2
scikit-learn==1.6.1
statsmodels==0.14.4
matplotlib==3.10.3
seaborn==0.13.2
```

```
Pillow==11.2.1
joblib==1.5.1
pypdf==5.5.0
sympy==1.14.0
```

simulation_config.json

```
{
  "random_seed": 20260809,
  "n_batteries": 360,
  "n_batches": 6,
  "max_cycles": 1800,
  "nominal_capacity_ah": 2.5,
  "nominal_internal_resistance_mohm": 35.0,
  "eol_soh_percent": 80.0,
  "reference_cycles": 1000.0,
  "base_fade_coefficient": 0.065,
  "base_fade_exponent": 1.08,
  "mid_acceleration_coefficient": 0.027,
  "mid_acceleration_exponent": 1.35,
  "knee_acceleration_coefficient": 0.055,
  "knee_acceleration_exponent": 1.85,
  "base_knee_cycle": 1180.0,
  "capacity_measurement_sd_ah": 0.0025,
  "resistance_measurement_sd_mohm": 0.18,
  "cycle_storage_step": 1,
  "analysis_cycle_stride": 12,
  "bootstrap_repetitions": 120,
  "permutation_repetitions": 12
}
```

parameter_registry.csv

```
parameter,symbol,value_or_range,unit,category,basis,source_url,notes
Random seed,s,20260809,dimensionless,simulation setting,Reproducibility convention,,Fixed across the complete pipeline
Population size,N,360,batteries,simulation setting,Task requirement,,Within the requested 300-1000 range
Batch count,B,6,batches,simulation setting,Experimental-design choice,,Creates manufacturing heterogeneity
Maximum observation cycle,n_max,1800,cycles,simulation setting,Computational and scenario design,,Right-censored if EOL is not reached
Nominal initial capacity,Q_nom,2.5,Ah,simulation setting,Representative cell-scale choice,,Not claimed as a measured cell specification
Nominal initial resistance,R_nom,35,mOhm,simulation setting,Representative cell-scale choice,,Not claimed as a measured cell specification
Temperature range,T,5-45,degC,simulation setting,Literature-supported stress range,https://doi.org/10.1016/j.jpowsour.2010.11.134,Exact sampling distribution is synthetic
Charge C-rate range,C_ch,0.5-2.0,C,simulation setting,Literature-supported stress variable,https://doi.org/10.1016/j.jpowsour.2010.11.134,Exact levels are synthetic
Discharge C-rate range,C_dis,0.5-2.0,C,simulation setting,Literature-supported stress variable,https://doi.org/10.1016/j.jpowsour.2010.11.134,Exact levels are synthetic
Depth-of-discharge range,DOD,50-100,percent,simulation setting,Literature-supported stress variable,https://doi.org/10.1016/j.jpowsour.2010.11.134,Exact distribution is synthetic
Simulated end-of-life threshold,SOH_EOL,80,percent,simulation setting,Common capacity-fade reference point in research and engineering analysis,https://doi.org/10.2172/2430790,Not a universal mandatory automotive retirement standard; real EOL also depends on power energy calendar life and application requirements
Reference cycles,n_ref,1000,cycles,simulation setting,Scaling choice,,Dimensionless cycle normalization
Base fade coefficient,a0,0.065,capacity fraction,simulation setting,Plausibility calibration,,Not a fitted literature coefficient
Base fade exponent,b0,1.08,dimensionless,simulation setting,Power-law aging structure,https://doi.org/10.1016/j.jpowsour.2010.11.134,Exponent is a simulation setting
Mid-stage coefficient,a1,0.027,capacity fraction,simulation setting,Plausibility calibration,,Introduces gradual acceleration before the knee
Knee coefficient,a2,0.055,capacity fraction,simulation setting,Knee-mechanism abstraction,https://arxiv.org/abs/2201.02891,Not a fitted electrochemical parameter
Baseline knee cycle,k0,1180,cycles,simulation setting,Plausibility calibration,,Battery-level knee is stress-adjusted and heterogeneous
Calendar/cycle stress separation,-,semi-empirical,dimensionless,literature structure,Holistic aging framework,https://doi.org/10.1016/j.jpowsour.2014.02.012,Only the structural idea is borrowed
High temperature/SOC aging direction,-,positive effect,dimensionless,literature direction,Calendar-aging evidence,https://mediatum.ub.tum.de/doc/1651485/1651485.pdf,No coefficient copied
Capacity-resistance coupling,-,positive resistance growth,dimensionless,simulation setting,Physical plausibility,,Association is imposed without claiming causality
```

```
22  Capacity measurement noise,sigma_Q,0.0025,Ah,simulation setting,Measurement-noise choice,,Zero-mean Gaussian noise
23  Resistance measurement noise,sigma_R,0.18,mOhm,simulation setting,Measurement-noise choice,,Zero-mean Gaussian noise
```